

SQL for Data Analyst Interviews

A complete handbook — fundamentals to interview-ready analytical SQL

- PostgreSQL as the primary dialect, with notes on SQL Server, MySQL, BigQuery and Snowflake
- 20 parts · 200 practice questions · 10 mock interviews · 22 analytical patterns
- Built around two realistic schemas: a UK e-commerce business and an NHS trust

Contents

SQL Data Analyst Interview Handbook (PostgreSQL)	9
How to use it	9
Schema A — RetailCo (e-commerce)	9
Schema B — NHS Trust service data	10
Sample data	11
Parts 1–5: Fundamentals, Filtering, Aggregation, Joins, CASE	13
PART 1 — SQL FUNDAMENTALS	14
1.1 What SQL is	14
1.2 Relational databases	14
1.3 Tables, rows, columns, and grain	14
1.4 Primary keys	15
1.5 Foreign keys and referential integrity	15
1.6 Relationships	16
1.7 NULL	16
1.8 Data types	16
1.9 SELECT and FROM	17
1.10 Logical order of evaluation	17
1.11 WHERE	18
1.12 ORDER BY	18
1.13 LIMIT and OFFSET	18
1.14 DISTINCT	19
PART 2 — FILTERING AND LOGICAL OPERATIONS	20
2.1 Three-valued logic	20
2.2 Comparison operators	20
2.3 AND, OR, NOT — and precedence	21
2.4 IN and NOT IN	21
2.5 BETWEEN	21
2.6 LIKE and ILIKE	21
2.7 IS NULL / IS NOT NULL	22
2.8 Progressive exercises	22
PART 3 — AGGREGATIONS	25
3.1 The aggregate functions	25
3.2 GROUP BY	26
3.3 HAVING, and WHERE vs HAVING	26

3.4 Worked business examples	27
3.5 Aggregation exercises	28
PART 4 — JOINS	30
4.1 Mental model	30
4.2 INNER JOIN	30
4.3 LEFT JOIN	30
4.4 RIGHT JOIN	31
4.5 FULL OUTER JOIN	31
4.6 CROSS JOIN	31
4.7 SELF JOIN	32
4.8 Join keys, duplicates and join explosion	32
4.9 Progressive join exercises	33
PART 5 — CASE AND CONDITIONAL LOGIC	36
5.1 Syntax	36
5.2 Customer segmentation	36
5.3 Revenue bands and age groups	36
5.4 Conditional aggregation	37
5.5 CASE elsewhere in the query	38
5.6 Nested CASE	38
5.7 Risk, KPI and pass/fail classification	38
5.8 CASE exercises	39
Parts 6–8: NULLs & Data Cleaning, Strings, Dates	41
PART 6 — NULLS AND DATA CLEANING	42
6.1 What NULL actually means	42
6.2 NULL vs 0 vs empty string	42
6.3 NULL propagation in arithmetic and concatenation	42
6.4 COALESCE	43
6.5 NULLIF	43
6.6 Aggregates, joins and NULL	43
6.7 The classic NULL interview traps	43
6.8 Duplicate records	44
6.9 Deduplication techniques	44
6.10 Invalid values and standardisation	45
6.11 A complete cleaning pipeline	45
6.12 NULL and cleaning exercises	46
PART 7 — STRING FUNCTIONS	48

7.1 Concatenation	48
7.2 Case and trimming	48
7.3 Length, padding, position	48
7.4 Substring extraction	48
7.5 REPLACE and TRANSLATE	49
7.6 SPLIT_PART and string_to_array	49
7.7 Regular expressions	49
7.8 Practical cleaning examples	49
7.9 String exercises	50
PART 8 — DATE AND TIME ANALYSIS	52
8.1 Types	52
8.2 Current date and time	52
8.3 INTERVAL and date arithmetic	52
8.4 DATE_TRUNC — the workhorse	53
8.5 EXTRACT and DATE_PART	53
8.6 AGE and durations	54
8.7 Working days	54
8.8 Period-over-period analysis	54
8.9 Recency, lifetime and time-to-event	55
8.10 Date exercises	56
Parts 9–11: Subqueries, CTEs, Window Functions	58
PART 9 — SUBQUERIES	59
9.1 Scalar subqueries	59
9.2 Subqueries in FROM (derived tables)	59
9.3 Subqueries in WHERE	59
9.4 Correlated subqueries	60
9.5 EXISTS and NOT EXISTS	60
9.6 Subquery vs JOIN — when to use which	61
9.7 Subquery exercises	61
PART 10 — COMMON TABLE EXPRESSIONS (CTEs)	63
10.1 Why analysts live in CTEs	63
10.2 Syntax and multiple CTEs	63
10.3 Chained CTEs — a full analytical pipeline	63
10.4 Materialisation and performance	64
10.5 Recursive CTEs	64
10.6 CTE exercises	65

PART 11 — WINDOW FUNCTIONS **67**

11.1 The core idea	67
11.2 OVER, PARTITION BY, ORDER BY	67
11.3 Window frames	67
11.4 Ranking: ROW_NUMBER, RANK, DENSE_RANK	68
11.5 Offset functions: LAG, LEAD, FIRST_VALUE, LAST_VALUE	69
11.6 Aggregate window functions	69
11.7 The problem patterns	69
11.8 Window function exercises	71

Part 12 — Advanced Analytical SQL **73**

12.1 Top N	73
12.2 Greatest-per-group	73
12.3 Deduplication	74
12.4 Gaps and islands	74
12.5 Running totals	75
12.6 Rolling averages	75
12.7 Percent of total	76
12.8 Ranking	76
12.9 Cohort analysis	76
12.10 Retention	77
12.11 Churn	77
12.12 Funnel analysis	78
12.13 Conversion rates	79
12.14 Customer segmentation (RFM)	79
12.15 Repeat purchases	80
12.16 First and last purchase	80
12.17 Session analysis (sessionisation)	80
12.18 Consecutive events	81
12.19 Anomaly detection	82
12.20 Pareto analysis	82
12.21 Contribution analysis	82
12.22 Putting it together — a churn early-warning query	83

Parts 13–15: Business KPIs, Performance, Database Design **85**

PART 13 — BUSINESS KPI SQL **86**

13.1 Revenue	86
13.2 Profit and gross margin	86
13.3 Average order value	87

13.4 Conversion rate	87
13.5 Retention rate	87
13.6 Churn rate	87
13.7 Customer acquisition	87
13.8 Customer lifetime value	88
13.9 Repeat purchase rate	88
13.10 Active users — DAU, MAU, stickiness	88
13.11 Growth rate	89
13.12 Market share	89
13.13 Average waiting time	89
13.14 SLA compliance and breach rate	90
PART 14 — PERFORMANCE AND QUERY OPTIMISATION	91
14.1 How a query executes	91
14.2 Indexes	91
14.3 Scan types	91
14.4 EXPLAIN and EXPLAIN ANALYZE	92
14.5 Filtering early	92
14.6 Join optimisation	93
14.7 Aggregation optimisation	93
14.8 CTE considerations	93
14.9 Avoiding SELECT *	93
14.10 Large table considerations	94
14.11 A realistic optimisation walkthrough	94
PART 15 — DATABASE DESIGN FUNDAMENTALS	95
15.1 Normalisation	95
15.2 Denormalisation	95
15.3 Star schema	95
15.4 Slowly changing dimensions	96
15.5 Referential integrity	97
15.6 OLTP vs OLAP	97
15.7 How design affects your SQL	97
15.8 Design interview questions	98
Part 16a — 50 Beginner SQL Interview Questions	99
Self-check	106
Part 16b — 75 Intermediate SQL Interview Questions	107
Self-check	119

Part 16c — 75 Advanced SQL Interview Questions	120
Self-check	132
Parts 17–19: Pattern Recognition, Mock Interviews, Debugging	133
PART 17 — SQL INTERVIEW PATTERN RECOGNITION	134
17.1 Master cheat sheet	134
17.2 Reflexes worth drilling	135
17.3 Phrase → clause quick map	136
PART 18 — TEN MOCK SQL INTERVIEWS	137
Mock 1 — Junior: retail basics	137
Mock 2 — Junior: aggregation and filtering	137
Mock 3 — Intermediate: the grain trap	138
Mock 4 — Intermediate: window functions	138
Mock 5 — Intermediate: the NULL trap	139
Mock 6 — Strong: cohort retention	139
Mock 7 — Strong: funnel and drop-off	140
Mock 8 — Strong: NHS waiting times	140
Mock 9 — Advanced: debugging someone else's query	141
Mock 10 — Advanced: open-ended business problem	142
PART 19 — SQL DEBUGGING	144
19.1 A debugging method	144
19.2 Broken queries — find the bug	144
19.3 Error messages and what they usually mean	147
19.4 Sanity checks to run before shipping any query	147
Part 20 — Cheat Sheets, and the SQL Mastery Checklist	149
20.1 Query syntax and evaluation order	149
20.2 JOIN decision tree	149
20.3 GROUP BY and HAVING	149
20.4 CASE	149
20.5 NULL handling	150
20.6 Date functions	150
20.7 String functions	151
20.8 Window functions	151
20.9 CTEs and subqueries	151
20.10 Common analytical patterns	152
20.11 KPI formulas	152
20.12 Dialect differences	153

FINAL SQL MASTERY CHECKLIST	154
Level 1 — Basic SQL	154
Level 2 — Job-ready SQL	154
Level 3 — Interview-ready SQL	154
Level 4 — Strong Data Analyst SQL	155
Level 5 — Advanced SQL	155
The last word	156

SQL Data Analyst Interview Handbook (PostgreSQL)

Built for UK Data Analyst / Junior Analyst / Graduate Analyst / Insight Analyst interviews.

Every example in this handbook uses one of two schemas defined below. Learn these two schemas once and you can read any query in the handbook without scrolling back.

How to use it

Read Parts 1–11 in order. They are cumulative. Parts 12–15 are the difference between "can write SQL" and "is a analyst". Part 16 is drill material — do not read the solutions first. Part 17 is what you revise the night before. Part 18 is rehearsal out loud.

File	Contents
01-fundamentals-to-case.md	Parts 1–5: fundamentals, filtering, aggregation, joins, CASE
02-cleaning-strings-dates.md	Parts 6–8: NULLs & cleaning, strings, dates
03-subqueries-ctes-windows.md	Parts 9–11: subqueries, CTEs, window functions
04-advanced-analytical-sql.md	Part 12: the 22 analyst patterns
05-kpis-performance-design.md	Parts 13–15: KPI SQL, optimisation, schema design
06-question-bank-beginner.md	Part 16a: 50 beginner questions
07-question-bank-intermediate.md	Part 16b: 75 intermediate questions
08-question-bank-advanced.md	Part 16c: 75 advanced questions
09-patterns-mocks-debugging.md	Parts 17–19: pattern recognition, 10 mock interviews, debugging
10-cheatsheets-and-checklist.md	Part 20 + the five-level mastery checklist

Dialect note: everything is PostgreSQL unless flagged. Blocks marked **Dialect** tell you what changes in SQL Server, MySQL, BigQuery or Snowflake — UK analyst roles are split roughly between Postgres, SQL Server and Snowflake, and interviewers frequently ask "how would that differ in T-SQL?"

Schema A — RetailCo (e-commerce)

This is the workhorse. Most retail, marketing, fintech and product-analyst interviews use a schema shaped like this.

```
CREATE TABLE customers (  
  customer_id integer PRIMARY KEY,  
  first_name  text          NOT NULL,  
  last_name   text          NOT NULL,  
  email       text          UNIQUE,  
  signup_date date          NOT NULL,  
  country     text,         -- 'UK', 'IE', 'FR', ...  
  city        text,  
  marketing_opt_in boolean DEFAULT false,  
  channel     text          -- 'organic', 'paid_search', 'email', 'referral'  
);  
  
CREATE TABLE products (  
  product_id integer PRIMARY KEY,  
  product_name text      NOT NULL,  
  category    text,      -- 'Electronics', 'Home', 'Apparel', 'Grocery'  
  subcategory text,  
  unit_price  numeric(10,2) NOT NULL,  
  unit_cost   numeric(10,2) NOT NULL,  
  is_active   boolean     DEFAULT true  
);  
  
CREATE TABLE orders (  
  order_id      integer PRIMARY KEY,  
  customer_id   integer REFERENCES customers(customer_id),  
  order_ts      timestamp NOT NULL,  
  status        text      NOT NULL, -- 'completed', 'cancelled', 'refunded', 'pending'
```

```

channel          text,                -- 'web', 'app', 'phone'
shipping_cost    numeric(10,2) DEFAULT 0,
discount_code    text
);

CREATE TABLE order_items (
  order_item_id  integer PRIMARY KEY,
  order_id       integer REFERENCES orders(order_id),
  product_id     integer REFERENCES products(product_id),
  quantity       integer NOT NULL,
  unit_price     numeric(10,2) NOT NULL, -- price AT TIME OF SALE
  discount_pct   numeric(5,4) DEFAULT 0 -- 0.15 = 15% off
);

CREATE TABLE web_events (
  event_id       bigint PRIMARY KEY,
  customer_id    integer,             -- NULL for anonymous visitors
  session_id     text NOT NULL,
  event_ts       timestamp NOT NULL,
  event_name     text NOT NULL,      -- 'page_view', 'product_view', 'add_to_cart', 'checkout_start', 'purchase'
  page_url       text,
  device         text                -- 'desktop', 'mobile', 'tablet'
);

CREATE TABLE employees (
  employee_id    integer PRIMARY KEY,
  full_name      text NOT NULL,
  department     text,
  manager_id     integer REFERENCES employees(employee_id), -- self-referencing
  hire_date      date,
  salary         numeric(10,2),
  location       text
);

```

Key facts to internalise, because interview questions hinge on them:

- `orders` is one row per order; `order_items` is one row per product line. **Joining orders to order_items multiplies order rows.** This single fact causes more wrong answers than anything else in analyst interviews.
- Revenue lives in `order_items`, not `orders`. Line revenue = `quantity * unit_price * (1 - discount_pct)`.
- `orders.status` matters. "Revenue" nearly always means `status = 'completed'`. If you don't filter it, an interviewer will ask why.
- `web_events.customer_id` is nullable — anonymous traffic. Counting customers off this table without handling NULL is a trap.
- `employees.manager_id` is a self-referencing FK, which is what self-join questions are built on.

Schema B — NHS Trust service data

UK public-sector and healthcare analyst roles (NHS trusts, ICBs, local authorities, care providers) interview on this shape. Waiting times, breaches and SLA compliance are the standard questions.

```

CREATE TABLE patients (
  patient_id     integer PRIMARY KEY,
  nhs_number     text UNIQUE,
  date_of_birth  date,
  sex            text,                -- 'M', 'F', 'Other', 'Unknown'
  postcode_sector text,              -- 'LS1 4', partial postcode
  imd_decile     integer,            -- 1 = most deprived, 10 = least
  registered_gp  integer
);

CREATE TABLE referrals (
  referral_id    integer PRIMARY KEY,
  patient_id     integer REFERENCES patients(patient_id),
  specialty      text,               -- 'Cardiology', 'Trauma & Orthopaedics', 'Dermatology'
  referral_date  date NOT NULL,
  source         text,               -- 'GP', 'A&E', 'Consultant', 'Self'
  priority       text,               -- 'Routine', 'Urgent', 'Two Week Wait'
  site_code      text
);

CREATE TABLE appointments (
  appointment_id integer PRIMARY KEY,

```

```

referral_id    integer REFERENCES referrals(referral_id),
patient_id     integer REFERENCES patients(patient_id),
scheduled_ts   timestamp,
attended_ts    timestamp, -- NULL if not attended
outcome        text,      -- 'Attended', 'DNA', 'Cancelled by patient', 'Cancelled by provider'
clinician_id   integer
);

CREATE TABLE ae_attendances (
  attendance_id integer PRIMARY KEY,
  patient_id     integer REFERENCES patients(patient_id),
  arrival_ts     timestamp NOT NULL,
  departure_ts   timestamp, -- NULL if still in department
  triage_category integer,  -- 1 (immediate) to 5 (non-urgent)
  admitted      boolean,
  site_code      text
);

CREATE TABLE waiting_list (
  waiting_id     integer PRIMARY KEY,
  referral_id    integer REFERENCES referrals(referral_id),
  added_date     date NOT NULL,
  removed_date   date,      -- NULL = still waiting
  removal_reason text       -- 'Treated', 'Removed - died', 'Removed - declined', 'Transferred'
);

```

Domain rules used throughout:

- **A&E four-hour standard:** patient should be admitted, transferred or discharged within 4 hours of `arrival_ts`. A breach is `departure_ts - arrival_ts > interval '4 hours'`.
- **RTT 18-week standard:** referral-to-treatment within 18 weeks of `referral_date`.
- **Two Week Wait:** suspected cancer referrals must be seen within 14 days.
- **DNA** = Did Not Attend. DNA rate is a standard reported metric and is deliberately ambiguous — see Part 13.

Sample data

Enough rows to reason about, small enough to trace by hand. Load this if you want to run the handbook's queries.

```

INSERT INTO customers VALUES
(1, 'Aisha', 'Khan', 'aisha@example.com', '2023-01-15', 'UK', 'Leeds', true, 'organic'),
(2, 'Tom', 'Brady', 'tom@example.com', '2023-01-20', 'UK', 'Leeds', false, 'paid_search'),
(3, 'Marie', 'Dubois', 'marie@example.com', '2023-02-03', 'FR', 'Lyon', true, 'email'),
(4, 'Sean', 'O'Neill', 'sean@example.com', '2023-02-18', 'IE', 'Cork', false, 'organic'),
(5, 'Priya', 'Patel', NULL, '2023-03-01', 'UK', 'London', true, 'referral'),
(6, 'Jack', 'Wilson', 'jack@example.com', '2023-03-14', 'UK', 'Manchester', false, 'paid_search'),
(7, 'Nina', 'Rossi', 'nina@example.com', '2024-01-09', 'UK', 'London', true, 'email');

INSERT INTO products VALUES
(10, 'Wireless Mouse', 'Electronics', 'Accessories', 24.99, 9.50, true),
(11, 'USB-C Hub', 'Electronics', 'Accessories', 49.99, 22.00, true),
(12, 'Desk Lamp', 'Home', 'Lighting', 34.50, 14.00, true),
(13, 'Cotton Throw', 'Home', 'Textiles', 29.00, 11.25, false),
(14, 'Running Shoes', 'Apparel', 'Footwear', 89.99, 38.00, true),
(15, 'Coffee Beans 1kg', 'Grocery', 'Beverages', 18.75, 7.10, true);

INSERT INTO orders VALUES
(1001, 1, '2024-01-05 10:12:00', 'completed', 'web', 3.99, NULL),
(1002, 1, '2024-02-11 14:03:00', 'completed', 'app', 0, 'SPRING10'),
(1003, 2, '2024-01-19 09:45:00', 'completed', 'web', 3.99, NULL),
(1004, 3, '2024-02-02 16:30:00', 'refunded', 'web', 5.99, NULL),
(1005, 4, '2024-02-20 11:00:00', 'completed', 'app', 0, NULL),
(1006, 1, '2024-03-08 19:22:00', 'completed', 'web', 3.99, 'SPRING10'),
(1007, 6, '2024-03-15 08:05:00', 'cancelled', 'web', 3.99, NULL),
(1008, 2, '2024-03-28 13:40:00', 'completed', 'web', 0, NULL),
(1009, 7, '2024-04-02 12:15:00', 'completed', 'app', 0, NULL),
(1010, 5, '2024-04-19 17:55:00', 'pending', 'web', 3.99, NULL);

INSERT INTO order_items VALUES
(1, 1001, 10, 2, 24.99, 0),
(2, 1001, 15, 1, 18.75, 0),
(3, 1002, 11, 1, 49.99, 0.10),
(4, 1003, 14, 1, 89.99, 0),
(5, 1004, 12, 2, 34.50, 0),
(6, 1005, 15, 3, 18.75, 0),

```

```
(7,1006,10,1,24.99,0.10),  
(8,1006,12,1,34.50,0.10),  
(9,1007,13,1,29.00,0),  
(10,1008,14,2,89.99,0),  
(11,1009,11,1,49.99,0),  
(12,1010,15,2,18.75,0);
```

Customer 5 has only a pending order. Customer 3's only order was refunded. Customer 6's only order was cancelled. Those three rows are what separate a correct answer from a plausible one on most of the questions in Part 16 — an interviewer seeds edge cases exactly like this.

Parts 1–5: Fundamentals, Filtering, Aggregation, Joins, CASE

PART 1 — SQL FUNDAMENTALS

1.1 What SQL is

SQL is a declarative language for asking a relational database for a set of rows. Declarative means you describe the result you want, not the steps to compute it. You write "give me completed orders from March grouped by customer"; the database's query planner decides whether to scan the table, use an index, hash the join or sort-merge it.

That has a practical consequence for interviews. When an interviewer asks "how would you make this faster?", they are asking what you can do to make the planner's job easier — filter earlier, avoid functions on indexed columns, reduce the rows entering a join. You are never optimising by rewriting loops, because there are no loops.

SQL is also **set-based**. A query operates on whole sets of rows at once, not row by row. Analysts coming from Excel tend to think one row at a time and reach for correlated subqueries where a window function or join is the natural tool. Breaking that habit is most of what Parts 9–11 are about.

Real analyst use case. Ninety percent of an analyst's SQL is: pull a filtered slice of transactional data, aggregate it to a business grain (customer, month, region), and hand it to a dashboard or a stakeholder.

Interview question. *"What does it mean that SQL is declarative, and why does it matter to you as an analyst?"* — Answer: you specify the what, the planner chooses the how; it matters because performance work is about giving the planner better options (indexes, selective filters, sensible join order) rather than micro-managing execution.

1.2 Relational databases

A relational database stores data as a collection of tables (relations) linked by shared key values, with the engine enforcing rules about those links. The point of the model is that each fact is stored once, in the table where it belongs, and assembled at query time by joining.

Customer name lives in `customers`, once. If it lived on every order row, changing a surname would mean updating thousands of rows and risking half of them disagreeing. That is the entire argument for normalisation (Part 15).

Common mistake. Assuming the database is one big flat table because that's how the CSV export looks. Always ask for, or inspect, the schema before writing a query. In an interview, asking "can I confirm the grain of this table — one row per what?" is a strong opening move.

Interview question. *"Why is data split across multiple tables rather than stored in one wide table?"* — Avoid duplication, keep updates consistent, enforce integrity with foreign keys, and let each table have a single clear grain. Then note the counter-case: analytical warehouses deliberately denormalise into star schemas because read performance and simplicity beat update efficiency when nothing is being updated.

1.3 Tables, rows, columns, and grain

- A **table** is a set of rows with a fixed set of typed columns.
- A **row** (tuple, record) is one instance of the thing the table describes.
- A **column** (field, attribute) is one property, with one data type, the same for every row.
- The **grain** is the answer to "one row per what?" — `orders` is one row per order; `order_items` is one row per product line within an order.

Grain is the single most useful concept in this handbook. Nearly every wrong analytical answer comes from computing a metric at the wrong grain.

```
-- grain: one row per order
SELECT order_id, customer_id, order_ts FROM orders;

-- grain: one row per order line
SELECT order_item_id, order_id, product_id, quantity FROM order_items;
```

Real analyst use case. Before writing anything, establish grain:

```
SELECT COUNT(*) AS rows, COUNT(DISTINCT order_id) AS orders FROM order_items;
```

If those two numbers differ, the table is not one row per order, and `SUM(shipping_cost)` after joining to it will double-count.

Common mistake. Summing an order-level column (`shipping_cost`) after joining to `order_items`. Shipping gets counted once per line, so a 3-line order contributes 3× its shipping.

Interview question. "You join orders to order_items and total revenue looks right but total shipping cost has tripled. What happened?" — The join changed the grain from one row per order to one row per line; order-level measures must be aggregated at order grain before or separately from line-level measures.

1.4 Primary keys

A primary key uniquely identifies a row. It cannot be NULL and cannot repeat. It may be a single column (`customer_id`) or several together (a composite key).

```
CREATE TABLE customers (  
  customer_id integer PRIMARY KEY,  
  ...  
);  
  
-- composite: one row per customer per month  
CREATE TABLE customer_monthly_summary (  
  customer_id integer,  
  month_start date,  
  order_count integer,  
  PRIMARY KEY (customer_id, month_start)  
);
```

A **surrogate key** is a meaningless generated id (`customer_id serial`). A **natural key** is a real-world identifier (`nhs_number`, `email`). Warehouses prefer surrogate keys because natural keys change — people change email addresses and surnames, NHS numbers get corrected — and a key that changes breaks every reference to it.

Real analyst use case. Checking whether a table you have been given actually has the key someone claims:

```
SELECT customer_id, COUNT(*)  
FROM customers  
GROUP BY customer_id  
HAVING COUNT(*) > 1;
```

Zero rows means the key holds. On an extract from a source system rather than a real table, it often doesn't.

Common mistake. Trusting that a column called `id` is unique in an extract, staging table, or CSV. Verify.

Interview question. "Difference between a primary key and a unique constraint?" — Both enforce uniqueness; a primary key additionally forbids NULL, there is only one per table, and it is the row's canonical identity. A unique column can be nullable, and in Postgres multiple NULLs are permitted in a unique column because NULLs are not equal to each other.

1.5 Foreign keys and referential integrity

A foreign key is a column whose values must exist as a primary key in another table. `orders.customer_id` references `customers.customer_id`, so an order cannot reference a customer who does not exist.

```
CREATE TABLE orders (  
  order_id integer PRIMARY KEY,  
  customer_id integer REFERENCES customers(customer_id),  
  ...  
);
```

Foreign keys can be NULL, which means "no relationship" — `employees.manager_id` is NULL for the CEO, `web_events.customer_id` is NULL for anonymous visitors. That distinction (no relationship vs unknown relationship) matters when you're deciding between an INNER and a LEFT JOIN.

Real analyst use case. Orphan detection in a warehouse where FKs were never enforced:

```
SELECT o.order_id, o.customer_id  
FROM orders o  
LEFT JOIN customers c ON c.customer_id = o.customer_id  
WHERE c.customer_id IS NULL;
```

Any rows returned are orders pointing at customers that don't exist. In a properly constrained OLTP database this is impossible; in a warehouse fed by nightly loads it happens constantly, usually because the dimension load ran before the

fact load.

Common mistake. Assuming referential integrity holds in an analytics warehouse. It usually doesn't. Run the orphan check before you promise a stakeholder that an inner join loses nothing.

Interview question. "You inner-join fact to dimension and row count drops by 3%. What do you check?" — Orphan foreign keys (late-arriving or missing dimension rows), NULL join keys, type mismatches between the key columns, and trailing whitespace or case differences if the key is text.

1.6 Relationships

One-to-one. Each row in A matches at most one row in B. Rare; usually a table split for security or sparsity — `patients` and `patient_sensitive_details`. Joining does not change row count.

One-to-many. One customer, many orders. This is the common case. Joining from the "one" side to the "many" side multiplies the one-side rows.

Many-to-many. Orders and products: an order has many products, a product appears in many orders. Relational databases cannot represent this directly, so a junction (bridge) table sits between them — that is exactly what `order_items` is. The junction table typically carries its own attributes (`quantity`, `unit_price` at time of sale), which is why it is a real table and not just plumbing.

```
customers —1:M—> orders —1:M—> order_items <—M:1— products
```

Reading that diagram tells you the answer to most join questions before you write anything. Going left to right, row counts multiply. `customers` joined all the way to `order_items` gives one row per customer per order line.

Interview question. "How do you model a many-to-many relationship?" — With a junction table holding a foreign key to each side, ideally with a composite primary key across both, plus any attributes specific to the pairing.

1.7 NULL

NULL is not zero, not an empty string, and not "false". It means *unknown or not applicable*. This gets its own full treatment in Part 6 because it accounts for a large share of interview traps, but three rules to hold from the start:

1. Any comparison with NULL yields NULL, not true or false. `NULL = NULL` is NULL. Use `IS NULL`.
2. WHERE keeps only rows where the condition is **true**. NULL is not true, so rows with NULL conditions are dropped.
3. Aggregate functions ignore NULLs — except `COUNT(*)`.

```
SELECT NULL = NULL;      -- NULL, not true
SELECT NULL IS NULL;     -- true
SELECT 100 + NULL;       -- NULL: any arithmetic with NULL is NULL
SELECT COUNT(*), COUNT(email) FROM customers; -- 7, 6 - Priya's email is NULL
```

Interview question. "What's the difference between `COUNT(*)` and `COUNT(column)`?" — `COUNT(*)` counts rows; `COUNT(col)` counts rows where `col` is not NULL. Asked in some form in most junior interviews.

1.8 Data types

Postgres type	Use for	Notes
<code>integer / bigint</code>	ids, counts	<code>bigint</code> past ~2.1 billion
<code>numeric(p,s)</code>	money	exact decimal; use this for currency
<code>real / double precision</code>	measurements, scientific	binary floating point, inexact
<code>text / varchar(n)</code>	strings	in Postgres <code>text</code> is not slower; length limits are validation, not performance
<code>boolean</code>	true/false/NULL	three-valued
<code>date</code>	calendar day	no time component
<code>timestamp</code>	date + time, no zone	"naive"
<code>timestampz</code>	date + time, zone-aware	stores UTC, renders in session zone
<code>interval</code>	duration	<code>interval '4 hours'</code>

Postgres type	Use for	Notes
jsonb	semi-structured	indexable, common in event tables

Two things that come up in real UK analyst work:

Money must be numeric, never float. $0.1 + 0.2 = 0.30000000000000004$ in floating point. Finance will notice.

```
SELECT 0.1::double precision + 0.2::double precision; -- 0.30000000000000004
SELECT 0.1::numeric + 0.2::numeric;                -- 0.3
```

timestamp vs timestamptz and British Summer Time. UK data spans BST (UTC+1, late March to late October) and GMT (UTC). If your timestamps are naive **timestamp** in UTC and you group by day, summer events between 23:00 and 00:00 UK time land on the previous day. That quietly shifts daily figures for seven months of the year.

```
SELECT DATE_TRUNC('day', event_ts AT TIME ZONE 'UTC' AT TIME ZONE 'Europe/London')
FROM web_events;
```

Integer division catches people out. In Postgres **integer / integer** truncates:

```
SELECT 7 / 2;           -- 3
SELECT 7.0 / 2;         -- 3.5
SELECT 7::numeric / 2;  -- 3.5
```

Conversion rate computed as `COUNT(purchases) / COUNT(sessions)` returns 0 for anything under 100%. Cast the numerator.

Interview question. "Why store currency as numeric rather than float?" — Floats are binary approximations of decimal fractions; repeated addition accumulates error and totals will not reconcile to the penny. **numeric** is exact decimal arithmetic.

1.9 SELECT and FROM

```
SELECT column_list
FROM table;
```

Column aliases with **AS**; double-quote them only if they need spaces or capitals. Postgres folds unquoted identifiers to lower case, so `SELECT x AS Revenue` gives you a column named `revenue`, while `AS "Revenue"` preserves the capital and then requires quoting forever after. Prefer `snake_case`.

```
SELECT
  first_name || ' ' || last_name AS customer_name,
  signup_date,
  country
FROM customers;
```

Table aliases keep multi-table queries readable. Use short meaningful ones (`o`, `oi`, `c`), and once you use an alias you must use it everywhere in that query.

Common mistake. `SELECT *` in production queries and saved views. It pulls columns you don't need, breaks when the upstream schema adds a column, makes the query's dependencies invisible to anyone reading it, and prevents index-only scans. Fine while exploring, wrong in anything anyone else will run.

Interview question. "Why avoid `SELECT *`?" — Network and memory cost of unused columns, brittleness to schema change, loss of index-only scans, and it hides what the query actually depends on.

1.10 Logical order of evaluation

Written order and execution order differ, and this explains most "why can't I use my alias there?" confusion.

Written: `SELECT` → `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `ORDER BY` → `LIMIT`

Logically evaluated:

```
1. FROM      (and JOINS)  -- build the working set
2. WHERE     -- filter individual rows
3. GROUP BY  -- collapse into groups
4. HAVING    -- filter groups
5. SELECT    -- compute output expressions, assign aliases
6. DISTINCT
7. ORDER BY  -- aliases are now visible
8. LIMIT / OFFSET
```

Consequences you will be asked about:

- You cannot reference a SELECT alias in WHERE — WHERE runs before SELECT.
- You *can* reference a SELECT alias in ORDER BY and GROUP BY in Postgres, because those are resolved after (GROUP BY alias support is a Postgres extension, not standard).
- You cannot put an aggregate in WHERE, because groups don't exist yet. That's what HAVING is for.
- LIMIT applies last, so `LIMIT 10` after an aggregation gives 10 groups, not 10 rows of input.

```
-- fails: revenue does not exist yet at WHERE time
SELECT quantity * unit_price AS revenue
FROM order_items
WHERE revenue > 50;

-- works
SELECT quantity * unit_price AS revenue
FROM order_items
WHERE quantity * unit_price > 50;

-- also works, and is more readable at scale
SELECT * FROM (
    SELECT quantity * unit_price AS revenue FROM order_items
) t
WHERE revenue > 50;
```

Interview question. "Why can I use an alias in ORDER BY but not WHERE?" — This is the answer they want, in one sentence: WHERE is evaluated before SELECT so the alias doesn't exist yet; ORDER BY is evaluated after.

1.11 WHERE

Filters rows before grouping. Keeps rows where the condition evaluates to true — not false, and not NULL.

```
SELECT order_id, customer_id, order_ts
FROM orders
WHERE status = 'completed'
    AND order_ts >= DATE '2024-03-01'
    AND order_ts < DATE '2024-04-01';
```

Note the half-open date range. `BETWEEN '2024-03-01' AND '2024-03-31'` on a *timestamp* column silently drops everything after midnight on the 31st, because `'2024-03-31'` is read as `2024-03-31 00:00:00`. That is a full day of data lost, and it is a genuinely common production bug. Half-open `>= start AND < next_start` is always right, for dates and timestamps alike.

Common mistake. Wrapping the filtered column in a function: `WHERE DATE(order_ts) = '2024-03-15'` cannot use an index on `order_ts`. Rewrite as a range: `WHERE order_ts >= '2024-03-15' AND order_ts < '2024-03-16'`.

1.12 ORDER BY

```
SELECT customer_id, order_ts
FROM orders
ORDER BY order_ts DESC, order_id ASC;
```

ASC is default. `NULLS FIRST/NULLS LAST` controls NULL placement — in Postgres, NULLs sort last for ASC and first for DESC by default, which is the opposite of some other engines and will silently change "top N" results.

```
SELECT product_name, unit_price
FROM products
ORDER BY unit_price DESC NULLS LAST;
```

Dialect. Ordering by position (`ORDER BY 2 DESC`) works widely but is fragile — inserting a column silently changes the sort. Fine in a scratch query, avoid in anything saved.

Common mistake. Assuming rows come back in a stable order without ORDER BY. There is no default order. A query that "always" returned data sorted will reorder the day the planner switches to a parallel scan.

1.13 LIMIT and OFFSET

```
SELECT product_name, unit_price
FROM products
ORDER BY unit_price DESC
LIMIT 3;
```

```
-- pagination: rows 11-20
SELECT ... ORDER BY order_ts DESC LIMIT 10 OFFSET 10;
```

LIMIT without ORDER BY returns an arbitrary 3 rows, not the top 3.

Ties are the interview point. LIMIT 3 on prices 90, 90, 85, 85 gives you three rows arbitrarily, cutting a tie in half. If the question says "the top 3 most expensive products", ask whether ties should all be returned — and if so use RANK() <= 3 (Part 11) rather than LIMIT.

Dialect. SQL Server uses SELECT TOP 3 ... or OFFSET n ROWS FETCH NEXT m ROWS ONLY. Oracle (12c+) uses FETCH FIRST n ROWS ONLY, which Postgres also supports. LIMIT is Postgres/MySQL/SQLite.

Interview question. "How do you return the top 3 products by price including ties?" — DENSE_RANK() OVER (ORDER BY unit_price DESC) <= 3; explain the difference between RANK and DENSE_RANK when you do.

1.14 DISTINCT

Removes duplicate rows across **all** selected columns, not just the first one.

```
SELECT DISTINCT country FROM customers;
SELECT DISTINCT country, city FROM customers; -- distinct combinations
```

DISTINCT ON is a Postgres-specific tool worth knowing cold, because it answers "latest record per group" in three lines:

```
SELECT DISTINCT ON (customer_id)
       customer_id, order_id, order_ts
FROM orders
ORDER BY customer_id, order_ts DESC;
```

Keeps the first row per customer_id after sorting. The leading ORDER BY columns must match the DISTINCT ON columns. Faster than a window function for this task and a nice thing to know in a Postgres shop — but say "the portable version is ROW_NUMBER" in an interview, because it doesn't exist outside Postgres.

Common mistake. Reaching for SELECT DISTINCT to fix duplicate rows caused by a bad join. It hides the symptom and leaves the wrong join in place; the moment you add a column with genuinely varying values, the duplicates come back. Find the join causing the fan-out instead.

Interview question. "Your query returns duplicates. Would you add DISTINCT?" — Only after diagnosing the source. Duplicates are either genuine data duplication (fix with deduplication logic, Part 12) or join fan-out (fix the join grain). DISTINCT is a last resort and also expensive — it forces a sort or hash of the entire result.

PART 2 — FILTERING AND LOGICAL OPERATIONS

2.1 Three-valued logic

SQL logic has three values: TRUE, FALSE, UNKNOWN. Any comparison involving NULL returns UNKNOWN. WHERE keeps a row only when the whole condition is TRUE.

AND

	TRUE	FALSE	NULL
TRUE	TRUE	FALSE	NULL
FALSE	FALSE	FALSE	FALSE
NULL	NULL	FALSE	NULL

OR

	TRUE	FALSE	NULL
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	NULL
NULL	TRUE	NULL	NULL

NOT: NOT TRUE = FALSE, NOT FALSE = TRUE, **NOT NULL = NULL**.

Read the two shaded corners: **FALSE AND NULL** is FALSE (one false condemns the whole AND), but **TRUE OR NULL** is TRUE (one true saves the whole OR). Those are the short-circuits, and they're why some NULL-containing conditions still behave sensibly.

The killer consequence: a row where the condition is UNKNOWN is excluded from **WHERE cond** **and** from **WHERE NOT cond**. The two do not partition your data.

```
SELECT COUNT(*) FROM customers WHERE email = 'x@y.com';      -- excludes Priya (NULL email)
SELECT COUNT(*) FROM customers WHERE email <> 'x@y.com';      -- ALSO excludes Priya
-- Priya appears in neither. To include her:
SELECT COUNT(*) FROM customers WHERE email IS DISTINCT FROM 'x@y.com';
```

IS DISTINCT FROM is NULL-safe inequality — it treats NULL as a comparable value. **IS NOT DISTINCT FROM** is NULL-safe equality. Knowing these two operators is a genuine differentiator in a Postgres interview.

Dialect. MySQL's NULL-safe equality is **<=>**. SQL Server has no direct equivalent; you write **WHERE (a = b OR (a IS NULL AND b IS NULL))** or use **EXCEPT/INTERSECT**.

2.2 Comparison operators

Operator	Meaning	Notes
=	equal	never true for NULL
!= or <>	not equal	<> is the standard; identical in Postgres
> <	greater / less	works on dates, timestamps, text (collation order)
>= <=	inclusive	

Text comparison uses the database collation. In a typical UK Postgres install with **en_GB.UTF-8**, sorting is case-insensitive-ish for ordering purposes but **=** remains case-**sensitive**: **'UK' = 'uk'** is false. This is why country and status filters need normalising (Part 6).

```
SELECT * FROM orders WHERE shipping_cost > 0;
SELECT * FROM orders WHERE order_ts >= CURRENT_DATE - INTERVAL '30 days';
SELECT * FROM customers WHERE last_name >= 'M'; -- M to Z
```

2.3 AND, OR, NOT — and precedence

NOT binds tighter than **AND**, which binds tighter than **OR**. Missing parentheses around an **OR** is one of the most damaging silent bugs in analyst SQL, because the query runs and returns plausible numbers.

```
-- WRONG: reads as (country='UK' AND status='completed') OR (country='IE')
SELECT * FROM orders o
JOIN customers c USING (customer_id)
WHERE c.country = 'UK' AND o.status = 'completed'
      OR c.country = 'IE';
-- returns every Irish order including cancelled ones

-- RIGHT
WHERE (c.country = 'UK' OR c.country = 'IE')
      AND o.status = 'completed';
```

Rule for life: whenever a WHERE clause contains both AND and OR, parenthesise the OR. Reviewers should not have to remember precedence.

2.4 IN and NOT IN

```
SELECT * FROM orders WHERE status IN ('completed', 'refunded');
SELECT * FROM customers WHERE country NOT IN ('UK', 'IE');
```

IN is shorthand for a chain of ORs. **NOT IN** is shorthand for a chain of ANDs — and that is where it breaks.

The NOT IN NULL trap. This is asked constantly, at every level.

```
-- if ANY row in the subquery returns NULL customer_id, this returns ZERO rows
SELECT * FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders);
```

Why: $x \text{ NOT IN } (1, 2, \text{NULL})$ expands to $x \neq 1 \text{ AND } x \neq 2 \text{ AND } x \neq \text{NULL}$. The last term is UNKNOWN, so the whole AND is at best UNKNOWN, never TRUE. No row can qualify. And it fails *silently* — an empty result, no error.

Three fixes, best first:

```
-- 1. NOT EXISTS — NULL-safe by construction, and usually the fastest
SELECT * FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);

-- 2. LEFT JOIN ... IS NULL (anti-join)
SELECT c.* FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.customer_id IS NULL;

-- 3. NOT IN with the NULLs explicitly removed
SELECT * FROM customers
WHERE customer_id NOT IN (
    SELECT customer_id FROM orders WHERE customer_id IS NOT NULL
);
```

Say "I'd use NOT EXISTS because NOT IN returns no rows if the subquery contains a NULL" and you have demonstrably passed the mid-level filter.

2.5 BETWEEN

Inclusive on both ends: $x \text{ BETWEEN } a \text{ AND } b$ is $x \geq a \text{ AND } x \leq b$.

Perfectly safe on integers and **date** columns. Dangerous on **timestamp**:

```
-- silently loses everything on 31 March after 00:00:00
WHERE order_ts BETWEEN '2024-03-01' AND '2024-03-31'

-- correct
WHERE order_ts >= DATE '2024-03-01' AND order_ts < DATE '2024-04-01'
```

Make half-open ranges your default habit and the class of bug disappears — it also handles month lengths, leap years and future timestamp precision changes without thought.

2.6 LIKE and ILIKE

LIKE is case-sensitive pattern matching; **ILIKE** is the Postgres case-insensitive version. **%** matches any sequence of characters (including none), **_** matches exactly one.

```
WHERE email LIKE '%gmail.com'      -- ends with
WHERE product_name ILIKE 'wireless%' -- starts with, any case
WHERE postcode_sector LIKE 'LS_ %'  -- LS1, LS2, ... single char then space
WHERE discount_code LIKE 'SPRING'
```

To match a literal **%** or **_**, escape it: **LIKE '100!%' ESCAPE '!'.**

Performance. A leading wildcard ('%gmail.com') cannot use a normal B-tree index; the database must scan every row. A trailing wildcard ('wireless%') can use an index if the column's collation permits it (**text_pattern_ops** in Postgres). For serious text search, use full-text search (**tsvector**) or a trigram index (**pg_trgm**) — knowing that these exist is enough for an analyst interview.

Dialect. **ILIKE** is Postgres-only. Portable equivalent: **WHERE LOWER(col) LIKE LOWER('pattern')** — but note this disables plain index use unless you have a functional index on **LOWER(col)**.

2.7 IS NULL / IS NOT NULL

The only correct way to test for NULL.

```
SELECT * FROM appointments WHERE attended_ts IS NULL; -- did not attend (yet)
SELECT * FROM waiting_list WHERE removed_date IS NULL; -- still waiting
SELECT * FROM ae_attendances WHERE departure_ts IS NULL; -- still in department
```

That last pattern — NULL as an open-ended "still in progress" marker — is everywhere in service data, and it's why **AVG(departure_ts - arrival_ts)** quietly excludes exactly the longest-staying patients. Part 6 covers the consequences.

2.8 Progressive exercises

Solutions are at the end of this section. Attempt them before looking.

Level 1

1. All customers who signed up in 2023.
2. All products priced over £30 that are still active.
3. Orders that are not completed.
4. Customers with no email recorded.
5. Products in Electronics or Home.

Level 2 6. UK or Irish customers who opted into marketing. 7. Orders placed in March 2024 that were completed, with a discount code applied. 8. Products where the margin (**unit_price - unit_cost**) exceeds £20. 9. Customers whose email is not a gmail address — including those with no email at all. 10. Orders with shipping cost of exactly zero, excluding cancelled orders.

Level 3 11. Products where the markup is more than 150% of cost. 12. Customers who signed up in Q1 of any year. 13. A&E attendances that breached the four-hour standard, excluding those still in the department. 14. Two Week Wait referrals from A&E or GP made in the last 90 days. 15. Order lines with a discount applied where the discounted line value still exceeds £40.

Level 4 (thinking required) 16. Customers in the UK, or customers anywhere who came through the email channel — completed orders only. Write it so no reviewer could misread the precedence. 17. Customers who have never placed an order. Do it three different ways and say which you'd ship. 18. Products whose name contains a digit. 19. Patients whose recorded sex is anything other than 'F', including where it is missing. 20. Orders where the discount code is either absent or is not one of the known-valid codes ('SPRING10', 'WELCOME5').

Solutions

```
-- 1
SELECT * FROM customers
WHERE signup_date >= DATE '2023-01-01' AND signup_date < DATE '2024-01-01';

-- 2
SELECT * FROM products WHERE unit_price > 30 AND is_active;
```

```

-- 3
SELECT * FROM orders WHERE status <> 'completed'; -- status is NOT NULL, so safe here

-- 4
SELECT * FROM customers WHERE email IS NULL;

-- 5
SELECT * FROM products WHERE category IN ('Electronics','Home');

-- 6
SELECT * FROM customers
WHERE country IN ('UK','IE') AND marketing_opt_in;

-- 7
SELECT * FROM orders
WHERE order_ts >= DATE '2024-03-01' AND order_ts < DATE '2024-04-01'
AND status = 'completed'
AND discount_code IS NOT NULL;

-- 8
SELECT product_name, unit_price - unit_cost AS margin
FROM products WHERE unit_price - unit_cost > 20;

-- 9 the "including NULLs" is the whole point
SELECT * FROM customers
WHERE email IS NULL OR email NOT LIKE '%@gmail.com';
-- or, more elegantly:
SELECT * FROM customers
WHERE COALESCE(email,'') NOT LIKE '%@gmail.com';

-- 10
SELECT * FROM orders WHERE shipping_cost = 0 AND status <> 'cancelled';

-- 11
SELECT product_name FROM products WHERE unit_price > unit_cost * 2.5;

-- 12
SELECT * FROM customers WHERE EXTRACT(MONTH FROM signup_date) BETWEEN 1 AND 3;

-- 13
SELECT * FROM ae_attendances
WHERE departure_ts IS NOT NULL
AND departure_ts - arrival_ts > INTERVAL '4 hours';

-- 14
SELECT * FROM referrals
WHERE priority = 'Two Week Wait'
AND source IN ('A&E','GP')
AND referral_date >= CURRENT_DATE - INTERVAL '90 days';

-- 15
SELECT * FROM order_items
WHERE discount_pct > 0
AND quantity * unit_price * (1 - discount_pct) > 40;

-- 16
SELECT o.*
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
WHERE o.status = 'completed'
AND (c.country = 'UK' OR c.channel = 'email');

-- 17 three ways; ship NOT EXISTS
SELECT c.* FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);

SELECT c.* FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;

SELECT * FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders WHERE customer_id IS NOT NULL);

-- 18
SELECT product_name FROM products WHERE product_name ~ '[0-9]';

-- 19 NULL-safe inequality
SELECT * FROM patients WHERE sex IS DISTINCT FROM 'F';

```

```
-- 20
SELECT * FROM orders
WHERE discount_code IS NULL
      OR discount_code NOT IN ('SPRING10', 'WELCOME5');
```

PART 3 — AGGREGATIONS

3.1 The aggregate functions

An aggregate collapses many rows into one value. **All aggregates except `COUNT(*)` ignore NULLs.** Internalise that; it drives every subtlety below.

Function	Ignores NULL?	Returns on zero rows
<code>COUNT(*)</code>	no — counts rows	0
<code>COUNT(col)</code>	yes	0
<code>COUNT(DISTINCT col)</code>	yes	0
<code>SUM(col)</code>	yes	NULL
<code>AVG(col)</code>	yes	NULL
<code>MIN/MAX(col)</code>	yes	NULL

`SUM` over no rows returning NULL rather than 0 is a real reporting bug: a month with no sales shows blank, not £0, and then any arithmetic on it becomes NULL. Wrap in `COALESCE(SUM(x), 0)` whenever the output goes to a dashboard.

COUNT variants

```
SELECT
  COUNT(*)           AS all_rows,      -- 7
  COUNT(email)       AS with_email,    -- 6
  COUNT(DISTINCT country) AS countries, -- 3
  COUNT(DISTINCT customer_id) AS customers -- 7
FROM customers;
```

`COUNT(1)` and `COUNT(*)` are identical in Postgres — same plan, same speed. Anyone claiming otherwise is repeating folklore from a different engine and a different decade.

`COUNT(DISTINCT ...)` is expensive: it must deduplicate, which means sorting or hashing. On very large tables, approximate counting (HyperLogLog via the `postgresql-hll` extension, or `APPROX_COUNT_DISTINCT` in BigQuery/Snowflake) trades exactness for speed. Worth mentioning if asked about scale.

AVG and the NULL-vs-zero distinction

This is the most important aggregation trap and it appears in interviews constantly.

```
-- suppose feedback_score is NULL for customers who never responded
SELECT AVG(feedback_score) FROM survey;
```

`AVG` divides by the count of **non-NULL** values. If half your respondents didn't answer, you get the average of those who did — which may be what you want, or may be badly wrong. If a non-response should count as zero:

```
SELECT AVG(COALESCE(feedback_score, 0)) FROM survey;
```

These give materially different numbers, and the correct choice is a business decision, not a technical one. In an interview: "Should non-responses be excluded or treated as zero?" is exactly the clarifying question they want to hear.

SUM

```
SELECT SUM(quantity * unit_price * (1 - discount_pct)) AS gross_revenue
FROM order_items;
```

Note the calculation happens per row, then sums. `SUM(quantity) * AVG(unit_price)` is not the same thing and is wrong whenever prices vary — a classic seeded error.

MIN / MAX

Work on numbers, dates, timestamps and text. On text they use collation order.

```
SELECT customer_id,
  MIN(order_ts) AS first_order,
  MAX(order_ts) AS latest_order
```

```
FROM orders
WHERE status = 'completed'
GROUP BY customer_id;
```

This is the cheapest way to get first/last **dates**. It does *not* give you the first/last **row** — for the product bought on the first order you need **DISTINCT ON** or **ROW_NUMBER** (Part 11), or the argmin trick:

```
-- product from each customer's earliest order, no window function
SELECT customer_id,
       (ARRAY_AGG(product_id ORDER BY order_ts))[1] AS first_product
FROM orders JOIN order_items USING (order_id)
GROUP BY customer_id;
```

3.2 GROUP BY

GROUP BY collapses rows into one row per distinct combination of the grouping columns.

```
SELECT
  c.country,
  COUNT(DISTINCT o.customer_id) AS customers,
  COUNT(*) AS orders,
  ROUND(AVG(o.shipping_cost), 2) AS avg_shipping
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
WHERE o.status = 'completed'
GROUP BY c.country
ORDER BY orders DESC;
```

The rule. Every column in SELECT must either appear in GROUP BY or be inside an aggregate. Postgres enforces this and errors out. MySQL historically allowed it and returned an arbitrary value from the group — the source of countless silent errors, now disabled by default under **ONLY_FULL_GROUP_BY**.

The one exception in Postgres: if you group by a table's primary key, you may select any other column from that table, since the key functionally determines them.

```
-- Legal in Postgres: customer_id is the PK of customers
SELECT c.customer_id, c.first_name, c.last_name, COUNT(o.order_id)
FROM customers c LEFT JOIN orders o USING (customer_id)
GROUP BY c.customer_id;
```

GROUP BY with no rows in a group. Groups that don't exist in the data don't appear in the output. A month with zero orders produces no row at all, not a zero. If a dashboard needs a continuous series, generate the calendar and LEFT JOIN to it:

```
SELECT d.month, COALESCE(COUNT(o.order_id), 0) AS orders
FROM generate_series(DATE '2024-01-01', DATE '2024-06-01', INTERVAL '1 month') AS d(month)
LEFT JOIN orders o
  ON DATE_TRUNC('month', o.order_ts) = d.month
  AND o.status = 'completed'
GROUP BY d.month
ORDER BY d.month;
```

Note the join condition, not a WHERE: putting **o.status='completed'** in WHERE would turn the LEFT JOIN back into an inner join and delete the empty months again. That mechanism is explained fully in Part 4.

GROUPING SETS, ROLLUP, CUBE. Multiple grouping levels in one pass — useful for subtotal rows.

```
SELECT c.country, p.category, SUM(oi.quantity * oi.unit_price) AS revenue
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
JOIN customers c ON c.customer_id = o.customer_id
JOIN products p ON p.product_id = oi.product_id
WHERE o.status = 'completed'
GROUP BY ROLLUP (c.country, p.category);
-- gives country+category, country subtotal, and grand total rows
```

Distinguish a subtotal NULL from a genuine data NULL with **GROUPING(c.country)**, which returns 1 for aggregated levels.

3.3 HAVING, and WHERE vs HAVING

WHERE filters rows before grouping. HAVING filters groups after aggregation. That is the entire distinction, and it follows directly from the evaluation order in 1.10.

```
SELECT customer_id, COUNT(*) AS completed_orders
FROM orders
WHERE status = 'completed'           -- row filter: happens first
GROUP BY customer_id
HAVING COUNT(*) >= 2;                -- group filter: happens after
```

- `WHERE COUNT(*) > 2` → error: aggregates aren't allowed in WHERE.
- Anything expressible in WHERE should be in WHERE, because it reduces rows before the (expensive) grouping. `HAVING country = 'UK'` works if country is grouped, but it filters later and does more work.

The classic exam question distills to: "Find customers with more than 2 completed orders." Both the status filter (WHERE, it's about individual orders) and the count filter (HAVING, it's about the group) are needed, and choosing correctly demonstrates you understand order of evaluation.

Compare these two — they answer different questions:

```
-- A: customers whose COMPLETED orders number 2+
SELECT customer_id FROM orders
WHERE status='completed' GROUP BY customer_id HAVING COUNT(*) >= 2;

-- B: customers with 2+ orders of any status, of which at least one completed
SELECT customer_id FROM orders
GROUP BY customer_id
HAVING COUNT(*) >= 2 AND COUNT(*) FILTER (WHERE status='completed') >= 1;
```

`FILTER (WHERE ...)` is the Postgres way to aggregate a subset — cleaner than `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` and it works on any aggregate. Portable code needs the CASE form (Part 5).

3.4 Worked business examples

Revenue by month, completed only.

```
SELECT
    DATE_TRUNC('month', o.order_ts)::date AS month,
    COUNT(DISTINCT o.order_id) AS orders,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct)), 2) AS revenue
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'completed'
GROUP BY 1
ORDER BY 1;
```

`COUNT(DISTINCT o.order_id)` and not `COUNT(*)`: after the join, each order appears once per line.

Average order value. AOV is order revenue averaged across orders, so aggregate to order grain first.

```
WITH order_revenue AS (
    SELECT o.order_id,
        SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct)) AS revenue
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.order_id
    WHERE o.status = 'completed'
    GROUP BY o.order_id
)
SELECT ROUND(AVG(revenue), 2) AS aov FROM order_revenue;
```

Doing `AVG(oi.quantity * oi.unit_price)` on the joined table gives average *line* value, a different and usually much smaller number. Interviewers seed this deliberately.

Employees: headcount and pay by department.

```
SELECT department,
    COUNT(*) AS headcount,
    ROUND(AVG(salary), 0) AS mean_salary,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median_salary,
    MIN(hire_date) AS longest_serving_hire
FROM employees
GROUP BY department
HAVING COUNT(*) >= 3
ORDER BY mean_salary DESC;
```

`PERCENTILE_CONT` is an ordered-set aggregate — the standard way to get a median in Postgres, and a good thing to have in your pocket, since salary and waiting-time distributions are skewed and the mean misleads.

NHS: DNA rate by specialty.

```
SELECT r.specialty,
       COUNT(*)
       COUNT(*) FILTER (WHERE a.outcome = 'DNA')
       ROUND(100.0 * COUNT(*) FILTER (WHERE a.outcome = 'DNA') / COUNT(*), 1) AS dna_rate_pct
AS appointments,
AS dnas,
FROM appointments a
JOIN referrals r ON r.referral_id = a.referral_id
WHERE a.scheduled_ts >= DATE '2024-01-01'
GROUP BY r.specialty
HAVING COUNT(*) >= 50
ORDER BY dna_rate_pct DESC;
```

Two things a strong candidate says unprompted: `100.0 *` forces numeric division (integer division would floor to 0), and the `HAVING COUNT(*) >= 50` suppresses tiny denominators where a 100% DNA rate means one missed appointment. Rate metrics on small denominators are how bad dashboards get built.

A&E four-hour performance by site.

```
SELECT site_code,
       COUNT(*) AS attendances,
       COUNT(*) FILTER (WHERE departure_ts - arrival_ts > INTERVAL '4 hours') AS breaches,
       ROUND(100.0 * COUNT(*) FILTER (WHERE departure_ts - arrival_ts <= INTERVAL '4 hours')
             / NULLIF(COUNT(*) FILTER (WHERE departure_ts IS NOT NULL), 0), 1) AS pct_within_4h
FROM ae_attendances
WHERE arrival_ts >= DATE_TRUNC('month', CURRENT_DATE)
GROUP BY site_code;
```

`NULLIF(x, 0)` prevents division by zero by turning a zero denominator into NULL, which propagates to a NULL result instead of an error. Standard defensive practice for any rate.

3.5 Aggregation exercises

1. Number of customers per country.
2. Total quantity sold per product.
3. Average unit price by category, two decimal places.
4. Categories with more than 2 products.
5. For each customer: order count, first order date, last order date — completed only.
6. Products never ordered. (Aggregation isn't the tool — say why.)
7. Monthly revenue with month-on-month order counts, 2024 only.
8. Customers whose total completed spend exceeds £100.
9. Share of orders by channel as a percentage of all orders.
10. Specialties where the DNA rate exceeds 10% on at least 100 appointments.

```
-- 1
SELECT country, COUNT(*) FROM customers GROUP BY country;

-- 2
SELECT p.product_name, SUM(oi.quantity) AS units
FROM order_items oi JOIN products p USING (product_id)
GROUP BY p.product_name ORDER BY units DESC;

-- 3
SELECT category, ROUND(AVG(unit_price),2) FROM products GROUP BY category;

-- 4
SELECT category, COUNT(*) FROM products GROUP BY category HAVING COUNT(*) > 2;

-- 5
SELECT customer_id, COUNT(*) AS orders, MIN(order_ts)::date, MAX(order_ts)::date
FROM orders WHERE status='completed' GROUP BY customer_id;

-- 6 aggregation can't find absence; you need an anti-join
SELECT p.product_id, p.product_name FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id);

-- 7
SELECT DATE_TRUNC('month', o.order_ts)::date AS month,
       COUNT(DISTINCT o.order_id) AS orders,
```

```

        ROUND(SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)),2) AS revenue
FROM orders o JOIN order_items oi USING (order_id)
WHERE o.status='completed' AND o.order_ts >= DATE '2024-01-01'
GROUP BY 1 ORDER BY 1;

-- 8
SELECT o.customer_id, ROUND(SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)),2) AS spend
FROM orders o JOIN order_items oi USING (order_id)
WHERE o.status='completed'
GROUP BY o.customer_id HAVING SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) > 100;

-- 9 window aggregate for the denominator, see Part 11
SELECT channel, COUNT(*) AS orders,
        ROUND(100.0 * COUNT(*) / SUM(COUNT(*) OVER (), 1) AS pct_of_total
FROM orders GROUP BY channel;

-- 10
SELECT r.specialty,
        ROUND(100.0*COUNT(*) FILTER (WHERE a.outcome='DNA')/COUNT(*),1) AS dna_rate
FROM appointments a JOIN referrals r USING (referral_id)
GROUP BY r.specialty
HAVING COUNT(*) >= 100
        AND 100.0*COUNT(*) FILTER (WHERE a.outcome='DNA')/COUNT(*) > 10;

```

Exercise 9 is worth staring at. `SUM(COUNT(*)) OVER ()` is an aggregate of an aggregate — the window runs *after* the GROUP BY, so `COUNT(*)` is already the per-channel count and `SUM(...) OVER ()` totals them across all groups. It replaces a self-join or a subquery, and being comfortable with it marks you out.

PART 4 — JOINS

Joins decide whether your numbers are right. Interviewers know this, so joins get more airtime than any other topic.

4.1 Mental model

A join takes two sets of rows and produces a new set by matching them on a condition. Start from the conceptual cross product (every row of A paired with every row of B), keep the pairs where the ON condition is true, then — depending on the join type — add back unmatched rows from one or both sides with NULLs in the other side's columns.

INNER	matched pairs only
LEFT	matched pairs + unmatched LEFT rows (right cols NULL)
RIGHT	matched pairs + unmatched RIGHT rows (left cols NULL)
FULL OUTER	matched pairs + unmatched from both sides
CROSS	every pair, no condition

4.2 INNER JOIN

```
SELECT o.order_id, c.first_name, c.country, o.order_ts
FROM orders o
INNER JOIN customers c ON c.customer_id = o.customer_id;
```

JOIN alone means **INNER JOIN**. Rows without a match on either side vanish. That is fine when you want only matched data, and a silent data-loss bug when you assumed everything matches. If you inner-join and the row count drops, you have found either orphans or NULL keys — never shrug it off.

USING (customer_id) is shorthand when the column names are identical on both sides; it also collapses the two key columns into one in the output, which is handy with **SELECT ***.

```
SELECT * FROM orders JOIN customers USING (customer_id);
```

NATURAL JOIN joins on *all* identically named columns automatically. Never use it. **orders** and **customers** both have a **channel** column, so a natural join would match on **customer_id** AND **channel** and return near-nonsense. It's a good interview answer to explain exactly this: natural joins break invisibly when someone adds a column upstream.

4.3 LEFT JOIN

Keeps every row from the left table.

```
SELECT c.customer_id, c.first_name, COUNT(o.order_id) AS orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name
ORDER BY orders;
```

COUNT(o.order_id) not **COUNT(*)**: for a customer with no orders, the joined row still exists (with NULLs on the right), so **COUNT(*)** returns 1 and you'd report that everyone has at least one order. **COUNT(o.order_id)** ignores the NULL and correctly returns 0. This is the single most common LEFT JOIN mistake and a favourite interview trap.

The WHERE-vs-ON rule for outer joins

Put the condition in the wrong place and your LEFT JOIN silently becomes an INNER JOIN.

```
-- INTENDED: all customers, with their 2024 order count (0 if none)
-- BROKEN: the WHERE clause drops the NULL rows the LEFT JOIN just created
SELECT c.customer_id, COUNT(o.order_id)
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_ts >= DATE '2024-01-01' -- <-- NULL >= date is UNKNOWN, row dropped
GROUP BY c.customer_id;

-- CORRECT: filter the right table inside the ON clause
SELECT c.customer_id, COUNT(o.order_id)
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.customer_id
  AND o.order_ts >= DATE '2024-01-01'
GROUP BY c.customer_id;
```

The rule: **conditions on the outer (preserved) table go in WHERE; conditions on the optional table go in ON**. Say that sentence in an interview and it lands.

The exception is deliberate: `WHERE right_table.key IS NULL` after a LEFT JOIN is the **anti-join**, used on purpose to find non-matches.

```
-- customers who have never ordered
SELECT c.* FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

Use a column that is NOT NULL in the right table for the IS NULL test — ideally its primary key. Testing a nullable column can't distinguish "no match" from "matched, value was NULL".

4.4 RIGHT JOIN

The mirror of LEFT. Keeps all rows from the right table.

```
SELECT c.first_name, o.order_id
FROM orders o
RIGHT JOIN customers c ON c.customer_id = o.customer_id;
```

Functionally there is no reason to use it — every RIGHT JOIN is a LEFT JOIN with the tables swapped, and LEFT reads better because the preserved table is the one you already named in FROM. In a chain of four joins, a stray RIGHT JOIN in the middle is very hard to reason about. Know what it does, don't write it.

4.5 FULL OUTER JOIN

Keeps unmatched rows from both sides. Its genuine use case is reconciliation — comparing two sources that should agree.

```
-- reconciling a finance extract against the order table
SELECT
  COALESCE(o.order_id, f.order_id) AS order_id,
  o.total AS system_total,
  f.total AS finance_total,
  CASE WHEN o.order_id IS NULL THEN 'missing from system'
        WHEN f.order_id IS NULL THEN 'missing from finance'
        WHEN o.total <> f.total THEN 'value mismatch'
        ELSE 'ok' END AS reconciliation_status
FROM order_totals o
FULL OUTER JOIN finance_extract f ON f.order_id = o.order_id
WHERE o.order_id IS NULL OR f.order_id IS NULL OR o.total IS DISTINCT FROM f.total;
```

Note `COALESCE` on the key — with a full outer join either side's key can be NULL — and `IS DISTINCT FROM` for the comparison, so that a NULL on one side registers as a mismatch instead of being silently skipped.

4.6 CROSS JOIN

Every row of A with every row of B. No ON clause. Output is `rows(A) × rows(B)`.

Deliberate uses:

```
-- 1. Scaffolding: every product × every month, so gaps become explicit zeroes
SELECT p.product_id, m.month, COALESCE(SUM(oi.quantity), 0) AS units
FROM products p
CROSS JOIN generate_series(DATE '2024-01-01', DATE '2024-06-01', INTERVAL '1 month') AS m(month)
LEFT JOIN order_items oi ON oi.product_id = p.product_id
LEFT JOIN orders o ON o.order_id = oi.order_id
  AND DATE_TRUNC('month', o.order_ts) = m.month
  AND o.status = 'completed'
GROUP BY p.product_id, m.month;

-- 2. Attaching a single scalar to every row
SELECT c.customer_id, c.country, t.grand_total
FROM customers c
CROSS JOIN (SELECT SUM(shipping_cost) AS grand_total FROM orders) t;
```

Accidental cross joins — a missing or mistyped ON — are how a query that should return 10,000 rows returns 40 million and takes down the warehouse. If a query is inexplicably slow and huge, suspect a missing join condition first.

LATERAL is the correlated cousin: a subquery on the right that can reference columns from the left. It's the neat way to do "top 2 per group" without a window function:

```
SELECT c.customer_id, r.order_id, r.order_ts
FROM customers c
CROSS JOIN LATERAL (
  SELECT o.order_id, o.order_ts
  FROM orders o
  WHERE o.customer_id = c.customer_id AND o.status = 'completed'
  ORDER BY o.order_ts DESC
  LIMIT 2
) r;
```

Use **LEFT JOIN LATERAL (...)** **ON true** if you want to keep customers with no orders. **LATERAL** is often faster than a window function when you need a small N per group and there's an index on the correlation column.

4.7 SELF JOIN

A table joined to itself, with two aliases. The canonical case is a hierarchy.

```
SELECT e.full_name AS employee,
       m.full_name AS manager
FROM employees e
LEFT JOIN employees m ON m.employee_id = e.manager_id;
```

LEFT, not **INNER** — the CEO has a **NULL manager_id** and an inner join would delete them from the org chart. Interviewers seed exactly this.

Self joins also do sequence work (though window functions usually do it better):

```
-- pairs of employees in the same department earning within £1,000 of each other
SELECT a.full_name, b.full_name, a.salary, b.salary
FROM employees a
JOIN employees b
  ON a.department = b.department
  AND a.employee_id < b.employee_id           -- avoids self-pairs and mirror duplicates
  AND ABS(a.salary - b.salary) <= 1000;
```

The **a.employee_id < b.employee_id** condition is the idiom that turns every pair from appearing twice (A-B and B-A) into once. Expect to be asked why it's there.

4.8 Join keys, duplicates and join explosion

Fan-out. Joining a one-side to a many-side multiplies the one-side rows by the number of matches. That is correct behaviour, not a bug — but any measure from the one-side is now duplicated.

```
-- WRONG: shipping counted once per line
SELECT SUM(o.shipping_cost) FROM orders o JOIN order_items oi USING (order_id);

-- RIGHT: aggregate each grain separately, then combine
WITH line_rev AS (
  SELECT order_id, SUM(quantity * unit_price * (1 - discount_pct)) AS revenue
  FROM order_items GROUP BY order_id
)
SELECT SUM(l.revenue) AS revenue, SUM(o.shipping_cost) AS shipping
FROM orders o JOIN line_rev l USING (order_id)
WHERE o.status = 'completed';
```

The pattern — **aggregate to a common grain first, then join** — is the general answer to fan-out, and it is what a strong candidate reaches for automatically.

Join explosion. Joining two many-side tables to a common parent multiplies them by each other. An order with 3 items and 2 shipments produces 6 rows, and both quantities and shipment weights are now wrong.

```
-- 3 items × 2 shipments = 6 rows; both sums inflated
SELECT o.order_id, SUM(oi.quantity), SUM(s.weight_kg)
FROM orders o
JOIN order_items oi USING (order_id)
JOIN shipments s USING (order_id)
GROUP BY o.order_id;
```

Fix: pre-aggregate each branch to order grain, then join the summaries. This comes up in nearly every mid-level interview in some disguise.

Duplicate keys. If the "one" side isn't actually unique — a slowly-changing dimension with multiple versions per customer, say — every join against it doubles rows. Always check:

```
SELECT customer_id, COUNT(*) FROM dim_customer GROUP BY customer_id HAVING COUNT(*) > 1;
```

For an SCD Type 2 dimension you must join on the key *and* the validity window:

```
JOIN dim_customer d
  ON d.customer_id = o.customer_id
 AND o.order_ts >= d.valid_from
 AND o.order_ts < COALESCE(d.valid_to, 'infinity')
```

NULL keys never match. A NULL join key matches nothing, including other NULLs — `web_events.customer_id` being NULL for anonymous sessions means those events disappear from any inner join to `customers`, which is usually correct, but you must know it's happening.

Type mismatches. Joining `varchar` to `integer` errors in Postgres (more permissive engines coerce silently and match nothing). Joining text keys with trailing whitespace or different case is worse — it runs and returns fewer rows than it should. `TRIM(LOWER(x))` on both sides diagnoses it; fixing it upstream is the real answer.

4.9 Progressive join exercises

Level 1

1. Every order with the customer's name and country.
2. Every customer with their order count, including customers who never ordered.
3. Order lines with product names and category.
4. Employees with their manager's name, keeping the CEO.
5. Products never ordered.

Level 2 6. Revenue per customer, including zero for those who never bought. 7. For each category, the number of distinct customers who bought from it. 8. Customers who bought Electronics but never Home. 9. Orders alongside the count of lines they contain. 10. Each customer's most recent completed order date and the channel of that order.

Level 3 11. Total revenue and total shipping cost by month, both correct. (Watch the grain.) 12. Products bought together: pairs of products appearing in the same order, ordered by frequency. 13. Employees earning more than their manager. 14. For each patient, their first referral and whether they were ever seen. 15. Customers whose only orders are cancelled or refunded.

Level 4 16. Month-by-month table of every category, showing zero for months a category sold nothing. 17. Customers who bought every product in the Electronics category. (Relational division.) 18. For each order, revenue, shipping, and item count — no double counting anywhere. 19. Referrals with no corresponding appointment after 18 weeks — the RTT breach list. 20. The second-most-recent order per customer.

Solutions

```
-- 1
SELECT o.order_id, c.first_name || ' ' || c.last_name AS customer, c.country, o.order_ts
FROM orders o JOIN customers c USING (customer_id);

-- 2
SELECT c.customer_id, c.first_name, COUNT(o.order_id) AS orders
FROM customers c LEFT JOIN orders o USING (customer_id)
GROUP BY c.customer_id, c.first_name;

-- 3
SELECT oi.order_item_id, p.product_name, p.category, oi.quantity
FROM order_items oi JOIN products p USING (product_id);

-- 4
SELECT e.full_name AS employee, COALESCE(m.full_name, '(no manager)') AS manager
FROM employees e LEFT JOIN employees m ON m.employee_id = e.manager_id;

-- 5
SELECT p.* FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id);

-- 6
```

```

SELECT c.customer_id,
       COALESCE(SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)),0) AS revenue
FROM customers c
LEFT JOIN orders o ON o.customer_id=c.customer_id AND o.status='completed'
LEFT JOIN order_items oi ON oi.order_id=o.order_id
GROUP BY c.customer_id;

-- 7
SELECT p.category, COUNT(DISTINCT o.customer_id) AS customers
FROM products p
JOIN order_items oi USING (product_id)
JOIN orders o ON o.order_id=oi.order_id AND o.status='completed'
GROUP BY p.category;

-- 8
SELECT DISTINCT o.customer_id
FROM orders o
JOIN order_items oi USING (order_id)
JOIN products p USING (product_id)
WHERE p.category='Electronics' AND o.status='completed'
AND NOT EXISTS (
    SELECT 1 FROM orders o2
    JOIN order_items oi2 USING (order_id)
    JOIN products p2 USING (product_id)
    WHERE o2.customer_id=o.customer_id AND p2.category='Home' AND o2.status='completed'
);

-- 9
SELECT o.order_id, o.order_ts, COUNT(oi.order_item_id) AS lines
FROM orders o LEFT JOIN order_items oi USING (order_id)
GROUP BY o.order_id, o.order_ts;

-- 10 DISTINCT ON is the Postgres shortcut
SELECT DISTINCT ON (customer_id) customer_id, order_ts::date AS last_order, channel
FROM orders WHERE status='completed'
ORDER BY customer_id, order_ts DESC;

-- 11 pre-aggregate to avoid fan-out
WITH line_rev AS (
    SELECT order_id, SUM(quantity*unit_price*(1-discount_pct)) AS revenue
    FROM order_items GROUP BY order_id
)
SELECT DATE_TRUNC('month',o.order_ts)::date AS month,
       ROUND(SUM(l.revenue),2) AS revenue,
       ROUND(SUM(o.shipping_cost),2) AS shipping
FROM orders o JOIN line_rev l USING (order_id)
WHERE o.status='completed'
GROUP BY 1 ORDER BY 1;

-- 12 self join on order_items with the < idiom
SELECT p1.product_name AS product_a, p2.product_name AS product_b, COUNT(*) AS times_together
FROM order_items a
JOIN order_items b ON b.order_id=a.order_id AND b.product_id > a.product_id
JOIN products p1 ON p1.product_id=a.product_id
JOIN products p2 ON p2.product_id=b.product_id
GROUP BY 1,2 ORDER BY times_together DESC;

-- 13
SELECT e.full_name, e.salary, m.full_name AS manager, m.salary AS manager_salary
FROM employees e JOIN employees m ON m.employee_id=e.manager_id
WHERE e.salary > m.salary;

-- 14
SELECT DISTINCT ON (r.patient_id)
       r.patient_id, r.referral_id, r.referral_date,
       (a.appointment_id IS NOT NULL) AS ever_seen
FROM referrals r
LEFT JOIN appointments a ON a.referral_id=r.referral_id AND a.outcome='Attended'
ORDER BY r.patient_id, r.referral_date, a.attended_ts;

-- 15
SELECT c.customer_id FROM customers c
JOIN orders o ON o.customer_id=c.customer_id
GROUP BY c.customer_id
HAVING COUNT(*) FILTER (WHERE o.status='completed') = 0;

-- 16
SELECT cat.category, m.month, COALESCE(SUM(oi.quantity*oi.unit_price),0) AS revenue
FROM (SELECT DISTINCT category FROM products) cat

```

```

CROSS JOIN generate_series(DATE '2024-01-01', DATE '2024-04-01', INTERVAL '1 month') m(month)
LEFT JOIN products p ON p.category=cat.category
LEFT JOIN order_items oi ON oi.product_id=p.product_id
LEFT JOIN orders o ON o.order_id=oi.order_id
    AND o.status='completed'
    AND DATE_TRUNC('month',o.order_ts)=m.month
GROUP BY 1,2 ORDER BY 1,2;

-- 17 relational division: count distinct matches = count of targets
SELECT o.customer_id
FROM orders o JOIN order_items oi USING (order_id) JOIN products p USING (product_id)
WHERE p.category='Electronics' AND o.status='completed'
GROUP BY o.customer_id
HAVING COUNT(DISTINCT p.product_id) =
    (SELECT COUNT(*) FROM products WHERE category='Electronics');

-- 18
WITH li AS (
    SELECT order_id,
        SUM(quantity*unit_price*(1-discount_pct)) AS revenue,
        SUM(quantity) AS items
    FROM order_items GROUP BY order_id
)
SELECT o.order_id, o.order_ts, li.revenue, o.shipping_cost, li.items
FROM orders o LEFT JOIN li USING (order_id);

-- 19
SELECT r.referral_id, r.patient_id, r.specialty, r.referral_date,
    CURRENT_DATE - r.referral_date AS days_waiting
FROM referrals r
WHERE r.referral_date < CURRENT_DATE - INTERVAL '18 weeks'
    AND NOT EXISTS (
        SELECT 1 FROM appointments a
        WHERE a.referral_id=r.referral_id AND a.outcome='Attended'
    )
ORDER BY days_waiting DESC;

-- 20
SELECT customer_id, order_id, order_ts FROM (
    SELECT customer_id, order_id, order_ts,
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts DESC) AS rn
    FROM orders WHERE status='completed'
) t WHERE rn = 2;

```

PART 5 — CASE AND CONDITIONAL LOGIC

5.1 Syntax

```
CASE WHEN condition1 THEN result1
      WHEN condition2 THEN result2
      ELSE default
END
```

Conditions are evaluated **top to bottom, first match wins**. Later branches implicitly exclude earlier ones, which means you don't need `WHEN x >= 100 AND x < 500` if `x >= 500` was already handled above.

Omitting `ELSE` means unmatched rows get `NULL`. That is sometimes intentional and often a bug — always write an explicit `ELSE`, even if it's `ELSE 'Unknown'`, so the intent is visible.

All branches must return compatible types. Mixing `THEN 0` with `THEN 'none'` errors.

There's also a simple form for equality against one expression:

```
CASE status
  WHEN 'completed' THEN 'Revenue'
  WHEN 'refunded'  THEN 'Reversal'
  ELSE 'Excluded'
END
```

It cannot express ranges or `NULL` tests, so the searched form above is what you'll use 90% of the time. Note `CASE x WHEN NULL THEN ...` never matches, because it compiles to `x = NULL`; use the searched form with `WHEN x IS NULL`.

5.2 Customer segmentation

```
WITH customer_spend AS (
  SELECT o.customer_id,
         SUM(o.quantity * oi.unit_price * (1 - oi.discount_pct)) AS lifetime_value,
         COUNT(DISTINCT o.order_id) AS orders,
         MAX(o.order_ts)::date AS last_order
  FROM orders o JOIN order_items oi USING (order_id)
  WHERE o.status = 'completed'
  GROUP BY o.customer_id
)
SELECT customer_id, lifetime_value, orders, last_order,
       CASE
         WHEN lifetime_value >= 1000 AND last_order >= CURRENT_DATE - INTERVAL '90 days'
           THEN 'VIP Active'
         WHEN lifetime_value >= 1000
           THEN 'VIP At Risk'
         WHEN orders >= 3
           THEN 'Loyal'
         WHEN orders = 1 AND last_order >= CURRENT_DATE - INTERVAL '30 days'
           THEN 'New'
         WHEN last_order < CURRENT_DATE - INTERVAL '365 days'
           THEN 'Churned'
         ELSE 'Occasional'
       END AS segment
FROM customer_spend;
```

Order matters enormously here. 'VIP At Risk' only catches high-value customers who failed the recency test above it. Rewriting the branches in a different order gives a different segmentation from identical logic — and interviewers test this by asking "what happens if I move this `WHEN` to the top?"

5.3 Revenue bands and age groups

```
SELECT order_id, order_value,
       CASE WHEN order_value < 25 THEN 'Under £25'
            WHEN order_value < 50 THEN '£25-49'
            WHEN order_value < 100 THEN '£50-99'
            WHEN order_value < 250 THEN '£100-249'
            ELSE '£250+'
       END AS value_band
FROM order_totals;
```

Ascending non-overlapping bands with only an upper bound per branch: because the first match wins, each **WHEN** implicitly starts where the previous ended. Cleaner and less error-prone than repeating both bounds — and it makes gaps impossible.

Band labels sort alphabetically, not numerically, which mangles dashboards (£100–249' before '£25–49'). Emit a sort key alongside:

```
SELECT band, sort_order, COUNT(*)
FROM (
  SELECT CASE WHEN v < 25 THEN 'Under £25' WHEN v < 50 THEN '£25–49' ELSE '£50+' END AS band,
         CASE WHEN v < 25 THEN 1          WHEN v < 50 THEN 2          ELSE 3          END AS sort_order
    FROM order_totals
) t GROUP BY band, sort_order ORDER BY sort_order;
```

Age bands from a date of birth, NHS style:

```
SELECT patient_id,
       DATE_PART('year', AGE(CURRENT_DATE, date_of_birth))::int AS age,
       CASE WHEN date_of_birth IS NULL THEN 'Unknown'
            WHEN AGE(CURRENT_DATE, date_of_birth) < INTERVAL '18 years' THEN '0-17'
            WHEN AGE(CURRENT_DATE, date_of_birth) < INTERVAL '40 years' THEN '18-39'
            WHEN AGE(CURRENT_DATE, date_of_birth) < INTERVAL '65 years' THEN '40-64'
            WHEN AGE(CURRENT_DATE, date_of_birth) < INTERVAL '80 years' THEN '65-79'
            ELSE '80+'
       END AS age_band
FROM patients;
```

The NULL branch goes **first**. Put it last and NULL dates fall through every comparison (all UNKNOWN) into the ELSE, silently labelling patients with unknown DOB as '80+'. This is a genuinely common real-world data quality error, and it's a great answer when asked "what would you check in this query?".

5.4 Conditional aggregation

This is the highest-value CASE technique. `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` counts a subset; nest it inside aggregates to pivot rows into columns.

```
SELECT
  DATE_TRUNC('month', order_ts)::date AS month,
  COUNT(*)                               AS all_orders,
  SUM(CASE WHEN status = 'completed' THEN 1 ELSE 0 END) AS completed,
  SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END) AS cancelled,
  SUM(CASE WHEN status = 'refunded' THEN 1 ELSE 0 END)  AS refunded,
  ROUND(100.0 * SUM(CASE WHEN status='completed' THEN 1 ELSE 0 END) / COUNT(*), 1) AS completion_rate_pct
FROM orders
GROUP BY 1 ORDER BY 1;
```

Two idioms with different behaviour:

- `SUM(CASE WHEN c THEN 1 ELSE 0 END)` — counts matches, returns 0 when none.
- `COUNT(CASE WHEN c THEN 1 END)` — same result, relying on COUNT ignoring the implicit NULL ELSE.

Postgres has the cleaner **FILTER** clause, which does the same thing and reads better:

```
COUNT(*) FILTER (WHERE status = 'completed')
SUM(revenue) FILTER (WHERE country = 'UK')
AVG(price) FILTER (WHERE category = 'Electronics')
```

Use **FILTER** in Postgres, know the CASE form because it's portable, and be able to say which is which.

Pivoting revenue by category into columns:

```
SELECT c.country,
       ROUND(SUM(oi.quantity*oi.unit_price) FILTER (WHERE p.category='Electronics'), 2) AS electronics,
       ROUND(SUM(oi.quantity*oi.unit_price) FILTER (WHERE p.category='Home'), 2) AS home,
       ROUND(SUM(oi.quantity*oi.unit_price) FILTER (WHERE p.category='Apparel'), 2) AS apparel,
       ROUND(SUM(oi.quantity*oi.unit_price), 2) AS total
FROM orders o
JOIN order_items oi USING (order_id)
JOIN products p USING (product_id)
JOIN customers c ON c.customer_id = o.customer_id
WHERE o.status='completed'
GROUP BY c.country;
```

SQL cannot pivot on values it doesn't know at parse time — the category names have to be hardcoded. Dynamic pivots need application code, `crosstab()` from the `tablefunc` extension, or a BI tool. Saying that plainly is the right answer when an interviewer asks "what if a new category is added?".

5.5 CASE elsewhere in the query

In **ORDER BY** — custom sort orders:

```
ORDER BY CASE priority WHEN 'Two Week Wait' THEN 1
                  WHEN 'Urgent' THEN 2
                  ELSE 3 END,
referral_date;
```

In **WHERE** — usually a sign you should use plain boolean logic instead, but legitimate for parameterised filters:

```
WHERE CASE WHEN :include_cancelled THEN true ELSE status <> 'cancelled' END
```

In **GROUP BY** — group by a derived band. Repeat the whole expression, or reference the output position/alias (Postgres allows the alias):

```
SELECT CASE WHEN imd_decile <= 3 THEN 'Most deprived'
           WHEN imd_decile <= 7 THEN 'Mid'
           WHEN imd_decile IS NULL THEN 'Unknown'
           ELSE 'Least deprived' END AS deprivation_group,
COUNT(*)
FROM patients
GROUP BY deprivation_group;
```

In a **JOIN condition** — legal, and usually slow, because it defeats index use. Prefer restructuring.

5.6 Nested CASE

Nesting is legal but hurts readability. Usually a compound condition in a flat CASE says the same thing more clearly.

```
-- nested
CASE WHEN country = 'UK'
      THEN CASE WHEN lifetime_value > 500 THEN 'UK High' ELSE 'UK Standard' END
      ELSE CASE WHEN lifetime_value > 500 THEN 'Intl High' ELSE 'Intl Standard' END
END

-- flat, preferred
CASE WHEN country='UK' AND lifetime_value > 500 THEN 'UK High'
      WHEN country='UK' THEN 'UK Standard'
      WHEN lifetime_value > 500 THEN 'Intl High'
      ELSE 'Intl Standard'
END
```

Nesting earns its place when the inner logic is genuinely a sub-decision reused across branches, or when the outer branches have very different inner structures. Otherwise flatten.

5.7 Risk, KPI and pass/fail classification

```
-- NHS: RTT position with a RAG rating
SELECT r.referral_id, r.specialty,
CURRENT_DATE - r.referral_date AS days_waiting,
CASE WHEN CURRENT_DATE - r.referral_date > 126 THEN 'Red' -- >18 weeks
      WHEN CURRENT_DATE - r.referral_date > 84 THEN 'Amber' -- >12 weeks
      ELSE 'Green' END AS rag_status,
CASE WHEN r.priority = 'Two Week Wait'
      AND CURRENT_DATE - r.referral_date > 14 THEN true
      ELSE false END AS twow_breach
FROM referrals r
JOIN waiting_list w ON w.referral_id = r.referral_id
WHERE w.removed_date IS NULL; -- still waiting

-- Active / inactive / dormant customers
SELECT customer_id,
CASE WHEN last_order_date IS NULL THEN 'Never purchased'
      WHEN last_order_date >= CURRENT_DATE - INTERVAL '90 days' THEN 'Active'
      WHEN last_order_date >= CURRENT_DATE - INTERVAL '365 days' THEN 'Lapsing'
      ELSE 'Dormant' END AS status
FROM customer_last_order;
```

```
-- SLA compliance, pass/fail per ticket then rolled up
SELECT team,
       COUNT(*) AS tickets,
       COUNT(*) FILTER (WHERE resolved_ts - created_ts <= sla_target) AS met,
       ROUND(100.0 * COUNT(*) FILTER (WHERE resolved_ts - created_ts <= sla_target)
            / NULLIF(COUNT(*) FILTER (WHERE resolved_ts IS NOT NULL), 0), 1) AS sla_pct
FROM tickets GROUP BY team;
```

Note the denominator excludes unresolved tickets. Whether that's right is a business question — an open ticket that's already blown its SLA arguably should count as a failure. Raise it; don't just pick one. Analysts who surface the definitional ambiguity rather than silently choosing get hired.

5.8 CASE exercises

1. Label products 'Budget' (<£25), 'Mid' (<£60), 'Premium' otherwise.
2. Count orders by status in a single row using conditional aggregation.
3. Flag customers as 'Opted in'/'Opted out', handling NULL.
4. Revenue split by channel across columns, by month.
5. Classify order margin per line as 'Loss','Thin','Healthy' (<0, <20%, else).
6. Bucket A&E attendances by wait time: under 2h, 2–4h, 4–12h, over 12h, still in department.
7. For each customer, columns for orders in 2023 and orders in 2024, plus a growth flag.
8. Assign a customer priority score combining recency, frequency and value into High/Medium/Low.

```
-- 1
SELECT product_name, unit_price,
       CASE WHEN unit_price < 25 THEN 'Budget'
            WHEN unit_price < 60 THEN 'Mid'
            ELSE 'Premium' END AS tier
FROM products;

-- 2
SELECT COUNT(*) FILTER (WHERE status='completed') AS completed,
       COUNT(*) FILTER (WHERE status='cancelled') AS cancelled,
       COUNT(*) FILTER (WHERE status='refunded') AS refunded,
       COUNT(*) FILTER (WHERE status='pending') AS pending
FROM orders;

-- 3
SELECT customer_id,
       CASE WHEN marketing_opt_in IS NULL THEN 'Unknown'
            WHEN marketing_opt_in THEN 'Opted in'
            ELSE 'Opted out' END AS marketing_status
FROM customers;

-- 4
SELECT DATE_TRUNC('month',o.order_ts)::date AS month,
       ROUND(SUM(oi.quantity*oi.unit_price) FILTER (WHERE o.channel='web'),2) AS web,
       ROUND(SUM(oi.quantity*oi.unit_price) FILTER (WHERE o.channel='app'),2) AS app,
       ROUND(SUM(oi.quantity*oi.unit_price) FILTER (WHERE o.channel='phone'),2) AS phone
FROM orders o JOIN order_items oi USING (order_id)
WHERE o.status='completed' GROUP BY 1 ORDER BY 1;

-- 5
SELECT oi.order_item_id,
       CASE WHEN oi.unit_price*(1-oi.discount_pct) < p.unit_cost THEN 'Loss'
            WHEN (oi.unit_price*(1-oi.discount_pct) - p.unit_cost)
              / NULLIF(oi.unit_price*(1-oi.discount_pct),0) < 0.20 THEN 'Thin'
            ELSE 'Healthy' END AS margin_band
FROM order_items oi JOIN products p USING (product_id);

-- 6
SELECT CASE WHEN departure_ts IS NULL THEN 'Still in department'
            WHEN departure_ts - arrival_ts < INTERVAL '2 hours' THEN 'Under 2h'
            WHEN departure_ts - arrival_ts <= INTERVAL '4 hours' THEN '2-4h'
            WHEN departure_ts - arrival_ts <= INTERVAL '12 hours' THEN '4-12h'
            ELSE 'Over 12h' END AS wait_band,
       COUNT(*)
FROM ae_attendances GROUP BY 1;

-- 7
SELECT customer_id,
```

```

COUNT(*) FILTER (WHERE order_ts >= '2023-01-01' AND order_ts < '2024-01-01') AS orders_2023,
COUNT(*) FILTER (WHERE order_ts >= '2024-01-01' AND order_ts < '2025-01-01') AS orders_2024,
CASE WHEN COUNT(*) FILTER (WHERE order_ts >= '2024-01-01') >
      COUNT(*) FILTER (WHERE order_ts >= '2023-01-01' AND order_ts < '2024-01-01')
      THEN 'Growing' ELSE 'Flat or declining' END AS trend
FROM orders WHERE status='completed' GROUP BY customer_id;

-- 8 simple RFM
WITH rfm AS (
  SELECT o.customer_id,
         CURRENT_DATE - MAX(o.order_ts)::date AS recency_days,
         COUNT(DISTINCT o.order_id) AS frequency,
         SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS monetary
  FROM orders o JOIN order_items oi USING (order_id)
  WHERE o.status='completed' GROUP BY o.customer_id
)
SELECT customer_id, recency_days, frequency, monetary,
       CASE WHEN recency_days <= 90 AND frequency >= 3 AND monetary >= 200 THEN 'High'
            WHEN recency_days <= 180 AND (frequency >= 2 OR monetary >= 100) THEN 'Medium'
            ELSE 'Low' END AS priority
FROM rfm;

```

Parts 6–8: NULLs & Data Cleaning, Strings, Dates

PART 6 — NULLS AND DATA CLEANING

6.1 What NULL actually means

NULL is a marker for "no value here". It is not a value itself, which is why it doesn't behave like one. There are at least four distinct real-world situations that all arrive in your table as NULL, and conflating them is where analysts go wrong:

Meaning	Example	Correct handling
Unknown	patient's date of birth not recorded	keep NULL, report separately
Not applicable	<code>manager_id</code> for the CEO	keep NULL, LEFT JOIN
Not yet	<code>departure_ts</code> for a patient still in A&E	keep NULL, it's a live state
Genuinely zero	no orders this month	should be 0, not NULL

The last row is the one that causes reporting errors. A month with no sales *has* revenue of £0. If your query returns NULL because no rows existed to sum, you must convert it.

6.2 NULL vs 0 vs empty string

Three different things, and interviewers ask about the distinction explicitly.

```
SELECT
  COUNT(*)           AS rows,      -- 3
  COUNT(val)         AS non_null,  -- 2 (NULL excluded)
  SUM(val)           AS total,     -- 5 (NULL ignored)
  AVG(val)           AS mean       -- 2.5, NOT 1.67
FROM (VALUES (5),(0),(NULL)) AS t(val);
```

AVG here divides 5 by 2, not by 3. If the NULL genuinely means zero, your average is 50% too high. That single example is worth memorising as your answer to "what's the difference between NULL and 0?".

Empty string is a *value*, and a different one from NULL:

```
SELECT '' IS NULL;           -- false
SELECT LENGTH('');          -- 0
SELECT LENGTH(NULL);        -- NULL
SELECT COUNT(*) FROM customers WHERE email = ''; -- counts empties only
SELECT COUNT(*) FROM customers WHERE email IS NULL; -- counts nulls only
```

Real extracts contain all of: NULL, '', ' ', 'NULL' (the literal string), 'N/A', '-', 'unknown'. Standardising them is step one of any cleaning job:

```
NULLIF(NULLIF(NULLIF(TRIM(email), ''), 'N/A'), 'NULL') AS email_clean
-- or more readably
CASE WHEN TRIM(COALESCE(email, '')) IN ('', 'N/A', 'NULL', '-', 'unknown') THEN NULL
      ELSE TRIM(email) END AS email_clean
```

Dialect. Oracle famously treats the empty string as NULL, so `'' IS NULL` is true there. Postgres, SQL Server and MySQL keep them distinct. Mention it if you're asked about portability.

6.3 NULL propagation in arithmetic and concatenation

Any operation involving NULL yields NULL.

```
SELECT 100 + NULL;           -- NULL
SELECT 'abc' || NULL;        -- NULL (!!)
SELECT CONCAT('abc', NULL);  -- 'abc' (CONCAT skips NULLs)
SELECT GREATEST(1, NULL, 3); -- 3 (GREATEST/LEAST skip NULLs in Postgres)
```

The `||` case bites constantly. Building an address by concatenating five fields returns NULL for any customer missing one of them — so a missing county wipes out the entire address. Use `CONCAT_WS` (Part 7) or `COALESCE` each part.

```
-- broken: any NULL component nulls the whole thing
SELECT address_line_1 || ', ' || city || ', ' || postcode FROM customers;

-- fine
SELECT CONCAT_WS(', ', address_line_1, NULLIF(TRIM(city), ''), postcode) FROM customers;
```

6.4 COALESCE

Returns the first non-NULL argument. Takes any number of arguments; they must be type-compatible.

```
COALESCE(email, 'no email on file')
COALESCE(discount_pct, 0)
COALESCE(preferred_name, first_name, 'Customer') -- fallback chain
COALESCE(removed_date, CURRENT_DATE) -- treat "still open" as today
COALESCE(SUM(revenue), 0) -- empty groups report zero
```

The `COALESCE(end_date, CURRENT_DATE)` idiom is how you compute durations for open-ended records — waiting list time for patients still waiting, tenure for current employees, session length for a session still in progress.

```
-- current waiting time, including people still on the list
SELECT w.waiting_id,
       COALESCE(w.removed_date, CURRENT_DATE) - w.added_date AS days_waiting,
       (w.removed_date IS NULL) AS still_waiting
FROM waiting_list w;
```

Be careful: `COALESCE` inside a `WHERE` clause on an indexed column blocks the index. `WHERE COALESCE(status, 'x') <> 'cancelled'` will scan. Rewrite as `WHERE status IS NULL OR status <> 'cancelled'`.

Dialect. `COALESCE` is standard and works everywhere. `ISNULL()` is SQL Server's two-argument version, `IFNULL()` is MySQL's, `NVL()` is Oracle's. Use `COALESCE`.

6.5 NULLIF

`NULLIF(a, b)` returns `NULL` if `a = b`, otherwise returns `a`. It has exactly two everyday uses.

Guarding division by zero — the important one:

```
SELECT category,
       SUM(revenue) / NULLIF(SUM(units), 0) AS revenue_per_unit
FROM sales GROUP BY category;
```

Division by zero raises an error and kills the whole query; dividing by `NULL` returns `NULL` and lets the rest of the report through. Every rate, ratio and percentage you write should have `NULLIF(denominator, 0)`. This is one of the fastest ways to look experienced.

Converting sentinel values to NULL:

```
NULLIF(TRIM(city), '') -- blank becomes NULL
NULLIF(age, -1) -- -1 used as "not recorded"
NULLIF(imd_decile, 0) -- 0 is not a valid decile
```

6.6 Aggregates, joins and NULL

Recap the behaviours that get tested:

```
-- 1. aggregates ignore NULL, COUNT(*) doesn't
SELECT COUNT(*), COUNT(email) FROM customers;

-- 2. GROUP BY treats all NULLs as ONE group
SELECT country, COUNT(*) FROM customers GROUP BY country; -- one row where country IS NULL

-- 3. DISTINCT treats all NULLs as one value
SELECT COUNT(DISTINCT country) FROM customers; -- NULL not counted at all

-- 4. NULL keys never join
-- 5. UNION deduplicates NULLs as if equal
-- 6. ORDER BY: NULLs last for ASC, first for DESC (Postgres default)
```

Points 2 and 3 look contradictory and are worth stating carefully: `GROUP BY` puts all `NULL`s in one group *and shows it*, while `COUNT(DISTINCT col)` excludes `NULL`s entirely. So a table with 3 countries plus some `NULL`s gives 4 groups from `GROUP BY` but `COUNT(DISTINCT country) = 3`.

6.7 The classic NULL interview traps

Trap 1 — NOT IN with NULLs. Covered in Part 2.4. Returns zero rows. Use `NOT EXISTS`.

Trap 2 — the inequality that loses rows.

```
SELECT COUNT(*) FROM orders WHERE discount_code <> 'SPRING10';
```

Excludes every order where `discount_code IS NULL` — usually the majority. The stakeholder asked "how many orders didn't use SPRING10?" and you've undercounted badly.

```
WHERE discount_code IS DISTINCT FROM 'SPRING10'
```

Trap 3 — NOT (condition) doesn't complement the condition. `WHERE x > 10` and `WHERE NOT (x > 10)` together do not return every row, because NULLs satisfy neither.

Trap 4 — CHECK constraints pass on NULL. `CHECK (age >= 0)` accepts NULL ages, because the check must evaluate to false to reject and NULL isn't false.

Trap 5 — LEFT JOIN then COUNT(*). Returns 1 for unmatched rows. Count the right table's key.

Trap 6 — LEFT JOIN then filter in WHERE. Turns it into an inner join. Part 4.3.

Trap 7 — UNIQUE allows multiple NULLs. In Postgres a unique column may contain many NULL rows, because NULLs aren't equal to each other. So a UNIQUE email column does not prevent 500 customers with no email.

Trap 8 — AVG excludes rather than zeroes. Part 6.2.

Trap 9 — NULL in a boolean. `WHERE is_active` drops rows where `is_active IS NULL` and rows where it's false, identically. If NULL means "not yet decided", use `WHERE is_active IS NOT FALSE` or be explicit.

Trap 10 — string concatenation with ||. Part 6.3.

6.8 Duplicate records

There are two kinds of duplicate and they need different fixes.

Exact duplicates — every column identical, typically from a load run twice.

```
-- detect
SELECT order_id, customer_id, order_ts, COUNT(*) AS copies
FROM orders_staging
GROUP BY order_id, customer_id, order_ts
HAVING COUNT(*) > 1;

-- count how bad it is
SELECT COUNT(*) AS rows, COUNT(DISTINCT order_id) AS distinct_orders FROM orders_staging;
```

Business duplicates — different rows representing the same real thing. Same customer registered twice with different emails; the same referral entered under two IDs. These are harder and need fuzzy matching on name + DOB + postcode, or a defined match key.

```
-- candidate duplicate customers on a normalised key
SELECT LOWER(TRIM(last_name)) AS ln, date_of_birth, postcode_sector,
COUNT(*) AS records, ARRAY_AGG(patient_id) AS ids
FROM patients
GROUP BY 1,2,3
HAVING COUNT(*) > 1;
```

6.9 Deduplication techniques

Keep one row per key — ROW_NUMBER, the general answer.

```
WITH ranked AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY customer_id -- the key that should be unique
      ORDER BY updated_at DESC, id DESC -- which copy wins: newest, tie-broken by id
    ) AS rn
  FROM customer_staging
)
SELECT * FROM ranked WHERE rn = 1;
```

The ORDER BY inside the window is the business rule — "keep the most recently updated record" — made explicit. Always include a deterministic tie-breaker (`id`), otherwise the same query can return different rows on different runs and your pipeline becomes non-reproducible. Interviewers who have been burned by this will ask.

Postgres shortcut:

```
SELECT DISTINCT ON (customer_id) *
FROM customer_staging
ORDER BY customer_id, updated_at DESC, id DESC;
```

Exact duplicates only — DISTINCT or GROUP BY:

```
SELECT DISTINCT * FROM orders_staging;
```

Deleting duplicates in place, keeping the lowest ctid (Postgres physical row identifier — useful when there is genuinely no unique column):

```
DELETE FROM orders_staging a
USING orders_staging b
WHERE a.ctid > b.ctid
AND a.order_id = b.order_id;
```

Comparing the three approaches — an interviewer may ask which you'd use:

Method	When	Caveat
DISTINCT	whole rows identical	can't choose which copy to keep
GROUP BY + aggregates	want to merge values across copies	must aggregate every column
ROW_NUMBER	need a rule for which copy wins	portable, most flexible
DISTINCT ON	same, Postgres only	shortest, fastest

6.10 Invalid values and standardisation

Real data validation checks, all of which make good things to volunteer in an interview:

```
SELECT
  COUNT(*) FILTER (WHERE quantity <= 0) AS bad_quantity,
  COUNT(*) FILTER (WHERE unit_price < 0) AS negative_price,
  COUNT(*) FILTER (WHERE discount_pct < 0 OR discount_pct > 1) AS impossible_discount,
  COUNT(*) FILTER (WHERE unit_price < 0.01) AS suspicious_free
FROM order_items;

SELECT
  COUNT(*) FILTER (WHERE date_of_birth > CURRENT_DATE) AS future_dob,
  COUNT(*) FILTER (WHERE date_of_birth < DATE '1900-01-01') AS implausible_dob,
  COUNT(*) FILTER (WHERE imd_decile NOT BETWEEN 1 AND 10) AS bad_decile,
  COUNT(*) FILTER (WHERE nhs_number !~ '^([0-9]{10})$') AS malformed_nhs_no
FROM patients;

-- referential and temporal sanity
SELECT COUNT(*) FROM ae_attendances WHERE departure_ts < arrival_ts; -- time travel
SELECT COUNT(*) FROM appointments WHERE attended_ts < scheduled_ts - INTERVAL '1 day';
```

Standardisation — collapsing variant spellings to one canonical value:

```
SELECT
  CASE
    WHEN UPPER(TRIM(country)) IN ('UK', 'GB', 'GBR', 'UNITED KINGDOM', 'GREAT BRITAIN', 'ENGLAND',
                                  'SCOTLAND', 'WALES', 'NORTHERN IRELAND') THEN 'UK'
    WHEN UPPER(TRIM(country)) IN ('IE', 'IRL', 'IRELAND', 'REPUBLIC OF IRELAND', 'EIRE') THEN 'IE'
    WHEN TRIM(COALESCE(country, '')) = '' THEN NULL
    ELSE INITCAP(TRIM(country))
  END AS country_clean,
  COUNT(*)
FROM customers
GROUP BY 1;
```

In production this belongs in a lookup/mapping table joined at load time, not a CASE statement buried in one query — otherwise every analyst writes their own slightly different version and the numbers stop agreeing. Saying that shows you think about maintainability, which is exactly what distinguishes an analyst from someone who writes queries.

6.11 A complete cleaning pipeline

Worth reading end to end; it's the shape of a real staging-to-clean transformation and a strong thing to sketch on a whiteboard.

```

WITH standardised AS (
  SELECT
    customer_id,
    INITCAP(TRIM(first_name)) AS first_name,
    INITCAP(TRIM(last_name)) AS last_name,
    NULLIF(LOWER(TRIM(email)), '') AS email,
    CASE WHEN UPPER(TRIM(country)) IN ('UK', 'GB', 'UNITED KINGDOM') THEN 'UK'
         WHEN TRIM(COALESCE(country, '')) = '' THEN NULL
         ELSE UPPER(TRIM(country)) END AS country,
    signup_date,
    COALESCE(marketing_opt_in, false) AS marketing_opt_in,
    updated_at
  FROM customers_raw
),
validated AS (
  SELECT *,
    CASE WHEN email !~ '^[^@\s]+@[^@\s]+\.[^@\s]+$' THEN false ELSE true END AS email_valid,
    (signup_date <= CURRENT_DATE) AS signup_plausible
  FROM standardised
),
deduplicated AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id
                                ORDER BY updated_at DESC NULLS LAST, customer_id) AS rn
  FROM validated
)
SELECT customer_id, first_name, last_name,
  CASE WHEN email_valid THEN email ELSE NULL END AS email,
  country, signup_date, marketing_opt_in
FROM deduplicated
WHERE rn = 1
  AND signup_plausible;

```

Each CTE does one job: standardise, validate, deduplicate, select. That structure is readable, individually testable (run any CTE alone to inspect it), and easy for a reviewer to follow — which is the argument for CTEs generally, made in Part 10.

6.12 NULL and cleaning exercises

1. Count customers with a missing email, and separately those with a blank one.
2. Compute the average feedback score twice — excluding non-responses, and treating them as zero.
3. Find orders whose discount code isn't SPRING10, including those with no code.
4. Safely compute conversion rate where some days have zero sessions.
5. Find duplicate patients on name + DOB + postcode sector.
6. Keep only the latest record per customer from a staging table with an `updated_at`.
7. Build a full display address from four possibly-NULL components.
8. Find order lines with impossible values.
9. Compute days waiting for everyone on the waiting list, including those still waiting.
10. Report customers per country, showing 'Unknown' rather than a blank row for missing countries.

```

-- 1
SELECT COUNT(*) FILTER (WHERE email IS NULL) AS missing,
  COUNT(*) FILTER (WHERE TRIM(email) = '') AS blank
FROM customers;

-- 2
SELECT AVG(score) AS excluding_nonresponse,
  AVG(COALESCE(score, 0)) AS treating_as_zero,
  COUNT(*) AS surveyed, COUNT(score) AS responded
FROM survey;

-- 3
SELECT * FROM orders WHERE discount_code IS DISTINCT FROM 'SPRING10';

-- 4
SELECT day, ROUND(100.0 * purchases / NULLIF(sessions, 0), 2) AS conversion_pct
FROM daily_funnel;

-- 5
SELECT LOWER(TRIM(p.postcode_sector)) AS pc, p.date_of_birth,
  COUNT(*) AS n, ARRAY_AGG(p.patient_id) AS patient_ids
FROM patients p GROUP BY 1, 2 HAVING COUNT(*) > 1;

```

```

-- 6
SELECT DISTINCT ON (customer_id) *
FROM customer_staging ORDER BY customer_id, updated_at DESC, customer_id;

-- 7
SELECT CONCAT_WS(' ', NULLIF(TRIM(line1), ''), NULLIF(TRIM(line2), ''),
                NULLIF(TRIM(city), ''), NULLIF(TRIM(postcode), '')) AS address
FROM addresses;

-- 8
SELECT * FROM order_items
WHERE quantity <= 0 OR unit_price < 0 OR discount_pct NOT BETWEEN 0 AND 1
      OR quantity IS NULL OR unit_price IS NULL;

-- 9
SELECT waiting_id, added_date,
       COALESCE(removed_date, CURRENT_DATE) - added_date AS days_waiting,
       removed_date IS NULL AS still_waiting
FROM waiting_list;

-- 10
SELECT COALESCE(NULLIF(TRIM(country), ''), 'Unknown') AS country, COUNT(*)
FROM customers GROUP BY 1 ORDER BY 2 DESC;

```

PART 7 — STRING FUNCTIONS

Analysts spend more time on strings than they expect: cleaning names, parsing URLs, extracting postcode areas, splitting delimited fields, matching messy free text.

7.1 Concatenation

```
SELECT first_name || ' ' || last_name      AS full_name;      -- NULL if either is NULL
SELECT CONCAT(first_name, ' ', last_name)  AS full_name;      -- NULLs become ''
SELECT CONCAT_WS(',', ' ', city, county, postcode) AS address; -- separator, skips NULLs
```

`CONCAT_WS` is the one to reach for with optional components: it puts the separator only between the values that exist, so a missing county doesn't leave a doubled comma.

```
SELECT CONCAT_WS(',', ' ', 'Leeds', NULL, 'LS1 4AB'); -- 'Leeds, LS1 4AB'
SELECT 'Leeds' || ' ', ' ' || NULL || ' ', ' ' || 'LS1 4AB'; -- NULL
```

Dialect. `||` is standard SQL and works in Postgres, Oracle, SQLite. SQL Server uses `+` (and `CONCAT`). MySQL's `||` means OR by default — use `CONCAT` there.

7.2 Case and trimming

```
LOWER(x)    UPPER(x)    INITCAP(x)    -- INITCAP: 'john SMITH' -> 'John Smith'
TRIM(x)     LTRIM(x)     RTRIM(x)     -- whitespace by default
TRIM(BOTH '0' FROM '00123') -- '123' — trim specific characters
BTRIM(x, '.,') -- Postgres: trim any of these chars from both ends
```

`TRIM(LOWER(x))` before comparing text is the default defensive move — `'UK '` and `'uk '` are different values and will fail to join, fail to group together, and quietly split your totals in two.

`INITCAP` is naive about real names: `'o'neill'` becomes `'O'Neill'` correctly by luck, but `'mcdonald'` becomes `'Mcdonald'` and `'ABC LTD'` becomes `'Abc Ltd'`. Don't apply it to company names or use it as a data fix; use it for display only.

7.3 Length, padding, position

```
LENGTH('hello')      -- 5 characters
OCTET_LENGTH('hello') -- 5 bytes (differs for non-ASCII)
LPAD('7', 3, '0')     -- '007'
RPAD('ab', 5, '.')    -- 'ab...'
POSITION('@' IN email) -- 1-based index, 0 if not found
STRPOS(email, '@')    -- Postgres synonym
```

`LPAD` fixes the classic problem of numeric codes stored as text losing leading zeros in Excel — cost centre `'00123'` arriving as `'123'`.

```
SELECT LPAD(cost_centre::text, 5, '0') FROM finance_extract;
```

7.4 Substring extraction

```
LEFT(str, n)          -- first n characters
RIGHT(str, n)         -- last n
SUBSTRING(str FROM 3 FOR 4) -- standard syntax
SUBSTRING(str, 3, 4)    -- Postgres shorthand: from position 3, 4 chars
SUBSTRING(str FROM '[0-9]+') -- regex form: first match
```

UK postcode handling, which comes up in any local-authority or NHS role:

```
SELECT
  postcode,
  UPPER(REPLACE(postcode, ' ', '')) AS normalised,
  SUBSTRING(UPPER(postcode) FROM '^[A-Z]{1,2}') AS area,      -- 'LS'
  SPLIT_PART(UPPER(TRIM(postcode)), ' ', 1) AS outcode,      -- 'LS1'
  LEFT(SPLIT_PART(UPPER(TRIM(postcode)), ' ', 2), 1) AS sector_digit
FROM addresses;
```

Postcodes vary in length (`M1 1AA` to `EC1A 1BB`), so fixed-position slicing is wrong. Split on the space, or regex it.

7.5 REPLACE and TRANSLATE

```
REPLACE('01234 567 890', ' ', '')      -- strip spaces from a phone number
REPLACE(product_name, '&', '&')          -- undo HTML escaping
TRANSLATE('£1,234.00', '£', '')         -- delete multiple chars at once -> '1234.00'
```

TRANSLATE maps characters one-to-one; when the "to" string is shorter, the extra "from" characters are deleted. It's the neat way to strip a set of characters without chaining REPLACES.

```
-- currency string to numeric
SELECT TRANSLATE('£1,234.56', '£', '')::numeric; -- 1234.56
```

7.6 SPLIT_PART and string_to_array

```
SPLIT_PART('leeds/cardiology/routine', '/', 2) -- 'cardiology'
SPLIT_PART(email, '@', 2)                      -- domain
STRING_TO_ARRAY('a,b,c', ',')                  -- {a,b,c}
UNNEST(STRING_TO_ARRAY(tags, ','))              -- one row per tag
```

SPLIT_PART returns an empty string, not an error, when the part doesn't exist — convenient, but wrap in **NULLIF(..., '')** if absence should read as NULL.

Exploding a delimited column into rows is a common cleaning task:

```
SELECT product_id, TRIM(tag) AS tag
FROM products, UNNEST(STRING_TO_ARRAY(tags, ',')) AS tag;
```

Dialect. **SPLIT_PART** is Postgres. SQL Server has **STRING_SPLIT** (returns rows, ordinal only from 2022), Snowflake has **SPLIT_PART** too, BigQuery has **SPLIT** returning an array.

7.7 Regular expressions

Postgres regex operators and functions:

```
str ~ pattern          -- matches, case sensitive
str ~* pattern         -- matches, case insensitive
str !~ pattern         -- does not match
REGEXP_REPLACE(str, pattern, replacement, 'g') -- 'g' = replace all occurrences
REGEXP_MATCHES(str, pattern)                  -- returns array of capture groups
SUBSTRING(str FROM pattern)                   -- first match as text
REGEXP_SPLIT_TO_TABLE(str, pattern)           -- split to rows
```

Practical cleaning:

```
-- keep digits only (phone numbers, NHS numbers, reference codes)
REGEXP_REPLACE(phone, '[^0-9]', '', 'g')

-- collapse runs of whitespace to a single space
REGEXP_REPLACE(TRIM(free_text), '\s+', ' ', 'g')

-- validate email shape
WHERE email ~* '^[^@\s]+@[^\s]+\.[a-z]{2,}$'

-- validate UK postcode (simplified but practical)
WHERE UPPER(REGEXP_REPLACE(postcode, ' ', '')) ~ '^[A-Z]{1,2}[0-9][A-Z0-9]?[0-9][A-Z]{2}$'

-- extract the first number from free text
SUBSTRING(notes FROM '[0-9]+')

-- strip everything except letters and spaces from a name
REGEXP_REPLACE(full_name, '[^A-Za-z '], '', 'g')
```

Regex is slow on large tables and cannot use a standard index. For filtering millions of rows prefer **LIKE** with a trailing wildcard where possible, or precompute a cleaned column.

Dialect. Regex support varies more than anything else in SQL. SQL Server has essentially none before 2025 (you use **LIKE** with bracket classes or CLR functions); MySQL uses **REGEXP/RLIKE**; BigQuery uses **REGEXP_CONTAINS/REGEXP_EXTRACT**. Flag this if asked about portability.

7.8 Practical cleaning examples

Parsing a URL into path and query parameters:

```

SELECT page_url,
       SPLIT_PART(SPLIT_PART(page_url, '?', 1), '://', 2) AS host_and_path,
       NULLIF(SPLIT_PART(page_url, '?', 2), '') AS query_string,
       SUBSTRING(page_url FROM 'utm_source=([^&]+)') AS utm_source,
       SUBSTRING(page_url FROM 'utm_campaign=([^&]+)') AS utm_campaign
FROM web_events
WHERE event_name = 'page_view';

```

Splitting a single name field:

```

SELECT full_name,
       SPLIT_PART(TRIM(full_name), ' ', 1) AS first_name,
       NULLIF(REGEXP_REPLACE(TRIM(full_name), '^S+\s*', ''), '') AS rest_of_name
FROM employees;

```

Caveat worth voicing in an interview: name splitting is culturally naive and loses information (double-barrelled surnames, patronymics, name orders where the family name comes first). If the source system has separate fields, use them.

Fuzzy matching for deduplication — Postgres ships `fuzzystrmatch` and `pg_trgm`:

```

CREATE EXTENSION IF NOT EXISTS pg_trgm;

SELECT a.patient_id, b.patient_id, a.surname, b.surname,
       SIMILARITY(a.surname, b.surname) AS score
FROM patients a JOIN patients b ON a.patient_id < b.patient_id
WHERE a.date_of_birth = b.date_of_birth
      AND SIMILARITY(a.surname, b.surname) > 0.6;

SELECT SOUNDEX('Smith'), SOUNDEX('Smyth');           -- both S530
SELECT LEVENSHTEIN('Smith', 'Smyth');                 -- 1 edit

```

Restricting the pairing by an exact key first (`date_of_birth` here) is essential — comparing every row to every other row is quadratic and will never finish on a real table.

7.9 String exercises

1. Build a full name from first and last, handling missing parts.
2. Extract the email domain and count customers per domain.
3. Normalise phone numbers to digits only.
4. Find products whose name contains a number.
5. Extract the UK postcode area (letters at the start).
6. Standardise country names to uppercase, trimmed, with blanks as NULL.
7. Split a comma-separated tag column into one row per tag.
8. Find customers whose email doesn't look valid.
9. Mask emails for a shared export: first character, then asterisks, then the domain.
10. Extract the campaign name from a URL query string.

```

-- 1
SELECT CONCAT_WS(' ', NULLIF(TRIM(first_name), ''), NULLIF(TRIM(last_name), '')) FROM customers;

-- 2
SELECT LOWER(SPLIT_PART(email, '@', 2)) AS domain, COUNT(*) AS customers
FROM customers WHERE email IS NOT NULL GROUP BY 1 ORDER BY 2 DESC;

-- 3
SELECT phone, REGEXP_REPLACE(phone, '^[0-9]', '', 'g') AS digits FROM contacts;

-- 4
SELECT product_name FROM products WHERE product_name ~ '[0-9]';

-- 5
SELECT postcode, SUBSTRING(UPPER(TRIM(postcode)) FROM '[A-Z]{1,2}') AS area FROM addresses;

-- 6
SELECT NULLIF(UPPER(TRIM(country)), '') AS country_clean, COUNT(*) FROM customers GROUP BY 1;

-- 7
SELECT p.product_id, TRIM(t) AS tag
FROM products p, UNNEST(STRING_TO_ARRAY(p.tags, ',')) AS t;

```

```
-- 8
SELECT customer_id, email FROM customers
WHERE email IS NOT NULL AND email !~* '^[^@\s]+@[^@\s]+\.[a-z]{2,}$';

-- 9
SELECT email,
    LEFT(email,1) || REPEAT('*', GREATEST(POSITION('@' IN email)-2, 0))
    || SUBSTRING(email FROM POSITION('@' IN email)) AS masked
FROM customers WHERE email IS NOT NULL;

-- 10
SELECT DISTINCT SUBSTRING(page_url FROM 'utm_campaign=([^&]+)' AS campaign
FROM web_events WHERE page_url LIKE '%utm_campaign=%';
```

PART 8 — DATE AND TIME ANALYSIS

Almost every analytical question has a time dimension. This section is long because dates generate more interview questions than any topic except joins.

8.1 Types

Type	Contains	Example
date	calendar day	2024-03-15
time	clock time	14:30:00
timestamp	date + time, no zone	2024-03-15 14:30:00
timestampz	date + time, zone-aware	stored UTC, displayed in session zone
interval	a duration	3 days 04:00:00

Subtracting two dates gives an **integer** (days). Subtracting two timestamps gives an **interval**. That difference trips people up constantly:

```
SELECT DATE '2024-03-15' - DATE '2024-03-01';           -- 14 (integer)
SELECT TIMESTAMP '2024-03-15 10:00' - TIMESTAMP '2024-03-01 08:00'; -- 14 days 02:00:00
SELECT EXTRACT(EPOCH FROM (t2 - t1)) / 3600 AS hours_between; -- numeric hours
```

`EXTRACT(EPOCH FROM interval)` gives seconds, and is how you turn an interval into a number you can average. `AVG` of an interval works in Postgres, but converting to a number first is more portable and easier to format.

8.2 Current date and time

```
CURRENT_DATE           -- 2024-03-15
CURRENT_TIMESTAMP / NOW() -- start of the current transaction, with zone
LOCALTIMESTAMP         -- without zone
STATEMENT_TIMESTAMP()  -- start of the current statement
CLOCK_TIMESTAMP()      -- actual wall clock, changes within a statement
```

`NOW()` is fixed for the whole transaction, so every row in a long query gets the same value — which is what you want for reproducibility.

Reproducibility warning. A saved query using `CURRENT_DATE` gives different answers on different days, which makes results impossible to reconcile with a report someone ran last week. For anything that will be re-run or audited, parameterise the reporting date rather than hardcoding "today":

```
WITH params AS (SELECT DATE '2024-03-31' AS as_at)
SELECT ... FROM orders, params WHERE order_ts < params.as_at;
```

8.3 INTERVAL and date arithmetic

```
CURRENT_DATE + INTERVAL '30 days'
CURRENT_DATE - INTERVAL '1 year'
order_ts + INTERVAL '2 hours 30 minutes'
DATE '2024-01-31' + INTERVAL '1 month' -- 2024-02-29 (clamps to month end)
CURRENT_DATE + 7                      -- date + integer = date, days
```

Month arithmetic clamps rather than overflowing: 31 January plus one month is 29 February in a leap year, 28 February otherwise. That's usually what you want, but note it isn't reversible — adding a month then subtracting one doesn't always return the start date.

Common windows:

```
WHERE order_ts >= CURRENT_DATE - INTERVAL '30 days' -- rolling 30 days
WHERE order_ts >= DATE_TRUNC('month', CURRENT_DATE) -- month to date
WHERE order_ts >= DATE_TRUNC('year', CURRENT_DATE)  -- year to date
WHERE order_ts >= DATE_TRUNC('month', CURRENT_DATE) - INTERVAL '1 month'
AND order_ts < DATE_TRUNC('month', CURRENT_DATE) -- last full month
WHERE order_ts >= DATE_TRUNC('quarter', CURRENT_DATE) -- quarter to date
```

Dialect. SQL Server uses `DATEADD(day, -30, GETDATE())` and `DATEDIFF(day, a, b)`. MySQL uses `DATE_SUB(NOW(), INTERVAL 30 DAY)`. BigQuery uses `DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)`. The Postgres `INTERVAL` literal syntax is the standard one.

8.4 DATE_TRUNC — the workhorse

Rounds a timestamp down to the start of a period. It's how you build every time series.

```
DATE_TRUNC('hour', ts) -- 2024-03-15 14:00:00
DATE_TRUNC('day', ts) -- 2024-03-15 00:00:00
DATE_TRUNC('week', ts) -- Monday of that week (ISO: weeks start Monday)
DATE_TRUNC('month', ts) -- 2024-03-01 00:00:00
DATE_TRUNC('quarter', ts) -- 2024-01-01
DATE_TRUNC('year', ts) -- 2024-01-01
```

The result is a timestamp; cast to `::date` for tidy output.

Postgres weeks start on **Monday**, matching ISO 8601 and UK business convention. Many US-origin tools start weeks on Sunday. If a stakeholder's numbers disagree with yours by a day at week boundaries, this is why.

```
SELECT DATE_TRUNC('week', o.order_ts)::date AS week_commencing,
       COUNT(DISTINCT o.order_id) AS orders,
       ROUND(SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)),2) AS revenue
FROM orders o JOIN order_items oi USING (order_id)
WHERE o.status='completed'
GROUP BY 1 ORDER BY 1;
```

UK financial year. Companies often report April–March; the public sector always does. Shift the date before truncating:

```
-- FY starting 1 April: subtract 3 months, truncate to year, add 3 back
SELECT DATE_TRUNC('year', order_ts - INTERVAL '3 months') + INTERVAL '3 months' AS fy_start,
       EXTRACT(YEAR FROM order_ts - INTERVAL '3 months') AS fy_starting_year
FROM orders;
```

For the tax year starting 6 April, subtract `INTERVAL '3 months 5 days'`. Knowing this pattern is genuinely useful in UK analyst work and rarely known by candidates.

8.5 EXTRACT and DATE_PART

Pull a component out as a number.

```
EXTRACT(YEAR FROM ts) -- 2024
EXTRACT(QUARTER FROM ts) -- 1
EXTRACT(MONTH FROM ts) -- 3
EXTRACT(DAY FROM ts) -- 15
EXTRACT(DOW FROM ts) -- 0=Sunday ... 6=Saturday
EXTRACT(ISODOW FROM ts) -- 1=Monday ... 7=Sunday <- use this in the UK
EXTRACT(DOY FROM ts) -- day of year
EXTRACT(WEEK FROM ts) -- ISO week number
EXTRACT(HOUR FROM ts) -- 14
EXTRACT(EPOCH FROM ts) -- Unix seconds
```

`DATE_PART('month', ts)` is the function-call equivalent and returns `double precision`; cast to `int` if you're using it as a label.

EXTRACT vs DATE_TRUNC — the distinction matters. `EXTRACT(MONTH FROM ts) = 3` gives March in every year combined. `DATE_TRUNC('month', ts)` gives each March separately. Grouping by `EXTRACT(MONTH ...)` when you meant a monthly time series silently merges years, and the chart looks fine.

```
-- seasonality: all Marches together (correct use of EXTRACT)
SELECT EXTRACT(MONTH FROM order_ts) AS month_of_year, COUNT(*)
FROM orders GROUP BY 1 ORDER BY 1;

-- time series: each month separately (correct use of DATE_TRUNC)
SELECT DATE_TRUNC('month', order_ts)::date AS month, COUNT(*)
FROM orders GROUP BY 1 ORDER BY 1;
```

Weekday and business-hours analysis:

```
SELECT TO_CHAR(order_ts, 'Day') AS day_name,
       EXTRACT(ISODOW FROM order_ts) AS dow,
       COUNT(*) AS orders,
       COUNT(*) FILTER (WHERE EXTRACT(HOUR FROM order_ts) BETWEEN 9 AND 17) AS in_hours
FROM orders
GROUP BY 1,2 ORDER BY 2;
```

`TO_CHAR` handles formatting: `'YYYY-MM'`, `'DD/MM/YYYY'` (UK format), `'Day'`, `'Mon'`, `'HH24:MI'`. Note `TO_CHAR(x, 'Day')` pads with spaces to a fixed width — `TRIM` it.

8.6 AGE and durations

```
AGE(TIMESTAMP '2024-03-15', TIMESTAMP '1990-06-20') -- 33 years 8 mons 23 days
AGE(date_of_birth) -- from today
EXTRACT(YEAR FROM AGE(CURRENT_DATE, date_of_birth)) -- age in whole years
```

`AGE` returns a human-readable interval accounting for varying month lengths. For "age in years", extract the year part — do **not** compute `(CURRENT_DATE - dob)/365.25`, which is off by a day for many people and will make your patient age bands disagree with clinical systems.

Duration metrics:

```
-- A&E Length of stay
SELECT attendance_id,
       departure_ts - arrival_ts AS los_interval,
       ROUND(EXTRACT(EPOCH FROM (departure_ts - arrival_ts))/60, 0) AS los_minutes,
       departure_ts - arrival_ts > INTERVAL '4 hours' AS breached
FROM ae_attendances
WHERE departure_ts IS NOT NULL;

-- average and median, by triage category
SELECT triage_category,
       ROUND(AVG(EXTRACT(EPOCH FROM (departure_ts-arrival_ts))/60)) AS mean_minutes,
       ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (
         ORDER BY EXTRACT(EPOCH FROM (departure_ts-arrival_ts))/60)) AS median_minutes,
       ROUND(PERCENTILE_CONT(0.95) WITHIN GROUP (
         ORDER BY EXTRACT(EPOCH FROM (departure_ts-arrival_ts))/60)) AS p95_minutes
FROM ae_attendances
WHERE departure_ts IS NOT NULL
GROUP BY triage_category ORDER BY triage_category;
```

Reporting mean *and* median *and* p95 for waiting times is what a good analyst does — these distributions are right-skewed, the mean flatters performance, and the p95 is what patients actually experience at the tail.

8.7 Working days

UK reporting frequently needs working days excluding weekends and bank holidays. There's no built-in function; you need a calendar table, which is the correct answer in an interview.

```
-- quick and dirty: weekdays only
SELECT COUNT(*) FROM generate_series(start_date, end_date, INTERVAL '1 day') d
WHERE EXTRACT(ISODOW FROM d) < 6;

-- proper: a date dimension with a bank holiday flag
CREATE TABLE dim_date (
  date_key      date PRIMARY KEY,
  is_weekend    boolean,
  is_bank_holiday boolean,
  is_working_day boolean,
  fy_start      date,
  fy_quarter    integer,
  iso_week      integer
);

SELECT COUNT(*) AS working_days
FROM dim_date
WHERE date_key >= r.referral_date AND date_key < a.attended_ts::date
      AND is_working_day;
```

A date dimension also solves the missing-periods problem for free: `LEFT JOIN` to it and every date exists whether or not there was activity.

8.8 Period-over-period analysis

The core skill this section exists for.

Month-on-month growth using LAG:

```

WITH monthly AS (
    SELECT DATE_TRUNC('month', o.order_ts)::date AS month,
           SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct)) AS revenue
    FROM orders o JOIN order_items oi USING (order_id)
    WHERE o.status = 'completed'
    GROUP BY 1
)
SELECT month,
       ROUND(revenue, 2) AS revenue,
       ROUND(LAG(revenue) OVER (ORDER BY month), 2) AS prev_month,
       ROUND(revenue - LAG(revenue) OVER (ORDER BY month), 2) AS change,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
            / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 1) AS mom_pct
FROM monthly
ORDER BY month;

```

The trap: if a month has no orders it produces no row, so `LAG` compares against two months ago while claiming it's last month. Generate the calendar and `LEFT JOIN` when gaps are possible.

Year-on-year, comparing the same month across years:

```

WITH monthly AS (
    SELECT DATE_TRUNC('month', order_ts)::date AS month, SUM(total) AS revenue
    FROM order_totals GROUP BY 1
)
SELECT m.month, m.revenue,
       p.revenue AS same_month_last_year,
       ROUND(100.0 * (m.revenue - p.revenue) / NULLIF(p.revenue, 0), 1) AS yoy_pct
FROM monthly m
LEFT JOIN monthly p ON p.month = m.month - INTERVAL '1 year'
ORDER BY m.month;

```

Self-join on a date offset is more robust than `LAG(revenue, 12)`, because `LAG` counts *rows*, not months — one missing month and every subsequent comparison is off by one. Say this if asked "why not just `LAG 12?`".

Rolling 7-day and 28-day averages:

```

SELECT day, orders,
       ROUND(AVG(orders) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW), 1)
       AS rolling_7d,
       ROUND(AVG(orders) OVER (ORDER BY day
                               RANGE BETWEEN INTERVAL '27 days' PRECEDING AND CURRENT ROW), 1)
       AS rolling_28d_gapsafe
FROM daily_orders
ORDER BY day;

```

`ROWS` counts rows; `RANGE` with an interval counts calendar time. On a dense daily series they agree. On a sparse one — days with no orders missing entirely — `ROWS 6 PRECEDING` reaches back further than a week and quietly inflates the average. `RANGE ... INTERVAL` is gap-safe. This distinction is a genuine senior-level discriminator.

Rolling 12-month total:

```

SELECT month,
       SUM(revenue) OVER (ORDER BY month
                          RANGE BETWEEN INTERVAL '11 months' PRECEDING AND CURRENT ROW)
       AS rolling_12m_revenue
FROM monthly;

```

8.9 Recency, lifetime and time-to-event

Days since last order per customer:

```

SELECT customer_id,
       MAX(order_ts)::date AS last_order,
       CURRENT_DATE - MAX(order_ts)::date AS days_since,
       CASE WHEN CURRENT_DATE - MAX(order_ts)::date <= 30 THEN 'Active'
            WHEN CURRENT_DATE - MAX(order_ts)::date <= 90 THEN 'Recent'
            WHEN CURRENT_DATE - MAX(order_ts)::date <= 365 THEN 'Lapsing'
            ELSE 'Dormant' END AS recency_band
FROM orders WHERE status='completed'
GROUP BY customer_id;

```

Customer lifetime and tenure:

```

SELECT c.customer_id,
       c.signup_date,
       MIN(o.order_ts)::date AS first_order,

```

```

MAX(o.order_ts)::date AS last_order,
MIN(o.order_ts)::date - c.signup_date AS days_to_first_order,
MAX(o.order_ts)::date - MIN(o.order_ts)::date AS active_lifespan_days,
CURRENT_DATE - c.signup_date AS tenure_days
FROM customers c
LEFT JOIN orders o ON o.customer_id=c.customer_id AND o.status='completed'
GROUP BY c.customer_id, c.signup_date;

```

`days_to_first_order` is a genuinely useful activation metric, and the `LEFT JOIN` correctly leaves it `NULL` for customers who never converted.

Time between consecutive orders (the purchase cadence):

```

SELECT customer_id, order_ts::date AS order_date,
       order_ts::date - LAG(order_ts::date) OVER (PARTITION BY customer_id ORDER BY order_ts)
       AS days_since_previous
FROM orders WHERE status='completed';

```

Averaging that per customer gives their purchase frequency; the population median tells you what "overdue" means, which is how you build a churn-risk flag that isn't guesswork.

Time-to-event, NHS style — referral to first appointment:

```

SELECT r.specialty,
       COUNT(*) AS referrals,
       ROUND(AVG(a.attended_ts::date - r.referral_date), 1) AS mean_days_to_seen,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY a.attended_ts::date - r.referral_date)
       AS median_days,
       COUNT(*) FILTER (WHERE a.attended_ts::date - r.referral_date > 126) AS over_18_weeks
FROM referrals r
JOIN LATERAL (
  SELECT attended_ts FROM appointments a2
  WHERE a2.referral_id = r.referral_id AND a2.outcome='Attended'
  ORDER BY attended_ts LIMIT 1
) a ON true
GROUP BY r.specialty;

```

The `LATERAL` picks the *first* attended appointment per referral, which is what "time to be seen" means — joining to all appointments and averaging would count follow-ups and understate nothing while overstating the count.

Censoring. Patients still waiting have no appointment and are excluded by that join. They are, by definition, the longest waiters. Reporting only completed pathways systematically flatters performance — this is survivorship bias, it's why NHS statistics report both "completed pathway" and "incomplete pathway" waits, and raising it unprompted in an interview is a strong signal.

8.10 Date exercises

1. Orders in the last 30 days.
2. Revenue by calendar quarter for 2024.
3. Each customer's first and last order date, and the gap between them.
4. Orders by day of the week.
5. Month-on-month revenue change with percentages.
6. Year-on-year comparison of the same month.
7. Rolling 7-day average order count.
8. Customers who haven't ordered in 90 days but had ordered before that.
9. A&E attendances breaching four hours, by month and site.
10. Average days from referral to first appointment by specialty, acknowledging still-waiting patients.
11. Revenue by UK financial year (April–March).
12. A daily series with zero-filled gaps for the last 30 days.

```

-- 1
SELECT * FROM orders WHERE order_ts >= CURRENT_DATE - INTERVAL '30 days';

-- 2
SELECT DATE_TRUNC('quarter', o.order_ts)::date AS quarter,
       ROUND(SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)),2) AS revenue

```

```

FROM orders o JOIN order_items oi USING (order_id)
WHERE o.status='completed' AND o.order_ts >= '2024-01-01' AND o.order_ts < '2025-01-01'
GROUP BY 1 ORDER BY 1;

-- 3
SELECT customer_id, MIN(order_ts)::date AS first, MAX(order_ts)::date AS last,
       MAX(order_ts)::date - MIN(order_ts)::date AS span_days
FROM orders WHERE status='completed' GROUP BY customer_id;

-- 4
SELECT TRIM(TO_CHAR(order_ts,'Day')) AS day, EXTRACT(ISODOW FROM order_ts) AS dow, COUNT(*)
FROM orders GROUP BY 1,2 ORDER BY 2;

-- 5 see 8.8

-- 6 see 8.8

-- 7
SELECT day, orders,
       ROUND(AVG(orders) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW),1)
FROM (SELECT order_ts::date AS day, COUNT(*) AS orders FROM orders GROUP BY 1) d;

-- 8
SELECT customer_id, MAX(order_ts)::date AS last_order
FROM orders WHERE status='completed'
GROUP BY customer_id
HAVING MAX(order_ts) < CURRENT_DATE - INTERVAL '90 days';

-- 9
SELECT DATE_TRUNC('month',arrival_ts)::date AS month, site_code,
       COUNT(*) AS attendances,
       COUNT(*) FILTER (WHERE departure_ts - arrival_ts > INTERVAL '4 hours') AS breaches,
       ROUND(100.0*COUNT(*) FILTER (WHERE departure_ts - arrival_ts > INTERVAL '4 hours')
             / NULLIF(COUNT(*) FILTER (WHERE departure_ts IS NOT NULL),0),1) AS breach_pct
FROM ae_attendances GROUP BY 1,2 ORDER BY 1,2;

-- 10 see 8.9, and report the still-waiting count alongside
SELECT r.specialty,
       COUNT(*) FILTER (WHERE a.attended_ts IS NOT NULL) AS seen,
       COUNT(*) FILTER (WHERE a.attended_ts IS NULL) AS still_waiting,
       ROUND(AVG(a.attended_ts::date - r.referral_date),1) AS mean_days_to_seen
FROM referrals r
LEFT JOIN LATERAL (
  SELECT attended_ts FROM appointments x
  WHERE x.referral_id=r.referral_id AND x.outcome='Attended'
  ORDER BY attended_ts LIMIT 1) a ON true
GROUP BY r.specialty;

-- 11
SELECT EXTRACT(YEAR FROM order_ts - INTERVAL '3 months')::int AS fy_start_year,
       ROUND(SUM(oi.quantity*oi.unit_price),2) AS revenue
FROM orders o JOIN order_items oi USING (order_id)
WHERE o.status='completed' GROUP BY 1 ORDER BY 1;

-- 12
SELECT d.day::date, COALESCE(COUNT(o.order_id),0) AS orders
FROM generate_series(CURRENT_DATE - 29, CURRENT_DATE, INTERVAL '1 day') d(day)
LEFT JOIN orders o ON o.order_ts::date = d.day::date AND o.status='completed'
GROUP BY 1 ORDER BY 1;

```

Parts 9–11: Subqueries, CTEs, Window Functions

PART 9 — SUBQUERIES

A subquery is a query nested inside another. Classify them two ways, because the two classifications answer different questions:

- **By what they return:** scalar (one value), row, or table.
- **By dependency:** uncorrelated (runs once, standalone) or correlated (references the outer query, conceptually runs per outer row).

9.1 Scalar subqueries

Return exactly one row and one column. Usable anywhere a single value is legal.

```
-- in SELECT: attach a global figure to every row
SELECT product_name, unit_price,
       (SELECT ROUND(AVG(unit_price),2) FROM products) AS avg_price,
       ROUND(unit_price - (SELECT AVG(unit_price) FROM products), 2) AS diff_from_avg
FROM products;

-- in WHERE: compare against an aggregate
SELECT product_name, unit_price FROM products
WHERE unit_price > (SELECT AVG(unit_price) FROM products);
```

If a scalar subquery returns more than one row, Postgres raises an error at runtime — which is safer than silently picking one, but it means a subquery that works on test data can fail in production when the data grows a duplicate. Add **LIMIT 1** with an explicit **ORDER BY** if "any one of them" is genuinely acceptable, but usually the multiple rows mean your logic is wrong.

If it returns no rows, you get NULL, not an error, and the NULL propagates silently.

9.2 Subqueries in FROM (derived tables)

A subquery in FROM acts as a temporary table. It **must** have an alias in Postgres.

```
SELECT country, ROUND(AVG(order_count),1) AS avg_orders_per_customer
FROM (
  SELECT c.customer_id, c.country, COUNT(o.order_id) AS order_count
  FROM customers c
  LEFT JOIN orders o ON o.customer_id = c.customer_id AND o.status='completed'
  GROUP BY c.customer_id, c.country
) AS per_customer
GROUP BY country;
```

This is the standard solution to "aggregate an aggregate" — you cannot nest aggregate functions directly (**AVG(COUNT(*))** is illegal outside a window context), so you aggregate once in the inner query and again in the outer.

Derived tables and CTEs are interchangeable here. Prefer CTEs (Part 10) for readability; derived tables are fine for a single small nesting.

9.3 Subqueries in WHERE

With IN:

```
SELECT * FROM customers
WHERE customer_id IN (
  SELECT customer_id FROM orders WHERE status='completed' AND order_ts >= '2024-01-01'
);
```

With a comparison operator and an aggregate:

```
SELECT * FROM orders
WHERE order_ts > (SELECT MAX(order_ts) FROM orders WHERE customer_id = 1);
```

With ANY / ALL — less common but they appear in exams:

```
WHERE unit_price > ALL (SELECT unit_price FROM products WHERE category='Grocery')
WHERE unit_price > ANY (SELECT unit_price FROM products WHERE category='Grocery')
```

> **ALL** means greater than the maximum; > **ANY** means greater than the minimum. = **ANY** is exactly **IN**. Note **ALL** has the same NULL problem as **NOT IN**, and both return true vacuously over an empty set — > **ALL (empty)** is true.

9.4 Correlated subqueries

A correlated subquery references a column from the outer query, so it is logically evaluated once per outer row.

```
-- customers who spent more than their country's average
SELECT c.customer_id, c.country, c.lifetime_value
FROM customer_summary c
WHERE c.lifetime_value > (
    SELECT AVG(c2.lifetime_value)
    FROM customer_summary c2
    WHERE c2.country = c.country      -- <-- the correlation
);
```

Readable, and for small tables perfectly fine. For large tables a window function does the same job in one pass:

```
SELECT * FROM (
    SELECT customer_id, country, lifetime_value,
        AVG(lifetime_value) OVER (PARTITION BY country) AS country_avg
    FROM customer_summary
) t WHERE lifetime_value > country_avg;
```

That rewrite — correlated aggregate subquery to window function — is one of the highest-value optimisations an analyst can know, and interviewers love asking for it. Postgres's planner often rewrites simple correlated subqueries into joins itself, but not reliably, and not when the subquery is in **SELECT**.

Correlated subquery in **SELECT**:

```
SELECT c.customer_id, c.first_name,
    (SELECT COUNT(*) FROM orders o
     WHERE o.customer_id = c.customer_id AND o.status='completed') AS orders,
    (SELECT MAX(o.order_ts) FROM orders o
     WHERE o.customer_id = c.customer_id) AS last_order
FROM customers c;
```

Each extra correlated column is another pass over **orders**. Two subqueries here means scanning orders twice; a single **LEFT JOIN** with **GROUP BY** does it once. Recognising that is worth saying out loud.

9.5 EXISTS and NOT EXISTS

EXISTS tests whether a subquery returns *any* row. It returns a boolean and **short-circuits** — the database stops as soon as it finds one match, which is why it's usually the fastest way to express "has at least one".

```
-- customers who have ordered
SELECT c.* FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o
              WHERE o.customer_id = c.customer_id AND o.status='completed');

-- customers who never ordered (the anti-join)
SELECT c.* FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

Points that come up:

- **SELECT 1** vs **SELECT *** inside **EXISTS** makes no difference — the select list is never evaluated. **SELECT 1** just signals intent.
- **EXISTS** is **NULL-safe**, unlike **NOT IN**. This is the headline reason to prefer it.
- **EXISTS** never multiplies rows. **JOIN** to a many-side to test existence duplicates the outer rows and forces you into **DISTINCT**; **EXISTS** doesn't.

Multi-condition existence is where **EXISTS** really earns its place:

```
-- customers who bought Electronics AND also bought Home, but never returned anything
SELECT c.customer_id FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o JOIN order_items oi USING (order_id)
              JOIN products p USING (product_id)
              WHERE o.customer_id=c.customer_id AND p.category='Electronics'
              AND o.status='completed')
AND EXISTS (SELECT 1 FROM orders o JOIN order_items oi USING (order_id)
              JOIN products p USING (product_id)
```

```
WHERE o.customer_id=c.customer_id AND p.category='Home'
AND o.status='completed')
AND NOT EXISTS (SELECT 1 FROM orders o
WHERE o.customer_id=c.customer_id AND o.status='refunded');
```

Doing that with joins would require three joins, careful DISTINCT handling, and would be much harder to read or amend.

9.6 Subquery vs JOIN — when to use which

Use	Because
EXISTS / NOT EXISTS	testing presence/absence without duplicating rows; NULL-safe
JOIN	you need columns from the other table
IN with a small static list	simple, readable
IN with a subquery	fine when the subquery key can't be NULL; otherwise EXISTS
Derived table / CTE	you need to aggregate before joining, or reuse a result
Correlated subquery	per-row logic that's genuinely hard to express otherwise — but check whether a window function does it
LATERAL	top-N per group, or a correlated subquery returning multiple columns

The decisive question is: **do I need data from the other table, or just a yes/no?** Need data → join. Yes/no → EXISTS.

Three ways to write "customers who ordered in 2024", all correct:

```
-- JOIN + DISTINCT: works, but DISTINCT is doing damage control after fan-out
SELECT DISTINCT c.* FROM customers c
JOIN orders o ON o.customer_id=c.customer_id
WHERE o.order_ts >= '2024-01-01';

-- IN: clear, safe here because orders.customer_id is a non-null FK
SELECT * FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders WHERE order_ts >= '2024-01-01');

-- EXISTS: usually the best plan, no fan-out, no NULL risk
SELECT c.* FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o
WHERE o.customer_id=c.customer_id AND o.order_ts >= '2024-01-01');
```

Say "I'd write EXISTS: it can't duplicate rows, it's NULL-safe, and it short-circuits" and you've answered the question behind the question.

9.7 Subquery exercises

1. Products priced above the overall average.
2. Products priced above their own category's average.
3. Customers who have ordered but never had a refund.
4. The most recent order for each customer, using a correlated subquery.
5. Customers whose spend is above the average customer spend.
6. Categories where every product is active.
7. Employees earning more than the average in their department.
8. Products that have never been ordered — three ways.
9. Customers who ordered in both 2023 and 2024.
10. The order containing the single most expensive line.

```
-- 1
SELECT * FROM products WHERE unit_price > (SELECT AVG(unit_price) FROM products);

-- 2 correlated
SELECT p.* FROM products p
WHERE p.unit_price > (SELECT AVG(p2.unit_price) FROM products p2 WHERE p2.category=p.category);
-- window equivalent
SELECT * FROM (SELECT *, AVG(unit_price) OVER (PARTITION BY category) AS cat_avg FROM products) t
WHERE unit_price > cat_avg;
```

```

-- 3
SELECT c.* FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id AND o.status='completed')
AND NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id AND o.status='refunded');

-- 4
SELECT o.* FROM orders o
WHERE o.order_ts = (SELECT MAX(o2.order_ts) FROM orders o2 WHERE o2.customer_id=o.customer_id);
-- caveat: ties return multiple rows; ROW_NUMBER guarantees exactly one

-- 5
WITH spend AS (
  SELECT o.customer_id, SUM(oi.quantity*oi.unit_price) AS total
  FROM orders o JOIN order_items oi USING (order_id) WHERE o.status='completed'
  GROUP BY o.customer_id)
SELECT * FROM spend WHERE total > (SELECT AVG(total) FROM spend);

-- 6
SELECT DISTINCT category FROM products p
WHERE NOT EXISTS (SELECT 1 FROM products p2 WHERE p2.category=p.category AND NOT p2.is_active);

-- 7
SELECT e.* FROM employees e
WHERE e.salary > (SELECT AVG(e2.salary) FROM employees e2 WHERE e2.department=e.department);

-- 8
SELECT * FROM products p WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id=p.product_id);
SELECT p.* FROM products p LEFT JOIN order_items oi USING (product_id) WHERE oi.order_item_id IS NULL;
SELECT * FROM products WHERE product_id NOT IN (SELECT product_id FROM order_items WHERE product_id IS NOT NULL);

-- 9
SELECT customer_id FROM orders WHERE status='completed'
GROUP BY customer_id
HAVING COUNT(*) FILTER (WHERE order_ts >= '2023-01-01' AND order_ts < '2024-01-01') > 0
AND COUNT(*) FILTER (WHERE order_ts >= '2024-01-01' AND order_ts < '2025-01-01') > 0;

-- 10
SELECT * FROM orders WHERE order_id = (
  SELECT order_id FROM order_items ORDER BY quantity*unit_price DESC LIMIT 1);

```

PART 10 — COMMON TABLE EXPRESSIONS (CTEs)

10.1 Why analysts live in CTEs

A CTE is a named subquery defined before the main query with **WITH**. Functionally it's a derived table with a name — but the naming is the whole point.

Analytical questions are pipelines: filter, aggregate to a grain, join to another aggregate, rank, filter again. Nesting that as subqueries produces a query you read inside-out. CTEs let you write it top-down, in the order you thought of it, with each step named after what it means.

```
-- nested: read from the inside out, and good luck debugging it
SELECT * FROM (
  SELECT customer_id, total, RANK() OVER (ORDER BY total DESC) AS r
  FROM (
    SELECT customer_id, SUM(revenue) AS total
    FROM (SELECT o.customer_id, oi.quantity*oi.unit_price AS revenue
          FROM orders o JOIN order_items oi USING (order_id)
          WHERE o.status='completed') a
    GROUP BY customer_id) b
  ) c WHERE r <= 10;

-- CTEs: read top to bottom, each step named
WITH line_revenue AS (
  SELECT o.customer_id, oi.quantity * oi.unit_price * (1-oi.discount_pct) AS revenue
  FROM orders o
  JOIN order_items oi USING (order_id)
  WHERE o.status = 'completed'
),
customer_totals AS (
  SELECT customer_id, SUM(revenue) AS lifetime_value
  FROM line_revenue GROUP BY customer_id
),
ranked AS (
  SELECT *, RANK() OVER (ORDER BY lifetime_value DESC) AS value_rank
  FROM customer_totals
)
SELECT * FROM ranked WHERE value_rank <= 10;
```

Same result. The second is reviewable, and — the practical benefit — you can debug it by replacing the final SELECT with **SELECT * FROM line_revenue LIMIT 20** to inspect any stage.

Reasons to give in an interview: readability, step-by-step debuggability, reuse of a result in multiple places, and they make the analytical logic legible to a reviewer who isn't a SQL expert. Then add the caveat about materialisation below, which shows you know the trade-off.

10.2 Syntax and multiple CTEs

```
WITH first_cte AS ( ... ),
    second_cte AS ( SELECT ... FROM first_cte ... ), -- can reference earlier CTEs
    third_cte AS ( SELECT ... FROM first_cte JOIN second_cte ... )
SELECT ... FROM third_cte;
```

WITH once, CTEs comma-separated, no comma before the final SELECT. A CTE can reference any CTE defined **before** it, not after (unless recursive). Each CTE can be referenced multiple times.

10.3 Chained CTEs — a full analytical pipeline

```
WITH completed_orders AS (
  SELECT order_id, customer_id, order_ts, channel
  FROM orders
  WHERE status = 'completed'
  AND order_ts >= DATE '2024-01-01'
),
order_values AS (
  SELECT co.order_id, co.customer_id, co.order_ts, co.channel,
    SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct)) AS order_value
  FROM completed_orders co
  JOIN order_items oi ON oi.order_id = co.order_id
```

```

GROUP BY co.order_id, co.customer_id, co.order_ts, co.channel
),
customer_metrics AS (
  SELECT customer_id,
         COUNT(*)           AS orders,
         SUM(order_value)    AS lifetime_value,
         AVG(order_value)    AS avg_order_value,
         MIN(order_ts)::date AS first_order,
         MAX(order_ts)::date AS last_order
  FROM order_values
  GROUP BY customer_id
),
segmented AS (
  SELECT cm.*, c.country, c.channel AS acquisition_channel,
         NTILE(4) OVER (ORDER BY cm.lifetime_value DESC) AS value_quartile,
         CASE WHEN cm.orders = 1 THEN 'One-time' ELSE 'Repeat' END AS repeat_status
  FROM customer_metrics cm
  JOIN customers c USING (customer_id)
)
SELECT country, acquisition_channel, repeat_status,
       COUNT(*) AS customers,
       ROUND(AVG(lifetime_value), 2) AS avg_ltv,
       ROUND(AVG(avg_order_value), 2) AS avg_aov
FROM segmented
GROUP BY 1,2,3
ORDER BY avg_ltv DESC;

```

Five named stages, each doing one thing. This is what "readable analytical pipeline" means, and being able to produce something with this shape under interview pressure is what a strong candidate looks like.

10.4 Materialisation and performance

The important caveat. **Before Postgres 12**, CTEs were always materialised — computed once into a temporary result, with no predicate push-down. That made them an "optimisation fence": a filter in the outer query couldn't be pushed into the CTE, so a CTE selecting a billion rows computed all billion even if you then filtered to ten.

Postgres 12+ inlines CTEs that are referenced exactly once and are not recursive and have no side effects, treating them like derived tables. You can force either behaviour:

```

WITH big AS MATERIALIZED ( SELECT ... ) -- compute once, reuse
WITH big AS NOT MATERIALIZED ( SELECT ... ) -- inline, allow push-down

```

Use **MATERIALIZED** when a CTE is expensive and referenced several times. Use **NOT MATERIALIZED** when a CTE is a simple scan referenced once and you want filters pushed in.

Dialect. SQL Server and Oracle always inline CTEs (they're purely syntactic there). MySQL 8+ behaves similarly to modern Postgres. In BigQuery and Snowflake a multiply-referenced CTE may be recomputed each time — if a CTE is used three times and is expensive, consider a temp table. Knowing this distinction is a real senior-level answer.

A CTE is **not** a temporary table: it exists only for the duration of the statement, isn't indexed, and isn't visible to other queries.

10.5 Recursive CTEs

For hierarchies and generated sequences. Structure:

```

WITH RECURSIVE cte_name AS (
  -- anchor: the starting rows
  SELECT ...
  UNION ALL
  -- recursive term: references cte_name, produces the next level
  SELECT ... FROM table JOIN cte_name ON ...
)
SELECT * FROM cte_name;

```

Org hierarchy — the standard interview example:

```

WITH RECURSIVE org AS (
  SELECT employee_id, full_name, manager_id, 1 AS level,
         full_name::text AS path
  FROM employees
  WHERE manager_id IS NULL
  -- anchor: the top of the tree

```

```

UNION ALL

SELECT e.employee_id, e.full_name, e.manager_id, o.level + 1,
       o.path || ' > ' || e.full_name
FROM employees e
JOIN org o ON o.employee_id = e.manager_id -- recursive: children of known nodes
)
SELECT level, REPEAT(' ', level-1) || full_name AS org_chart, path
FROM org
ORDER BY path;

```

UNION ALL not **UNION** — **UNION** deduplicates on every iteration, which is slow and usually wrong. If the hierarchy might contain a cycle (bad data where A manages B manages A), add a depth guard: **WHERE o.level < 20**. Volunteering that guard is the thing that marks out someone who's actually run one of these in anger.

Generating a date series without **generate_series** (useful in engines that lack it):

```

WITH RECURSIVE dates AS (
  SELECT DATE '2024-01-01' AS d
  UNION ALL
  SELECT d + 1 FROM dates WHERE d < DATE '2024-12-31'
)
SELECT * FROM dates;

```

In Postgres just use **generate_series(DATE '2024-01-01', DATE '2024-12-31', INTERVAL '1 day')**.

Patient pathway / referral chains — following linked records:

```

WITH RECURSIVE pathway AS (
  SELECT referral_id, patient_id, referral_date, onward_referral_id, 1 AS step
  FROM referrals WHERE source <> 'Consultant' -- pathway starts

  UNION ALL

  SELECT r.referral_id, r.patient_id, r.referral_date, r.onward_referral_id, p.step + 1
  FROM referrals r JOIN pathway p ON r.referral_id = p.onward_referral_id
  WHERE p.step < 10
)
SELECT patient_id, COUNT(*) AS steps_in_pathway,
       MAX(referral_date) - MIN(referral_date) AS pathway_length_days
FROM pathway GROUP BY patient_id;

```

For a junior analyst role, being able to write the org-chart recursion and explain anchor/recursive/termination is enough. Nobody expects graph algorithms.

10.6 CTE exercises

1. Rewrite the nested query in 10.1 from scratch using CTEs.
2. Build a pipeline: monthly revenue → month-on-month change → flag months that fell.
3. Use one CTE twice in the same query.
4. Compute each customer's LTV, then their quartile, then average LTV per quartile.
5. Write the org chart recursion with indentation.
6. Generate every month of 2024 and LEFT JOIN revenue to it.
7. Build a cleaning pipeline: standardise → validate → dedupe → output.
8. Find the top 3 products per category using CTEs plus a window function.

```

-- 2
WITH monthly AS (
  SELECT DATE_TRUNC('month', o.order_ts)::date AS month,
         SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS revenue
  FROM orders o JOIN order_items oi USING (order_id)
  WHERE o.status='completed' GROUP BY 1),
with_change AS (
  SELECT month, revenue,
         LAG(revenue) OVER (ORDER BY month) AS prev,
         revenue - LAG(revenue) OVER (ORDER BY month) AS delta
  FROM monthly)
SELECT month, ROUND(revenue,2) AS revenue, ROUND(delta,2) AS change,
       ROUND(100.0*delta/NULLIF(prev,0),1) AS pct_change,
       CASE WHEN delta < 0 THEN 'Declined' WHEN delta > 0 THEN 'Grew' ELSE 'Flat' END AS direction
FROM with_change ORDER BY month;

```

```

-- 3 same CTE referenced twice: this month vs the same month last year
WITH monthly AS (
    SELECT DATE_TRUNC('month', order_ts)::date AS month, COUNT(*) AS orders
    FROM orders WHERE status='completed' GROUP BY 1)
SELECT m.month, m.orders, p.orders AS last_year,
    ROUND(100.0*(m.orders-p.orders)/NULLIF(p.orders,0),1) AS yoy_pct
FROM monthly m LEFT JOIN monthly p ON p.month = m.month - INTERVAL '1 year'
ORDER BY m.month;

-- 4
WITH ltv AS (
    SELECT o.customer_id, SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS lifetime_value
    FROM orders o JOIN order_items oi USING (order_id) WHERE o.status='completed'
    GROUP BY o.customer_id),
quartiles AS (
    SELECT *, NTILE(4) OVER (ORDER BY lifetime_value DESC) AS quartile FROM ltv)
SELECT quartile, COUNT(*) AS customers,
    ROUND(AVG(lifetime_value),2) AS avg_ltv,
    ROUND(SUM(lifetime_value),2) AS total_ltv,
    ROUND(100.0*SUM(lifetime_value)/SUM(SUM(lifetime_value)) OVER (),1) AS pct_of_revenue
FROM quartiles GROUP BY quartile ORDER BY quartile;

-- 6
WITH months AS (
    SELECT generate_series(DATE '2024-01-01', DATE '2024-12-01', INTERVAL '1 month')::date AS month),
rev AS (
    SELECT DATE_TRUNC('month', o.order_ts)::date AS month,
        SUM(oi.quantity*oi.unit_price) AS revenue
    FROM orders o JOIN order_items oi USING (order_id) WHERE o.status='completed' GROUP BY 1)
SELECT m.month, COALESCE(ROUND(r.revenue,2),0) AS revenue
FROM months m LEFT JOIN rev r USING (month) ORDER BY m.month;

-- 8
WITH product_sales AS (
    SELECT p.category, p.product_id, p.product_name,
        SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS revenue
    FROM order_items oi
    JOIN products p USING (product_id)
    JOIN orders o ON o.order_id=oi.order_id AND o.status='completed'
    GROUP BY 1,2,3),
ranked AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn
    FROM product_sales)
SELECT category, product_name, ROUND(revenue,2) AS revenue, rn
FROM ranked WHERE rn <= 3 ORDER BY category, rn;

```

PART 11 — WINDOW FUNCTIONS

The single biggest differentiator in analyst interviews. Candidates who are fluent here get offers.

11.1 The core idea

A window function computes a value across a set of rows related to the current row, **without collapsing them**. GROUP BY gives you one row per group; a window function gives you every row, with the group-level value attached.

```
-- GROUP BY: 4 rows out (one per category)
SELECT category, AVG(unit_price) FROM products GROUP BY category;

-- window: 6 rows out, each with its category's average alongside
SELECT product_name, category, unit_price,
       AVG(unit_price) OVER (PARTITION BY category) AS category_avg
FROM products;
```

That difference — keeping row-level detail while adding group-level context — is what makes comparisons like "this order versus the customer's average" possible in one pass.

Evaluation order matters. Window functions run *after* WHERE, GROUP BY and HAVING, and *before* ORDER BY and LIMIT. Two consequences tested constantly:

- You cannot filter on a window function in WHERE. `WHERE ROW_NUMBER() OVER (...) = 1` is a syntax error. Wrap it in a subquery or CTE and filter outside. There is no **HAVING** equivalent for windows.
- Because windows run after grouping, you can apply one *to an aggregate*: `SUM(COUNT(*)) OVER ()` is legal and useful (Part 3.5).

11.2 OVER, PARTITION BY, ORDER BY

```
function() OVER (
  PARTITION BY col1, col2      -- optional: split into independent groups
  ORDER BY col3                -- optional: order within each partition
  [frame]                     -- optional: which rows in the partition count
)
```

- **OVER ()** — empty: the whole result set is one window.
- **PARTITION BY** — restart the calculation for each group. Like GROUP BY, but non-collapsing.
- **ORDER BY** — order within the partition. Required for ranking and offset functions. Adding it to an aggregate **changes its meaning** — see 11.3.

```
SELECT customer_id, order_id, order_ts, order_value,
       SUM(order_value) OVER () AS grand_total,
       SUM(order_value) OVER (PARTITION BY customer_id) AS customer_total,
       SUM(order_value) OVER (PARTITION BY customer_id ORDER BY order_ts) AS running_total
FROM order_values;
```

Three windows, three different numbers, one pass. Reading that block and understanding why the third differs from the second is the core skill.

Named windows keep repeated definitions DRY:

```
SELECT customer_id, order_ts,
       ROW_NUMBER() OVER w AS seq,
       LAG(order_value) OVER w AS prev_value,
       SUM(order_value) OVER w AS running_total
FROM order_values
WINDOW w AS (PARTITION BY customer_id ORDER BY order_ts);
```

11.3 Window frames

The frame defines which rows within the partition the function sees. This is the concept most candidates skip and most interviewers probe.

```
ROWS BETWEEN <start> AND <end> -- counts physical rows
RANGE BETWEEN <start> AND <end> -- counts by value of the ORDER BY column
GROUPS BETWEEN <start> AND <end> -- counts peer groups (Postgres 11+)
```

Bounds: UNBOUNDED PRECEDING, n PRECEDING, CURRENT ROW, n FOLLOWING, UNBOUNDED FOLLOWING.

The defaults, which cause real bugs:

- No ORDER BY in the window → frame is the entire partition. SUM(x) OVER (PARTITION BY c) is the partition total.
- ORDER BY present, no explicit frame → the default is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. That's a running total — but with RANGE, "current row" means "all rows with the same ORDER BY value", so tied rows all get the same cumulative value.

```
-- ties: with RANGE (default), two orders on the same day share one running total
SUM(v) OVER (ORDER BY order_date)
-- with ROWS, each row increments separately
SUM(v) OVER (ORDER BY order_date ROWS UNBOUNDED PRECEDING)
```

If you want a strict row-by-row running total, say ROWS explicitly. This is exactly the sort of detail that separates a strong answer.

Common frames:

```
-- running total
SUM(x) OVER (ORDER BY d ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)

-- 7-day moving average (dense series)
AVG(x) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)

-- 7-day moving average (gap-safe, calendar-based)
AVG(x) OVER (ORDER BY d RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW)

-- centred 3-period average
AVG(x) OVER (ORDER BY d ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)

-- partition total on every row
SUM(x) OVER (PARTITION BY c ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)

-- remaining total from here on
SUM(x) OVER (ORDER BY d ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING)
```

11.4 Ranking: ROW_NUMBER, RANK, DENSE_RANK

The difference is entirely about how they handle ties, and you will be asked.

values	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
90	2	2	2
90	3	2	2
80	4	4	3

- **ROW_NUMBER** — always 1,2,3,4. No ties, arbitrary tie-breaking. Use for deduplication and "pick exactly one".
- **RANK** — ties share a rank, then it skips. Use for competition-style ranking, "joint second".
- **DENSE_RANK** — ties share a rank, no gaps. Use for "top 3 distinct values".

```
SELECT product_name, category, unit_price,
       ROW_NUMBER() OVER (PARTITION BY category ORDER BY unit_price DESC) AS rn,
       RANK() OVER (PARTITION BY category ORDER BY unit_price DESC) AS rnk,
       DENSE_RANK() OVER (PARTITION BY category ORDER BY unit_price DESC) AS dense,
       PERCENT_RANK() OVER (PARTITION BY category ORDER BY unit_price DESC) AS pct_rank,
       CUME_DIST() OVER (PARTITION BY category ORDER BY unit_price DESC) AS cume
FROM products;
```

Determinism. ROW_NUMBER() over tied values assigns arbitrarily, and the assignment can change between runs. Always add a unique tie-breaker to the ORDER BY when the result must be reproducible:

```
ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts DESC, order_id DESC)
```

NTILE(n) splits a partition into n roughly equal buckets:

```
SELECT customer_id, lifetime_value,
       NTILE(4) OVER (ORDER BY lifetime_value DESC) AS quartile,
       NTILE(10) OVER (ORDER BY lifetime_value DESC) AS decile
FROM customer_ltv;
```

NTILE distributes by row count, not by value, so bucket boundaries fall at odd places when values are lumpy and identical values can land in different buckets. For value-based bands use CASE or **PERCENT_RANK**. Mention it; it shows you know the tool's limits.

11.5 Offset functions: LAG, LEAD, FIRST_VALUE, LAST_VALUE

```
LAG(expr, offset, default) OVER (PARTITION BY ... ORDER BY ...)
LEAD(expr, offset, default) OVER (...)
FIRST_VALUE(expr) OVER (...)
LAST_VALUE(expr) OVER (...)
NTH_VALUE(expr, n) OVER (...)
```

offset defaults to 1, **default** to NULL — supplying a default (**LAG(revenue,1,0)**) saves a COALESCE.

```
SELECT customer_id, order_ts::date AS order_date, order_value,
       LAG(order_value) OVER w AS previous_order_value,
       LEAD(order_value) OVER w AS next_order_value,
       order_ts::date - LAG(order_ts::date) OVER w AS days_since_previous,
       FIRST_VALUE(order_value) OVER w AS first_ever_order_value
FROM order_values
WINDOW w AS (PARTITION BY customer_id ORDER BY order_ts);
```

The **LAST_VALUE** trap, asked often. With the default frame (**UNBOUNDED PRECEDING TO CURRENT ROW**), **LAST_VALUE** returns the current row — because the current row is the last one in the frame so far.

```
-- WRONG: returns the current row's value every time
LAST_VALUE(order_value) OVER (PARTITION BY customer_id ORDER BY order_ts)

-- RIGHT: extend the frame to the end of the partition
LAST_VALUE(order_value) OVER (PARTITION BY customer_id ORDER BY order_ts
                             ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)

-- or sidestep it entirely
FIRST_VALUE(order_value) OVER (PARTITION BY customer_id ORDER BY order_ts DESC)
```

FIRST_VALUE is unaffected because the frame's start is already the partition's start. Being able to explain *why* — not just recite the fix — is the strong answer.

11.6 Aggregate window functions

Any aggregate can be a window function: **SUM**, **AVG**, **COUNT**, **MIN**, **MAX**, plus **FILTER**.

```
SELECT order_id, customer_id, order_ts::date AS d, order_value,
       SUM(order_value) OVER (PARTITION BY customer_id ORDER BY order_ts
                             ROWS UNBOUNDED PRECEDING) AS cumulative_spend,
       AVG(order_value) OVER (PARTITION BY customer_id) AS customer_avg_order,
       COUNT(*) OVER (PARTITION BY customer_id) AS total_orders,
       MAX(order_value) OVER (PARTITION BY customer_id) AS biggest_order,
       ROUND(100.0 * order_value
             / SUM(order_value) OVER (PARTITION BY customer_id), 1) AS pct_of_customer_spend,
       ROUND(100.0 * order_value
             / SUM(order_value) OVER (), 2) AS pct_of_all_revenue
FROM order_values;
```

COUNT(*) FILTER (WHERE ...) OVER (...) works too, and is the neat way to do a running count of a subset.

11.7 The problem patterns

These are the shapes that appear in interviews. Learn to recognise each from its phrasing.

Top N per group

"Top 3 products by revenue in each category."

```
WITH product_revenue AS (
  SELECT p.category, p.product_name,
```

```

        SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS revenue
    FROM order_items oi
    JOIN products p USING (product_id)
    JOIN orders o ON o.order_id=oi.order_id AND o.status='completed'
    GROUP BY 1,2
)
SELECT * FROM (
    SELECT *, DENSE_RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rnk
    FROM product_revenue
) t WHERE rnk <= 3
ORDER BY category, rnk;

```

Choose the ranking function deliberately and say why: `ROW_NUMBER` gives exactly 3 and arbitrarily cuts ties; `DENSE_RANK` gives all products at the top 3 revenue levels. Ask the interviewer which they want — that question alone is worth marks.

Latest record per group

"Each customer's most recent order."

```

-- portable
SELECT * FROM (
    SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts DESC, order_id DESC) rn
    FROM orders o WHERE status='completed'
) t WHERE rn = 1;

-- Postgres, shorter and often faster
SELECT DISTINCT ON (customer_id) *
FROM orders WHERE status='completed'
ORDER BY customer_id, order_ts DESC, order_id DESC;

-- correlated, no window function
SELECT o.* FROM orders o
WHERE o.order_ts = (SELECT MAX(o2.order_ts) FROM orders o2 WHERE o2.customer_id=o.customer_id);

```

The third returns duplicates on ties — a real difference, not a stylistic one.

Running total

```

SELECT order_ts::date AS d, order_value,
    SUM(order_value) OVER (ORDER BY order_ts ROWS UNBOUNDED PRECEDING) AS cumulative
FROM order_values ORDER BY order_ts;

```

Moving average

```

SELECT d, orders,
    ROUND(AVG(orders) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW), 1) AS ma7
FROM daily_orders;

```

The first six rows average fewer than seven days. If the report shouldn't show a partial window, null them out:

```

CASE WHEN COUNT(*) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) = 7
    THEN AVG(orders) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) END

```

Percentage of total

```

SELECT category,
    ROUND(SUM(revenue), 2) AS revenue,
    ROUND(100.0 * SUM(revenue) / SUM(SUM(revenue)) OVER (), 1) AS pct_of_total,
    ROUND(100.0 * SUM(SUM(revenue)) OVER (ORDER BY SUM(revenue) DESC
        ROWS UNBOUNDED PRECEDING)
        / SUM(SUM(revenue)) OVER (), 1) AS cumulative_pct
FROM category_sales GROUP BY category
ORDER BY revenue DESC;

```

That cumulative percentage column is Pareto analysis — "which categories make up 80% of revenue" — in one expression. `SUM(SUM(x)) OVER ()` looks strange the first time; the inner SUM is the GROUP BY aggregate, the outer is the window over those results.

Purchase sequence and first/repeat

```

SELECT customer_id, order_id, order_ts,
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts) AS purchase_number,
    CASE WHEN ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts) = 1
    THEN 'First' ELSE 'Repeat' END AS purchase_type,
    COUNT(*) OVER (PARTITION BY customer_id) AS lifetime_orders
FROM orders WHERE status='completed';

```

Month-over-month change

```
WITH monthly AS (  
    SELECT DATE_TRUNC('month', order_ts)::date AS month, SUM(order_value) AS revenue  
    FROM order_values GROUP BY 1)  
SELECT month, revenue,  
    LAG(revenue) OVER (ORDER BY month) AS prev,  
    ROUND(100.0*(revenue - LAG(revenue) OVER (ORDER BY month))  
        / NULLIF(LAG(revenue) OVER (ORDER BY month),0), 1) AS mom_pct  
FROM monthly;
```

Comparing a row to its group

```
SELECT e.full_name, e.department, e.salary,  
    ROUND(AVG(e.salary) OVER (PARTITION BY e.department)) AS dept_avg,  
    ROUND(e.salary - AVG(e.salary) OVER (PARTITION BY e.department)) AS vs_dept_avg,  
    RANK() OVER (PARTITION BY e.department ORDER BY e.salary DESC) AS salary_rank_in_dept,  
    ROUND(100.0*e.salary / SUM(e.salary) OVER (PARTITION BY e.department),1) AS pct_of_dept_paybill  
FROM employees e;
```

Gaps and islands — consecutive events

The hardest common pattern, covered fully in Part 12. The trick in one line: for consecutive integers or dates, `value - ROW_NUMBER()` is constant within a consecutive run, so grouping by that difference identifies each run.

```
WITH numbered AS (  
    SELECT customer_id, activity_date,  
        activity_date - (ROW_NUMBER() OVER (PARTITION BY customer_id  
                                            ORDER BY activity_date))::int AS grp  
    FROM daily_activity  
)  
SELECT customer_id, MIN(activity_date) AS streak_start, MAX(activity_date) AS streak_end,  
    COUNT(*) AS consecutive_days  
FROM numbered GROUP BY customer_id, grp  
HAVING COUNT(*) >= 3  
ORDER BY consecutive_days DESC;
```

11.8 Window function exercises

1. Rank customers by lifetime value.
2. For each product, its price and its category's average price.
3. Each customer's second order.
4. Running total of revenue by day.
5. 7-day moving average of orders.
6. Each order's value as a percentage of that customer's total.
7. Days between consecutive orders per customer.
8. Top 2 employees by salary in each department.
9. Month-on-month revenue growth percentage.
10. Each customer's first and last order value on every row.
11. Products in the top decile by revenue.
12. Cumulative share of revenue by category, ranked (Pareto).
13. Customers whose latest order is smaller than their previous one.
14. The longest streak of consecutive days with at least one order.
15. For each order, how many orders that customer had already placed.

```
-- 1  
SELECT customer_id, lifetime_value, RANK() OVER (ORDER BY lifetime_value DESC) FROM customer_ltv;  
  
-- 2  
SELECT product_name, category, unit_price,  
    ROUND(AVG(unit_price) OVER (PARTITION BY category),2) AS cat_avg,  
    ROUND(unit_price - AVG(unit_price) OVER (PARTITION BY category),2) AS diff  
FROM products;  
  
-- 3  
SELECT * FROM (SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts) rn
```

```

FROM orders o WHERE status='completed') t WHERE rn=2;

-- 4
SELECT d, revenue, SUM(revenue) OVER (ORDER BY d ROWS UNBOUNDED PRECEDING) AS cumulative
FROM daily_revenue;

-- 5 see 11.7

-- 6
SELECT order_id, customer_id, order_value,
       ROUND(100.0*order_value/SUM(order_value) OVER (PARTITION BY customer_id),1) AS pct
FROM order_values;

-- 7
SELECT customer_id, order_ts::date,
       order_ts::date - LAG(order_ts::date) OVER (PARTITION BY customer_id ORDER BY order_ts) AS gap
FROM orders WHERE status='completed';

-- 8
SELECT * FROM (SELECT e.*, DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) r
              FROM employees e) t WHERE r <= 2;

-- 9 see 11.7

-- 10
SELECT customer_id, order_ts, order_value,
       FIRST_VALUE(order_value) OVER w AS first_order_value,
       LAST_VALUE(order_value) OVER (PARTITION BY customer_id ORDER BY order_ts
                                     ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
                                     AS last_order_value
FROM order_values WINDOW w AS (PARTITION BY customer_id ORDER BY order_ts);

-- 11
SELECT * FROM (SELECT product_id, revenue, NTILE(10) OVER (ORDER BY revenue DESC) d
              FROM product_revenue) t WHERE d = 1;

-- 12 see 11.7 percentage of total

-- 13
WITH seq AS (
  SELECT customer_id, order_id, order_ts, order_value,
         LAG(order_value) OVER (PARTITION BY customer_id ORDER BY order_ts) AS prev_value,
         ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts DESC) AS rn_desc
  FROM order_values)
SELECT * FROM seq WHERE rn_desc=1 AND prev_value IS NOT NULL AND order_value < prev_value;

-- 14
WITH days AS (SELECT DISTINCT order_ts::date AS d FROM orders WHERE status='completed'),
grp AS (SELECT d, d - (ROW_NUMBER() OVER (ORDER BY d))::int AS g FROM days)
SELECT MIN(d) AS streak_start, MAX(d) AS streak_end, COUNT(*) AS days
FROM grp GROUP BY g ORDER BY days DESC LIMIT 1;

-- 15
SELECT order_id, customer_id, order_ts,
       COUNT(*) OVER (PARTITION BY customer_id ORDER BY order_ts
                     ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING) AS prior_orders
FROM orders WHERE status='completed';

```

Exercise 15's frame — `UNBOUNDED PRECEDING AND 1 PRECEDING` — excludes the current row, which is exactly "how many came before". Being able to construct a frame like that on demand means you actually understand frames rather than having memorised three recipes.

Part 12 — Advanced Analytical SQL

Twenty-two patterns. Each one: the problem, how to think about it, the SQL, why it works, an alternative, and the variation an interviewer will throw at you.

12.1 Top N

Problem. The 5 highest-spending customers.

Thought process. Aggregate to customer grain, order, limit. The only real decision is ties — `LIMIT` cuts them arbitrarily.

```
WITH customer_spend AS (  
    SELECT o.customer_id, SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS spend  
    FROM orders o JOIN order_items oi USING (order_id)  
    WHERE o.status='completed'  
    GROUP BY o.customer_id  
)  
SELECT customer_id, ROUND(spend,2) AS spend  
FROM customer_spend  
ORDER BY spend DESC  
LIMIT 5;
```

Explanation. Aggregate first, sort second, cut third. The status filter goes in WHERE because it's a row-level condition.

Alternative — ties included:

```
SELECT * FROM (SELECT *, RANK() OVER (ORDER BY spend DESC) r FROM customer_spend) t WHERE r <= 5;
```

Interview variation. "What if two customers tie for fifth?" Answer: `LIMIT` returns one of them arbitrarily and non-deterministically; `RANK() <= 5` returns both. Which is right depends on whether the output feeds a fixed-size report or a fair ranking — ask.

12.2 Greatest-per-group

Problem. The single most expensive product in each category, with its name.

Thought process. `MAX(price)` per category gives the value but not the row. You need the whole row that carries the max — that's an argmax, and window functions are the general answer.

```
SELECT category, product_name, unit_price  
FROM (  
    SELECT category, product_name, unit_price,  
           ROW_NUMBER() OVER (PARTITION BY category ORDER BY unit_price DESC, product_id) AS rn  
    FROM products  
) t  
WHERE rn = 1;
```

Explanation. Partition by the group, order by the metric descending, keep row 1. The `product_id` tie-breaker makes it deterministic.

Alternatives:

```
-- Postgres  
SELECT DISTINCT ON (category) category, product_name, unit_price  
FROM products ORDER BY category, unit_price DESC, product_id;  
  
-- portable, no window function - returns ALL tied rows  
SELECT p.* FROM products p  
JOIN (SELECT category, MAX(unit_price) mx FROM products GROUP BY category) m  
ON m.category=p.category AND m.mx=p.unit_price;  
  
-- correlated  
SELECT p.* FROM products p  
WHERE p.unit_price = (SELECT MAX(p2.unit_price) FROM products p2 WHERE p2.category=p.category);
```

Interview variation. "Now the top 3 per category" — change `= 1` to `<= 3` and pick your ranking function. "Now do it without a window function" — the self-join to the MAX subquery is the answer they're looking for.

12.3 Deduplication

Problem. A staging table has duplicate customer records. Keep the most recently updated one for each customer.

Thought process. Define the key that should be unique, define the rule for which duplicate wins, rank within the key, keep rank 1.

```
WITH ranked AS (  
    SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id  
                                ORDER BY updated_at DESC NULLS LAST, id DESC) AS rn  
    FROM customers_staging  
)  
SELECT * FROM ranked WHERE rn = 1;
```

Explanation. `ROW_NUMBER` not `RANK` — `RANK` would return all tied rows and you'd still have duplicates. `NULLS LAST` so a record with a missing timestamp doesn't win. The `id DESC` tie-breaker guarantees reproducibility.

Alternative — exact duplicates only: `SELECT DISTINCT *`. Deleting in place:

```
DELETE FROM customers_staging WHERE id IN (  
    SELECT id FROM (SELECT id, ROW_NUMBER() OVER (PARTITION BY customer_id  
                                                ORDER BY updated_at DESC, id DESC) rn  
    FROM customers_staging) t WHERE rn > 1);
```

Interview variation. "The duplicates aren't identical — one has an email, the other a phone. What now?" That's a merge, not a dedupe: group by the key and coalesce column by column.

```
SELECT customer_id,  
    MAX(email) FILTER (WHERE email IS NOT NULL) AS email,  
    MAX(phone) FILTER (WHERE phone IS NOT NULL) AS phone,  
    MIN(created_at) AS created_at  
FROM customers_staging GROUP BY customer_id;
```

12.4 Gaps and islands

Problem. Find each customer's longest streak of consecutive days with activity.

Thought process. Consecutive dates increase by 1; so does `ROW_NUMBER`. Their difference is therefore **constant within a consecutive run** and changes at every gap. Group by that difference and each group is one island.

```
WITH activity AS (  
    SELECT DISTINCT customer_id, order_ts::date AS activity_date  
    FROM orders WHERE status='completed'  
)  
grouped AS (  
    SELECT customer_id, activity_date,  
        activity_date - (ROW_NUMBER() OVER (PARTITION BY customer_id  
                                            ORDER BY activity_date))::int AS island_key  
    FROM activity  
)  
islands AS (  
    SELECT customer_id, island_key,  
        MIN(activity_date) AS streak_start,  
        MAX(activity_date) AS streak_end,  
        COUNT(*) AS streak_length  
    FROM grouped GROUP BY customer_id, island_key  
)  
SELECT DISTINCT ON (customer_id) customer_id, streak_start, streak_end, streak_length  
FROM islands ORDER BY customer_id, streak_length DESC, streak_start;
```

Explanation. Trace it: dates 1,2,3,7,8 get row numbers 1,2,3,4,5; differences are 0,0,0,3,3. Two groups. `DISTINCT` on the activity dates first is essential — two orders on one day would break the arithmetic.

Alternative — the LAG method, which generalises to non-daily gaps:

```
WITH marked AS (  
    SELECT customer_id, activity_date,  
        CASE WHEN activity_date - LAG(activity_date) OVER  
            (PARTITION BY customer_id ORDER BY activity_date) = 1  
        THEN 0 ELSE 1 END AS is_new_streak  
    FROM activity  
)
```

```

numbered AS (
  SELECT *, SUM(is_new_streak) OVER (PARTITION BY customer_id ORDER BY activity_date) AS streak_id
  FROM marked
)
SELECT customer_id, streak_id, MIN(activity_date), MAX(activity_date), COUNT(*)
FROM numbered GROUP BY customer_id, streak_id;

```

The "flag then cumulative-sum the flag to create a group id" idiom is worth learning on its own — it solves sessionisation, status-change grouping, and anything of the form "start a new group when X happens".

Interview variations. "Gaps instead of islands" — find missing dates by LEAD-ing and looking for jumps > 1. "Consecutive months, not days" — convert to a month index ($\text{year} * 12 + \text{month}$) so consecutive months differ by 1. "Consecutive logins allowing a one-day gap" — change the LAG condition to ≤ 2 .

12.5 Running totals

Problem. Cumulative revenue by day, for the year to date.

```

WITH daily AS (
  SELECT o.order_ts::date AS d, SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS revenue
  FROM orders o JOIN order_items oi USING (order_id)
  WHERE o.status='completed' AND o.order_ts >= DATE_TRUNC('year', CURRENT_DATE)
  GROUP BY 1
)
SELECT d, ROUND(revenue,2) AS revenue,
       ROUND(SUM(revenue) OVER (ORDER BY d ROWS UNBOUNDED PRECEDING),2) AS cumulative
FROM daily ORDER BY d;

```

Explanation. `ROWS UNBOUNDED PRECEDING` is shorthand for `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. Specifying `ROWS` rather than relying on the `RANGE` default matters if the `ORDER BY` column has ties.

Alternative — the self-join version, which is what you'd write in a very old engine and is $O(n^2)$:

```

SELECT a.d, SUM(b.revenue) FROM daily a JOIN daily b ON b.d <= a.d GROUP BY a.d;

```

Interview variation. "Running total per customer, resetting each year" — `PARTITION BY customer_id, DATE_TRUNC('year', d)`. "Running total that resets when a target is hit" — that needs recursion or procedural logic; say so rather than contorting a window function.

12.6 Rolling averages

Problem. 7-day and 28-day moving averages of daily orders.

```

WITH daily AS (
  SELECT d::date, COALESCE(COUNT(o.order_id),0) AS orders
  FROM generate_series(CURRENT_DATE - 89, CURRENT_DATE, INTERVAL '1 day') d
  LEFT JOIN orders o ON o.order_ts::date = d::date AND o.status='completed'
  GROUP BY 1
)
SELECT d, orders,
       ROUND(AVG(orders) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW),1) AS ma7,
       ROUND(AVG(orders) OVER (ORDER BY d ROWS BETWEEN 27 PRECEDING AND CURRENT ROW),1) AS ma28
FROM daily ORDER BY d;

```

Explanation. The `generate_series` LEFT JOIN is doing real work: without it, days with zero orders are absent and `ROWS 6 PRECEDING` spans more than a week. Zero-fill first, then window.

Alternative — gap-safe without zero-filling:

```

AVG(orders) OVER (ORDER BY d RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW)

```

This still differs subtly: it averages only the days that exist, so a week with three missing days averages 4 values, not 7. Zero-filling and `RANGE` answer different questions — say which you mean.

Interview variation. "Why is the 7-day average smoother than the daily figure?" Because it removes day-of-week seasonality — which is also the reason to use 7 and 28 rather than 5 and 30.

12.7 Percent of total

Problem. Each category's share of total revenue.

```
SELECT p.category,
       ROUND(SUM(oi.quantity*oi.unit_price),2) AS revenue,
       ROUND(100.0*SUM(oi.quantity*oi.unit_price)
            / SUM(SUM(oi.quantity*oi.unit_price)) OVER (), 1) AS pct_of_total
FROM order_items oi
JOIN products p USING (product_id)
JOIN orders o ON o.order_id=oi.order_id AND o.status='completed'
GROUP BY p.category
ORDER BY revenue DESC;
```

Explanation. `SUM(SUM(x)) OVER ()` — inner SUM is the group aggregate, outer window SUM totals the groups. One pass, no self-join, no subquery.

Alternative: a CROSS JOIN to a scalar total, or a CTE plus a second aggregation. Both work; the window version is shorter and faster.

Interview variation. "Share within each country rather than overall" — `OVER (PARTITION BY country)`. "Cumulative share" — add `ORDER BY revenue DESC ROWS UNBOUNDED PRECEDING` to the numerator's window. That's Pareto (12.22).

12.8 Ranking

Problem. Rank sales reps within their region, showing joint positions correctly.

```
SELECT region, rep_name, sales,
       RANK() OVER (PARTITION BY region ORDER BY sales DESC) AS rank_with_gaps,
       DENSE_RANK() OVER (PARTITION BY region ORDER BY sales DESC) AS rank_no_gaps,
       ROUND(100*PERCENT_RANK() OVER (PARTITION BY region ORDER BY sales DESC),1) AS pct_rank,
       COUNT(*) OVER (PARTITION BY region) AS reps_in_region
FROM rep_sales;
```

Interview variation. "Rank across regions but show the region rank too" — two windows, one partitioned, one not. "Reps in the top 10% of their region" — `PERCENT_RANK() <= 0.1`.

12.9 Cohort analysis

Problem. Group customers by the month of their first purchase, then track how many are still active in each subsequent month.

Thought process. Three steps, and naming them is half the interview answer: (1) assign each customer a cohort — their first-purchase month; (2) find every month each customer was active; (3) join the two and count distinct customers per cohort per month-offset.

```
WITH first_purchase AS (
  SELECT customer_id,
         DATE_TRUNC('month', MIN(order_ts))::date AS cohort_month
  FROM orders WHERE status='completed'
  GROUP BY customer_id
),
activity AS (
  SELECT DISTINCT customer_id,
                 DATE_TRUNC('month', order_ts)::date AS activity_month
  FROM orders WHERE status='completed'
),
cohort_activity AS (
  SELECT f.cohort_month,
         a.activity_month,
         (EXTRACT(YEAR FROM a.activity_month) - EXTRACT(YEAR FROM f.cohort_month)) * 12
         + (EXTRACT(MONTH FROM a.activity_month) - EXTRACT(MONTH FROM f.cohort_month)) AS month_number,
         a.customer_id
  FROM first_purchase f
  JOIN activity a USING (customer_id)
),
cohort_sizes AS (
  SELECT cohort_month, COUNT(*) AS cohort_size FROM first_purchase GROUP BY cohort_month
)
SELECT ca.cohort_month,
```

```

    cs.cohort_size,
    ca.month_number,
    COUNT(DISTINCT ca.customer_id) AS active_customers,
    ROUND(100.0 * COUNT(DISTINCT ca.customer_id) / cs.cohort_size, 1) AS retention_pct
FROM cohort_activity ca
JOIN cohort_sizes cs USING (cohort_month)
GROUP BY ca.cohort_month, cs.cohort_size, ca.month_number
ORDER BY ca.cohort_month, ca.month_number;

```

Explanation. `month_number` is computed as a month difference, not $(date - date)/30$, which drifts. Month 0 is always 100% by construction — that's the sanity check that your query is right. The output is a long-format cohort table; pivot it in the BI tool, or with conditional aggregation if the interviewer wants the triangle:

```

SELECT cohort_month, cohort_size,
    MAX(retention_pct) FILTER (WHERE month_number=1) AS m1,
    MAX(retention_pct) FILTER (WHERE month_number=2) AS m2,
    MAX(retention_pct) FILTER (WHERE month_number=3) AS m3
FROM cohort_retention GROUP BY 1,2 ORDER BY 1;

```

Interview variations. "Cohort by signup month rather than first purchase" — use `customers.signup_date`; this measures activation as well as retention, and the two answer different questions. "Revenue cohorts" — sum revenue instead of counting customers, and divide by cohort size for revenue-per-original-customer. "Why is the most recent cohort's month-3 retention blank?" Because three months haven't elapsed — right-censoring, and mistaking it for a collapse in retention is a classic bad-analysis story.

12.10 Retention

Problem. What proportion of customers active in month N are still active in month N+1?

```

WITH monthly_active AS (
    SELECT DISTINCT customer_id, DATE_TRUNC('month', order_ts)::date AS month
    FROM orders WHERE status='completed'
)
SELECT curr.month,
    COUNT(DISTINCT curr.customer_id) AS active_this_month,
    COUNT(DISTINCT nxt.customer_id) AS retained_next_month,
    ROUND(100.0 * COUNT(DISTINCT nxt.customer_id)
        / NULLIF(COUNT(DISTINCT curr.customer_id),0), 1) AS retention_pct
FROM monthly_active curr
LEFT JOIN monthly_active nxt
    ON nxt.customer_id = curr.customer_id
    AND nxt.month = curr.month + INTERVAL '1 month'
GROUP BY curr.month
ORDER BY curr.month;

```

Explanation. Self-join on the same table offset by one month. LEFT JOIN so that non-returning customers still count in the denominator — an inner join would give you 100% retention every month, which is the seeded error.

Alternative — new/retained/resurrected/churned decomposition, which is what a product analyst actually reports:

```

WITH ma AS (SELECT DISTINCT customer_id, DATE_TRUNC('month',order_ts)::date AS month
    FROM orders WHERE status='completed'),
first_month AS (SELECT customer_id, MIN(month) AS first_month FROM ma GROUP BY customer_id),
classified AS (
    SELECT m.month, m.customer_id,
        CASE WHEN m.month = f.first_month THEN 'new'
            WHEN EXISTS (SELECT 1 FROM ma p WHERE p.customer_id=m.customer_id
                AND p.month = m.month - INTERVAL '1 month') THEN 'retained'
            ELSE 'resurrected' END AS status
    FROM ma m JOIN first_month f USING (customer_id)
SELECT month, COUNT(*) FILTER (WHERE status='new') AS new_customers,
    COUNT(*) FILTER (WHERE status='retained') AS retained,
    COUNT(*) FILTER (WHERE status='resurrected') AS resurrected
FROM classified GROUP BY month ORDER BY month;

```

Interview variation. "Rolling 30-day retention rather than calendar month" — replace calendar months with day-offset windows relative to each customer's own first activity.

12.11 Churn

Problem. Which customers have churned, and what's the monthly churn rate?

Thought process. Churn needs a definition before it needs SQL. For a subscription business it's an explicit cancellation. For retail there is no cancel event, so churn is a rule: no purchase in N days, where N comes from the observed repurchase distribution.

```
-- pick N from the data rather than guessing
WITH gaps AS (
  SELECT customer_id,
         order_ts::date - LAG(order_ts::date) OVER (PARTITION BY customer_id ORDER BY order_ts) AS gap
  FROM orders WHERE status='completed'
)
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY gap) AS median_gap,
       PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY gap) AS p90_gap
FROM gaps WHERE gap IS NOT NULL;

-- classify against a 180-day rule
WITH last_order AS (
  SELECT customer_id, MAX(order_ts)::date AS last_order_date, COUNT(*) AS lifetime_orders
  FROM orders WHERE status='completed' GROUP BY customer_id
)
SELECT CASE WHEN last_order_date >= CURRENT_DATE - 180 THEN 'Active'
           WHEN lifetime_orders = 1 THEN 'Churned - never repeated'
           ELSE 'Churned - lapsed repeat buyer' END AS churn_status,
       COUNT(*) AS customers,
       ROUND(100.0*COUNT(*)/SUM(COUNT(*)) OVER (,1) AS pct
FROM last_order GROUP BY 1;
```

Monthly churn rate:

```
WITH ma AS (SELECT DISTINCT customer_id, DATE_TRUNC('month',order_ts)::date AS month
            FROM orders WHERE status='completed')
SELECT curr.month,
       COUNT(DISTINCT curr.customer_id) AS active_start,
       COUNT(DISTINCT curr.customer_id) FILTER (WHERE nxt.customer_id IS NULL) AS churned,
       ROUND(100.0*COUNT(DISTINCT curr.customer_id) FILTER (WHERE nxt.customer_id IS NULL)
            / NULLIF(COUNT(DISTINCT curr.customer_id),0),1) AS churn_rate_pct
FROM ma curr
LEFT JOIN ma nxt ON nxt.customer_id=curr.customer_id AND nxt.month=curr.month+INTERVAL '1 month'
GROUP BY curr.month ORDER BY curr.month;
```

Interview variation. "Churn rate is rising but revenue is flat — explain." Possible: churn concentrated in low-value customers; or high-value customers buying more to compensate; or the denominator shrank. Segment churn by value decile before concluding anything. That answer demonstrates analytical thinking rather than SQL, which is what a senior interviewer is testing at this point.

12.12 Funnel analysis

Problem. Conversion through page view → product view → add to cart → checkout → purchase.

Thought process. Two kinds of funnel and you must ask which: **unordered** (did the user ever do each step?) or **ordered** (did they do them in sequence, each after the previous?). Unordered is much easier and often what's wanted.

```
-- unordered funnel by session
WITH session_steps AS (
  SELECT session_id,
         MAX(CASE WHEN event_name='page_view' THEN 1 ELSE 0 END) AS s1,
         MAX(CASE WHEN event_name='product_view' THEN 1 ELSE 0 END) AS s2,
         MAX(CASE WHEN event_name='add_to_cart' THEN 1 ELSE 0 END) AS s3,
         MAX(CASE WHEN event_name='checkout_start' THEN 1 ELSE 0 END) AS s4,
         MAX(CASE WHEN event_name='purchase' THEN 1 ELSE 0 END) AS s5
  FROM web_events
  WHERE event_ts >= CURRENT_DATE - 30
  GROUP BY session_id
)
SELECT SUM(s1) AS visited,
       SUM(s2) AS viewed_product,
       SUM(s3) AS added_to_cart,
       SUM(s4) AS started_checkout,
       SUM(s5) AS purchased,
       ROUND(100.0*SUM(s2)/NULLIF(SUM(s1),0),1) AS pct_view_product,
       ROUND(100.0*SUM(s3)/NULLIF(SUM(s2),0),1) AS pct_add_given_view,
       ROUND(100.0*SUM(s4)/NULLIF(SUM(s3),0),1) AS pct_checkout_given_add,
       ROUND(100.0*SUM(s5)/NULLIF(SUM(s4),0),1) AS pct_purchase_given_checkout,
```

```
ROUND(100.0*SUM(s5)/NULLIF(SUM(s1),0),1) AS overall_conversion
FROM session_steps;
```

Ordered funnel — each step must occur after the previous one:

```
WITH steps AS (
  SELECT session_id,
    MIN(event_ts) FILTER (WHERE event_name='product_view') AS t_view,
    MIN(event_ts) FILTER (WHERE event_name='add_to_cart') AS t_cart,
    MIN(event_ts) FILTER (WHERE event_name='checkout_start') AS t_checkout,
    MIN(event_ts) FILTER (WHERE event_name='purchase') AS t_purchase
  FROM web_events GROUP BY session_id
)
SELECT COUNT(*) FILTER (WHERE t_view IS NOT NULL) AS viewed,
  COUNT(*) FILTER (WHERE t_cart > t_view) AS carted_after_view,
  COUNT(*) FILTER (WHERE t_checkout > t_cart) AS checkout_after_cart,
  COUNT(*) FILTER (WHERE t_purchase > t_checkout) AS purchased_after_checkout
FROM steps;
```

Explanation. `MIN(ts) FILTER (WHERE ...)` gives the first occurrence of each step; comparing timestamps enforces order. NULLs propagate correctly — a comparison with a NULL step is UNKNOWN, so the row isn't counted, which is what you want.

Interview variations. "Where's the biggest drop-off?" Report step-to-step rates, not just overall — the largest absolute loss and the worst rate can be different steps. "Funnel by device" — add `device` to the GROUP BY and compare. "What if a user adds to cart on Monday and buys on Thursday?" Then session-level funnelling undercounts; switch the grain to customer and define an attribution window.

12.13 Conversion rates

```
SELECT DATE_TRUNC('week', e.event_ts)::date AS week,
  e.device,
  COUNT(DISTINCT e.session_id) AS sessions,
  COUNT(DISTINCT e.session_id) FILTER (WHERE e.event_name='purchase') AS converting,
  ROUND(100.0*COUNT(DISTINCT e.session_id) FILTER (WHERE e.event_name='purchase')
    / NULLIF(COUNT(DISTINCT e.session_id),0), 2) AS conversion_rate_pct
FROM web_events e
GROUP BY 1,2 ORDER BY 1,2;
```

Interview variation. "Mobile converts at 1.2%, desktop at 3.4% — should we deprioritise mobile?" No: mobile likely has more top-of-funnel browsing traffic, and users may research on mobile and buy on desktop. Cross-device attribution and traffic-source mix are confounders. Recognising Simpson's paradox risk here is the answer that lands.

12.14 Customer segmentation (RFM)

```
WITH rfm_base AS (
  SELECT o.customer_id,
    CURRENT_DATE - MAX(o.order_ts)::date AS recency_days,
    COUNT(DISTINCT o.order_id) AS frequency,
    SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS monetary
  FROM orders o JOIN order_items oi USING (order_id)
  WHERE o.status='completed'
  GROUP BY o.customer_id
),
scored AS (
  SELECT *,
    NTILE(5) OVER (ORDER BY recency_days ASC) AS r_score, -- Lower recency = better
    NTILE(5) OVER (ORDER BY frequency DESC) AS f_score,
    NTILE(5) OVER (ORDER BY monetary DESC) AS m_score
  FROM rfm_base
)
SELECT customer_id, recency_days, frequency, ROUND(monetary,2) AS monetary,
  r_score, f_score, m_score,
  CASE WHEN r_score<=2 AND f_score<=2 AND m_score<=2 THEN 'Champions'
    WHEN r_score<=2 AND f_score<=3 THEN 'Loyal'
    WHEN r_score<=2 THEN 'Promising'
    WHEN r_score>=4 AND f_score<=2 THEN 'At risk - was valuable'
    WHEN r_score>=4 THEN 'Hibernating'
    ELSE 'Needs attention' END AS segment
FROM scored;
```

Note on NTILE direction. With `ORDER BY recency_days ASC`, bucket 1 is the *most recent* customers — good. Getting the direction backwards on any of the three silently inverts your segments, and the output still looks plausible, so state the direction in a comment as done above.

Interview variation. "Segment sizes are wildly uneven — why?" NTILE forces equal *counts*, so uneven sizes mean you used CASE thresholds rather than quintiles, or ties are collapsing buckets. "How would you validate the segmentation?" Check that Champions have materially higher forward-looking revenue than other segments over the next quarter — a segmentation that doesn't predict anything is decoration.

12.15 Repeat purchases

```
WITH customer_orders AS (
  SELECT customer_id, COUNT(*) AS orders,
         MIN(order_ts)::date AS first_order, MAX(order_ts)::date AS last_order
  FROM orders WHERE status='completed' GROUP BY customer_id
)
SELECT COUNT(*) AS all_customers,
       COUNT(*) FILTER (WHERE orders >= 2) AS repeat_customers,
       ROUND(100.0*COUNT(*) FILTER (WHERE orders>=2)/COUNT(*),1) AS repeat_rate_pct,
       ROUND(AVG(orders),2) AS avg_orders_per_customer,
       ROUND(AVG(last_order - first_order) FILTER (WHERE orders>=2),0) AS avg_days_between_first_and_last
FROM customer_orders;
```

Time to second purchase, which is the actionable version:

```
WITH seq AS (
  SELECT customer_id, order_ts,
         ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts) AS n
  FROM orders WHERE status='completed'
)
SELECT ROUND(AVG(s2.order_ts::date - s1.order_ts::date),1) AS mean_days_to_second,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY s2.order_ts::date - s1.order_ts::date)
       AS median_days_to_second
FROM seq s1 JOIN seq s2 ON s2.customer_id=s1.customer_id AND s1.n=1 AND s2.n=2;
```

Interview variation. "Repeat rate is 30% — is that good?" It's meaningless without a cohort: customers who signed up last week haven't had time to repeat. Compute repeat rate by cohort with a fixed observation window (e.g. repeat within 90 days of first purchase) so cohorts are comparable. That's the answer.

12.16 First and last purchase

```
SELECT DISTINCT ON (o.customer_id)
       o.customer_id, o.order_id AS first_order_id, o.order_ts::date AS first_order_date,
       p.product_name AS first_product, p.category AS first_category
FROM orders o
JOIN order_items oi USING (order_id)
JOIN products p USING (product_id)
WHERE o.status='completed'
ORDER BY o.customer_id, o.order_ts, oi.order_item_id;
```

Interview variation. "Which first-purchase category leads to the highest lifetime value?" Join first category to total LTV and group — a genuinely useful acquisition insight, and a natural follow-up they'll enjoy you volunteering.

```
WITH first_cat AS (
  SELECT DISTINCT ON (o.customer_id) o.customer_id, p.category
  FROM orders o JOIN order_items oi USING (order_id) JOIN products p USING (product_id)
  WHERE o.status='completed' ORDER BY o.customer_id, o.order_ts, oi.order_item_id),
ltv AS (
  SELECT o.customer_id, SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS lifetime_value
  FROM orders o JOIN order_items oi USING (order_id) WHERE o.status='completed'
  GROUP BY o.customer_id)
SELECT f.category, COUNT(*) AS customers, ROUND(AVG(l.lifetime_value),2) AS avg_ltv
FROM first_cat f JOIN ltv l USING (customer_id)
GROUP BY f.category ORDER BY avg_ltv DESC;
```

12.17 Session analysis (sessionisation)

Problem. Events have no session id; define a session as activity with no more than 30 minutes between consecutive events.

Thought process. Same idiom as gaps and islands: flag the rows that start a new session, then cumulative-sum the flag to produce a session id.

```
WITH ordered AS (
    SELECT customer_id, event_ts, event_name,
           LAG(event_ts) OVER (PARTITION BY customer_id ORDER BY event_ts) AS prev_ts
    FROM web_events WHERE customer_id IS NOT NULL
),
flagged AS (
    SELECT *, CASE WHEN prev_ts IS NULL
                    OR event_ts - prev_ts > INTERVAL '30 minutes'
                    THEN 1 ELSE 0 END AS is_session_start
    FROM ordered
),
sessionised AS (
    SELECT *, SUM(is_session_start) OVER (PARTITION BY customer_id ORDER BY event_ts)
           AS session_number
    FROM flagged
)
SELECT customer_id, session_number,
       MIN(event_ts) AS session_start,
       MAX(event_ts) AS session_end,
       ROUND(EXTRACT(EPOCH FROM (MAX(event_ts) - MIN(event_ts)))/60,1) AS duration_minutes,
       COUNT(*) AS events,
       BOOL_OR(event_name='purchase') AS converted
FROM sessionised
GROUP BY customer_id, session_number
ORDER BY customer_id, session_number;
```

Explanation. `BOOL_OR` is the neat aggregate for "did any row in this group satisfy X". The cumulative sum of a 0/1 flag creating a group id is worth committing to memory — it's the same trick as 12.4.

Interview variation. "Single-event sessions have zero duration — is that right?" Technically yes, analytically misleading: bounce sessions have unknown duration, not zero, and including them drags the average down. Either exclude them or report bounce rate separately.

12.18 Consecutive events

Problem. Customers who placed orders in three consecutive months.

```
WITH monthly AS (
    SELECT DISTINCT customer_id, DATE_TRUNC('month', order_ts)::date AS month
    FROM orders WHERE status='completed'
),
indexed AS (
    SELECT customer_id, month,
           (EXTRACT(YEAR FROM month)*12 + EXTRACT(MONTH FROM month))::int AS month_index
    FROM monthly
),
grouped AS (
    SELECT *, month_index - ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY month_index)
           AS island
    FROM indexed
)
SELECT customer_id, MIN(month) AS run_start, MAX(month) AS run_end, COUNT(*) AS consecutive_months
FROM grouped
GROUP BY customer_id, island
HAVING COUNT(*) >= 3;
```

Alternative — LAG twice, fine for a fixed run length and easier to read:

```
SELECT DISTINCT customer_id FROM (
    SELECT customer_id, month,
           LAG(month,1) OVER (PARTITION BY customer_id ORDER BY month) AS m1,
           LAG(month,2) OVER (PARTITION BY customer_id ORDER BY month) AS m2
    FROM monthly) t
WHERE month - INTERVAL '1 month' = m1 AND month - INTERVAL '2 months' = m2;
```

Interview variation. "Three consecutive *days* of login" — same shape, day index. "N consecutive, where N is a parameter" — the island method scales; the LAG method doesn't. Say that.

12.19 Anomaly detection

Problem. Flag days where orders deviate sharply from the recent norm.

```
WITH daily AS (  
    SELECT order_ts::date AS d, COUNT(*) AS orders  
    FROM orders WHERE status='completed' GROUP BY 1  
)  
stats AS (  
    SELECT d, orders,  
        AVG(orders) OVER (ORDER BY d ROWS BETWEEN 28 PRECEDING AND 1 PRECEDING) AS baseline,  
        STDDEV(orders) OVER (ORDER BY d ROWS BETWEEN 28 PRECEDING AND 1 PRECEDING) AS sd  
    FROM daily  
)  
SELECT d, orders, ROUND(baseline,1) AS baseline_28d,  
    ROUND((orders - baseline)/NULLIF(sd,0), 2) AS z_score,  
    CASE WHEN ABS(orders - baseline) > 3*sd THEN 'Extreme'  
        WHEN ABS(orders - baseline) > 2*sd THEN 'Unusual'  
        ELSE 'Normal' END AS flag  
FROM stats  
WHERE sd IS NOT NULL AND ABS(orders - baseline) > 2*sd  
ORDER BY d;
```

Explanation. The frame ends at 1 PRECEDING so the day being tested isn't part of its own baseline — otherwise an extreme value pulls the mean towards itself and hides the anomaly. That detail is the whole point of the query.

Interview variation. "Every Saturday flags as an anomaly." Day-of-week seasonality: compare each day to the same weekday in prior weeks (PARTITION BY EXTRACT(ISODOW FROM d)), or use a 7-day-multiple lag. "Z-scores assume normality" — true; for count data, or heavily skewed metrics, prefer a median/MAD approach or percentile bounds. Raising that unprompted is a strong signal.

12.20 Pareto analysis

Problem. How many customers account for 80% of revenue?

```
WITH customer_revenue AS (  
    SELECT o.customer_id, SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS revenue  
    FROM orders o JOIN order_items oi USING (order_id)  
    WHERE o.status='completed' GROUP BY o.customer_id  
)  
ranked AS (  
    SELECT customer_id, revenue,  
        ROW_NUMBER() OVER (ORDER BY revenue DESC) AS rn,  
        SUM(revenue) OVER (ORDER BY revenue DESC ROWS UNBOUNDED PRECEDING) AS cumulative_revenue,  
        SUM(revenue) OVER () AS total_revenue,  
        COUNT(*) OVER () AS total_customers  
    FROM customer_revenue  
)  
SELECT customer_id, ROUND(revenue,2) AS revenue,  
    ROUND(100.0*cumulative_revenue/total_revenue,1) AS cumulative_pct_revenue,  
    ROUND(100.0*rn/total_customers,1) AS pct_of_customers  
FROM ranked  
WHERE cumulative_revenue - revenue < 0.8*total_revenue -- include the row that crosses 80%  
ORDER BY rn;
```

Explanation. The WHERE condition uses the cumulative total *before* the current row, so the customer who tips the total past 80% is included rather than excluded. Off-by-one here is the standard mistake.

Interview variation. "Is it really 80/20?" Rarely exactly; report the actual figure ("the top 12% of customers drive 80% of revenue"). "Do it by product" — same query, group by product. "What's the risk?" Concentration: if the top 12% churn, revenue collapses. That framing turns a SQL answer into an analyst answer.

12.21 Contribution analysis

Problem. Revenue fell 8% month on month. Which categories caused it?

Thought process. Decompose the total change into per-segment contributions that sum back to the total. Each segment's contribution is its own change divided by the *previous total*, so the parts add up to the overall percentage change.

```

WITH monthly_category AS (
    SELECT DATE_TRUNC('month', o.order_ts)::date AS month, p.category,
           SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS revenue
    FROM orders o JOIN order_items oi USING (order_id) JOIN products p USING (product_id)
    WHERE o.status='completed' GROUP BY 1,2
),
with_prev AS (
    SELECT month, category, revenue,
           LAG(revenue) OVER (PARTITION BY category ORDER BY month) AS prev_revenue
    FROM monthly_category
),
totals AS (
    SELECT month, SUM(revenue) AS total_revenue, SUM(prev_revenue) AS prev_total
    FROM with_prev GROUP BY month
)
SELECT w.month, w.category,
       ROUND(w.revenue,2) AS revenue,
       ROUND(w.prev_revenue,2) AS prev_revenue,
       ROUND(w.revenue - w.prev_revenue,2) AS absolute_change,
       ROUND(100.0*(w.revenue-w.prev_revenue)/NULLIF(w.prev_revenue,0),1) AS category_growth_pct,
       ROUND(100.0*(w.revenue-w.prev_revenue)/NULLIF(t.prev_total,0),2) AS contribution_to_total_pct
FROM with_prev w JOIN totals t USING (month)
WHERE w.prev_revenue IS NOT NULL
ORDER BY w.month, contribution_to_total_pct;

```

Explanation. Two different percentages, and confusing them is the classic error. `category_growth_pct` says how much that category moved. `contribution_to_total_pct` says how much of the overall movement it accounts for — and those sum to the total change. A small category falling 60% may contribute almost nothing; a large category falling 4% may be the entire story.

Interview variation. "Separate price effect from volume effect." Decompose: revenue = units × average price, so change ≈ (Δunits × prior price) + (Δprice × prior units) + interaction. Being able to sketch that shows commercial literacy.

12.22 Putting it together — a churn early-warning query

The kind of thing that wins a take-home task. Every pattern above appears in it.

```

WITH order_values AS (
    SELECT o.order_id, o.customer_id, o.order_ts,
           SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS order_value
    FROM orders o JOIN order_items oi USING (order_id)
    WHERE o.status='completed'
    GROUP BY o.order_id, o.customer_id, o.order_ts
),
customer_history AS (
    SELECT customer_id,
           COUNT(*) AS orders,
           SUM(order_value) AS lifetime_value,
           AVG(order_value) AS avg_order_value,
           MIN(order_ts)::date AS first_order,
           MAX(order_ts)::date AS last_order,
           CURRENT_DATE - MAX(order_ts)::date AS days_since_last
    FROM order_values GROUP BY customer_id
),
cadence AS (
    SELECT customer_id,
           AVG(gap) AS avg_days_between_orders,
           STDDEV(gap) AS gap_variability
    FROM (SELECT customer_id,
                 order_ts::date - LAG(order_ts::date) OVER (PARTITION BY customer_id ORDER BY order_ts) AS gap
    FROM order_values) g
    WHERE gap IS NOT NULL
    GROUP BY customer_id
),
scored AS (
    SELECT h.*, c.avg_days_between_orders,
           NTILE(10) OVER (ORDER BY h.lifetime_value DESC) AS value_decile,
           CASE WHEN h.orders = 1 AND h.days_since_last > 90 THEN 'Failed to activate'
                WHEN c.avg_days_between_orders IS NULL THEN 'Insufficient history'
                WHEN h.days_since_last > 3*c.avg_days_between_orders THEN 'High risk'
                WHEN h.days_since_last > 2*c.avg_days_between_orders THEN 'Medium risk'
                ELSE 'Healthy' END AS churn_risk
    FROM customer_history h LEFT JOIN cadence c USING (customer_id)
)
SELECT churn_risk, value_decile,

```

```
COUNT(*) AS customers,  
ROUND(SUM(lifetime_value),2) AS revenue_at_risk,  
ROUND(AVG(days_since_last),0) AS avg_days_since_last  
FROM scored  
GROUP BY churn_risk, value_decile  
HAVING churn_risk IN ('High risk','Medium risk')  
ORDER BY value_decile, churn_risk;
```

The risk rule compares each customer's silence to *their own* purchase cadence rather than a single global threshold — a monthly buyer silent for 90 days is in trouble; an annual buyer isn't. Crossing that with value decile answers the question a commercial stakeholder actually has: not "who might churn" but "where should we spend retention budget".

Parts 13–15: Business KPIs, Performance, Database Design

PART 13 — BUSINESS KPI SQL

The skill this section teaches isn't SQL. It's translating a business definition into a query, and — more importantly — noticing where the definition is ambiguous and saying so. Interviewers ask "how would you calculate retention rate?" specifically to see whether you ask a clarifying question or just start typing.

Every KPI below is given as: **definition** → **decisions** → **SQL** → **what to challenge**.

13.1 Revenue

Definition. Value of goods sold in a period.

Decisions. Which order statuses count? Gross or net of discounts? Does shipping count as revenue? Are refunds deducted in the month of the order or the month of the refund? Ex-VAT or inc-VAT? Every one of these changes the number, and finance will have a fixed answer.

```
SELECT DATE_TRUNC('month', o.order_ts)::date AS month,
       ROUND(SUM(oi.quantity * oi.unit_price), 2) AS gross_revenue,
       ROUND(SUM(oi.quantity * oi.unit_price * oi.discount_pct), 2) AS discounts,
       ROUND(SUM(oi.quantity * oi.unit_price * (1-oi.discount_pct)), 2) AS net_product_revenue,
       ROUND(SUM(DISTINCT_SHIPPING.shipping), 2) AS shipping_revenue
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN LATERAL (SELECT o.shipping_cost AS shipping) DISTINCT_SHIPPING ON true
WHERE o.status = 'completed'
GROUP BY 1 ORDER BY 1;
```

Shipping is order-level, so summing it after the item join double-counts. The correct version separates the grains:

```
WITH order_revenue AS (
  SELECT o.order_id,
         DATE_TRUNC('month', o.order_ts)::date AS month,
         SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS product_revenue,
         MAX(o.shipping_cost) AS shipping_revenue
  FROM orders o JOIN order_items oi USING (order_id)
  WHERE o.status='completed'
  GROUP BY o.order_id, 2
)
SELECT month,
       ROUND(SUM(product_revenue),2) AS product_revenue,
       ROUND(SUM(shipping_revenue),2) AS shipping_revenue,
       ROUND(SUM(product_revenue+shipping_revenue),2) AS total_revenue
FROM order_revenue GROUP BY month ORDER BY month;
```

Challenge. "Revenue is up 5% but net revenue is flat" — you're discounting more heavily to hold volume. Always report gross, discount and net together; a single revenue line hides the mechanism.

13.2 Profit and gross margin

Definition. Gross profit = revenue – cost of goods sold. Gross margin % = gross profit ÷ revenue.

Decision. Which costs? Gross margin uses direct product cost only. Including shipping, payment fees or marketing gives contribution margin, a different metric.

```
SELECT p.category,
       ROUND(SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)), 2) AS revenue,
       ROUND(SUM(oi.quantity*p.unit_cost), 2) AS cogs,
       ROUND(SUM(oi.quantity*(oi.unit_price*(1-oi.discount_pct) - p.unit_cost)), 2) AS gross_profit,
       ROUND(100.0*SUM(oi.quantity*(oi.unit_price*(1-oi.discount_pct) - p.unit_cost))
             / NULLIF(SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)),0), 1) AS gross_margin_pct
FROM order_items oi
JOIN products p USING (product_id)
JOIN orders o ON o.order_id=oi.order_id AND o.status='completed'
GROUP BY p.category
ORDER BY gross_profit DESC;
```

Challenge. `products.unit_cost` is the *current* cost, but `order_items.unit_price` is the price *at time of sale*. Margins on historical orders are computed against today's costs, which is wrong when costs move. The proper fix is a cost history table or a cost snapshot on the order line. Spotting this asymmetry in a schema is a genuinely senior observation and this

schema contains it deliberately.

13.3 Average order value

```
WITH order_totals AS (  
  SELECT o.order_id, DATE_TRUNC('month', o.order_ts)::date AS month,  
         SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS order_value  
  FROM orders o JOIN order_items oi USING (order_id)  
  WHERE o.status='completed' GROUP BY 1,2  
)  
SELECT month,  
       COUNT(*) AS orders,  
       ROUND(AVG(order_value),2) AS aov,  
       ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY order_value)::numeric,2) AS median_order_value  
FROM order_totals GROUP BY month ORDER BY month;
```

Challenge. Report the median alongside. Order values are right-skewed; one £5,000 B2B order moves the mean and nothing else. If mean and median diverge sharply, the mean is not describing a typical order.

13.4 Conversion rate

Definition. Converting units ÷ eligible units.

Decision. Sessions or users? A user with five sessions and one purchase is 20% at session level and 100% at user level. Both are legitimate; they answer different questions.

```
SELECT DATE_TRUNC('week', event_ts)::date AS week,  
       COUNT(DISTINCT session_id) AS sessions,  
       COUNT(DISTINCT session_id) FILTER (WHERE event_name='purchase') AS converting_sessions,  
       ROUND(100.0*COUNT(DISTINCT session_id) FILTER (WHERE event_name='purchase')  
             / NULLIF(COUNT(DISTINCT session_id),0),2) AS session_cvr_pct,  
       COUNT(DISTINCT customer_id) FILTER (WHERE event_name='purchase') AS converting_users  
FROM web_events GROUP BY 1 ORDER BY 1;
```

13.5 Retention rate

Definition. Proportion of a starting group still active after a period.

Decisions. Cohort by signup or first purchase? Calendar month or rolling window? Does "active" mean logged in, or purchased? N-day retention (active exactly on day N) or unbounded (active at any point since)?

See Part 12.9–12.10 for the full queries. The one-line version:

```
ROUND(100.0 * COUNT(DISTINCT returning.customer_id) / NULLIF(COUNT(DISTINCT base.customer_id),0), 1)
```

Challenge. "Retention improved from 40% to 55%" — check whether the cohort composition changed. Stopping a bad paid-acquisition channel raises retention without any product improvement, because you removed the customers who were never going to stay.

13.6 Churn rate

Churn = 1 – retention, when measured over the same population and window. Not interchangeable when the denominators differ (retention often measured on a cohort, churn on the active base).

```
WITH ma AS (SELECT DISTINCT customer_id, DATE_TRUNC('month', order_ts)::date AS month  
            FROM orders WHERE status='completed')  
SELECT c.month,  
       COUNT(DISTINCT c.customer_id) AS active,  
       COUNT(DISTINCT c.customer_id) FILTER (WHERE n.customer_id IS NULL) AS churned,  
       ROUND(100.0*COUNT(DISTINCT c.customer_id) FILTER (WHERE n.customer_id IS NULL)  
             / NULLIF(COUNT(DISTINCT c.customer_id),0),1) AS churn_rate_pct  
FROM ma c LEFT JOIN ma n ON n.customer_id=c.customer_id AND n.month=c.month+INTERVAL '1 month'  
GROUP BY c.month ORDER BY c.month;
```

Challenge. The most recent month always shows 100% churn because next month hasn't happened. Exclude incomplete periods, or your dashboard will terrify someone.

13.7 Customer acquisition

```

SELECT DATE_TRUNC('month', signup_date)::date AS month,
       channel,
       COUNT(*) AS new_customers,
       COUNT(*) FILTER (WHERE EXISTS (
         SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id AND o.status='completed'))
       AS activated,
       ROUND(100.0*COUNT(*) FILTER (WHERE EXISTS (
         SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id AND o.status='completed'))
         / COUNT(*),1) AS activation_rate_pct
FROM customers c
GROUP BY 1,2 ORDER BY 1,2;

```

CAC needs marketing spend, which isn't in this schema:

```

SELECT m.month, m.channel, m.spend, a.new_customers,
       ROUND(m.spend/NULLIF(a.new_customers,0),2) AS cac
FROM marketing_spend m JOIN acquisitions a USING (month, channel);

```

Challenge. Attribution. Last-click credits the final touch, first-click the first; a customer who saw a paid ad then searched the brand name is attributed entirely differently under each. Say that CAC by channel is only as good as the attribution model behind it.

13.8 Customer lifetime value

Historic LTV — what they've actually spent, and the version you can compute from this schema:

```

SELECT c.customer_id, c.channel AS acquisition_channel,
       COUNT(DISTINCT o.order_id) AS orders,
       ROUND(COALESCE(SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)),0),2) AS historic_ltv,
       ROUND(COALESCE(SUM(oi.quantity*(oi.unit_price*(1-oi.discount_pct)-p.unit_cost)),0),2)
       AS gross_profit_ltv,
       CURRENT_DATE - c.signup_date AS tenure_days
FROM customers c
LEFT JOIN orders o ON o.customer_id=c.customer_id AND o.status='completed'
LEFT JOIN order_items oi USING (order_id)
LEFT JOIN products p USING (product_id)
GROUP BY c.customer_id, c.channel, c.signup_date;

```

Predicted LTV, simple version: $AOV \times \text{purchase frequency} \times \text{expected lifespan}$, ideally on gross profit not revenue.

```

WITH metrics AS (
  SELECT AVG(order_value) AS aov,
         COUNT(*)::numeric / COUNT(DISTINCT customer_id) AS orders_per_customer
  FROM order_values
)
SELECT ROUND(aov * orders_per_customer / NULLIF(churn_rate,0), 2) AS predicted_ltv
FROM metrics CROSS JOIN (SELECT 0.15 AS churn_rate) c;

```

Challenge. Historic LTV is biased by tenure — a customer who joined last month cannot have high LTV. Always compare LTV within cohorts of similar age, or normalise to LTV-at-90-days. Volunteering that is the difference between reciting a formula and understanding it.

13.9 Repeat purchase rate

```

SELECT DATE_TRUNC('month', first_order)::date AS cohort,
       COUNT(*) AS customers,
       COUNT(*) FILTER (WHERE orders >= 2) AS repeaters,
       ROUND(100.0*COUNT(*) FILTER (WHERE orders>=2)/COUNT(*),1) AS repeat_rate_pct
FROM (SELECT customer_id, COUNT(*) AS orders, MIN(order_ts)::date AS first_order
      FROM orders WHERE status='completed' GROUP BY customer_id) t
GROUP BY 1 ORDER BY 1;

```

Cohorting is essential here for the reason in 12.15 — recent cohorts haven't had time to repeat, so an uncohorted repeat rate falls every month you grow.

13.10 Active users — DAU, MAU, stickiness

```

-- DAU
SELECT event_ts::date AS day, COUNT(DISTINCT customer_id) AS dau
FROM web_events WHERE customer_id IS NOT NULL GROUP BY 1;

-- MAU, calendar month
SELECT DATE_TRUNC('month',event_ts)::date AS month, COUNT(DISTINCT customer_id) AS mau

```

```

FROM web_events WHERE customer_id IS NOT NULL GROUP BY 1;

-- rolling 28-day MAU and the DAU/MAU stickiness ratio
WITH dau AS (
  SELECT event_ts::date AS day, COUNT(DISTINCT customer_id) AS dau
  FROM web_events WHERE customer_id IS NOT NULL GROUP BY 1
),
rolling AS (
  SELECT d.day, d.dau,
         (SELECT COUNT(DISTINCT e.customer_id) FROM web_events e
          WHERE e.event_ts::date > d.day - 28 AND e.event_ts::date <= d.day) AS mau_28d
  FROM dau d
)
SELECT day, dau, mau_28d, ROUND(100.0*dau/NULLIF(mau_28d,0),1) AS stickiness_pct
FROM rolling ORDER BY day;

```

Note. MAU is *not* the sum or average of DAU — a user active 20 days counts once in MAU and 20 times across DAU. `COUNT(DISTINCT)` must be recomputed at each grain, which is why rolling MAU needs a correlated subquery or a self-join rather than a window function. Explaining why distinct counts aren't additive is a good interview moment.

Challenge. DAU/MAU near 100% means daily-habit usage; near 3% means monthly. Neither is inherently good — it depends on what the product is for. A tax-filing app with 5% stickiness is fine.

13.11 Growth rate

```

WITH monthly AS (
  SELECT DATE_TRUNC('month', order_ts)::date AS month, COUNT(*) AS orders
  FROM orders WHERE status='completed' GROUP BY 1)
SELECT month, orders,
       ROUND(100.0*(orders - LAG(orders) OVER (ORDER BY month))
             / NULLIF(LAG(orders) OVER (ORDER BY month),0),1) AS mom_growth_pct,
       ROUND(100.0*(orders - LAG(orders,12) OVER (ORDER BY month))
             / NULLIF(LAG(orders,12) OVER (ORDER BY month),0),1) AS yoy_growth_pct,
       ROUND((POWER(orders::numeric / NULLIF(FIRST_VALUE(orders) OVER (ORDER BY month),0),
                    1.0/NULLIF(ROW_NUMBER() OVER (ORDER BY month)-1,0)) - 1)*100, 2) AS cagr_pct
FROM monthly ORDER BY month;

```

Challenge. Percentage growth off a small base is meaningless — 2 orders to 6 is "200% growth". Always show the absolute numbers next to the percentage, and suppress percentages below a minimum base.

13.12 Market share

```

SELECT p.category, p.product_name,
       ROUND(SUM(oi.quantity*oi.unit_price),2) AS revenue,
       ROUND(100.0*SUM(oi.quantity*oi.unit_price)
             / SUM(SUM(oi.quantity*oi.unit_price) OVER (PARTITION BY p.category),1)
             AS share_of_category_pct,
       ROUND(100.0*SUM(oi.quantity*oi.unit_price)
             / SUM(SUM(oi.quantity*oi.unit_price) OVER (),1) AS share_of_total_pct
FROM order_items oi JOIN products p USING (product_id)
JOIN orders o ON o.order_id=oi.order_id AND o.status='completed'
GROUP BY p.category, p.product_name
ORDER BY p.category, revenue DESC;

```

True market share needs competitor data you don't have; internal share of category is what's computable. Be precise about which you're reporting.

13.13 Average waiting time

```

SELECT r.specialty,
       COUNT(*) AS completed_pathways,
       ROUND(AVG(a.attended_ts::date - r.referral_date),1) AS mean_wait_days,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY a.attended_ts::date - r.referral_date)
       AS median_wait_days,
       PERCENTILE_CONT(0.92) WITHIN GROUP (ORDER BY a.attended_ts::date - r.referral_date)
       AS p92_wait_days,
       MAX(a.attended_ts::date - r.referral_date) AS longest_wait
FROM referrals r
JOIN LATERAL (SELECT attended_ts FROM appointments x
               WHERE x.referral_id=r.referral_id AND x.outcome='Attended'
               ORDER BY attended_ts LIMIT 1) a ON true
GROUP BY r.specialty ORDER BY median_wait_days DESC;

```

The 92nd percentile is there because NHS RTT reporting uses a 92% within-18-weeks standard, so p92 is the number that tells you whether you're meeting it.

Challenge — this one matters. This measures *completed* pathways only. Patients still waiting are excluded, and they are the longest waiters. A trust can cut its average completed wait by treating the easy cases first while the queue lengthens. Always report incomplete-pathway waits alongside:

```
SELECT r.specialty,
COUNT(*) AS still_waiting,
ROUND(AVG(CURRENT_DATE - r.referral_date),1) AS mean_wait_so_far,
COUNT(*) FILTER (WHERE CURRENT_DATE - r.referral_date > 126) AS over_18_weeks
FROM referrals r
JOIN waiting_list w ON w.referral_id=r.referral_id AND w.removed_date IS NULL
GROUP BY r.specialty;
```

13.14 SLA compliance and breach rate

```
SELECT site_code,
DATE_TRUNC('month', arrival_ts)::date AS month,
COUNT(*) AS attendances,
COUNT(*) FILTER (WHERE departure_ts IS NULL) AS still_in_dept,
COUNT(*) FILTER (WHERE departure_ts - arrival_ts <= INTERVAL '4 hours') AS within_standard,
COUNT(*) FILTER (WHERE departure_ts - arrival_ts > INTERVAL '4 hours') AS breaches,
ROUND(100.0*COUNT(*) FILTER (WHERE departure_ts - arrival_ts <= INTERVAL '4 hours')
/ NULLIF(COUNT(*) FILTER (WHERE departure_ts IS NOT NULL),0),1) AS pct_within_4h,
ROUND(100.0*COUNT(*) FILTER (WHERE departure_ts - arrival_ts > INTERVAL '4 hours')
/ NULLIF(COUNT(*) FILTER (WHERE departure_ts IS NOT NULL),0),1) AS breach_rate_pct
FROM ae_attendances
GROUP BY 1,2 ORDER BY 1,2;
```

The decisions embedded here, all worth raising:

- Patients still in the department are excluded from the denominator. If someone has been waiting six hours and hasn't left, they've already breached — arguably they should count as a breach immediately. This choice flatters performance.
- Is the clock measured to departure, or to a decision to admit? Different definitions exist and give different numbers.
- Should the standard apply equally to triage category 1 and category 5?

An interviewer asking for "the four-hour performance figure" is often testing whether you'll produce a number silently or interrogate the definition first. Interrogate it.

A defensible version that counts current breaches:

```
COUNT(*) FILTER (
  WHERE COALESCE(departure_ts, CURRENT_TIMESTAMP) - arrival_ts > INTERVAL '4 hours'
) AS breaches_including_current
```

PART 14 — PERFORMANCE AND QUERY OPTIMISATION

What an analyst realistically needs: enough to write queries that don't melt the warehouse, read a query plan well enough to find the problem, and have a sensible conversation with a data engineer. Not index internals.

14.1 How a query executes

1. **Parse** — syntax check.
2. **Rewrite** — apply views, expand rules.
3. **Plan** — the optimiser estimates costs for alternative strategies and picks one, using table statistics gathered by **ANALYZE**.
4. **Execute** — run the chosen plan.

Two consequences worth knowing. First, *estimates* drive the plan: if statistics are stale, the planner picks badly, which is why a query can suddenly get 100× slower after a big data load. Second, the planner rewrites your SQL aggressively — subqueries become joins, filters move around — so cosmetic rewrites often change nothing.

14.2 Indexes

An index is a sorted structure mapping column values to row locations. It converts "scan every row" into "look it up".

```
CREATE INDEX idx_orders_customer ON orders(customer_id);
CREATE INDEX idx_orders_ts      ON orders(order_ts);
CREATE INDEX idx_orders_cust_ts ON orders(customer_id, order_ts); -- composite
CREATE INDEX idx_orders_recent  ON orders(order_ts) WHERE status='completed'; -- partial
CREATE INDEX idx_cust_email_lower ON customers(LOWER(email)); -- functional
```

What an analyst needs to know:

- **Indexes speed reads, slow writes**, and consume disk. They're a trade-off, not free.
- **Column order in a composite index matters.** (*customer_id*, *order_ts*) serves queries filtering on *customer_id* alone, or both. It does *not* efficiently serve a query filtering only on *order_ts* — think of a phone book sorted by surname then first name.
- **Foreign key columns are usually worth indexing;** Postgres does not create these automatically (it does for primary keys and unique constraints).
- **Small tables don't benefit.** Scanning 500 rows is faster than an index lookup, and the planner knows it.

What kills an index — this is the practically useful part:

```
-- function on the column: index unusable
WHERE DATE(order_ts) = '2024-03-15'
WHERE UPPER(email) = 'X@Y.COM'
WHERE customer_id::text = '42'

-- rewrite as a sargable range/predicate: index usable
WHERE order_ts >= '2024-03-15' AND order_ts < '2024-03-16'
WHERE email = 'x@y.com' -- or index LOWER(email) and query LOWER(email)
WHERE customer_id = 42

-- Leading wildcard: index unusable
WHERE email LIKE '%@gmail.com'

-- trailing wildcard: usable
WHERE product_name LIKE 'Wireless%'
```

"Sargable" (search-argument-able) is the term; using it correctly signals you've read beyond a tutorial.

14.3 Scan types

Plan node	Meaning	When it's fine
Seq Scan	read every row	small tables, or when returning most rows anyway

Plan node	Meaning	When it's fine
Index Scan	walk the index, fetch matching rows	highly selective filters
Index Only Scan	answer entirely from the index	best case; needs all selected columns in the index
Bitmap Heap Scan	build a bitmap of matching pages, then read them	medium selectivity, several conditions

A Seq Scan is not automatically bad. Fetching 60% of a table via an index is *slower* than scanning it, because of random I/O per row. The planner switching from Index Scan to Seq Scan as a filter gets less selective is correct behaviour. Saying "I saw a Seq Scan so I added an index" without checking selectivity is the junior answer.

Join algorithms you'll see:

- **Nested Loop** — for each row of A, look up matches in B. Great when A is tiny and B is indexed; catastrophic when both are large.
- **Hash Join** — build a hash table of the smaller side, probe with the larger. The workhorse for big equality joins.
- **Merge Join** — sort both sides, walk them together. Good when inputs are already sorted.

14.4 EXPLAIN and EXPLAIN ANALYZE

```
EXPLAIN SELECT ...;           -- estimated plan, doesn't run
EXPLAIN ANALYZE SELECT ...;   -- actually runs it, shows real timings
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) SELECT ...; -- adds I/O detail
```

Warning: `EXPLAIN ANALYZE` executes the statement. Never run it on an `UPDATE` or `DELETE` outside a transaction you intend to roll back.

Reading a plan:

```
Hash Join (cost=1.16..2.34 rows=10 width=64) (actual time=0.05..0.09 rows=847 loops=1)
  Hash Cond: (o.customer_id = c.customer_id)
    -> Seq Scan on orders o (cost=0.00..1.10 rows=10 width=32) (actual ... rows=847 ...)
        Filter: (status = 'completed'::text)
        Rows Removed by Filter: 153
    -> Hash (cost=1.07..1.07 rows=7 width=36) (actual ...)
        -> Seq Scan on customers c ...
```

Read it inside-out and bottom-up: indented children run first, feeding their parents.

What to look for, in priority order:

1. **rows estimated vs actual.** The plan above estimates 10 rows and gets 847. An estimate off by 100× means the planner chose its strategy on bad information — usually stale statistics (`ANALYZE orders;`) or a correlation it can't see. This is the number one cause of mysteriously slow queries.
2. **The node with the largest actual time**, remembering that a child's time is included in its parent's.
3. **Rows Removed by Filter** in the millions — you're reading a lot of rows to throw them away. Candidate for an index.
4. **Nested Loop with a large outer row count** — often a missing index on the inner side.
5. **loops=N** — the node ran N times; the displayed time is *per loop*, so multiply.
6. **Sort or Hash spilling to disk** (`external merge Disk: 52428kB`) — the operation exceeded `work_mem`.

Practical advice for an interview: you're not expected to tune a plan. You're expected to say "I'd run `EXPLAIN ANALYZE`, compare estimated to actual rows to see whether the planner is misinformed, and find the node consuming the most time." That answer is entirely sufficient for an analyst role and better than most candidates give.

14.5 Filtering early

The single most effective thing an analyst can do.

```
-- BAD: join everything, then filter
SELECT ... FROM orders o
JOIN order_items oi USING (order_id)
JOIN products p USING (product_id)
JOIN customers c USING (customer_id)
WHERE o.order_ts >= '2024-01-01';
```

```
-- BETTER: reduce the driving table first
WITH recent_orders AS (
  SELECT order_id, customer_id, order_ts
  FROM orders
  WHERE order_ts >= '2024-01-01' AND status='completed'
)
SELECT ... FROM recent_orders o
JOIN order_items oi USING (order_id)
...
```

In practice Postgres's planner usually pushes that filter down itself, so the two often produce the same plan. Where it genuinely matters: distributed engines (Spark, BigQuery, Redshift) with poorer push-down, CTEs marked **MATERIALIZED**, filters involving non-inlinable functions, and partitioned tables where an early filter enables partition pruning.

Partition pruning is worth knowing by name: if a table is partitioned by month, a `WHERE order_ts >= '2024-03-01'` filter lets the engine skip every other partition entirely. But only if the filter is directly on the partition key and isn't wrapped in a function.

14.6 Join optimisation

- **Join on indexed keys.** Foreign key columns usually want an index.
- **Reduce before you join.** Aggregate or filter a large table down before joining it to another large table.
- **Avoid joining on expressions.** `ON UPPER(a.code)=UPPER(b.code)` can't use ordinary indexes; normalise the data or add functional indexes to both sides.
- **Match types.** A join between `text` and `integer` may force casts that defeat indexes.
- **Watch fan-out.** The cheapest optimisation is often producing fewer rows: pre-aggregating one side from 10 million rows to 100 thousand changes everything downstream.
- **Join order** is the planner's job, and it will reorder inner joins freely. Don't try to outsmart it by rewriting; give it good statistics instead. (`LEFT JOIN` order is semantically fixed, so it has less freedom there — another reason not to use outer joins where inner ones suffice.)

14.7 Aggregation optimisation

- `COUNT(DISTINCT x)` is expensive — it must deduplicate. If exactness isn't needed at scale, approximate counting is available (`postgresql-hll`; `APPROX_COUNT_DISTINCT` in BigQuery/Snowflake).
- `GROUP BY` on many columns, or on wide text columns, forces large hash tables. Group by ids and join to labels afterwards.
- Filter before grouping (`WHERE`, not `HAVING`) whenever the condition is row-level.
- Repeated heavy aggregations belong in a materialised view or a scheduled summary table, not re-run per dashboard load:

```
CREATE MATERIALIZED VIEW mv_daily_sales AS
SELECT order_ts::date AS d, COUNT(*) AS orders, SUM(order_value) AS revenue
FROM order_values GROUP BY 1;

CREATE UNIQUE INDEX ON mv_daily_sales(d);
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_sales;
```

CONCURRENTLY keeps the view readable during the refresh and requires that unique index. That's a nicely specific detail to have.

14.8 CTE considerations

Covered in Part 10.4. The short version for an interview: in Postgres 12+, single-use CTEs are inlined; multiply-used ones are materialised. Use **MATERIALIZED** deliberately when a CTE is expensive and reused, **NOT MATERIALIZED** when you want filters pushed in. In older Postgres, CTEs are always an optimisation fence. In some cloud warehouses a repeated CTE is *recomputed* each reference, so a temp table may be cheaper.

14.9 Avoiding SELECT *

- Transfers columns you don't need, over the network and through memory.
- Prevents index-only scans (the index can't cover columns it doesn't contain).
- Breaks views and downstream code when the schema changes.
- Hides the query's real dependencies from anyone reading it.

On wide tables — event tables with 200 columns and JSON blobs are common — selecting five columns instead of all of them can be an order of magnitude faster.

14.10 Large table considerations

- **Always bound by time.** An unbounded `SELECT ... FROM events` on a billion-row table is how analysts get their warehouse access reviewed.
- **LIMIT while exploring**, but remember `LIMIT` with `ORDER BY` on an unindexed column still sorts everything first.
- **Sample** rather than scan when exploring distributions: `TABLESAMPLE SYSTEM (1)` reads roughly 1% of pages, very fast.
- **Understand the storage model.** Columnar stores (Redshift, BigQuery, Snowflake, Parquet) read only the columns you select, so `SELECT *` is disproportionately expensive there and column pruning matters more than row filtering.
- **Know your cost model.** BigQuery bills by bytes scanned: selecting fewer columns and partitioning by date directly reduces the bill. That's a very concrete answer if asked "how do you optimise in BigQuery?".
- **Set a statement timeout** in your session so a runaway query fails instead of running for six hours: `SET statement_timeout = '5min';`

14.11 A realistic optimisation walkthrough

Slow query:

```
SELECT c.first_name, c.last_name, COUNT(*) AS orders
FROM customers c, orders o, order_items oi
WHERE c.customer_id = o.customer_id
      AND o.order_id = oi.order_id
      AND DATE(o.order_ts) >= '2024-01-01'
      AND UPPER(c.country) = 'UK'
GROUP BY c.first_name, c.last_name;
```

Problems, in order of impact:

1. `DATE(o.order_ts)` wraps the indexed column — no index use on the largest table.
2. `UPPER(c.country)` likewise, though `customers` is small so it matters less.
3. Joining `order_items` at all — it's never selected from, it only multiplies rows and inflates `COUNT(*)`. The count is *wrong*, not just slow.
4. Grouping by name rather than id — two customers named Tom Brady merge, and text grouping is more expensive.
5. Implicit comma joins — legal, but they hide the join conditions among the filters.

Rewritten:

```
SELECT c.customer_id, c.first_name, c.last_name, COUNT(*) AS orders
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_ts >= DATE '2024-01-01'
      AND c.country = 'UK'
      AND o.status = 'completed'
GROUP BY c.customer_id, c.first_name, c.last_name;
```

Faster *and* correct. That's the point worth making in an interview: the optimisation and the bug fix were the same edit. Performance problems and correctness problems come from the same root — not thinking about grain.

PART 15 — DATABASE DESIGN FUNDAMENTALS

You won't design a production database as a junior analyst. You will be asked about normalisation and star schemas, because they explain why the tables you query look the way they do.

15.1 Normalisation

Organising data to eliminate redundancy and update anomalies.

Unnormalised:

order_id	customer	customer_email	products	total
1	Aisha Khan	aisha@ex.com	Mouse, Coffee	68.73

Three problems: the products column holds multiple values (can't query it); the customer's email is repeated on every order (update one, miss another, now they disagree); delete the only order for a customer and you lose their details entirely.

1NF — atomic values, no repeating groups. Each cell holds one value; each row is unique.

```
orders(order_id, customer_name, customer_email, order_date)
order_items(order_id, product_name, quantity, price)
```

2NF — 1NF plus no partial dependencies. Every non-key column depends on the *whole* primary key, not part of it. Only relevant with composite keys.

If `order_items` had key (`order_id`, `product_id`) and also carried `product_name`, that's a violation: `product_name` depends on `product_id` alone, not the pair. Move it to a `products` table.

3NF — 2NF plus no transitive dependencies. Non-key columns depend on the key and nothing but the key.

If `orders` carried `customer_name` and `customer_email`, those depend on `customer_id`, which depends on `order_id` — transitive. Move them to `customers`. Our RetailCo schema is in 3NF, which is why customer details live in exactly one place.

The shorthand that gets you through the question: *"every non-key attribute depends on the key, the whole key, and nothing but the key."*

Beyond 3NF: BCNF, 4NF, 5NF exist. Nobody will ask a junior analyst about them; knowing they exist and that 3NF is the practical stopping point for OLTP is enough.

15.2 Denormalisation

Deliberately reintroducing redundancy to make reads faster and simpler.

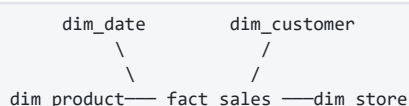
When it's right:

- Analytical warehouses where data is written once and read constantly.
- Avoiding a six-table join on every dashboard load.
- Storing a value that must not change with its source — `order_items.unit_price` is denormalised *on purpose*, because the price at the time of sale must survive a later price change. That's not a design flaw; it's a historical record.

Costs: more storage, and the risk of copies disagreeing. In a warehouse that's acceptable because the ETL is the single writer.

15.3 Star schema

The dominant analytical design. A central **fact** table surrounded by **dimension** tables.



```
CREATE TABLE fact_sales (
    sale_id          bigserial PRIMARY KEY,
```

```

date_key      integer REFERENCES dim_date(date_key),
customer_key  integer REFERENCES dim_customer(customer_key),
product_key   integer REFERENCES dim_product(product_key),
store_key     integer REFERENCES dim_store(store_key),
quantity      integer,                -- measures
gross_amount  numeric(12,2),
discount_amount numeric(12,2),
net_amount    numeric(12,2),
cost_amount   numeric(12,2)
);

CREATE TABLE dim_customer (
  customer_key serial PRIMARY KEY,    -- surrogate key
  customer_id  integer,              -- natural/business key
  full_name    text,
  country      text,
  segment      text,
  valid_from   date,                -- SCD Type 2
  valid_to     date,
  is_current   boolean
);

```

Fact tables hold measures (numeric, additive) and foreign keys to dimensions. Long and narrow — billions of rows, few columns. **Dimension tables** hold descriptive attributes you filter and group by. Short and wide — thousands of rows, many columns.

Why analysts should care: it makes queries uniform. Every question becomes "join fact to the dimensions I'm slicing by, filter, aggregate". No deep join chains, no guessing the path between tables.

Snowflake schema normalises the dimensions further (`dim_product` → `dim_category` → `dim_department`). It saves space and adds joins. Star is usually preferred for analytics precisely because the redundancy buys simplicity.

Grain is the first thing to define in a fact table: one row per what? Per order line, per order, per day per store? Every subsequent design decision follows from it, and mixing grains in one fact table is the classic warehouse design failure.

Additivity — worth knowing the vocabulary:

- *Additive* measures sum across every dimension (revenue, quantity).
- *Semi-additive* sum across some but not time (account balance, stock level — summing Monday's and Tuesday's stock is meaningless; you average or take the latest).
- *Non-additive* never sum (ratios, percentages, margin %) — you must recompute them from their components after aggregating.

That last point is a real and common analyst error: averaging a set of margin percentages gives a different, wrong answer versus recomputing total profit ÷ total revenue.

15.4 Slowly changing dimensions

A customer moves from Leeds to London. What happens to last year's orders?

- **Type 0** — never change it.
- **Type 1** — overwrite. History is lost; all past orders now appear to come from London.
- **Type 2** — add a new row with new validity dates, mark the old one not current. History preserved; the dimension grows. This is why you join on the key *and* the date range.
- **Type 3** — keep a `previous_country` column. Only remembers one change back.

Type 2 is the standard for anything where historical accuracy matters, and it's why `dim_customer` above has `valid_from/valid_to/is_current`.

```

-- as-at-the-time attributes
SELECT d.country, SUM(f.net_amount)
FROM fact_sales f
JOIN dim_date dt ON dt.date_key = f.date_key
JOIN dim_customer d
  ON d.customer_key = f.customer_key    -- surrogate key already points at the right version
GROUP BY d.country;

-- current attributes instead
JOIN dim_customer d ON d.customer_id = <business key> AND d.is_current

```

The distinction — "revenue by the country they lived in at the time" vs "by where they live now" — is a real business question with two different right answers, and knowing that the schema determines which you get is a strong thing to say.

15.5 Referential integrity

Foreign key constraints guarantee that references point at rows that exist. They also prevent deleting a parent that still has children, unless you specify `ON DELETE CASCADE` (delete the children too) or `ON DELETE SET NULL`.

In OLTP databases, constraints are enforced and you can trust them. In warehouses they're often declared but `NOT ENFORCED`, or absent entirely, because enforcement slows bulk loads. **So check for orphans yourself** before promising anyone that a join loses nothing:

```
SELECT COUNT(*) FROM fact_sales f
LEFT JOIN dim_customer d ON d.customer_key = f.customer_key
WHERE d.customer_key IS NULL;
```

A standard warehouse practice is an "unknown member" row (key = -1) in each dimension, so facts with a missing dimension reference still join and appear in reports as 'Unknown' rather than vanishing from an inner join. If you've seen that, say so — it's a detail that only comes from real exposure.

15.6 OLTP vs OLAP

	OLTP	OLAP
Purpose	run the business	analyse the business
Workload	many small reads/writes	few large reads
Design	normalised (3NF)	denormalised (star)
Grain	one row per transaction	fact rows plus aggregates
Indexes	many, for point lookups	fewer; partitioning, clustering, columnar
Storage	row-oriented	column-oriented
Typical query	"get order 1234"	"revenue by region by month for 3 years"
Examples	Postgres, MySQL, SQL Server	BigQuery, Snowflake, Redshift, Databricks

Why column storage matters to you. In a row store, reading one column still reads whole rows off disk. In a column store, `SELECT SUM(revenue)` reads only the revenue column — often 50× less I/O — and compresses far better because similar values sit together. This is why `SELECT *` is disproportionately punished in a warehouse, and why analysts get a columnar warehouse rather than access to the production OLTP database.

The other reason for the separation: analytical queries scanning millions of rows would lock up and slow down the database that's taking customer orders. Never run a heavy analytical query against a production OLTP system. Saying this out loud in an interview shows operational awareness.

15.7 How design affects your SQL

Design fact	Effect on your query
Normalised source	more joins; watch fan-out and grain
Star schema	join fact to dimensions; grain is fixed by the fact table
SCD Type 2 dimension	must join on key and validity window, or you duplicate rows
No enforced FKs	check for orphans before trusting an inner join
Denormalised price on the line	historical accuracy preserved — use it, don't join to current price
Partitioned by date	filter on the partition key directly, unwrapped, to get pruning
Columnar storage	select only the columns you need; row filtering matters less
Pre-aggregated summary tables	use them instead of re-aggregating the fact table

15.8 Design interview questions

1. *"What's the difference between OLTP and OLAP?"* — Table above; lead with purpose and workload, then design and storage.
2. *"Why is a star schema used for analytics rather than a fully normalised model?"* — Fewer joins, predictable query shape, better performance for scan-and-aggregate workloads; the redundancy is safe because ETL is the only writer.
3. *"What's the grain of a fact table and why does it matter?"* — What one row represents. It determines which measures are valid at that row, and mixing grains breaks every aggregate.
4. *"How would you handle a customer changing address?"* — Type 1 vs Type 2, and the business question of whether history matters.
5. *"When would you denormalise?"* — Read-heavy analytics, avoiding repeated expensive joins, and capturing point-in-time values that must not change.
6. *"What's 3NF in one sentence?"* — Every non-key attribute depends on the key, the whole key, and nothing but the key.
7. *"Why not put everything in one big table?"* — Update anomalies, storage, and the impossibility of maintaining consistency — then note that a wide denormalised table is exactly what many modern warehouses do build for consumption, because the trade-offs invert when nothing is being updated.

Part 16a — 50 Beginner SQL Interview Questions

All questions use the schemas in `00-index-and-schema.md`. Sample data and expected outputs refer to the seed rows there, so load them if you want to check your answers by running them.

Format. Every question gives: problem, solution, line-by-line explanation, common mistakes, and a follow-up. Questions marked * get the full treatment including sample data, expected output, alternative solution and performance notes — those are the ones most likely to be asked verbatim, so work through them properly.

Use it as drill material. Cover the solution. Write your answer. Then compare. Reading solutions builds recognition, not recall, and interviews test recall.

* 1. Select all customers from the UK

Problem. Return every customer based in the UK.

Sample data. 7 customers: 5 UK, 1 FR, 1 IE.

Expected output. 5 rows — Aisha, Tom, Priya, Jack, Nina.

Solution.

```
SELECT customer_id, first_name, last_name, city
FROM customers
WHERE country = 'UK';
```

Explanation. `SELECT` names the columns to return — listing them rather than `*` documents intent. `FROM customers` is the source. `WHERE country = 'UK'` keeps rows where the condition is true; string literals use single quotes.

Alternative. If the data isn't clean, `WHERE UPPER(TRIM(country)) = 'UK'` handles `'uk '` and `'UK'`. Costs the index unless you have a functional index on that expression.

Performance. With an index on `country` and UK being a minority, an index scan. If most customers are UK, the planner correctly picks a sequential scan.

Common mistakes. Double quotes around `'UK'` — in Postgres double quotes mean an identifier, so `"UK"` is read as a column name and errors. Assuming case-insensitive matching.

Follow-up. *"What if country is sometimes 'United Kingdom'?"* → Normalise with a CASE or a mapping table, and note that this belongs upstream in the pipeline, not in every analyst's query.

2. Count the total number of orders

```
SELECT COUNT(*) AS total_orders FROM orders;
```

Explanation. `COUNT(*)` counts rows including those with NULLs in any column. Output: 10. **Mistake.** `COUNT(discount_code)` would return 3 — it counts non-NULL values only. **Follow-up.** *"Now count only completed orders."* → add `WHERE status='completed'`.

3. List products priced over £30

```
SELECT product_name, unit_price FROM products
WHERE unit_price > 30 ORDER BY unit_price DESC;
```

Explanation. Numeric comparison, no quotes on the number. `ORDER BY DESC` puts the most expensive first. **Follow-up.** *"Include only active products."* → `AND is_active` — booleans need `no = true`.

4. Find the 3 most expensive products

```
SELECT product_name, unit_price FROM products
ORDER BY unit_price DESC LIMIT 3;
```

Mistake. `LIMIT 3` without `ORDER BY` returns three arbitrary rows. **Follow-up.** "What if two products tie for third?" → `LIMIT` cuts arbitrarily; use `DENSE_RANK() <= 3` to include ties.

5. List the distinct order statuses

```
SELECT DISTINCT status FROM orders;
```

Follow-up. "With a count of each." → `SELECT status, COUNT(*) FROM orders GROUP BY status;` — and note that `GROUP BY` is usually better than `DISTINCT` because you get the counts for free.

6. Customers who signed up in 2023

```
SELECT * FROM customers
WHERE signup_date >= DATE '2023-01-01' AND signup_date < DATE '2024-01-01';
```

Explanation. Half-open range. Safe whether the column is `date` or `timestamp`. **Mistake.** `BETWEEN '2023-01-01' AND '2023-12-31'` — correct for a `date` column, silently loses 31 December on a `timestamp` column. Build the habit now.

Follow-up. "Use `EXTRACT` instead." → `WHERE EXTRACT(YEAR FROM signup_date)=2023` — works, but wraps the column in a function and blocks index use.

7. Customers with no email

```
SELECT * FROM customers WHERE email IS NULL;
```

Mistake. `WHERE email = NULL` returns zero rows always — comparison with `NULL` yields `UNKNOWN`, never true. **Follow-up.** "Also catch blank strings." → `WHERE email IS NULL OR TRIM(email)=''`.

8. Orders sorted by date, newest first

```
SELECT order_id, customer_id, order_ts FROM orders ORDER BY order_ts DESC;
```

Follow-up. "Break ties consistently." → add `, order_id DESC`. Without a tie-breaker the order of tied rows isn't guaranteed between runs.

9. Count customers per country

```
SELECT country, COUNT(*) AS customers FROM customers GROUP BY country ORDER BY customers DESC;
```

Explanation. `GROUP BY` collapses rows into one per distinct country. Any non-aggregated `SELECT` column must be in the `GROUP BY`. **Mistake.** Selecting `city` without grouping by it → error in Postgres. **Follow-up.** "What happens to customers with a `NULL` country?" → They form their own group, shown as an empty/`NULL` row. Use `COALESCE(country, 'Unknown')` for a readable label.

* 10. Total revenue from completed orders

Problem. Total revenue, where revenue is quantity × price less discount, counting only completed orders.

Expected output. One row, one number.

Solution.

```
SELECT ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct)), 2) AS total_revenue
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
WHERE o.status = 'completed';
```

Explanation. Revenue lives on the line, so `order_items` drives. The join to `orders` exists solely to reach `status`. Multiplication happens per row, then `SUM` aggregates. `ROUND` to pennies for presentation.

Alternative. `WHERE oi.order_id IN (SELECT order_id FROM orders WHERE status='completed')` — same result, and a fine answer; the join is more conventional and lets you add more order-level filters easily.

Performance. Needs an index on `order_items.order_id` (the FK) for the join, and ideally on `orders.status` if completed orders are a minority.

Common mistakes. Using `orders` alone — it has no monetary column at all. Forgetting the status filter and including cancelled and refunded orders. Forgetting the discount. Using `SUM(quantity) * AVG(unit_price)`, which is wrong whenever prices differ.

Follow-up. "Break it down by month." → add `DATE_TRUNC('month', o.order_ts)::date` to SELECT and GROUP BY.

11. Average product price

```
SELECT ROUND(AVG(unit_price), 2) AS avg_price FROM products;
```

Follow-up. "By category." → `SELECT category, ROUND(AVG(unit_price), 2) FROM products GROUP BY category;`

12. Cheapest and most expensive product in one row

```
SELECT MIN(unit_price) AS cheapest, MAX(unit_price) AS dearest FROM products;
```

Follow-up. "Now show their names." → MIN/MAX give the values, not the rows. You need `ORDER BY ... LIMIT 1, DISTINCT ON`, or `ROW_NUMBER`. This is the greatest-per-group problem (12.2).

13. Orders placed in March 2024

```
SELECT * FROM orders
WHERE order_ts >= DATE '2024-03-01' AND order_ts < DATE '2024-04-01';
```

14. Products in Electronics or Home

```
SELECT * FROM products WHERE category IN ('Electronics', 'Home');
```

Follow-up. "Everything except those." → `NOT IN` here is safe only because `category` has no NULLs in the list; if the column itself is NULL for a row, that row is excluded from both. Use `WHERE category IS NULL OR category NOT IN (...)` if NULLs should be kept.

15. Customers whose surname starts with 'K'

```
SELECT * FROM customers WHERE last_name LIKE 'K%';
```

Follow-up. "Case-insensitively." → `ILIKE 'k%'` in Postgres; `LOWER(last_name) LIKE 'k%'` portably.

16. Order count by status

```
SELECT status, COUNT(*) AS orders FROM orders GROUP BY status ORDER BY orders DESC;
```

17. Customers who have placed at least one order

```
SELECT DISTINCT c.customer_id, c.first_name, c.last_name
FROM customers c JOIN orders o ON o.customer_id = c.customer_id;
```

Better.

```
SELECT * FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

Explanation. The join version needs DISTINCT because a customer with 3 orders appears 3 times. EXISTS can't duplicate rows, so no DISTINCT and no sort. **Follow-up.** "Which is faster?" → Usually EXISTS: it short-circuits at the first match and avoids the deduplication.

* 18. Customers who have never placed an order

Problem. List customers with no orders at all.

Expected output. Any customer_id absent from `orders`.

Solution.

```
SELECT c.customer_id, c.first_name, c.last_name
FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

Explanation. `NOT EXISTS` returns true when the correlated subquery finds nothing. The correlation `o.customer_id = c.customer_id` links it to the current outer row.

Alternatives.

```
-- anti-join
SELECT c.* FROM customers c
LEFT JOIN orders o ON o.customer_id=c.customer_id
WHERE o.order_id IS NULL;

-- NOT IN, only safe with the null guard
SELECT * FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders WHERE customer_id IS NOT NULL);
```

Performance. `NOT EXISTS` and the `LEFT JOIN` anti-join usually produce the same plan (a hash anti-join). `NOT IN` can be worse because the planner can't use an anti-join when NULLs are possible.

Common mistakes. Plain `NOT IN` without the NULL guard — returns **zero rows** if any `customer_id` in `orders` is NULL, silently. Testing `IS NULL` on a nullable column of the right table instead of its primary key.

Follow-up. "Explain the `NOT IN` trap." → `x NOT IN (1,2,NULL)` expands to `x<>1 AND x<>2 AND x<>NULL`; the last is UNKNOWN so the whole AND can never be true. This is one of the two or three most-asked SQL interview questions anywhere.

19. Number of orders per customer

```
SELECT customer_id, COUNT(*) AS orders FROM orders GROUP BY customer_id ORDER BY orders DESC;
```

Follow-up. "Include customers with zero." → `LEFT JOIN` from `customers` and `COUNT(o.order_id)`, not `COUNT(*)`.

20. Customers with more than one order

```
SELECT customer_id, COUNT(*) AS orders
FROM orders GROUP BY customer_id HAVING COUNT(*) > 1;
```

Mistake. `WHERE COUNT(*) > 1` — aggregates aren't allowed in `WHERE`, which runs before grouping. **Follow-up.** "WHERE vs HAVING?" → `WHERE` filters rows before grouping; `HAVING` filters groups after.

21. Each order with its customer's name

```
SELECT o.order_id, o.order_ts, c.first_name || ' ' || c.last_name AS customer
FROM orders o JOIN customers c ON c.customer_id = o.customer_id;
```

Mistake. `||` returns NULL if either name is NULL. Use `CONCAT_WS(' ', first_name, last_name)` when they're nullable.

22. Order lines with product names

```
SELECT oi.order_id, p.product_name, oi.quantity, oi.unit_price
FROM order_items oi JOIN products p ON p.product_id = oi.product_id;
```

23. Total quantity sold per product

```
SELECT p.product_name, SUM(oi.quantity) AS units_sold
FROM order_items oi JOIN products p USING (product_id)
GROUP BY p.product_name ORDER BY units_sold DESC;
```

Follow-up. "Only completed orders." → join `orders` and filter status. Without it you're counting cancelled orders as sales.

24. Products never ordered

```
SELECT p.* FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id);
```

25. Orders with no discount code

```
SELECT * FROM orders WHERE discount_code IS NULL;
```

26. Customers in Leeds or Manchester

```
SELECT * FROM customers WHERE city IN ('Leeds', 'Manchester');
```

27. Products between £20 and £50

```
SELECT * FROM products WHERE unit_price BETWEEN 20 AND 50;
```

Explanation. BETWEEN is inclusive at both ends. Safe here — the column is numeric, not a timestamp.

28. Count distinct customers who have ordered

```
SELECT COUNT(DISTINCT customer_id) AS ordering_customers FROM orders;
```

Follow-up. "Why not COUNT()?" → That counts orders, not customers. Distinct counts are the whole difference between "how much did we sell" and "how many people bought".

29. Full name and signup year

```
SELECT first_name || ' ' || last_name AS full_name,
       EXTRACT(YEAR FROM signup_date) AS signup_year
FROM customers;
```

30. * Average order value

Problem. Average total value of a completed order.

Solution.

```
WITH order_totals AS (
  SELECT o.order_id, SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS order_value
  FROM orders o JOIN order_items oi USING (order_id)
  WHERE o.status='completed'
  GROUP BY o.order_id
)
SELECT ROUND(AVG(order_value), 2) AS average_order_value,
       COUNT(*) AS orders
FROM order_totals;
```

Explanation. Two-step by necessity. Step one aggregates lines to order grain. Step two averages across orders. You cannot nest aggregates (`AVG(SUM(x))` is illegal outside a window context), so the CTE is doing structural work, not just cosmetics.

Alternative. `SELECT SUM(line_value)/COUNT(DISTINCT order_id) FROM ...` in one pass — mathematically identical, arguably harder to read, but a legitimate answer worth mentioning.

Common mistakes. `AVG(quantity*unit_price)` on the joined table gives average *line* value — a different, smaller number. This is the single most common seeded error in AOV questions.

Follow-up. "Report the median too." → `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY order_value)`. Order values are skewed; the mean alone misleads.

31. Orders in the last 30 days

```
SELECT * FROM orders WHERE order_ts >= CURRENT_DATE - INTERVAL '30 days';
```

Follow-up. "Why might this be a problem in a saved report?" → It's relative to today, so re-running it later gives different numbers and you can't reconcile against last week's output. Parameterise the as-at date.

32. Employees and their managers

```
SELECT e.full_name AS employee, m.full_name AS manager
FROM employees e LEFT JOIN employees m ON m.employee_id = e.manager_id;
```

Mistake. INNER JOIN drops the CEO (NULL `manager_id`).

33. Departments with more than 5 employees

```
SELECT department, COUNT(*) AS headcount
FROM employees GROUP BY department HAVING COUNT(*) > 5;
```

34. Highest-paid employee per department

```
SELECT DISTINCT ON (department) department, full_name, salary
FROM employees ORDER BY department, salary DESC, employee_id;
```

Portable version.

```
SELECT * FROM (
  SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC, employee_id) rn
  FROM employees e) t WHERE rn = 1;
```

35. Customers by signup date, most recent first, top 5

```
SELECT * FROM customers ORDER BY signup_date DESC, customer_id DESC LIMIT 5;
```

36. Products with 'Wireless' in the name

```
SELECT * FROM products WHERE product_name ILIKE '%wireless%';
```

Note. The leading wildcard prevents index use. Fine on a small products table; on millions of rows you'd want a trigram index or full-text search.

37. Total shipping cost collected

```
SELECT ROUND(SUM(shipping_cost),2) FROM orders WHERE status='completed';
```

Mistake. Joining `order_items` first — shipping is order-level and would be counted once per line.

38. Orders per month in 2024

```
SELECT DATE_TRUNC('month', order_ts)::date AS month, COUNT(*) AS orders
FROM orders
WHERE order_ts >= DATE '2024-01-01' AND order_ts < DATE '2025-01-01'
GROUP BY 1 ORDER BY 1;
```

Follow-up. "Why `DATE_TRUNC` and not `EXTRACT(MONTH)`?" → `EXTRACT` would merge March 2023 and March 2024 into one row.

39. Customers who opted into marketing

```
SELECT * FROM customers WHERE marketing_opt_in;
```

Follow-up. "What about NULLs?" → Excluded, same as false. If NULL means "not yet asked", use `IS NOT FALSE` or handle it explicitly.

40. Number of products per category

```
SELECT category, COUNT(*) AS products FROM products GROUP BY category ORDER BY products DESC;
```

41. First and last order date overall

```
SELECT MIN(order_ts)::date AS first_order, MAX(order_ts)::date AS last_order FROM orders;
```

42. Line value for each order item

```
SELECT order_item_id, order_id, quantity, unit_price, discount_pct,  
       ROUND(quantity * unit_price * (1 - discount_pct), 2) AS line_value  
FROM order_items;
```

Mistake. `WHERE line_value > 50` fails — the alias doesn't exist at WHERE time. Repeat the expression or wrap in a subquery.

43. Orders not placed via the web

```
SELECT * FROM orders WHERE channel <> 'web';
```

Follow-up. "What if channel can be NULL?" → Those rows are excluded. Use `IS DISTINCT FROM 'web'` to include them.

44. Products by category then price

```
SELECT category, product_name, unit_price  
FROM products ORDER BY category ASC, unit_price DESC;
```

45. Count patients by sex, handling missing values

```
SELECT COALESCE(NULLIF(TRIM(sex), ''), 'Not recorded') AS sex, COUNT(*)  
FROM patients GROUP BY 1 ORDER BY 2 DESC;
```

46. Referrals made in the last 90 days

```
SELECT * FROM referrals WHERE referral_date >= CURRENT_DATE - INTERVAL '90 days';
```

47. Appointments that were not attended

```
SELECT * FROM appointments WHERE outcome = 'DNA';
```

Follow-up. "Is 'not attended' the same as DNA?" → No. Cancellations by the patient or the provider are also non-attendances but are reported separately, and lumping them together inflates the DNA rate. Ask before you compute.

48. Patients still on the waiting list

```
SELECT * FROM waiting_list WHERE removed_date IS NULL;
```

49. A&E attendances lasting over four hours

```
SELECT * FROM ae_attendances  
WHERE departure_ts IS NOT NULL  
AND departure_ts - arrival_ts > INTERVAL '4 hours';
```

Explanation. The `IS NOT NULL` guard is optional for correctness (a NULL comparison is UNKNOWN and excluded anyway) but stating it makes the intent explicit and forces you to think about the still-present patients. **Follow-up.** "What about patients still in the department who've already been there 6 hours?" → They've breached but aren't counted. Use `COALESCE(departure_ts, CURRENT_TIMESTAMP)`.

* 50. Revenue by product category

Problem. Total revenue by category, completed orders only, highest first.

Expected output. One row per category with a rounded revenue figure.

Solution.

```
SELECT p.category,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct)), 2) AS revenue,
       COUNT(DISTINCT o.order_id) AS orders,
       SUM(oi.quantity) AS units
FROM   order_items oi
JOIN   products p ON p.product_id = oi.product_id
JOIN   orders o ON o.order_id = oi.order_id
WHERE  o.status = 'completed'
GROUP BY p.category
ORDER BY revenue DESC;
```

Explanation. Three tables: lines carry the money and quantity, products carry the category, orders carry the status.

`COUNT(DISTINCT o.order_id)` because one order can contribute several lines within a category. Grouping happens after all joins and the WHERE filter.

Alternative. Add `ROUND(100.0*SUM(...)/SUM(SUM(...)) OVER (),1)` for share of total — a natural extension the interviewer may ask for next, so have it ready.

Performance. Indexes on `order_items.product_id`, `order_items.order_id`, and `orders.status` if completed is selective. Three-way hash join then a hash aggregate.

Common mistakes. `COUNT(*)` for orders (counts lines). Omitting the status filter. Omitting the discount. Joining products to orders directly — there is no relationship between them; the path runs through `order_items`.

Follow-up. *"Which category has the best margin?"* → Bring in `p.unit_cost` and compute gross profit and margin percentage — see Part 13.2, including the caveat that `unit_cost` is current cost rather than cost at time of sale.

Self-check

You should be able to write questions 1–50 unaided in under three minutes each. If any of these still make you hesitate, that's your revision list:

- WHERE vs HAVING (20)
- COUNT(*) vs COUNT(col) (2, 19)
- NOT IN with NULLs (18)
- LEFT JOIN then COUNT (19, 32)
- Grain and fan-out (10, 37, 50)
- Alias visibility in WHERE (42)
- Half-open date ranges (6, 13, 38)

Part 16b — 75 Intermediate SQL Interview Questions

Same schemas as before. These are the questions that decide most Data Analyst interviews — joins with grain traps, window functions, date logic, and metrics with ambiguous definitions.

* = fuller treatment (sample data, expected output, alternatives, performance).

* 1. Revenue per customer including customers who never bought

Problem. Every customer with their total completed revenue; zero if they never bought.

Expected output. All 7 customers. Customers 3 (refunded only), 5 (pending only) and 6 (cancelled only) show 0.00 — that's the point of the question.

Solution.

```
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer,
       COALESCE(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct)), 0) AS revenue
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.customer_id
  AND o.status = 'completed'
LEFT JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY revenue DESC;
```

Explanation. The status filter sits in the **ON clause**, not WHERE. In WHERE it would eliminate the NULL rows the LEFT JOIN created, converting it into an inner join and dropping customers 3, 5, 6 entirely. `COALESCE(SUM(...), 0)` turns the NULL sum for non-buyers into 0.

Alternative. Pre-aggregate then LEFT JOIN — often clearer and avoids double LEFT JOIN chains:

```
WITH rev AS (
  SELECT o.customer_id, SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS revenue
  FROM orders o JOIN order_items oi USING (order_id)
  WHERE o.status='completed' GROUP BY o.customer_id
  SELECT c.customer_id, COALESCE(r.revenue,0) AS revenue
  FROM customers c LEFT JOIN rev r USING (customer_id);
```

Performance. The alternative is usually faster: it aggregates the large table first and joins a small result to a small table.

Common mistakes. Status filter in WHERE (the big one). `COUNT(*)` instead of `COUNT(o.order_id)` if counting orders. Forgetting COALESCE and shipping NULLs to a dashboard.

Follow-up. "How would you tell 'never ordered' apart from 'ordered but nothing completed'?" → Add `COUNT(o_all.order_id)` from a second LEFT JOIN without the status filter, or a boolean `EXISTS` flag.

2. Customers with more than 2 completed orders

```
SELECT customer_id, COUNT(*) AS completed_orders
FROM orders WHERE status='completed'
GROUP BY customer_id HAVING COUNT(*) > 2;
```

Explanation. Row filter in WHERE, group filter in HAVING. Both are needed and choosing correctly is the test.

3. Second-highest product price

```
SELECT MAX(unit_price) FROM products
WHERE unit_price < (SELECT MAX(unit_price) FROM products);
```

Alternatives. `ORDER BY unit_price DESC LIMIT 1 OFFSET 1`, or `DENSE_RANK() = 2`. **Follow-up.** "What if the top two prices are equal?" → The subquery version and `DENSE_RANK` return the true second distinct price; `OFFSET` returns the duplicate top price. Say which the business wants.

4. Month-on-month revenue growth

```
WITH monthly AS (  
  SELECT DATE_TRUNC('month', o.order_ts)::date AS month,  
         SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS revenue  
  FROM orders o JOIN order_items oi USING (order_id)  
 WHERE o.status='completed' GROUP BY 1)  
SELECT month, ROUND(revenue,2) AS revenue,  
       ROUND(LAG(revenue) OVER (ORDER BY month),2) AS prev_month,  
       ROUND(100.0*(revenue-LAG(revenue) OVER (ORDER BY month))  
             /NULLIF(LAG(revenue) OVER (ORDER BY month),0),1) AS mom_pct  
FROM monthly ORDER BY month;
```

Mistake. Missing months break LAG — it compares to the previous *row*, not the previous month. Zero-fill with `generate_series` when gaps are possible.

5. Top 3 products by revenue in each category

```
WITH pr AS (  
  SELECT p.category, p.product_name,  
         SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS revenue  
  FROM order_items oi JOIN products p USING (product_id)  
  JOIN orders o ON o.order_id=oi.order_id AND o.status='completed'  
  GROUP BY 1,2)  
SELECT * FROM (SELECT *, DENSE_RANK() OVER (PARTITION BY category ORDER BY revenue DESC) r  
              FROM pr) t  
WHERE r <= 3 ORDER BY category, r;
```

Follow-up. "ROW_NUMBER or DENSE_RANK?" → ROW_NUMBER gives exactly 3 and cuts ties arbitrarily; DENSE_RANK keeps all products at the top 3 revenue levels. Ask which is wanted.

6. Each customer's most recent order

```
SELECT DISTINCT ON (customer_id) customer_id, order_id, order_ts, channel  
FROM orders WHERE status='completed'  
ORDER BY customer_id, order_ts DESC, order_id DESC;
```

7. Running total of daily revenue

```
WITH daily AS (  
  SELECT o.order_ts::date AS d, SUM(oi.quantity*oi.unit_price) AS revenue  
  FROM orders o JOIN order_items oi USING (order_id) WHERE o.status='completed' GROUP BY 1)  
SELECT d, ROUND(revenue,2) AS revenue,  
       ROUND(SUM(revenue) OVER (ORDER BY d ROWS UNBOUNDED PRECEDING),2) AS cumulative  
FROM daily ORDER BY d;
```

8. Customers who bought Electronics but never Home

```
SELECT DISTINCT o.customer_id FROM orders o  
JOIN order_items oi USING (order_id) JOIN products p USING (product_id)  
WHERE p.category='Electronics' AND o.status='completed'  
AND NOT EXISTS (  
  SELECT 1 FROM orders o2 JOIN order_items oi2 USING (order_id) JOIN products p2 USING (product_id)  
  WHERE o2.customer_id=o.customer_id AND o2.status='completed' AND p2.category='Home');
```

9. Percentage of orders by channel

```
SELECT channel, COUNT(*) AS orders,  
       ROUND(100.0*COUNT(*)/SUM(COUNT(*) OVER ()),1) AS pct  
FROM orders GROUP BY channel ORDER BY orders DESC;
```

Explanation. `SUM(COUNT(*)) OVER ()` — the window runs after grouping, summing the group counts. Avoids a subquery entirely.

10. * Average order value by month with order count

Solution.

```

WITH order_totals AS (
  SELECT o.order_id,
         DATE_TRUNC('month', o.order_ts)::date AS month,
         SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS order_value
  FROM orders o JOIN order_items oi USING (order_id)
  WHERE o.status='completed'
  GROUP BY o.order_id, 2
)
SELECT month,
       COUNT(*) AS orders,
       ROUND(AVG(order_value),2) AS aov,
       ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY order_value)::numeric,2) AS median,
       ROUND(SUM(order_value),2) AS revenue
FROM order_totals GROUP BY month ORDER BY month;

```

Explanation. Aggregate to order grain first, then to month. The month must be in the first GROUP BY so it survives to the second stage. **Performance.** One scan of the join, two hash aggregates. Fine to tens of millions of rows. **Mistake.** Averaging line values. Also: **AVG** of already-averaged values (averaging monthly AOVs to get a yearly AOV) is wrong unless the months have equal order counts — a genuinely common reporting error. **Follow-up.** "AOV rose but revenue fell — explain." → Fewer, larger orders. Decompose: revenue = orders × AOV, and check which factor moved.

11. Customers whose spend is above average

```

WITH spend AS (
  SELECT o.customer_id, SUM(oi.quantity*oi.unit_price) AS total
  FROM orders o JOIN order_items oi USING (order_id) WHERE o.status='completed'
  GROUP BY o.customer_id)
SELECT * FROM spend WHERE total > (SELECT AVG(total) FROM spend) ORDER BY total DESC;

```

12. Days between each customer's consecutive orders

```

SELECT customer_id, order_ts::date AS order_date,
       order_ts::date - LAG(order_ts::date) OVER (PARTITION BY customer_id ORDER BY order_ts) AS days_since_prev
FROM orders WHERE status='completed' ORDER BY customer_id, order_ts;

```

13. Products bought together most often

```

SELECT p1.product_name AS a, p2.product_name AS b, COUNT(*) AS times
FROM order_items i1
JOIN order_items i2 ON i2.order_id=i1.order_id AND i2.product_id > i1.product_id
JOIN products p1 ON p1.product_id=i1.product_id
JOIN products p2 ON p2.product_id=i2.product_id
GROUP BY 1,2 ORDER BY times DESC LIMIT 10;

```

Explanation. > on product_id prevents self-pairs and stops each pair appearing twice.

14. Revenue and shipping by month, both correct

```

WITH li AS (SELECT order_id, SUM(quantity*unit_price*(1-discount_pct)) AS revenue
            FROM order_items GROUP BY order_id)
SELECT DATE_TRUNC('month',o.order_ts)::date AS month,
       ROUND(SUM(li.revenue),2) AS revenue, ROUND(SUM(o.shipping_cost),2) AS shipping
FROM orders o JOIN li USING (order_id) WHERE o.status='completed'
GROUP BY 1 ORDER BY 1;

```

Explanation. The grain fix. Joining `order_items` directly would multiply shipping by the line count.

15. Customers who ordered in consecutive months

```

WITH m AS (SELECT DISTINCT customer_id, DATE_TRUNC('month',order_ts)::date AS month
            FROM orders WHERE status='completed')
SELECT DISTINCT customer_id FROM (
  SELECT customer_id, month, LAG(month) OVER (PARTITION BY customer_id ORDER BY month) AS prev
  FROM m) t
WHERE month - INTERVAL '1 month' = prev;

```

16. Each order's value as a share of its customer's total

```
SELECT order_id, customer_id, ROUND(order_value,2) AS order_value,
       ROUND(100.0*order_value/SUM(order_value) OVER (PARTITION BY customer_id),1) AS pct_of_customer
FROM order_values;
```

17. Rank customers by spend within their country

```
WITH spend AS (
  SELECT c.customer_id, c.country, SUM(oi.quantity*oi.unit_price) AS total
  FROM customers c JOIN orders o ON o.customer_id=c.customer_id AND o.status='completed'
  JOIN order_items oi USING (order_id) GROUP BY 1,2)
SELECT *, RANK() OVER (PARTITION BY country ORDER BY total DESC) AS country_rank,
       RANK() OVER (ORDER BY total DESC) AS overall_rank
FROM spend;
```

18. 7-day moving average of orders

```
WITH d AS (
  SELECT g::date AS day, COUNT(o.order_id) AS orders
  FROM generate_series(CURRENT_DATE-89, CURRENT_DATE, INTERVAL '1 day') g
  LEFT JOIN orders o ON o.order_ts::date=g::date AND o.status='completed'
  GROUP BY 1)
SELECT day, orders,
       ROUND(AVG(orders) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW),1) AS ma7
FROM d ORDER BY day;
```

Explanation. The generate_series zero-fill is load-bearing — without it, ROWS 6 PRECEDING reaches back further than seven calendar days.

19. Customers whose latest order was smaller than their previous one

```
WITH seq AS (
  SELECT customer_id, order_id, order_value, order_ts,
         LAG(order_value) OVER (PARTITION BY customer_id ORDER BY order_ts) AS prev,
         ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts DESC) AS rn
  FROM order_values)
SELECT customer_id, order_value, prev FROM seq
WHERE rn=1 AND prev IS NOT NULL AND order_value < prev;
```

20. Monthly new vs returning customers

```
WITH first_order AS (
  SELECT customer_id, DATE_TRUNC('month',MIN(order_ts))::date AS first_month
  FROM orders WHERE status='completed' GROUP BY customer_id),
activity AS (
  SELECT DISTINCT customer_id, DATE_TRUNC('month',order_ts)::date AS month
  FROM orders WHERE status='completed')
SELECT a.month,
       COUNT(*) FILTER (WHERE a.month = f.first_month) AS new_customers,
       COUNT(*) FILTER (WHERE a.month > f.first_month) AS returning_customers
FROM activity a JOIN first_order f USING (customer_id)
GROUP BY a.month ORDER BY a.month;
```

* 21. Monthly retention rate

Problem. Of customers active in a month, what proportion are active the following month?

Solution.

```
WITH monthly_active AS (
  SELECT DISTINCT customer_id, DATE_TRUNC('month', order_ts)::date AS month
  FROM orders WHERE status = 'completed'
)
SELECT curr.month,
       COUNT(DISTINCT curr.customer_id) AS active,
       COUNT(DISTINCT nxt.customer_id) AS retained,
       ROUND(100.0*COUNT(DISTINCT nxt.customer_id)
            / NULLIF(COUNT(DISTINCT curr.customer_id),0), 1) AS retention_pct
FROM monthly_active curr
LEFT JOIN monthly_active nxt
  ON nxt.customer_id = curr.customer_id
  AND nxt.month = curr.month + INTERVAL '1 month'
```

```
GROUP BY curr.month ORDER BY curr.month;
```

Explanation. Self-join offset by a month. **DISTINCT** in the CTE reduces multiple orders per month to one row. **LEFT JOIN** keeps non-returns in the denominator.

Alternative. **EXISTS** instead of the self-join, which avoids any risk of fan-out:

```
SELECT month, COUNT(*) AS active,
       COUNT(*) FILTER (WHERE EXISTS (
         SELECT 1 FROM monthly_active n
         WHERE n.customer_id=m.customer_id AND n.month=m.month+INTERVAL '1 month')) AS retained
FROM monthly_active m GROUP BY month;
```

Common mistakes. **INNER JOIN** → always 100%. Forgetting the final month is structurally incomplete (no "next month" exists yet) and reporting it as 0% retention.

Follow-up. "Why does the last month show 0%?" → Censoring. Exclude incomplete periods from the output.

22. Cohort retention by first-purchase month

See Part 12.9 for the full query. Expect this as a whiteboard question; practise saying the three steps aloud before writing: assign cohort, list activity months, compute month offset.

23. Customers who spent more than £500 lifetime

```
SELECT o.customer_id, ROUND(SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)),2) AS ltv
FROM orders o JOIN order_items oi USING (order_id) WHERE o.status='completed'
GROUP BY o.customer_id
HAVING SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) > 500;
```

24. Gross margin by category

```
SELECT p.category,
       ROUND(SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)),2) AS revenue,
       ROUND(SUM(oi.quantity*p.unit_cost),2) AS cogs,
       ROUND(100.0*SUM(oi.quantity*(oi.unit_price*(1-oi.discount_pct)-p.unit_cost))
            /NULLIF(SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)),0),1) AS margin_pct
FROM order_items oi JOIN products p USING (product_id)
JOIN orders o ON o.order_id=oi.order_id AND o.status='completed'
GROUP BY p.category ORDER BY margin_pct DESC;
```

Follow-up. "Any concern with this?" → **unit_cost** is current, **unit_price** is historic. Margins on old orders are computed against today's costs.

25. Products whose price is above their category average

```
SELECT * FROM (
  SELECT p.*, AVG(unit_price) OVER (PARTITION BY category) AS cat_avg FROM products p) t
WHERE unit_price > cat_avg;
```

26. Orders containing more than 2 distinct products

```
SELECT order_id, COUNT(DISTINCT product_id) AS products
FROM order_items GROUP BY order_id HAVING COUNT(DISTINCT product_id) > 2;
```

27. Employees earning more than their department average

```
SELECT * FROM (
  SELECT e.*, AVG(salary) OVER (PARTITION BY department) AS dept_avg FROM employees e) t
WHERE salary > dept_avg;
```

28. Employees earning more than their manager

```
SELECT e.full_name, e.salary, m.full_name AS manager, m.salary AS manager_salary
FROM employees e JOIN employees m ON m.employee_id=e.manager_id
WHERE e.salary > m.salary;
```

29. Duplicate customer records by email

```
SELECT LOWER(TRIM(email)) AS email, COUNT(*), ARRAY_AGG(customer_id) AS ids
FROM customers WHERE email IS NOT NULL
GROUP BY 1 HAVING COUNT(*) > 1;
```

30. * Deduplicate keeping the most recent record

Problem. A staging table has multiple rows per customer. Keep only the latest.

Solution.

```
WITH ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id
                                ORDER BY updated_at DESC NULLS LAST, id DESC) AS rn
  FROM customers_staging
)
SELECT * FROM ranked WHERE rn = 1;
```

Explanation. PARTITION BY names the key that should be unique. The ORDER BY encodes the business rule for which copy wins. NULLS LAST stops a record with a missing timestamp winning. id DESC makes it deterministic.

Alternatives. DISTINCT ON (customer_id) in Postgres. DELETE where rn > 1 to fix in place.

Common mistakes. RANK instead of ROW_NUMBER — ties keep all rows and you still have duplicates. No tie-breaker — non-reproducible output, which breaks pipelines subtly and is very hard to debug later.

Follow-up. "What if duplicates hold complementary data?" → Merge rather than dedupe: GROUP BY customer_id with MAX(col) FILTER (WHERE col IS NOT NULL) per column.

31. Customers by acquisition channel with conversion rate

```
SELECT c.channel,
       COUNT(*) AS customers,
       COUNT(*) FILTER (WHERE EXISTS (SELECT 1 FROM orders o
                                       WHERE o.customer_id=c.customer_id AND o.status='completed')) AS converted,
       ROUND(100.0*COUNT(*) FILTER (WHERE EXISTS (SELECT 1 FROM orders o
                                       WHERE o.customer_id=c.customer_id AND o.status='completed'))/COUNT(*),1) AS conv_pct
FROM customers c GROUP BY c.channel ORDER BY conv_pct DESC;
```

32. Days from signup to first order

```
SELECT c.customer_id, c.signup_date, MIN(o.order_ts)::date AS first_order,
       MIN(o.order_ts)::date - c.signup_date AS days_to_activate
FROM customers c LEFT JOIN orders o ON o.customer_id=c.customer_id AND o.status='completed'
GROUP BY c.customer_id, c.signup_date;
```

33. Revenue by day of week

```
SELECT TRIM(TO_CHAR(o.order_ts,'Day')) AS day_name, EXTRACT(ISODOW FROM o.order_ts) AS dow,
       ROUND(SUM(oi.quantity*oi.unit_price),2) AS revenue
FROM orders o JOIN order_items oi USING (order_id) WHERE o.status='completed'
GROUP BY 1,2 ORDER BY 2;
```

34. Orders per customer distribution

```
SELECT orders_placed, COUNT(*) AS customers
FROM (SELECT customer_id, COUNT(*) AS orders_placed FROM orders
      WHERE status='completed' GROUP BY customer_id) t
GROUP BY orders_placed ORDER BY orders_placed;
```

35. Refund rate by category

```
SELECT p.category,
       COUNT(DISTINCT o.order_id) AS orders,
       COUNT(DISTINCT o.order_id) FILTER (WHERE o.status='refunded') AS refunded,
```

```
ROUND(100.0*COUNT(DISTINCT o.order_id) FILTER (WHERE o.status='refunded')
/NULLIF(COUNT(DISTINCT o.order_id),0),1) AS refund_rate_pct
FROM orders o JOIN order_items oi USING (order_id) JOIN products p USING (product_id)
WHERE o.status IN ('completed','refunded')
GROUP BY p.category ORDER BY refund_rate_pct DESC;
```

36. Top customer in each country

```
SELECT DISTINCT ON (country) country, customer_id, total
FROM (SELECT c.country, c.customer_id, SUM(oi.quantity*oi.unit_price) AS total
FROM customers c JOIN orders o ON o.customer_id=c.customer_id AND o.status='completed'
JOIN order_items oi USING (order_id) GROUP BY 1,2) t
ORDER BY country, total DESC, customer_id;
```

37. Cumulative revenue share by product (Pareto)

See Part 12.20. The key detail is the WHERE condition using the cumulative sum *excluding* the current row so the crossing row is included.

38. Customers active in 2023 but not 2024

```
SELECT customer_id FROM orders WHERE status='completed'
GROUP BY customer_id
HAVING COUNT(*) FILTER (WHERE order_ts >= '2023-01-01' AND order_ts < '2024-01-01') > 0
AND COUNT(*) FILTER (WHERE order_ts >= '2024-01-01') = 0;
```

39. Average basket size (items per order)

```
SELECT ROUND(AVG(items),2) AS avg_items_per_order
FROM (SELECT o.order_id, SUM(oi.quantity) AS items
FROM orders o JOIN order_items oi USING (order_id) WHERE o.status='completed'
GROUP BY o.order_id) t;
```

40. * Funnel conversion by step

Problem. Conversion from product view → add to cart → checkout → purchase, by session.

Solution.

```
WITH session_steps AS (
SELECT session_id,
MAX(CASE WHEN event_name='product_view' THEN 1 ELSE 0 END) AS viewed,
MAX(CASE WHEN event_name='add_to_cart' THEN 1 ELSE 0 END) AS carted,
MAX(CASE WHEN event_name='checkout_start' THEN 1 ELSE 0 END) AS checkout,
MAX(CASE WHEN event_name='purchase' THEN 1 ELSE 0 END) AS purchased
FROM web_events
WHERE event_ts >= CURRENT_DATE - 30
GROUP BY session_id
)
SELECT SUM(viewed) AS viewed, SUM(carted) AS carted,
SUM(checkout) AS checkout, SUM(purchased) AS purchased,
ROUND(100.0*SUM(carted)/NULLIF(SUM(viewed),0),1) AS view_to_cart_pct,
ROUND(100.0*SUM(checkout)/NULLIF(SUM(carted),0),1) AS cart_to_checkout_pct,
ROUND(100.0*SUM(purchased)/NULLIF(SUM(checkout),0),1) AS checkout_to_purchase_pct,
ROUND(100.0*SUM(purchased)/NULLIF(SUM(viewed),0),1) AS overall_pct
FROM session_steps;
```

Explanation. `MAX(CASE ...)` collapses many events per session into a 0/1 flag per step. Summing flags counts sessions reaching each step. Step-to-step percentages find the drop-off; the overall percentage alone hides where the problem is.

Alternative. Postgres `FILTER`, or timestamp comparison for a strictly ordered funnel (12.12).

Common mistakes. `COUNT(*)` per event name without collapsing to session grain — a user viewing 10 products counts 10 times. Assuming the funnel is ordered when this query doesn't enforce order.

Follow-up. "Split by device." → Add `device` to both GROUP BYs. "A user adds to cart on mobile and buys on desktop — what breaks?" → Session-level funnels miss cross-device journeys; switch to customer grain with an attribution window.

41. Sessionise events with a 30-minute timeout

See Part 12.17.

42. Customers who bought in 3 consecutive months

See Part 12.18.

43. Longest streak of consecutive active days

See Part 12.4.

44. Revenue by UK financial year

```
SELECT EXTRACT(YEAR FROM o.order_ts - INTERVAL '3 months')::int AS fy_starting,
       ROUND(SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)),2) AS revenue
FROM orders o JOIN order_items oi USING (order_id) WHERE o.status='completed'
GROUP BY 1 ORDER BY 1;
```

45. Year-on-year monthly comparison

```
WITH m AS (SELECT DATE_TRUNC('month',order_ts)::date AS month, COUNT(*) AS orders
            FROM orders WHERE status='completed' GROUP BY 1)
SELECT c.month, c.orders, p.orders AS last_year,
       ROUND(100.0*(c.orders-p.orders)/NULLIF(p.orders,0),1) AS yoy_pct
FROM m c LEFT JOIN m p ON p.month = c.month - INTERVAL '1 year' ORDER BY c.month;
```

Follow-up. "Why join rather than LAG(12)?" → LAG counts rows; a missing month shifts every subsequent comparison.

46. Fill in months with no sales

```
SELECT g::date AS month, COALESCE(ROUND(SUM(oi.quantity*oi.unit_price),2),0) AS revenue
FROM generate_series(DATE '2024-01-01', DATE '2024-12-01', INTERVAL '1 month') g
LEFT JOIN orders o ON DATE_TRUNC('month',o.order_ts)=g AND o.status='completed'
LEFT JOIN order_items oi ON oi.order_id=o.order_id
GROUP BY 1 ORDER BY 1;
```

47. Customers whose only orders were cancelled or refunded

```
SELECT customer_id FROM orders GROUP BY customer_id
HAVING COUNT(*) FILTER (WHERE status='completed') = 0;
```

48. Median order value

```
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY order_value) AS median FROM order_values;
```

Follow-up. "Without PERCENTILE_CONT?" → Rank rows and take the middle one(s):

```
SELECT AVG(order_value) FROM (
  SELECT order_value, ROW_NUMBER() OVER (ORDER BY order_value) rn, COUNT(*) OVER () n
  FROM order_values) t
WHERE rn IN ((n+1)/2, (n+2)/2);
```

49. Percentile bands of customer value

```
SELECT customer_id, lifetime_value,
       NTILE(10) OVER (ORDER BY lifetime_value DESC) AS decile,
       ROUND(100*PERCENT_RANK() OVER (ORDER BY lifetime_value DESC),1) AS pct_rank
FROM customer_ltv;
```

50. * Customers at risk of churn

Problem. Flag customers whose silence exceeds their own typical purchase gap.

Solution.

```
WITH gaps AS (
  SELECT customer_id, order_ts::date AS d,
         order_ts::date - LAG(order_ts::date) OVER (PARTITION BY customer_id ORDER BY order_ts) AS gap
  FROM orders WHERE status='completed'
),
cadence AS (
  SELECT customer_id, AVG(gap) AS avg_gap, COUNT(*)+1 AS orders, MAX(d) AS last_order
  FROM gaps WHERE gap IS NOT NULL GROUP BY customer_id
)
SELECT customer_id, orders, last_order, ROUND(avg_gap,0) AS typical_gap_days,
       CURRENT_DATE - last_order AS days_silent,
       ROUND((CURRENT_DATE - last_order)/NULLIF(avg_gap,0),1) AS gap_ratio,
       CASE WHEN CURRENT_DATE - last_order > 3*avg_gap THEN 'High risk'
            WHEN CURRENT_DATE - last_order > 2*avg_gap THEN 'Medium risk'
            ELSE 'Healthy' END AS risk
FROM cadence
WHERE CURRENT_DATE - last_order > 2*avg_gap
ORDER BY gap_ratio DESC;
```

Explanation. Each customer is compared to their own cadence rather than a global threshold — a weekly buyer silent for a month is in trouble; an annual buyer isn't. Customers with one order have no gap and are excluded; handle them as a separate "failed to activate" segment.

Common mistakes. A single global "no order in 90 days" rule, which misclassifies both frequent and infrequent buyers. Dividing by a possibly-zero average gap without NULLIF.

Follow-up. "How would you validate this?" → Backtest: compute risk flags as at six months ago and check whether flagged customers actually stopped buying. A churn model nobody has validated is a guess with a query attached.

51. Discount impact on margin

```
SELECT CASE WHEN oi.discount_pct = 0 THEN 'No discount'
           WHEN oi.discount_pct <= 0.10 THEN 'Up to 10%'
           ELSE 'Over 10%' END AS discount_band,
       COUNT(*) AS lines,
       ROUND(AVG(oi.unit_price*(1-oi.discount_pct) - p.unit_cost),2) AS avg_unit_margin,
       ROUND(SUM(oi.quantity*(oi.unit_price*(1-oi.discount_pct)-p.unit_cost),2) AS total_profit
FROM order_items oi JOIN products p USING (product_id)
JOIN orders o ON o.order_id=oi.order_id AND o.status='completed'
GROUP BY 1;
```

52. Repeat purchase rate by cohort

```
SELECT DATE_TRUNC('month',first_order)::date AS cohort, COUNT(*) AS customers,
       ROUND(100.0*COUNT(*) FILTER (WHERE orders>=2)/COUNT(*),1) AS repeat_rate_pct
FROM (SELECT customer_id, COUNT(*) AS orders, MIN(order_ts)::date AS first_order
      FROM orders WHERE status='completed' GROUP BY customer_id) t
GROUP BY 1 ORDER BY 1;
```

53. Time to second purchase

```
WITH seq AS (SELECT customer_id, order_ts, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts) n
            FROM orders WHERE status='completed')
SELECT ROUND(AVG(b.order_ts::date - a.order_ts::date),1) AS mean_days,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY b.order_ts::date - a.order_ts::date) AS median_days
FROM seq a JOIN seq b ON b.customer_id=a.customer_id AND a.n=1 AND b.n=2;
```

54. First product each customer bought

```
SELECT DISTINCT ON (o.customer_id) o.customer_id, p.product_name, p.category, o.order_ts::date
FROM orders o JOIN order_items oi USING (order_id) JOIN products p USING (product_id)
WHERE o.status='completed'
ORDER BY o.customer_id, o.order_ts, oi.order_item_id;
```

55. Which first-purchase category drives highest LTV

See Part 12.16.

56. Weekly active customers with week-on-week change

```
WITH w AS (SELECT DATE_TRUNC('week',order_ts)::date AS week,
                  COUNT(DISTINCT customer_id) AS active
            FROM orders WHERE status='completed' GROUP BY 1)
SELECT week, active, active - LAG(active) OVER (ORDER BY week) AS change,
       ROUND(100.0*(active-LAG(active) OVER (ORDER BY week))
            /NULLIF(LAG(active) OVER (ORDER BY week),0),1) AS wow_pct
FROM w ORDER BY week;
```

57. Category revenue pivoted by month

```
SELECT p.category,
       ROUND(SUM(oi.quantity*oi.unit_price) FILTER (WHERE DATE_TRUNC('month',o.order_ts)='2024-01-01'),2) AS jan,
       ROUND(SUM(oi.quantity*oi.unit_price) FILTER (WHERE DATE_TRUNC('month',o.order_ts)='2024-02-01'),2) AS feb,
       ROUND(SUM(oi.quantity*oi.unit_price) FILTER (WHERE DATE_TRUNC('month',o.order_ts)='2024-03-01'),2) AS mar
FROM order_items oi JOIN products p USING (product_id)
JOIN orders o ON o.order_id=oi.order_id AND o.status='completed'
GROUP BY p.category;
```

Follow-up. "What if a new month is added?" → SQL can't pivot on values unknown at parse time. Use the BI tool, `crosstab()`, or dynamic SQL from application code.

58. Customers who bought every Electronics product

```
SELECT o.customer_id
FROM orders o JOIN order_items oi USING (order_id) JOIN products p USING (product_id)
WHERE p.category='Electronics' AND o.status='completed'
GROUP BY o.customer_id
HAVING COUNT(DISTINCT p.product_id) = (SELECT COUNT(*) FROM products WHERE category='Electronics');
```

Explanation. Relational division. Count distinct matches per customer and compare to the total number of targets.

59. Orders with above-average line count

```
SELECT order_id, lines FROM (
  SELECT order_id, COUNT(*) AS lines FROM order_items GROUP BY order_id) t
WHERE lines > (SELECT AVG(cnt) FROM (SELECT COUNT(*) cnt FROM order_items GROUP BY order_id) x);
```

60. * A&E four-hour performance by site and month

Problem. Percentage of A&E attendances within four hours, by site and month.

Solution.

```
SELECT site_code,
       DATE_TRUNC('month', arrival_ts)::date AS month,
       COUNT(*) AS attendances,
       COUNT(*) FILTER (WHERE departure_ts IS NULL) AS still_present,
       COUNT(*) FILTER (WHERE departure_ts - arrival_ts <= INTERVAL '4 hours') AS within_4h,
       ROUND(100.0 * COUNT(*) FILTER (WHERE departure_ts - arrival_ts <= INTERVAL '4 hours')
            / NULLIF(COUNT(*) FILTER (WHERE departure_ts IS NOT NULL), 0), 1) AS pct_within_4h
FROM ae_attendances
GROUP BY site_code, 2
ORDER BY site_code, 2;
```

Explanation. The denominator counts only completed attendances, since a patient still present has no measurable total time. Reporting `still_present` separately makes that exclusion visible rather than hidden.

Alternative (arguably more honest). Count anyone already past four hours as a breach whether or not they've left:

```
COUNT(*) FILTER (WHERE COALESCE(departure_ts, CURRENT_TIMESTAMP) - arrival_ts > INTERVAL '4 hours')
```

Common mistakes. Including still-present patients in the denominator with no departure time — they fail the `<= 4 hours` test (NULL comparison is UNKNOWN) and silently depress the percentage. Using `BETWEEN` on the month boundary. Ignoring that a NULL departure might mean a data quality problem rather than a live patient.

Follow-up. "Performance improved 3 points this month — what would you check before reporting it?" → Whether attendance volume changed, whether the case mix (triage categories) shifted, whether more records have missing departure times, and whether it's within normal month-to-month variation. Three points on a noisy metric may be nothing.

61. DNA rate by specialty and deprivation decile

```
SELECT r.specialty,
       CASE WHEN p.imd_decile <= 3 THEN 'Most deprived'
            WHEN p.imd_decile <= 7 THEN 'Mid'
            WHEN p.imd_decile IS NULL THEN 'Unknown'
            ELSE 'Least deprived' END AS deprivation,
       COUNT(*) AS appointments,
       ROUND(100.0*COUNT(*) FILTER (WHERE a.outcome='DNA')/COUNT(*),1) AS dna_rate_pct
FROM appointments a JOIN referrals r USING (referral_id) JOIN patients p ON p.patient_id=a.patient_id
GROUP BY 1,2 HAVING COUNT(*) >= 30
ORDER BY 1,2;
```

62. Referral to first appointment waiting times

See Part 13.13, including the completed-vs-incomplete pathway caveat.

63. Patients with multiple referrals to the same specialty

```
SELECT patient_id, specialty, COUNT(*) AS referrals,
       MIN(referral_date) AS first, MAX(referral_date) AS latest
FROM referrals GROUP BY 1,2 HAVING COUNT(*) > 1 ORDER BY referrals DESC;
```

64. RTT 18-week breaches, still waiting

```
SELECT r.specialty, COUNT(*) AS waiting,
       COUNT(*) FILTER (WHERE CURRENT_DATE - r.referral_date > 126) AS over_18_weeks,
       ROUND(100.0*COUNT(*) FILTER (WHERE CURRENT_DATE - r.referral_date <= 126)/COUNT(*),1) AS pct_within
FROM referrals r JOIN waiting_list w ON w.referral_id=r.referral_id AND w.removed_date IS NULL
GROUP BY r.specialty ORDER BY pct_within;
```

65. Two Week Wait compliance

```
SELECT DATE_TRUNC('month',r.referral_date)::date AS month,
       COUNT(*) AS twow_referrals,
       COUNT(*) FILTER (WHERE a.attended_ts::date - r.referral_date <= 14) AS seen_within_14d,
       ROUND(100.0*COUNT(*) FILTER (WHERE a.attended_ts::date - r.referral_date <= 14)
            /NULLIF(COUNT(*) FILTER (WHERE a.attended_ts IS NOT NULL),0),1) AS compliance_pct
FROM referrals r
LEFT JOIN LATERAL (SELECT attended_ts FROM appointments x
                  WHERE x.referral_id=r.referral_id AND x.outcome='Attended'
                  ORDER BY attended_ts LIMIT 1) a ON true
WHERE r.priority='Two Week Wait'
GROUP BY 1 ORDER BY 1;
```

66. Anomalous days in order volume

See Part 12.19. The key detail: the baseline frame must end at 1 PRECEDING so the tested day isn't in its own baseline.

67. Contribution to revenue change by category

See Part 12.21. Be ready to explain the difference between a category's own growth rate and its contribution to the total change.

68. Basket analysis: what do buyers of X also buy?

```
SELECT p2.product_name, COUNT(DISTINCT o.order_id) AS co_occurrences
FROM orders o
JOIN order_items i1 ON i1.order_id=o.order_id AND i1.product_id=10
JOIN order_items i2 ON i2.order_id=o.order_id AND i2.product_id<>10
```

```
JOIN products p2 ON p2.product_id=i2.product_id
WHERE o.status='completed'
GROUP BY p2.product_name ORDER BY co_occurrences DESC;
```

Follow-up. "Raw co-occurrence favours popular products." → Compute lift: observed co-occurrence divided by what you'd expect if the two were independent.

69. Customers ranked by value with their percentile

```
SELECT customer_id, lifetime_value,
       RANK() OVER (ORDER BY lifetime_value DESC) AS rank,
       ROUND(100*CUME_DIST() OVER (ORDER BY lifetime_value),1) AS percentile
FROM customer_ltv;
```

70. Orders where the discount exceeded the margin

```
SELECT oi.order_item_id, p.product_name,
       ROUND(oi.unit_price*oi.discount_pct,2) AS discount_value,
       ROUND(oi.unit_price - p.unit_cost,2) AS full_margin
FROM order_items oi JOIN products p USING (product_id)
WHERE oi.unit_price*(1-oi.discount_pct) < p.unit_cost;
```

71. Active customer count on a rolling 28-day basis

```
SELECT d::date AS day,
       (SELECT COUNT(DISTINCT o.customer_id) FROM orders o
        WHERE o.status='completed' AND o.order_ts::date > d::date - 28
        AND o.order_ts::date <= d::date) AS rolling_28d_active
FROM generate_series(CURRENT_DATE-89, CURRENT_DATE, INTERVAL '1 day') d;
```

Explanation. A correlated subquery is needed because distinct counts aren't additive — you can't window-sum daily distinct counts into a rolling distinct count.

72. Compare two periods side by side

```
SELECT p.category,
       ROUND(SUM(oi.quantity*oi.unit_price) FILTER (
         WHERE o.order_ts >= CURRENT_DATE-30),2) AS last_30d,
       ROUND(SUM(oi.quantity*oi.unit_price) FILTER (
         WHERE o.order_ts >= CURRENT_DATE-60 AND o.order_ts < CURRENT_DATE-30),2) AS prev_30d
FROM orders o JOIN order_items oi USING (order_id) JOIN products p USING (product_id)
WHERE o.status='completed' AND o.order_ts >= CURRENT_DATE-60
GROUP BY p.category;
```

73. Identify orders with data quality problems

```
SELECT o.order_id,
       CASE WHEN NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.order_id=o.order_id)
         THEN 'No line items'
         WHEN o.order_ts > CURRENT_TIMESTAMP THEN 'Future dated'
         WHEN o.customer_id IS NULL THEN 'No customer'
         WHEN o.shipping_cost < 0 THEN 'Negative shipping'
       END AS issue
FROM orders o
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.order_id=o.order_id)
       OR o.order_ts > CURRENT_TIMESTAMP OR o.customer_id IS NULL OR o.shipping_cost < 0;
```

74. Customer segments by RFM

See Part 12.14. Be prepared to explain the NTILE direction for each of R, F and M — reversing one silently inverts the segments.

75. * Which channel produces the most valuable customers?

Problem. Compare acquisition channels on volume, conversion, AOV and lifetime value.

Solution.

```
WITH customer_value AS (
  SELECT c.customer_id, c.channel, c.signup_date,
         COUNT(DISTINCT o.order_id) AS orders,
         COALESCE(SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)),0) AS lifetime_value
  FROM customers c
  LEFT JOIN orders o ON o.customer_id=c.customer_id AND o.status='completed'
  LEFT JOIN order_items oi ON oi.order_id=o.order_id
  GROUP BY c.customer_id, c.channel, c.signup_date
)
SELECT channel,
       COUNT(*) AS customers,
       COUNT(*) FILTER (WHERE orders > 0) AS converted,
       ROUND(100.0*COUNT(*) FILTER (WHERE orders>0)/COUNT(*),1) AS conversion_pct,
       ROUND(AVG(lifetime_value),2) AS avg_ltv_all,
       ROUND(AVG(lifetime_value) FILTER (WHERE orders>0),2) AS avg_ltv_buyers,
       ROUND(AVG(orders) FILTER (WHERE orders>0),2) AS avg_orders_per_buyer,
       ROUND(AVG(CURRENT_DATE - signup_date),0) AS avg_tenure_days
FROM customer_value
GROUP BY channel
ORDER BY avg_ltv_all DESC;
```

Explanation. Two LTV columns because they answer different questions: `avg_ltv_all` includes non-converters and is the right number for comparing acquisition spend; `avg_ltv_buyers` tells you about the quality of customers who do convert. A channel can win on one and lose on the other, and that's a real finding, not noise.

Common mistakes. Inner joining and thereby dropping non-converters, which flatters every channel equally but distorts their relative ranking. Ignoring tenure — an older channel's customers have had longer to accumulate value, so a fair comparison normalises to LTV at a fixed number of days since signup.

Follow-up. "Referral has the highest LTV — should we shift all budget to it?" → No. Referral volume is usually capped and not directly buyable; and there's selection bias, since referred customers are pre-qualified by an existing happy customer. Also compare CAC by channel, not just LTV. This question is testing commercial judgement, not SQL, and a purely technical answer scores badly.

Self-check

You should be fluent in all of these. The ones that most often separate offers from rejections:

- LEFT JOIN with the filter in ON, not WHERE (1)
- Aggregating to the right grain before averaging (10, 14)
- Cohort and retention construction (21, 22, 52)
- Window functions for ranking, LAG and running totals (5, 7, 12, 19)
- Funnel construction at the right grain (40)
- Knowing when a metric definition is ambiguous and saying so (50, 60, 75)

Part 16c — 75 Advanced SQL Interview Questions

These are what you get in a second-round technical, a take-home task, or an interview for a role that says "junior" but expects real analytical work. Many have no single right answer — the assessment is as much about the questions you ask and the caveats you volunteer as the SQL you write.

* = fuller treatment.

* 1. Full cohort retention table

Problem. Monthly cohort retention: rows are cohorts by first-purchase month, columns are months since acquisition, values are the percentage still active.

Solution. See Part 12.9 for the complete query. The structure to reproduce under pressure:

```
WITH first_purchase AS (
    SELECT customer_id, DATE_TRUNC('month', MIN(order_ts))::date AS cohort_month
    FROM orders WHERE status='completed' GROUP BY customer_id
),
activity AS (
    SELECT DISTINCT customer_id, DATE_TRUNC('month', order_ts)::date AS activity_month
    FROM orders WHERE status='completed'
),
joined AS (
    SELECT f.cohort_month, a.customer_id,
        (EXTRACT(YEAR FROM a.activity_month)-EXTRACT(YEAR FROM f.cohort_month))*12
        + (EXTRACT(MONTH FROM a.activity_month)-EXTRACT(MONTH FROM f.cohort_month)) AS month_number
    FROM first_purchase f JOIN activity a USING (customer_id)
),
sizes AS (SELECT cohort_month, COUNT(*) AS cohort_size FROM first_purchase GROUP BY 1)
SELECT j.cohort_month, s.cohort_size, j.month_number,
    COUNT(DISTINCT j.customer_id) AS active,
    ROUND(100.0*COUNT(DISTINCT j.customer_id)/s.cohort_size,1) AS retention_pct
FROM joined j JOIN sizes s USING (cohort_month)
GROUP BY 1,2,3 ORDER BY 1,3;
```

Sanity check. Month 0 must be 100% for every cohort. If it isn't, your cohort assignment and your activity table disagree.

Common mistakes. Computing month offset as $(date - date)/30$, which drifts and eventually misassigns months. Dividing by the current cohort size rather than the original. Forgetting DISTINCT and counting orders rather than customers.

Follow-up. "Month 3 retention for the newest cohort is blank — is retention collapsing?" → No, three months haven't elapsed. Right-censoring. Only compare cohorts at equal maturity; this mistake has caused real, expensive misreadings.

2. Revenue cohorts rather than count cohorts

```
SELECT cohort_month, month_number,
    ROUND(SUM(revenue),2) AS cohort_revenue,
    ROUND(SUM(revenue)/MAX(cohort_size),2) AS revenue_per_original_customer
FROM cohort_revenue_base GROUP BY 1,2 ORDER BY 1,2;
```

Explanation. Dividing by the *original* cohort size, not by active customers, gives cumulative value per acquired customer — which is what you compare against CAC.

3. Gaps and islands: longest active streak

See Part 12.4 for both methods. Be able to explain the $date - ROW_NUMBER()$ trick in one sentence: consecutive dates and consecutive row numbers increase in step, so their difference is constant within a run.

4. Find missing dates in a series

```
WITH d AS (SELECT DISTINCT order_ts::date AS day FROM orders),
    bounds AS (SELECT MIN(day) lo, MAX(day) hi FROM d)
SELECT g::date AS missing_day
FROM bounds, generate_series(bounds.lo, bounds.hi, INTERVAL '1 day') g
WHERE NOT EXISTS (SELECT 1 FROM d WHERE d.day = g::date);
```

Alternative using LEAD to find gap ranges rather than individual days:

```
SELECT day AS gap_starts_after, next_day, next_day - day - 1 AS days_missing
FROM (SELECT day, LEAD(day) OVER (ORDER BY day) AS next_day FROM d) t
WHERE next_day - day > 1;
```

5. Sessionise with a 30-minute inactivity timeout

See Part 12.17. Expect a follow-up on why the cumulative sum of a boolean flag produces a group id.

6. Nth order per customer, generalised

```
SELECT * FROM (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts, order_id) AS n
  FROM orders o WHERE status='completed') t
WHERE n = 3;
```

7. Customers whose spend increased every month

```
WITH m AS (
  SELECT o.customer_id, DATE_TRUNC('month', o.order_ts)::date AS month,
         SUM(oi.quantity*oi.unit_price) AS spend
  FROM orders o JOIN order_items oi USING (order_id) WHERE o.status='completed'
  GROUP BY 1,2),
flagged AS (
  SELECT *, spend > LAG(spend) OVER (PARTITION BY customer_id ORDER BY month) AS increased
  FROM m)
SELECT customer_id FROM flagged
GROUP BY customer_id
HAVING COUNT(*) FILTER (WHERE increased IS FALSE) = 0 AND COUNT(*) >= 3;
```

Explanation. `increased` is NULL for the first month (no previous), so counting FALSE rather than NOT TRUE correctly ignores it.

8. Median by group without PERCENTILE_CONT

```
SELECT category, AVG(unit_price) AS median FROM (
  SELECT category, unit_price,
         ROW_NUMBER() OVER (PARTITION BY category ORDER BY unit_price) rn,
         COUNT(*) OVER (PARTITION BY category) n
  FROM products) t
WHERE rn IN ((n+1)/2, (n+2)/2) GROUP BY category;
```

Explanation. The $(n+1)/2$, $(n+2)/2$ pair uses integer division to select the single middle row for odd n and the two middle rows for even n .

9. Mode (most common value) per group

```
SELECT category, MODE() WITHIN GROUP (ORDER BY subcategory) AS most_common_subcategory
FROM products GROUP BY category;
```

10. * Year-on-year growth with missing periods handled

Problem. Monthly revenue with YoY growth, robust to months with no sales.

Solution.

```
WITH calendar AS (
  SELECT generate_series(DATE '2023-01-01', DATE '2024-12-01', INTERVAL '1 month')::date AS month
),
actual AS (
  SELECT DATE_TRUNC('month', o.order_ts)::date AS month,
         SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS revenue
  FROM orders o JOIN order_items oi USING (order_id)
  WHERE o.status='completed' GROUP BY 1
),
filled AS (
  SELECT c.month, COALESCE(a.revenue, 0) AS revenue
```

```

FROM calendar c LEFT JOIN actual a USING (month)
)
SELECT f.month, ROUND(f.revenue,2) AS revenue,
       ROUND(p.revenue,2) AS same_month_last_year,
       ROUND(100.0*(f.revenue - p.revenue)/NULLIF(p.revenue,0),1) AS yoy_pct
FROM filled f
LEFT JOIN filled p ON p.month = f.month - INTERVAL '1 year'
WHERE f.month >= DATE '2024-01-01'
ORDER BY f.month;

```

Explanation. Three defences against the same failure mode: generate the calendar so gaps become explicit zeros; join on the date offset rather than `LAG(12)` so a missing row can't shift the comparison; `NULLIF` on the denominator so a zero prior year doesn't error.

Common mistakes. `LAG(revenue, 12)` on a table with gaps. Dividing by a zero baseline. Reporting "+∞%" growth when the prior period was zero — better to show the absolute values and suppress the percentage.

Follow-up. "Prior year was zero. What do you display?" → Not a percentage. Show "n/a (no prior year activity)" and the absolute figures. A percentage change from zero is undefined, not infinite.

11. Rolling 12-month revenue

```

SELECT month, ROUND(SUM(revenue) OVER (ORDER BY month
    RANGE BETWEEN INTERVAL '11 months' PRECEDING AND CURRENT ROW),2) AS rolling_12m
FROM monthly_revenue;

```

12. Rank with custom tie handling

```

SELECT product_name, revenue,
       RANK() OVER (ORDER BY revenue DESC) AS competition_rank,
       DENSE_RANK() OVER (ORDER BY revenue DESC) AS dense,
       ROW_NUMBER() OVER (ORDER BY revenue DESC, product_id) AS deterministic_row
FROM product_revenue;

```

13. Top N per group with ties included

```

SELECT * FROM (
  SELECT p.*, DENSE_RANK() OVER (PARTITION BY category ORDER BY unit_price DESC) r
  FROM products p) t WHERE r <= 3;

```

14. Customers ranked within multiple dimensions at once

```

SELECT customer_id, country, channel, lifetime_value,
       RANK() OVER (ORDER BY lifetime_value DESC) AS overall,
       RANK() OVER (PARTITION BY country ORDER BY lifetime_value DESC) AS in_country,
       RANK() OVER (PARTITION BY channel ORDER BY lifetime_value DESC) AS in_channel,
       ROUND(100*PERCENT_RANK() OVER (ORDER BY lifetime_value),1) AS percentile
FROM customer_ltv;

```

15. Pareto: customers driving 80% of revenue

See Part 12.20. The off-by-one on the crossing row is the detail being tested.

16. Contribution analysis of a revenue decline

See Part 12.21. Be ready to distinguish a segment's growth rate from its contribution to the total change.

17. Price/volume decomposition

```

WITH m AS (
  SELECT DATE_TRUNC('month',o.order_ts)::date AS month, p.category,
         SUM(oi.quantity) AS units,
         SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct))/NULLIF(SUM(oi.quantity),0) AS avg_price,
         SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS revenue
  FROM orders o JOIN order_items oi USING (order_id) JOIN products p USING (product_id)
  WHERE o.status='completed' GROUP BY 1,2)

```

```

SELECT month, category,
       ROUND(revenue - LAG(revenue) OVER w, 2) AS total_change,
       ROUND((units - LAG(units) OVER w) * LAG(avg_price) OVER w, 2) AS volume_effect,
       ROUND((avg_price - LAG(avg_price) OVER w) * LAG(units) OVER w, 2) AS price_effect,
       ROUND((units - LAG(units) OVER w) * (avg_price - LAG(avg_price) OVER w), 2) AS interaction
FROM m WINDOW w AS (PARTITION BY category ORDER BY month) ORDER BY month, category;

```

Explanation. The three effects sum exactly to the total change. Being able to derive that decomposition is a genuine commercial-analysis skill and rare in candidates.

18. Ordered funnel with time constraints

See Part 12.12, the timestamp-comparison version.

19. Funnel with a 7-day attribution window

```

WITH first_touch AS (
  SELECT customer_id, MIN(event_ts) AS first_view
  FROM web_events WHERE event_name='product_view' GROUP BY customer_id)
SELECT COUNT(*) AS viewers,
       COUNT(*) FILTER (WHERE EXISTS (
         SELECT 1 FROM orders o WHERE o.customer_id=f.customer_id AND o.status='completed'
         AND o.order_ts BETWEEN f.first_view AND f.first_view + INTERVAL '7 days')) AS converted_7d
FROM first_touch f;

```

20. * Customer lifetime value with cohort normalisation

Problem. Compare LTV across acquisition cohorts fairly, given that older cohorts have had longer to spend.

Solution.

```

WITH orders_with_age AS (
  SELECT c.customer_id,
         DATE_TRUNC('month', c.signup_date)::date AS cohort,
         c.channel,
         o.order_ts::date - c.signup_date AS days_since_signup,
         oi.quantity*oi.unit_price*(1-oi.discount_pct) AS line_revenue
  FROM customers c
  JOIN orders o ON o.customer_id=c.customer_id AND o.status='completed'
  JOIN order_items oi ON oi.order_id=o.order_id
),
cohort_sizes AS (
  SELECT DATE_TRUNC('month', signup_date)::date AS cohort, channel, COUNT(*) AS customers
  FROM customers GROUP BY 1,2
)
SELECT o.cohort, o.channel, s.customers,
       ROUND(SUM(o.line_revenue) FILTER (WHERE o.days_since_signup <= 30)/s.customers, 2) AS ltv_30d,
       ROUND(SUM(o.line_revenue) FILTER (WHERE o.days_since_signup <= 90)/s.customers, 2) AS ltv_90d,
       ROUND(SUM(o.line_revenue) FILTER (WHERE o.days_since_signup <= 365)/s.customers, 2) AS ltv_365d
FROM orders_with_age o
JOIN cohort_sizes s ON s.cohort=o.cohort AND s.channel=o.channel
GROUP BY o.cohort, o.channel, s.customers
ORDER BY o.cohort, o.channel;

```

Explanation. LTV is measured at fixed ages since signup, so every cohort is compared at the same maturity. Dividing by cohort size (all acquired customers, not just buyers) makes it directly comparable to CAC.

Common mistakes. Raw lifetime totals — older cohorts always "win". Dividing by buyers rather than acquired customers, which silently removes the conversion rate from the comparison.

Follow-up. "The newest cohort has a blank ltv_365d." → It's censored. Only compare columns where every cohort has had time to mature, and grey out the rest rather than treating blanks as zeros.

21. Predicted LTV from cadence

```

WITH c AS (
  SELECT customer_id, COUNT(*) AS orders, AVG(order_value) AS aov,
         MAX(order_ts)::date - MIN(order_ts)::date AS span_days
  FROM order_values GROUP BY customer_id HAVING COUNT(*) >= 2)
SELECT customer_id, orders, ROUND(aov,2) AS aov,
       ROUND(orders::numeric/NULLIF(span_days,0)*365,2) AS predicted_annual_orders,

```

```
ROUND(aov * orders::numeric/NULLIF(span_days,0)*365,2) AS predicted_annual_value
FROM c ORDER BY predicted_annual_value DESC;
```

Caveat to volunteer. Extrapolating from a short span is unstable — two orders a week apart implies 52 orders a year. Require a minimum span and order count before trusting it.

22. Churn prediction features

```
SELECT c.customer_id,
       COUNT(DISTINCT o.order_id) AS orders,
       CURRENT_DATE - MAX(o.order_ts)::date AS recency,
       AVG(o.order_value) AS aov,
       STDDEV(o.order_value) AS order_value_variability,
       COUNT(DISTINCT DATE_TRUNC('month',o.order_ts)) AS active_months,
       COUNT(DISTINCT p.category) AS categories_bought,
       COUNT(*) FILTER (WHERE o.status='refunded') AS refunds,
       MAX(o.order_ts)::date - MIN(o.order_ts)::date AS tenure_days
FROM customers c
LEFT JOIN orders o ON o.customer_id=c.customer_id
LEFT JOIN order_values ov ON ov.order_id=o.order_id
LEFT JOIN order_items oi ON oi.order_id=o.order_id
LEFT JOIN products p ON p.product_id=oi.product_id
GROUP BY c.customer_id;
```

Follow-up. "Any leakage risk?" → Yes, if features are computed over a window that includes the churn label's period. Features must come strictly from before the prediction date. Spotting leakage is a strong answer for any analyst role touching modelling.

23. Market basket lift

```
WITH totals AS (SELECT COUNT(DISTINCT order_id)::numeric AS n FROM order_items),
pair AS (
  SELECT i1.product_id a, i2.product_id b, COUNT(DISTINCT i1.order_id)::numeric AS together
  FROM order_items i1 JOIN order_items i2 ON i2.order_id=i1.order_id AND i2.product_id>i1.product_id
  GROUP BY 1,2),
single AS (SELECT product_id, COUNT(DISTINCT order_id)::numeric AS cnt FROM order_items GROUP BY 1)
SELECT pa.product_name, pb.product_name, p.together,
       ROUND((p.together/t.n) / ((sa.cnt/t.n)*(sb.cnt/t.n)), 2) AS lift
FROM pair p
CROSS JOIN totals t
JOIN single sa ON sa.product_id=p.a JOIN single sb ON sb.product_id=p.b
JOIN products pa ON pa.product_id=p.a JOIN products pb ON pb.product_id=p.b
WHERE p.together >= 5
ORDER BY lift DESC;
```

Explanation. Lift > 1 means the pair co-occurs more than chance. Raw co-occurrence counts just rank popular products; lift is what you'd act on.

24. Recursive: full organisational hierarchy

See Part 10.5. Include the depth guard and be able to say why **UNION ALL** rather than **UNION**.

25. Count all reports beneath each manager (direct and indirect)

```
WITH RECURSIVE tree AS (
  SELECT employee_id AS root, employee_id, 0 AS depth FROM employees
  UNION ALL
  SELECT t.root, e.employee_id, t.depth+1
  FROM employees e JOIN tree t ON e.manager_id = t.employee_id
  WHERE t.depth < 20
)
SELECT e.full_name, COUNT(*)-1 AS total_reports
FROM tree t JOIN employees e ON e.employee_id = t.root
GROUP BY e.employee_id, e.full_name ORDER BY total_reports DESC;
```

26. Detect cycles in a hierarchy

```
WITH RECURSIVE walk AS (
  SELECT employee_id, manager_id, ARRAY[employee_id] AS path, false AS cycle
```

```

FROM employees
UNION ALL
SELECT e.employee_id, e.manager_id, w.path || e.employee_id, e.employee_id = ANY(w.path)
FROM employees e JOIN walk w ON e.employee_id = w.manager_id
WHERE NOT w.cycle
)
SELECT DISTINCT path FROM walk WHERE cycle;

```

27. Slowly changing dimension: point-in-time join

```

SELECT f.order_id, f.order_ts, d.country AS country_at_time_of_order
FROM orders f
JOIN dim_customer d
  ON d.customer_id = f.customer_id
 AND f.order_ts >= d.valid_from
 AND f.order_ts < COALESCE(d.valid_to, TIMESTAMP 'infinity');

```

Mistake. Joining on the key alone against a Type 2 dimension multiplies every fact row by the number of versions. This is a real and expensive production bug.

28. Reconcile two data sources

```

SELECT COALESCE(a.order_id,b.order_id) AS order_id, a.total AS system, b.total AS finance,
       CASE WHEN a.order_id IS NULL THEN 'Missing from system'
            WHEN b.order_id IS NULL THEN 'Missing from finance'
            WHEN a.total IS DISTINCT FROM b.total THEN 'Value mismatch'
       END AS issue
FROM system_totals a FULL OUTER JOIN finance_totals b USING (order_id)
WHERE a.order_id IS NULL OR b.order_id IS NULL OR a.total IS DISTINCT FROM b.total;

```

Explanation. `IS DISTINCT FROM` so a NULL on one side registers as a mismatch instead of being skipped.

29. Deduplicate with fuzzy matching

```

SELECT a.patient_id, b.patient_id, a.nhs_number, b.nhs_number,
       SIMILARITY(a.surname, b.surname) AS name_similarity
FROM patients a JOIN patients b ON a.patient_id < b.patient_id
WHERE a.date_of_birth = b.date_of_birth
     AND LEFT(a.postcode_sector,3) = LEFT(b.postcode_sector,3)
     AND SIMILARITY(a.surname, b.surname) > 0.7;

```

Explanation. Block on exact fields first (DOB, partial postcode), then fuzzy-match within blocks. Without blocking, this is $O(n^2)$ and will never finish.

30. * Identify and quantify the impact of duplicate rows

Problem. You inherit a revenue report you suspect is inflated by duplicates. Quantify it.

Solution.

```

WITH dup_check AS (
  SELECT order_id, product_id, quantity, unit_price,
         COUNT(*) AS copies,
         SUM(quantity*unit_price) AS reported_value,
         MIN(quantity*unit_price) AS true_value
  FROM order_items
  GROUP BY order_id, product_id, quantity, unit_price
)
SELECT COUNT(*) FILTER (WHERE copies > 1) AS duplicated_groups,
       SUM(copies - 1) FILTER (WHERE copies > 1) AS excess_rows,
       ROUND(SUM(reported_value),2) AS reported_total,
       ROUND(SUM(true_value),2) AS deduplicated_total,
       ROUND(SUM(reported_value) - SUM(true_value),2) AS overstatement,
       ROUND(100.0*(SUM(reported_value)-SUM(true_value))/NULLIF(SUM(true_value),0),2) AS overstatement_pct
FROM dup_check;

```

Explanation. Group by every column that defines a genuine business record, count copies, and compare the reported sum against one-copy-per-group.

The judgement call. Two identical lines might be a genuine duplicate load, or a customer legitimately ordering the same product on two lines. Without a reliable unique key you cannot distinguish them from the data alone — you have to ask whether the source system permits split lines. Saying that, rather than assuming, is the answer.

Follow-up. *"How would you stop it recurring?"* → A unique constraint on the natural key, an idempotent load (delete-then-insert by batch, or upsert on the key), and a row-count reconciliation check in the pipeline that fails loudly.

31. Data quality scorecard

```
SELECT 'customers' AS table_name, COUNT(*) AS rows,
       ROUND(100.0*COUNT(email)/COUNT(*),1) AS pct_email_populated,
       ROUND(100.0*COUNT(country)/COUNT(*),1) AS pct_country_populated,
       COUNT(*) - COUNT(DISTINCT customer_id) AS duplicate_keys,
       COUNT(*) FILTER (WHERE signup_date > CURRENT_DATE) AS future_dates,
       COUNT(*) FILTER (WHERE email !~* '^[^@\\s]+@[^@\\s]+\\. [a-z]{2,}$') AS malformed_emails
FROM customers;
```

32. Detect a broken data pipeline

```
WITH daily AS (SELECT order_ts::date AS d, COUNT(*) AS rows FROM orders GROUP BY 1)
SELECT d, rows,
       ROUND(AVG(rows) OVER (ORDER BY d ROWS BETWEEN 14 PRECEDING AND 1 PRECEDING),1) AS baseline,
       CASE WHEN rows = 0 THEN 'No data loaded'
            WHEN rows < 0.5*AVG(rows) OVER (ORDER BY d ROWS BETWEEN 14 PRECEDING AND 1 PRECEDING)
            THEN 'Suspiciously low'
            WHEN rows > 2.0*AVG(rows) OVER (ORDER BY d ROWS BETWEEN 14 PRECEDING AND 1 PRECEDING)
            THEN 'Suspiciously high - possible double load'
       END AS alert
FROM daily ORDER BY d DESC LIMIT 30;
```

33. Anomaly detection with seasonality

```
WITH daily AS (SELECT order_ts::date AS d, COUNT(*) AS orders FROM orders
                WHERE status='completed' GROUP BY 1),
seasonal AS (
  SELECT d, orders, EXTRACT(ISODOW FROM d) AS dow,
         AVG(orders) OVER (PARTITION BY EXTRACT(ISODOW FROM d)
                          ORDER BY d ROWS BETWEEN 4 PRECEDING AND 1 PRECEDING) AS same_dow_baseline,
         STDEV(orders) OVER (PARTITION BY EXTRACT(ISODOW FROM d)
                             ORDER BY d ROWS BETWEEN 4 PRECEDING AND 1 PRECEDING) AS same_dow_sd
  FROM daily)
SELECT d, orders, ROUND(same_dow_baseline,1) AS expected,
       ROUND((orders-same_dow_baseline)/NULLIF(same_dow_sd,0),2) AS z
FROM seasonal
WHERE same_dow_sd > 0 AND ABS(orders-same_dow_baseline) > 2*same_dow_sd
ORDER BY d DESC;
```

Explanation. Comparing each Saturday to the last four Saturdays removes day-of-week seasonality, which otherwise flags every weekend as anomalous.

34. A/B test results with confidence

```
SELECT variant,
       COUNT(*) AS users,
       COUNT(*) FILTER (WHERE converted) AS conversions,
       ROUND(100.0*COUNT(*) FILTER (WHERE converted)/COUNT(*),2) AS cvr_pct,
       ROUND(100.0*1.96*SQRT(
         (COUNT(*) FILTER (WHERE converted)::numeric/COUNT(*))
         * (1 - COUNT(*) FILTER (WHERE converted)::numeric/COUNT(*)) / COUNT(*)),2) AS ci_half_width_pp
FROM experiment_assignments GROUP BY variant;
```

Follow-up. *"Confidence intervals overlap — what do you conclude?"* → Not significant at this sample size; report the observed difference with its interval and the power to detect the minimum effect worth caring about. Also check for sample ratio mismatch (unequal assignment counts) before trusting anything, since that usually indicates a broken experiment.

35. Sample ratio mismatch check

```
SELECT variant, COUNT(*) AS users,
       ROUND(100.0*COUNT(*)/SUM(COUNT(*)) OVER (),2) AS pct,
       ROUND(POWER(COUNT(*) - SUM(COUNT(*)) OVER (),2.0, 2)
            / (SUM(COUNT(*)) OVER (),2.0), 3) AS chi_sq_component
FROM experiment_assignments GROUP BY variant;
```

36. Attribution: first touch vs last touch

```
WITH touches AS (
  SELECT customer_id, event_ts, SUBSTRING(page_url FROM 'utm_source=([^&]+)') AS source
  FROM web_events WHERE page_url LIKE '%utm_source=%' AND customer_id IS NOT NULL),
conv AS (SELECT customer_id, MIN(order_ts) AS conversion_ts FROM orders
        WHERE status='completed' GROUP BY customer_id)
SELECT c.customer_id,
       (SELECT source FROM touches t WHERE t.customer_id=c.customer_id
        AND t.event_ts <= c.conversion_ts ORDER BY t.event_ts LIMIT 1) AS first_touch,
       (SELECT source FROM touches t WHERE t.customer_id=c.customer_id
        AND t.event_ts <= c.conversion_ts ORDER BY t.event_ts DESC LIMIT 1) AS last_touch
FROM conv c;
```

37. Multi-touch attribution, linear

```
WITH touches AS (
  SELECT customer_id, source, COUNT(*) OVER (PARTITION BY customer_id) AS touch_count
  FROM marketing_touches)
SELECT source, ROUND(SUM(1.0/touch_count),2) AS attributed_conversions
FROM touches GROUP BY source ORDER BY attributed_conversions DESC;
```

38. Rolling retention (day N)

```
WITH signups AS (SELECT customer_id, signup_date FROM customers),
acts AS (SELECT DISTINCT customer_id, order_ts::date AS d FROM orders WHERE status='completed')
SELECT s.signup_date,
       COUNT(DISTINCT s.customer_id) AS cohort,
       COUNT(DISTINCT a.customer_id) FILTER (WHERE a.d - s.signup_date BETWEEN 1 AND 7) AS active_d1_7,
       COUNT(DISTINCT a.customer_id) FILTER (WHERE a.d - s.signup_date BETWEEN 28 AND 34) AS active_d28_34
FROM signups s LEFT JOIN acts a USING (customer_id)
GROUP BY s.signup_date ORDER BY s.signup_date;
```

39. Customer journey: order of categories purchased

```
SELECT customer_id, STRING_AGG(category, ' > ' ORDER BY first_bought) AS journey
FROM (SELECT o.customer_id, p.category, MIN(o.order_ts) AS first_bought
      FROM orders o JOIN order_items oi USING (order_id) JOIN products p USING (product_id)
      WHERE o.status='completed' GROUP BY 1,2) t
GROUP BY customer_id;
```

Follow-up. "Most common journey?" → Group by the aggregated string and count. Useful for spotting typical cross-sell paths.

40. * Waiting list: current position and projected wait

Problem. For each patient still waiting, their queue position within their specialty and a projected wait based on recent throughput.

Solution.

```
WITH still_waiting AS (
  SELECT r.referral_id, r.patient_id, r.specialty, r.priority, r.referral_date,
         CURRENT_DATE - r.referral_date AS days_waiting
  FROM referrals r
  JOIN waiting_list w ON w.referral_id = r.referral_id AND w.removed_date IS NULL
),
positions AS (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY specialty
    ORDER BY CASE priority WHEN 'Two Week Wait' THEN 1
                WHEN 'Urgent' THEN 2 ELSE 3 END,
                referral_date) AS queue_position
  FROM still_waiting
```

```

),
throughput AS (
  SELECT r.specialty,
         COUNT(*)::numeric / 12.0 AS avg_weekly_removals
  FROM waiting_list w JOIN referrals r ON r.referral_id = w.referral_id
  WHERE w.removed_date >= CURRENT_DATE - 84 -- Last 12 weeks
  GROUP BY r.specialty
)
SELECT p.specialty, p.priority, p.patient_id, p.days_waiting, p.queue_position,
       ROUND(t.avg_weekly_removals,1) AS weekly_throughput,
       ROUND(p.queue_position / NULLIF(t.avg_weekly_removals,0), 1) AS projected_weeks_to_treatment,
       CASE WHEN p.days_waiting > 126 THEN 'Already breached 18 weeks'
            WHEN p.days_waiting + 7*p.queue_position/NULLIF(t.avg_weekly_removals,0) > 126
              THEN 'Projected to breach'
            ELSE 'On track' END AS rtt_outlook
FROM positions p LEFT JOIN throughput t USING (specialty)
ORDER BY p.specialty, p.queue_position;

```

Explanation. The queue order encodes clinical priority before referral date, which is how the list actually works. Throughput comes from observed removals over a recent window rather than a planning assumption. The projection is a simple queue model: position ÷ throughput.

Caveats to state. It assumes throughput stays constant and no one jumps the queue clinically — both false, and the projection is indicative rather than a promise to a patient. Removals include deaths, declines and transfers, not just treatments, so filtering `removal_reason='Treated'` gives a more honest throughput figure.

Follow-up. "How would you improve this?" → Split throughput by priority, use a percentile of recent waits rather than a mean, and validate the projection against what actually happened for patients who have since been treated.

41. Time-to-event with censoring

```

SELECT r.specialty,
       COUNT(*) AS total_referrals,
       COUNT(*) FILTER (WHERE a.attended_ts IS NOT NULL) AS seen,
       COUNT(*) FILTER (WHERE a.attended_ts IS NULL) AS censored_still_waiting,
       ROUND(AVG(a.attended_ts::date - r.referral_date) FILTER (WHERE a.attended_ts IS NOT NULL),1)
       AS mean_wait_completed,
       ROUND(AVG(CURRENT_DATE - r.referral_date) FILTER (WHERE a.attended_ts IS NULL),1)
       AS mean_wait_so_far_incomplete
FROM referrals r
LEFT JOIN LATERAL (SELECT attended_ts FROM appointments x
                     WHERE x.referral_id=r.referral_id AND x.outcome='Attended'
                     ORDER BY attended_ts LIMIT 1) a ON true
GROUP BY r.specialty;

```

Explanation. Reporting both completed and incomplete waits is the correct treatment of censored data — averaging only completed pathways systematically understates the wait.

42. Repeat A&E attendances within 7 days

```

SELECT a1.patient_id, a1.attendance_id, a1.arrival_ts, a2.arrival_ts AS reattendance,
       ROUND(EXTRACT(EPOCH FROM (a2.arrival_ts-a1.arrival_ts))/86400,1) AS days_later
FROM ae_attendances a1
JOIN ae_attendances a2 ON a2.patient_id=a1.patient_id
  AND a2.arrival_ts > a1.arrival_ts
  AND a2.arrival_ts <= a1.arrival_ts + INTERVAL '7 days'
ORDER BY a1.patient_id, a1.arrival_ts;

```

Follow-up. "Why is this a quality metric?" → Unplanned reattendance suggests the first episode didn't resolve the problem. It's a standard NHS indicator.

43. Seasonal decomposition, simple

```

WITH d AS (SELECT order_ts::date AS day, COUNT(*) AS orders FROM orders
            WHERE status='completed' GROUP BY 1),
t AS (SELECT day, orders,
            AVG(orders) OVER (ORDER BY day ROWS BETWEEN 3 PRECEDING AND 3 FOLLOWING) AS trend
      FROM d)
SELECT day, orders, ROUND(trend,1) AS trend, ROUND(orders-trend,1) AS residual,
       ROUND(AVG(orders-trend) OVER (PARTITION BY EXTRACT(ISODOW FROM day)),1) AS dow_effect
FROM t ORDER BY day;

```

Explanation. A centred 7-day moving average estimates trend; the average residual by weekday estimates the seasonal component. Crude but genuinely useful and entirely doable in SQL.

44. Correlation between two metrics

```
SELECT ROUND(CORR(discount_pct, quantity)::numeric,3) AS correlation,
       ROUND(REGR_SLOPE(quantity, discount_pct)::numeric,3) AS slope,
       COUNT(*) AS n
FROM order_items;
```

Follow-up. *"Does discounting cause higher volume?"* → Correlation isn't causation, and here it's confounded: discounts are applied to products that were already selling slowly, or during peaks when demand is high anyway. You'd need an experiment or a natural one.

45. Percentile bands with equal value ranges

```
SELECT WIDTH_BUCKET(lifetime_value, 0, 1000, 10) AS value_bucket,
       COUNT(*), ROUND(MIN(lifetime_value),2), ROUND(MAX(lifetime_value),2)
FROM customer_ltv GROUP BY 1 ORDER BY 1;
```

Explanation. `WIDTH_BUCKET` makes equal-width value bands; `NTILE` makes equal-count bands. Different tools for different questions, and confusing them produces misleading charts.

46. Cross join for a complete reporting matrix

```
SELECT c.category, m.month, COALESCE(r.revenue, 0) AS revenue
FROM (SELECT DISTINCT category FROM products) c
CROSS JOIN generate_series(DATE '2024-01-01', DATE '2024-06-01', INTERVAL '1 month') m(month)
LEFT JOIN category_monthly_revenue r ON r.category=c.category AND r.month=m.month
ORDER BY c.category, m.month;
```

47. Self-join to find overlapping date ranges

```
SELECT a.referral_id, b.referral_id, a.specialty
FROM referrals a JOIN referrals b
  ON a.patient_id=b.patient_id AND a.referral_id < b.referral_id
 AND a.referral_date <= b.referral_date + 30
 AND b.referral_date <= a.referral_date + 30;
```

Explanation. Two ranges overlap when each starts before the other ends — the general overlap condition, worth memorising.

48. Running distinct count (approximation)

```
SELECT d, COUNT(*) FILTER (WHERE is_first) OVER (ORDER BY d ROWS UNBOUNDED PRECEDING) AS cumulative_customers
FROM (SELECT order_ts::date AS d, customer_id,
       ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts) = 1 AS is_first
FROM orders WHERE status='completed') t;
```

Explanation. Cumulative distinct counts aren't windowable directly, but flagging each customer's first appearance and running-summing the flags gives an exact cumulative distinct count. A neat trick worth having.

49. Find the top N with a dynamic N per group

```
SELECT * FROM (
  SELECT p.*, ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) rn,
         COUNT(*) OVER (PARTITION BY category) cat_size
  FROM product_revenue p) t
WHERE rn <= GREATEST(1, CEIL(cat_size * 0.2));
```

50. * Build a complete customer 360 table

Problem. One row per customer with everything a stakeholder might ask for.

Solution.

```
WITH order_values AS (
  SELECT o.order_id, o.customer_id, o.order_ts, o.channel, o.status,
         SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct)) AS order_value,
         SUM(oi.quantity) AS items
  FROM orders o JOIN order_items oi USING (order_id)
  GROUP BY o.order_id, o.customer_id, o.order_ts, o.channel, o.status
),
completed AS (SELECT * FROM order_values WHERE status='completed'),
core AS (
  SELECT customer_id,
         COUNT(*) AS orders,
         SUM(order_value) AS lifetime_value,
         AVG(order_value) AS aov,
         SUM(items) AS total_items,
         MIN(order_ts)::date AS first_order,
         MAX(order_ts)::date AS last_order,
         MODE() WITHIN GROUP (ORDER BY channel) AS preferred_channel
  FROM completed GROUP BY customer_id
),
categories AS (
  SELECT c.customer_id, COUNT(DISTINCT p.category) AS categories_bought,
         MODE() WITHIN GROUP (ORDER BY p.category) AS favourite_category
  FROM completed c JOIN order_items oi USING (order_id) JOIN products p USING (product_id)
  GROUP BY c.customer_id
),
returns AS (
  SELECT customer_id, COUNT(*) AS refunded_orders
  FROM order_values WHERE status='refunded' GROUP BY customer_id
),
cadence AS (
  SELECT customer_id, AVG(gap) AS avg_days_between_orders
  FROM (SELECT customer_id,
               order_ts::date - LAG(order_ts::date) OVER (PARTITION BY customer_id ORDER BY order_ts) AS gap
        FROM completed) g
  WHERE gap IS NOT NULL GROUP BY customer_id
)
SELECT cu.customer_id, cu.first_name, cu.last_name, cu.country, cu.city,
       cu.channel AS acquisition_channel, cu.signup_date,
       COALESCE(co.orders,0) AS orders,
       ROUND(COALESCE(co.lifetime_value,0),2) AS lifetime_value,
       ROUND(co.aov,2) AS aov,
       co.first_order, co.last_order,
       co.first_order - cu.signup_date AS days_to_activate,
       CURRENT_DATE - co.last_order AS days_since_last_order,
       ROUND(ca.avg_days_between_orders,0) AS typical_gap_days,
       COALESCE(cat.categories_bought,0) AS categories_bought,
       cat.favourite_category, co.preferred_channel,
       COALESCE(r.refunded_orders,0) AS refunded_orders,
       NTILE(10) OVER (ORDER BY COALESCE(co.lifetime_value,0) DESC) AS value_decile,
       CASE WHEN co.orders IS NULL THEN 'Never purchased'
            WHEN CURRENT_DATE - co.last_order <= 90 THEN 'Active'
            WHEN CURRENT_DATE - co.last_order <= 365 THEN 'Lapsing'
            ELSE 'Dormant' END AS lifecycle_stage
FROM customers cu
LEFT JOIN core co USING (customer_id)
LEFT JOIN categories cat USING (customer_id)
LEFT JOIN returns r USING (customer_id)
LEFT JOIN cadence ca USING (customer_id)
ORDER BY lifetime_value DESC;
```

Explanation. Each CTE computes one facet at customer grain, then a single LEFT JOIN chain assembles them. Because every branch is already at customer grain, nothing fans out. This is the correct general pattern for wide summary tables, and being able to produce it is close to a take-home-task-passing standard.

Performance. Several scans of `completed`. In production this becomes a materialised view or a nightly summary table with an index on `customer_id`, refreshed on a schedule.

Follow-up. "How would you keep this fresh?" → Incremental refresh: recompute only customers with activity since the last run, and merge into the target table. Full rebuild nightly is fine until it isn't.

51–75. Rapid-fire advanced problems

Each of these should take under ten minutes. Write your answer before reading the note.

- 51.** Customers whose order values are consistently above average — every single order above the global mean. (*HAVING with `MIN(order_value) > the global average`.*)
- 52.** The order that pushed cumulative revenue past £10,000. (*Running total, then filter where the running total crosses the threshold and the previous one didn't.*)
- 53.** Rank products by revenue but exclude the top customer's contribution. (*Filter in the aggregate with `FILTER`, or anti-join the top customer first.*)
- 54.** Products whose sales fell in three consecutive months. (*LAG twice, or gaps-and-islands on the "declined" flag.*)
- 55.** Customers who have bought in every month since their first order. (*Count distinct active months = months elapsed since first order.*)
- 56.** Median time between first and second order, by acquisition channel. (*ROW_NUMBER to pick orders 1 and 2, self-join, PERCENTILE_CONT grouped by channel.*)
- 57.** Share of revenue from new vs returning customers each month. (*Classify each order by whether it's the customer's first, then conditional aggregation.*)
- 58.** Products that are only ever bought alone. (*NOT EXISTS another line on the same order.*)
- 59.** Orders where every line was discounted. (*GROUP BY order, HAVING COUNT() FILTER (WHERE discount_pct=0) = 0.*)*
- 60.** The most common pair of consecutive categories in a customer's purchase sequence. (*LAG the category over the customer's ordered purchases, group by the pair.*)
- 61.** Customers whose spending is more variable than average. (*STDDEV per customer versus the average of those standard deviations.*)
- 62.** Weekly cohort retention rather than monthly. (*Same as monthly with `DATE_TRUNC('week')` and a week offset.*)
- 63.** Detect price changes over time from order history. (*DISTINCT product_id, unit_price with MIN/MAX order date per price; LAG to find changes.*)
- 64.** Revenue per active customer per month. (*Monthly revenue ÷ monthly distinct active customers — note this is not the same as averaging customer-level values.*)
- 65.** Customers who upgraded to a higher-value category over time. (*Compare first-purchase category average price against latest.*)
- 66.** Longest gap between orders for each customer. (*MAX of the LAG-derived gaps.*)
- 67.** Orders placed outside business hours by channel. (*EXTRACT(HOUR) and ISODOW filters, conditional aggregation.*)
- 68.** Specialties where waiting times worsened month on month for three months. (*Monthly median wait, LAG twice or the islands method.*)
- 69.** Patients appearing in both A&E and the elective waiting list within 30 days. (*Self-join across tables with an interval condition.*)
- 70.** The proportion of revenue from customers acquired in the last 12 months. (*Conditional aggregation on signup date relative to the reporting date.*)
- 71.** Products where revenue is concentrated in a single customer. (*Max customer share per product via a window over product totals — a concentration risk metric.*)
- 72.** Simulate a "what if we removed all discounts" revenue figure. (*Sum quantity × unit_price without the discount factor, and state clearly that it assumes demand is unchanged, which it isn't.*)
- 73.** Identify orders likely to be fraudulent by simple rules. (*Unusually high value versus the customer's history, new account, multiple orders within minutes — combine flags and score.*)
- 74.** Build a daily snapshot table of the waiting list. (*CROSS JOIN a date series with the waiting list, keeping rows where the date falls between added and removed — this is how you reconstruct history from event data.*)

75. Given a query someone else wrote that runs for 40 minutes, describe your diagnostic process. (*EXPLAIN ANALYZE; compare estimated vs actual rows to spot bad statistics; find the node with the largest actual time; check for accidental cross joins, functions on filtered columns, unnecessary DISTINCT, and unbounded date ranges; check whether the result grain is even correct, because slow and wrong often share a cause.*)

Self-check

By this point you should be able to:

- Build a cohort retention table from scratch without notes.
- Solve gaps and islands two different ways and explain both.
- Recognise censoring and survivorship bias in a metric and say so unprompted.
- Assemble a wide summary table without fan-out.
- Decompose a change into contributions that sum to the total.
- Read an EXPLAIN ANALYZE plan well enough to name the problem.
- Say "that metric definition is ambiguous, here's what I'd need to confirm" without it sounding like stalling.

That last one matters more than any query in this file.

Parts 17–19: Pattern Recognition, Mock Interviews, Debugging

PART 17 — SQL INTERVIEW PATTERN RECOGNITION

Interview questions are phrased in business language. The skill is mapping the phrase to the technique before you start typing. Read the left column until the right column is automatic.

17.1 Master cheat sheet

Selection and ranking

When you hear	Think	Core construct
"latest / most recent record per X"	greatest-per-group	<code>ROW_NUMBER() OVER (PARTITION BY x ORDER BY ts DESC) = 1</code> , or <code>DISTINCT ON (x)</code>
"first / earliest per X"	same, ascending	<code>ROW_NUMBER() ... ORDER BY ts</code>
"top 3 per department/category"	top N per group	<code>ROW_NUMBER/DENSE_RANK() <= 3</code> in a subquery
"the single highest/lowest"	argmax	<code>ORDER BY ... LIMIT 1</code> , or <code>DISTINCT ON</code>
"second highest"	offset ranking	<code>DENSE_RANK() = 2</code> , or <code>LIMIT 1 OFFSET 1</code>
"including ties"	RANK family	<code>DENSE_RANK()</code> not <code>ROW_NUMBER</code>
"top 10%"	percentile	<code>NTILE(10) = 1</code> or <code>PERCENT_RANK() <= 0.1</code>
"rank them"	ranking window	<code>RANK()</code> , and ask about tie behaviour
"nth record"	positional	<code>ROW_NUMBER() = n</code>

Comparison across rows

When you hear	Think	Core construct
"previous month / prior value"	offset	<code>LAG(x) OVER (ORDER BY month)</code>
"next transaction"	forward offset	<code>LEAD(x) OVER (...)</code>
"month-on-month change"	LAG + arithmetic	<code>(x - LAG(x))/NULLIF(LAG(x),0)</code>
"year-on-year"	date self-join	join on month - INTERVAL '1 year', not <code>LAG(12)</code>
"compared to their first order"	FIRST_VALUE	<code>FIRST_VALUE(x) OVER (PARTITION BY c ORDER BY ts)</code>
"compared to the group average"	partitioned aggregate	<code>AVG(x) OVER (PARTITION BY g)</code>
"days between events"	LAG on a date	<code>d - LAG(d) OVER (...)</code>
"consecutive / streak / in a row"	gaps and islands	<code>d - ROW_NUMBER()</code> grouping, or flag-and-cumsum

Aggregation shapes

When you hear	Think	Core construct
"running / cumulative total"	window sum	<code>SUM(x) OVER (ORDER BY d ROWS UNBOUNDED PRECEDING)</code>
"moving / rolling average"	framed window	<code>AVG(x) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)</code>
"percentage of total"	window aggregate	<code>x / SUM(x) OVER ()</code>
"share within each group"	partitioned window	<code>x / SUM(x) OVER (PARTITION BY g)</code>
"cumulative share / 80-20"	Pareto	running sum ÷ grand total, both as windows
"customers who have more than N..."	group filter	<code>GROUP BY ... HAVING COUNT(*) > N</code>
"average per customer"	two-stage aggregation	aggregate to customer grain in a CTE, then average
"count of X and Y in one row"	conditional aggregation	<code>COUNT(*) FILTER (WHERE ...)</code>
"pivot months into columns"	conditional aggregation	<code>SUM(x) FILTER (WHERE month = ...)</code>

Presence and absence

When you hear	Think	Core construct
"customers who never..."	anti-join	<code>NOT EXISTS</code> , or <code>LEFT JOIN ... WHERE key IS NULL</code>
"who have at least one..."	semi-join	<code>EXISTS</code>
"bought X but not Y"	<code>EXISTS</code> + <code>NOT EXISTS</code>	two correlated subqueries
"bought both X and Y"	two <code>EXISTS</code> , or <code>HAVING</code>	<code>HAVING COUNT(DISTINCT category) = 2</code> with a filter
"bought every product in..."	relational division	<code>HAVING COUNT(DISTINCT p) = (SELECT COUNT(*) FROM targets)</code>
"including those with zero"	outer join	<code>LEFT JOIN + COALESCE(...,0)</code> , filter in <code>ON</code> not <code>WHERE</code>
"missing dates / gaps"	calendar scaffold	<code>generate_series</code> + <code>LEFT JOIN</code> , or <code>LEAD</code> to find jumps

Business metrics

When you hear	Think	Watch out for
"cohort"	first-event month + activity months + offset	month 0 must be 100%
"retention"	self-join on period offset	<code>LEFT JOIN</code> , or you get 100%
"churn"	define the rule first	no cancel event in retail — it's a threshold you choose
"funnel"	flags per session, then step ratios	collapse to session grain first
"conversion rate"	numerator ÷ denominator	sessions or users? <code>100.0 *</code> for float division
"AOV"	order-grain aggregate then average	not line-level average
"LTV"	sum per customer	biased by tenure — normalise by cohort age
"DAU/MAU"	distinct counts at two grains	distinct counts are not additive
"growth rate"	LAG or date self-join	tiny base makes percentages meaningless
"market share"	window sum as denominator	internal share ≠ market share
"SLA / breach rate"	interval comparison	what about still-open cases?
"waiting time"	date difference	completed-only averages hide the longest waits

Data quality

When you hear	Think	Core construct
"duplicates"	<code>ROW_NUMBER</code> dedupe	define the key and the winner rule
"clean this data"	standardise → validate → dedupe	one CTE per step
"missing values"	<code>COALESCE</code> / <code>NULLIF</code>	is <code>NULL</code> a zero, or genuinely unknown?
"the numbers don't match"	reconciliation	<code>FULL OUTER JOIN + IS DISTINCT FROM</code>
"row counts are too high"	fan-out	pre-aggregate to a common grain before joining
"this query is slow"	<code>EXPLAIN ANALYZE</code>	estimated vs actual rows first

17.2 Reflexes worth drilling

Five things to say without thinking, because each one signals experience:

1. **"What's the grain of this table?"** before writing any join.
2. **"Should cancelled and refunded orders count?"** before any revenue figure.
3. **"I'd use `NOT EXISTS` rather than `NOT IN`, because `NOT IN` returns nothing if the subquery has a `NULL`."**
4. **"I'll put that filter in the `ON` clause so the `LEFT JOIN` stays a `LEFT JOIN`."**
5. **"That'll be integer division — I'll multiply by 100.0."**

17.3 Phrase → clause quick map

- "for each" → `GROUP BY` or `PARTITION BY`
 - "per" → same
 - "at least / more than N" → `HAVING`
 - "only / exclusively" → `NOT EXISTS` on the complement
 - "in the last 30 days" → `>= CURRENT_DATE - INTERVAL '30 days'`
 - "by month" → `DATE_TRUNC('month', ...)`
 - "in March" (all years) → `EXTRACT(MONTH ...) = 3`
 - "excluding" → `NOT EXISTS` or `<>` with a `NULL` check
 - "including zero" → `LEFT JOIN + COALESCE`
 - "consecutive" → gaps and islands
 - "compared to" → window function or self-join
 - "trend" → `LAG` or moving average
 - "distribution" → `GROUP BY` the bucketed value, or percentiles
-

PART 18 — TEN MOCK SQL INTERVIEWS

Do these out loud. Say your reasoning as you would in the room, then write the query, then read the follow-up before looking at the improved solution.

Mock 1 — Junior: retail basics

Interviewer. "Here's our orders and customers tables. Can you tell me how many orders each customer placed last month, and include customers who didn't order?"

Schema. `customers(customer_id, first_name, last_name, country), orders(order_id, customer_id, order_ts, status).`

Candidate thinking. Every customer must appear, so LEFT JOIN from customers. "Last month" is a date range on the orders side, so it belongs in the ON clause, not WHERE, otherwise I lose the zero-order customers. Count the order id, not *.

Candidate SQL.

```
SELECT c.customer_id, c.first_name, c.last_name,
       COUNT(o.order_id) AS orders_last_month
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.customer_id
 AND o.order_ts >= DATE_TRUNC('month', CURRENT_DATE) - INTERVAL '1 month'
 AND o.order_ts < DATE_TRUNC('month', CURRENT_DATE)
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY orders_last_month DESC;
```

Expected output. Every customer, with 0 for the inactive ones.

Follow-up. "What would happen if you'd put the date conditions in WHERE?"

Answer. Customers with no matching order get NULL in every `orders` column; `NULL >= date` is UNKNOWN, so WHERE drops those rows and the LEFT JOIN silently becomes an INNER JOIN.

Junior vs strong. A junior writes `COUNT(*)` and reports that everyone placed at least one order. A strong candidate uses `COUNT(o.order_id)`, explains the ON-vs-WHERE choice before being asked, and mentions that `DATE_TRUNC` makes "last month" reproducible regardless of when it's run.

Mock 2 — Junior: aggregation and filtering

Interviewer. "Which product categories generated more than £10,000 in revenue this year?"

Candidate thinking. Revenue is on `order_items`. Categories are on `products`. Status is on `orders` — I should check whether cancelled and refunded orders count. I'll ask.

Candidate. "Should I include refunded orders in revenue?" — Interviewer: "No, completed only."

Candidate SQL.

```
SELECT p.category,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct)), 2) AS revenue
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
JOIN orders o ON o.order_id = oi.order_id
WHERE o.status = 'completed'
 AND o.order_ts >= DATE_TRUNC('year', CURRENT_DATE)
GROUP BY p.category
HAVING SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct)) > 10000
ORDER BY revenue DESC;
```

Follow-up. "Why HAVING and not WHERE?"

Answer. The threshold is on an aggregate, which doesn't exist until after GROUP BY. WHERE runs first and can only see individual rows.

Follow-up 2. "Can you reference the alias `revenue` in HAVING?"

Answer. Not portably. Postgres allows aliases in GROUP BY and ORDER BY but not HAVING; repeat the expression, or wrap the aggregation in a CTE and filter outside — which reads better anyway.

Junior vs strong. The strong candidate asks about statuses before writing, and uses a CTE to avoid repeating the revenue expression twice.

Mock 3 — Intermediate: the grain trap

Interviewer. "Give me total revenue and total shipping cost by month."

Candidate thinking. Revenue is on the line, shipping is on the order. If I join them and sum both, shipping gets multiplied by the number of lines. I need to aggregate lines to order grain first.

Candidate SQL.

```
WITH order_lines AS (
  SELECT order_id, SUM(quantity * unit_price * (1 - discount_pct)) AS product_revenue
  FROM order_items
  GROUP BY order_id
)
SELECT DATE_TRUNC('month', o.order_ts)::date AS month,
       ROUND(SUM(l.product_revenue), 2) AS product_revenue,
       ROUND(SUM(o.shipping_cost), 2) AS shipping_revenue,
       COUNT(*) AS orders
FROM orders o
JOIN order_lines l ON l.order_id = o.order_id
WHERE o.status = 'completed'
GROUP BY 1 ORDER BY 1;
```

Follow-up. "What would the naive version have given you?"

Answer. Correct revenue, inflated shipping — each order's shipping counted once per line. A three-line order contributes triple.

Follow-up 2. "How would you have caught that in review?"

Answer. Compare `SUM(shipping_cost)` from `orders` alone against the joined version. Any difference means fan-out. More generally: check `COUNT(*)` before and after every join and know why it changed.

Junior vs strong. A junior writes the single join and ships a number that's 2.4× too high with no way of noticing. A strong candidate names the grain problem before writing and volunteers the reconciliation check.

Mock 4 — Intermediate: window functions

Interviewer. "For each customer, show their orders in sequence with the value of the previous order and the change."

Candidate SQL.

```
WITH order_values AS (
  SELECT o.order_id, o.customer_id, o.order_ts,
         SUM(o.quantity * oi.unit_price * (1 - oi.discount_pct)) AS order_value
  FROM orders o JOIN order_items oi USING (order_id)
  WHERE o.status = 'completed'
  GROUP BY o.order_id, o.customer_id, o.order_ts
)
SELECT customer_id, order_id, order_ts::date AS order_date,
       ROW_NUMBER() OVER w AS order_number,
       ROUND(order_value, 2) AS order_value,
       ROUND(LAG(order_value) OVER w, 2) AS previous_order_value,
       ROUND(order_value - LAG(order_value) OVER w, 2) AS change,
       order_ts::date - LAG(order_ts::date) OVER w AS days_since_previous
FROM order_values
WINDOW w AS (PARTITION BY customer_id ORDER BY order_ts)
ORDER BY customer_id, order_ts;
```

Follow-up. "Why is the first row's change NULL, and is that right?"

Answer. There's no previous order, so LAG returns NULL. It's correct — a change of zero would be a lie. If a default is needed for a dashboard, `LAG(order_value, 1, 0)` supplies one, but I'd only do that if the business wants it.

Follow-up 2. "Now flag customers whose most recent order was smaller than the one before."

Answer. Add `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts DESC) AS rn_desc`, wrap in a subquery, filter `rn_desc = 1 AND order_value < previous_order_value`.

Junior vs strong. The `WINDOW` clause, naming the window once instead of repeating it four times, is a small thing that reads as fluency. So is knowing LAG's third argument exists.

Mock 5 — Intermediate: the NULL trap

Interviewer. "Find customers who have never placed an order. Here's what a colleague wrote — it returns nothing. Why?"

```
SELECT * FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders);
```

Candidate. "If any row in `orders` has a NULL `customer_id`, `NOT IN` returns no rows at all. `x NOT IN (1,2,NULL)` expands to `x<>1 AND x<>2 AND x<>NULL`; the last comparison is UNKNOWN, so the whole AND can never be TRUE. It fails silently — an empty result, no error."

Candidate SQL.

```
SELECT c.* FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

Follow-up. "Any other way?"

Answer. LEFT JOIN with `WHERE o.order_id IS NULL` — an anti-join, same plan usually. Or keep `NOT IN` and add `WHERE customer_id IS NOT NULL` to the subquery, though I'd still prefer `NOT EXISTS` because it can't break again if the data changes.

Follow-up 2. "Which is fastest?"

Answer. Usually `NOT EXISTS` or the anti-join — both let the planner use a hash anti-join. `NOT IN` often can't, because it has to preserve the NULL semantics.

Junior vs strong. The junior says "add `DISTINCT`" or "I'm not sure". The strong candidate explains three-valued logic in two sentences and, unprompted, checks whether `orders.customer_id` actually contains NULLs.

Mock 6 — Strong: cohort retention

Interviewer. "Build me a monthly cohort retention table. Talk me through it as you go."

Candidate thinking, spoken. "Three pieces. First, each customer's cohort — the month of their first completed order. Second, every month each customer was active. Third, join those and compute the offset in months, then count distinct customers per cohort per offset, divided by the original cohort size. Month 0 should come out at 100% for every cohort, which is my check that it's right."

Candidate SQL. As in Part 12.9.

Follow-up. "The most recent cohort has nothing beyond month 0. Has retention collapsed?"

Answer. No — those months haven't happened yet. Right-censoring. I'd only compare cohorts at ages every cohort has reached, and grey out or blank the immature cells rather than letting them read as zeros. Treating censored cells as zeros is how people conclude a product is dying when it isn't.

Follow-up 2. "Cohort by signup date instead of first purchase — what changes?"

Answer. It becomes activation plus retention combined. Customers who never buy are in the denominator, so month 0 is no longer 100% and the figures drop. Both are legitimate; signup cohorts are right for evaluating acquisition, purchase cohorts for evaluating the product experience.

Follow-up 3. "Retention improved from 40% to 55% between two cohorts. What would you check before celebrating?"

Answer. Cohort composition. If a poor-quality paid channel was switched off, retention improves without anything getting better — you just stopped buying customers who were never going to stay. I'd segment by acquisition channel and see whether within-channel retention moved at all.

Junior vs strong. The junior produces a correct table. The strong candidate produces the table, states the month-0 sanity check unprompted, and immediately identifies censoring and mix effects. The third answer is what gets the offer.

Mock 7 — Strong: funnel and drop-off

Interviewer. "Our checkout conversion has dropped. Investigate with SQL."

Candidate thinking. First, quantify the drop and locate the step. Then segment to find where it's concentrated. Then check whether it's real or a tracking artefact.

Step 1 — funnel by week.

```
WITH session_steps AS (
  SELECT session_id, DATE_TRUNC('week', MIN(event_ts))::date AS week,
         MAX((event_name='product_view')::int) AS viewed,
         MAX((event_name='add_to_cart')::int) AS carted,
         MAX((event_name='checkout_start')::int) AS checkout,
         MAX((event_name='purchase')::int) AS purchased,
         MAX(device) AS device
  FROM web_events
  WHERE event_ts >= CURRENT_DATE - 84
  GROUP BY session_id
)
SELECT week,
       SUM(viewed) AS viewed, SUM(carted) AS carted,
       SUM(checkout) AS checkout, SUM(purchased) AS purchased,
       ROUND(100.0*SUM(carted)/NULLIF(SUM(viewed),0),1) AS view_to_cart,
       ROUND(100.0*SUM(checkout)/NULLIF(SUM(carted),0),1) AS cart_to_checkout,
       ROUND(100.0*SUM(purchased)/NULLIF(SUM(checkout),0),1) AS checkout_to_purchase
FROM session_steps GROUP BY week ORDER BY week;
```

Step 2 — segment the failing step by device and channel. Add `device` to the GROUP BY; join sessions to acquisition source if available.

Step 3 — rule out tracking. Did total event volume for that step drop at the same time? If `checkout_start` events fell but purchases held steady, the event stopped firing rather than users stopping.

```
SELECT DATE_TRUNC('day', event_ts)::date AS day, event_name, COUNT(*) AS events
FROM web_events WHERE event_ts >= CURRENT_DATE - 60
GROUP BY 1,2 ORDER BY 1,2;
```

Follow-up. "Checkout-to-purchase fell from 60% to 45% on mobile only, three weeks ago. Next steps?"

Answer. Correlate with a release date. Check whether it's all mobile or specific browsers/OS versions if that's tracked. Check payment failure rates if there's a payments table. And check whether mobile traffic *mix* changed — a surge of low-intent paid mobile traffic lowers the rate without anything breaking.

Junior vs strong. The junior computes one overall conversion number. The strong candidate segments to localise the drop, and — the differentiator — checks whether the metric broke rather than the product. Instrumentation failures cause a large share of apparent metric drops, and knowing to rule that out first saves everyone a week.

Mock 8 — Strong: NHS waiting times

Interviewer. "Report the average wait from referral to first appointment by specialty."

Candidate. "Before I write it — do you want completed pathways only, or should patients still waiting be included? And by 'first appointment', do you mean the first attended one, or the first booked, including DNAs?"

Interviewer. "Good question. First attended. And let's see both completed and incomplete."

Candidate SQL.

```
WITH first_seen AS (
  SELECT r.referral_id, r.specialty, r.priority, r.referral_date,
```

```

        (SELECT MIN(a.attended_ts)::date FROM appointments a
         WHERE a.referral_id = r.referral_id AND a.outcome = 'Attended') AS first_attended
FROM referrals r
)
SELECT specialty,
COUNT(*) FILTER (WHERE first_attended IS NOT NULL) AS completed_pathways,
COUNT(*) FILTER (WHERE first_attended IS NULL) AS still_waiting,
ROUND(AVG(first_attended - referral_date) FILTER (WHERE first_attended IS NOT NULL),1)
AS mean_wait_completed,
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY first_attended - referral_date)
FILTER (WHERE first_attended IS NOT NULL) AS median_wait_completed,
ROUND(AVG(CURRENT_DATE - referral_date) FILTER (WHERE first_attended IS NULL),1)
AS mean_wait_so_far_incomplete,
COUNT(*) FILTER (WHERE COALESCE(first_attended, CURRENT_DATE) - referral_date > 126)
AS over_18_weeks
FROM first_seen
GROUP BY specialty
ORDER BY median_wait_completed DESC;

```

Follow-up. "Why report the median as well as the mean?"

Answer. Waiting times are right-skewed. A handful of very long waits pull the mean up, so the mean overstates the typical wait while the median understates the tail. For a standard measured at the 92nd percentile, I'd report p92 too — that's the number that actually determines compliance.

Follow-up 2. "One specialty's average wait fell 20% this quarter. Is that good?"

Answer. Possibly, but check the queue. If they cleared a backlog of short-wait routine cases while long-waiters stayed on the list, the completed-pathway average falls while the situation gets worse. I'd look at the incomplete-pathway distribution and the total list size alongside it. Waiting time and list size have to be read together.

Junior vs strong. Asking the two clarifying questions up front is most of the marks. The rest is knowing that censored data flatters completed-only averages, and saying so before being asked.

Mock 9 — Advanced: debugging someone else's query

Interviewer. "This is meant to give revenue per customer for 2024. Finance says it's wrong. Find the problems."

```

SELECT c.first_name, c.last_name,
SUM(oi.quantity * oi.unit_price) AS revenue,
COUNT(*) AS orders,
o.shipping_cost
FROM customers c, orders o, order_items oi
WHERE c.customer_id = o.customer_id
AND o.order_id = oi.order_id
AND YEAR(o.order_ts) = 2024
GROUP BY c.first_name, c.last_name;

```

Candidate, working through it.

1. `YEAR()` doesn't exist in Postgres — it's `EXTRACT(YEAR FROM ...)`. So this doesn't even run. And wrapping the column in a function blocks index use; a half-open range is better.
2. `o.shipping_cost` is selected but not grouped or aggregated — that's a `GROUP BY` error in Postgres.
3. Grouping by name, not id — two customers with the same name merge into one row.
4. `COUNT(*)` counts order *lines*, not orders. Should be `COUNT(DISTINCT o.order_id)`.
5. No status filter — cancelled and refunded orders are counted as revenue.
6. `discount_pct` is ignored, so revenue is overstated on every discounted line.
7. Implicit comma joins hide the join conditions among the filters. Fine syntactically, poor practice.
8. Customers with no 2024 orders disappear entirely — may or may not be intended, worth confirming.

Corrected.

```

SELECT c.customer_id, c.first_name, c.last_name,
ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct)), 2) AS revenue,
COUNT(DISTINCT o.order_id) AS orders,
ROUND(SUM(DISTINCT_SHIP.shipping), 2) AS shipping
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id

```

```
JOIN order_items oi ON oi.order_id = o.order_id
JOIN LATERAL (SELECT o.shipping_cost AS shipping) DISTINCT_SHIP ON true
WHERE o.status = 'completed'
AND o.order_ts >= DATE '2024-01-01' AND o.order_ts < DATE '2025-01-01'
GROUP BY c.customer_id, c.first_name, c.last_name;
```

Candidate. "Actually, that shipping column is still wrong — the LATERAL doesn't fix the fan-out. Shipping is order-level, so it needs a separate aggregation:"

```
WITH order_lines AS (
  SELECT order_id, SUM(quantity*unit_price*(1-discount_pct)) AS revenue
  FROM order_items GROUP BY order_id
)
SELECT c.customer_id, c.first_name, c.last_name,
       ROUND(SUM(l.revenue),2) AS revenue,
       COUNT(*) AS orders,
       ROUND(SUM(o.shipping_cost),2) AS shipping
FROM customers c
JOIN orders o ON o.customer_id=c.customer_id
JOIN order_lines l ON l.order_id=o.order_id
WHERE o.status='completed'
AND o.order_ts >= DATE '2024-01-01' AND o.order_ts < DATE '2025-01-01'
GROUP BY c.customer_id, c.first_name, c.last_name;
```

Junior vs strong. The junior fixes the syntax error and stops. The strong candidate finds all eight issues, prioritises them by business impact (the status filter and the discount are the ones changing the number finance sees), and — notably — catches their own incomplete first fix rather than defending it.

Mock 10 — Advanced: open-ended business problem

Interviewer. "Revenue was down 12% last month. You have full data access. Walk me through what you'd do."

Candidate. "I'd decompose before I explain. Revenue is orders × average order value, so first: which one moved?"

```
WITH monthly AS (
  SELECT DATE_TRUNC('month', o.order_ts)::date AS month,
         COUNT(DISTINCT o.order_id) AS orders,
         SUM(ov.order_value) AS revenue
  FROM orders o JOIN order_values ov USING (order_id)
  WHERE o.status='completed' GROUP BY 1
)
SELECT month, orders, ROUND(revenue,2) AS revenue,
       ROUND(revenue/NULLIF(orders,0),2) AS aov,
       ROUND(100.0*(orders - LAG(orders) OVER (ORDER BY month))
             /NULLIF(LAG(orders) OVER (ORDER BY month),0),1) AS order_growth_pct,
       ROUND(100.0*(revenue/NULLIF(orders,0) - LAG(revenue/NULLIF(orders,0)) OVER (ORDER BY month))
             /NULLIF(LAG(revenue/NULLIF(orders,0)) OVER (ORDER BY month),0),1) AS aov_growth_pct
FROM monthly ORDER BY month;
```

"Then: is it volume or value, and where is it concentrated? I'd run contribution analysis by category, channel and customer segment — the version in Part 12.21 that decomposes the total change into parts that sum back to it, so I can say 'Electronics accounts for 9 of the 12 points' rather than 'Electronics is down'."

"Then: new versus returning customers. If new customer orders fell, it's acquisition. If returning fell, it's retention or a product problem."

```
SELECT DATE_TRUNC('month',o.order_ts)::date AS month,
       COUNT(*) FILTER (WHERE seq = 1) AS first_orders,
       COUNT(*) FILTER (WHERE seq > 1) AS repeat_orders,
       ROUND(SUM(ov.order_value) FILTER (WHERE seq=1),2) AS new_customer_revenue,
       ROUND(SUM(ov.order_value) FILTER (WHERE seq>1),2) AS returning_revenue
FROM (SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts) AS seq
      FROM orders o WHERE status='completed') o
JOIN order_values ov USING (order_id)
GROUP BY 1 ORDER BY 1;
```

"Then the boring checks, which I'd actually do first in practice: was last month shorter? Did it have fewer weekends or a bank holiday? Is the data complete — did a load fail? Is this outside normal month-to-month variation at all, or is 12% within the usual range? A 12% drop against a series that routinely swings 10% is not a finding."

```
-- variation check
SELECT ROUND(STDDEV(mom_pct),1) AS typical_monthly_swing, ROUND(AVG(mom_pct),1) AS mean_growth
FROM (SELECT 100.0*(revenue-LAG(revenue) OVER (ORDER BY month))
```

```
        /NULLIF(LAG(revenue) OVER (ORDER BY month),0) AS mom_pct
FROM monthly) t WHERE mom_pct IS NOT NULL;

-- completeness check
SELECT order_ts::date AS d, COUNT(*) FROM orders
WHERE order_ts >= CURRENT_DATE - 60 GROUP BY 1 ORDER BY 1;
```

Follow-up. "You find paid search orders fell 40% and everything else is flat. What now?"

Answer. That localises it to one channel. Check spend — if the budget was cut, that's the whole story and it's a finance question, not an analytics one. If spend held, look at cost per click and impressions for a bidding or tracking change, and check whether the UTM tagging changed, which would move orders into 'unattributed' rather than losing them. I'd want to distinguish "we sold less" from "we labelled it differently" before anyone reacts.

Junior vs strong. The junior writes one query showing revenue by month and says revenue is down. The strong candidate decomposes systematically, checks the boring explanations before the interesting ones, quantifies whether the change is even outside normal variation, and separates a real decline from a measurement change. Notice how little of that answer is SQL technique.

PART 19 — SQL DEBUGGING

19.1 A debugging method

1. **Does it run?** Syntax errors are the easy case — read the position marker in the error.
2. **Is the row count plausible?** Compare against the base table. More rows than expected means fan-out; fewer means an inner join or a filter dropping things.
3. **Is the grain right?** "One row per what?" applied to your output.
4. **Spot-check one entity.** Pick one customer, compute their number by hand from the raw rows, compare. This catches more real bugs than anything else.
5. **Check the edges.** Zero, NULL, ties, the first and last period, entities with no activity.
6. **Reconcile totals.** Your revenue total should match the total from a simpler query.
7. **Only then worry about speed.**

19.2 Broken queries — find the bug

Work through each before reading the answer.

Bug 1

```
SELECT customer_id, COUNT(*) FROM orders
WHERE COUNT(*) > 5 GROUP BY customer_id;
```

Answer

Aggregates can't appear in WHERE, which runs before grouping. Use `HAVING COUNT(*) > 5`.

Bug 2

```
SELECT category, product_name, AVG(unit_price) FROM products GROUP BY category;
```

Answer

`product_name` is neither grouped nor aggregated — an error in Postgres. Either add it to GROUP BY (changing the meaning entirely) or drop it. If you want the name alongside the category average, use a window function: `AVG(unit_price) OVER (PARTITION BY category)`.

Bug 3

```
SELECT c.customer_id, COUNT(*) AS orders
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id;
```

Answer

`COUNT(*)` returns 1 for customers with no orders, because the LEFT JOIN still produces a row. Use `COUNT(o.order_id)`.

Bug 4

```
SELECT c.* FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'completed';
```

Answer

The WHERE filter on the right table eliminates the NULL-extended rows, so this is an inner join with extra steps. Move the condition into ON if you want to preserve all customers.

Bug 5

```
SELECT * FROM orders WHERE order_ts BETWEEN '2024-03-01' AND '2024-03-31';
```

Answer

`order_ts` is a timestamp; '2024-03-31' is midnight, so everything on 31 March after 00:00:00 is lost. Use `>= '2024-03-01'` AND `< '2024-04-01'`.

Bug 6

```
SELECT * FROM customers WHERE customer_id NOT IN (SELECT customer_id FROM orders);
```

Answer

Returns zero rows if any `orders.customer_id` is NULL. Use `NOT EXISTS`.

Bug 7

```
SELECT COUNT(*) FROM orders WHERE discount_code <> 'SPRING10';
```

Answer

Excludes every row where `discount_code` IS NULL — usually most of them. Use `IS DISTINCT FROM 'SPRING10'`.

Bug 8

```
SELECT o.order_id, SUM(oi.quantity), SUM(s.weight_kg)
FROM orders o
JOIN order_items oi USING (order_id)
JOIN shipments s USING (order_id)
GROUP BY o.order_id;
```

Answer

Join explosion. Two one-to-many joins off the same parent multiply each other: 3 items × 2 shipments = 6 rows, and both sums are inflated. Pre-aggregate each branch to order grain, then join the summaries.

Bug 9

```
SELECT COUNT(purchases) / COUNT(sessions) AS conversion_rate FROM funnel;
```

Answer

Two bugs. Integer division truncates to 0 for anything under 100% — multiply by `100.0`. And `COUNT(col)` counts non-NULL rows, not sums; if these are numeric columns you want `SUM`, not `COUNT`. Also add `NULLIF(denominator, 0)`.

Bug 10

```
SELECT customer_id, order_ts,
       LAST_VALUE(order_value) OVER (PARTITION BY customer_id ORDER BY order_ts) AS latest_value
FROM order_values;
```

Answer

The default frame ends at the current row, so `LAST_VALUE` returns the current row's own value on every row. Add `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`, or use `FIRST_VALUE(...)` OVER (... ORDER BY order_ts DESC).

Bug 11

```
SELECT month, revenue, revenue - LAG(revenue) OVER (ORDER BY month) AS change
FROM monthly_revenue;
```

Answer

Only wrong if months can be missing. LAG returns the previous *row*, not the previous month, so a gap makes it compare across two months while labelling it as one. Zero-fill with `generate_series` first, or join on the date offset.

Bug 12

```
SELECT patient_id,
       CASE WHEN AGE(CURRENT_DATE, date_of_birth) < INTERVAL '18 years' THEN '0-17'
            WHEN AGE(CURRENT_DATE, date_of_birth) < INTERVAL '65 years' THEN '18-64'
            ELSE '65+' END AS age_band
FROM patients;
```

Answer

Patients with a NULL date of birth fail every comparison (all UNKNOWN) and fall into `ELSE`, silently labelled '65+'. Add `WHEN date_of_birth IS NULL THEN 'Unknown'` as the **first** branch.

Bug 13

```
SELECT * FROM (
  SELECT customer_id, order_ts,
         ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts DESC) rn
  FROM orders) t
WHERE rn = 1;
```

Answer

Not wrong exactly, but non-deterministic: if a customer has two orders with identical timestamps, which one wins changes between runs. Add a unique tie-breaker: `ORDER BY order_ts DESC, order_id DESC`. Also, no status filter — is that intended?

Bug 14

```
SELECT AVG(feedback_score) FROM survey_responses;
```

Answer

`AVG` ignores NULLs, so this averages only respondents. If non-response should count as zero, you need `AVG(COALESCE(feedback_score,0))`. Which is right is a business decision — but silently picking one without knowing is the bug.

Bug 15

```
SELECT DISTINCT c.customer_id, c.first_name, SUM(oi.quantity*oi.unit_price) AS revenue
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY c.customer_id, c.first_name;
```

Answer

The `DISTINCT` is pointless — `GROUP BY` already guarantees one row per group. Its presence suggests the author didn't understand why they had duplicates and added `DISTINCT` hopefully. Harmless here, but a red flag in review. Also missing a status filter.

Bug 16

```
SELECT e.full_name, m.full_name AS manager
FROM employees e JOIN employees m ON m.employee_id = e.manager_id;
```

Answer

Inner join drops the CEO, whose `manager_id` is NULL. Use `LEFT JOIN`. Any org chart missing exactly one person at the top is this bug.

Bug 17

```
SELECT category, SUM(revenue) / SUM(units) AS revenue_per_unit
FROM sales GROUP BY category;
```

Answer

Division by zero if any category has zero units — the whole query errors out, not just that row. `NULLIF(SUM(units), 0)`.

Bug 18

```
SELECT DATE_TRUNC('month', order_ts) AS month, COUNT(*)
FROM orders GROUP BY EXTRACT(MONTH FROM order_ts);
```

Answer

SELECT and GROUP BY disagree — grouping merges all Marches across years while the SELECT shows a specific month, so Postgres errors (the selected expression isn't grouped). Even if it ran, the intent is ambiguous. Pick one:

`DATE_TRUNC` for a time series, `EXTRACT` for seasonality.

Bug 19

```
SELECT * FROM orders o
JOIN dim_customer d ON d.customer_id = o.customer_id;
```

Answer

If `dim_customer` is an SCD Type 2 dimension with one row per version, this multiplies every order by the number of versions of that customer. Join on the key **and** the validity window, or restrict to `d.is_current`.

Bug 20

```
SELECT ROUND(100 * COUNT(*) FILTER (WHERE outcome='DNA') / COUNT(*), 1) AS dna_rate
FROM appointments GROUP BY specialty;
```

Answer

Three problems. `100 *` with two integer counts gives integer division — every rate rounds to 0 unless it's exactly 100%. Use `100.0`. `specialty` isn't a column of `appointments` (it's on `referrals`) so this needs a join. And there's no minimum-denominator guard, so a specialty with two appointments and one DNA reports a 50% rate.

19.3 Error messages and what they usually mean

Postgres error	Usual cause
column "x" must appear in the GROUP BY clause	selected a column that's neither grouped nor aggregated
aggregate functions are not allowed in WHERE	use HAVING
column "revenue" does not exist in WHERE	referencing a SELECT alias too early
subquery must return only one column	scalar subquery selecting several columns
more than one row returned by a subquery used as an expression	scalar subquery matched multiple rows — your key isn't unique
division by zero	missing <code>NULLIF(denominator, 0)</code>
operator does not exist: text = integer	type mismatch on a join or comparison
syntax error at or near ")"	trailing comma in a column list, or an unclosed bracket above
subquery in FROM must have an alias	derived table with no alias
window functions are not allowed in WHERE	wrap in a subquery/CTE and filter outside
relation "x" does not exist	typo, wrong schema, or missing <code>search_path</code>
INSERT has more expressions than target columns	column list and values out of step

19.4 Sanity checks to run before shipping any query

```

-- 1. row counts before and after each join
SELECT COUNT(*) FROM orders; -- baseline
SELECT COUNT(*) FROM orders JOIN order_items USING (order_id); -- expect more, know why

-- 2. is the key unique?
SELECT key, COUNT(*) FROM your_result GROUP BY key HAVING COUNT(*) > 1;

-- 3. any unexpected NULLs in the output?
SELECT COUNT(*) FILTER (WHERE metric IS NULL) FROM your_result;

-- 4. does the total reconcile to a simpler query?
SELECT SUM(revenue) FROM your_result;
SELECT SUM(quantity*unit_price*(1-discount_pct)) FROM order_items
WHERE order_id IN (SELECT order_id FROM orders WHERE status='completed');

-- 5. spot-check one entity by hand
SELECT * FROM orders WHERE customer_id = 1;

```

If you do nothing else before sending a number to a stakeholder, do check 5. Picking one customer and verifying their figure by hand catches grain errors, filter errors and join errors all at once, in about ninety seconds.

Part 20 — Cheat Sheets, and the SQL Mastery Checklist

Revision material. Read this the night before, not for the first time.

20.1 Query syntax and evaluation order

```
WITH cte AS ( ... )
SELECT DISTINCT col, AGG(col) AS alias
FROM table t
JOIN other o ON o.key = t.key
WHERE row_condition
GROUP BY col
HAVING group_condition
WINDOW w AS (PARTITION BY x ORDER BY y)
ORDER BY col DESC NULLS LAST
LIMIT n OFFSET m;
```

Logical evaluation order — the source of most "why can't I..." questions:

FROM/JOIN → WHERE → GROUP BY → HAVING → window functions → SELECT → DISTINCT → ORDER BY → LIMIT

- Aliases from SELECT are visible in **ORDER BY** and (in Postgres) **GROUP BY**, never in **WHERE** or **HAVING**.
- Aggregates: not in WHERE. Window functions: not in WHERE or HAVING — wrap in a subquery.

20.2 JOIN decision tree

```
Do I need columns from the other table?
├ No, just testing presence
│   ├── "has at least one" → EXISTS
│   └── "has none" → NOT EXISTS (never NOT IN with nullable keys)
└ Yes, I need its columns
    ├── Only rows that match both sides → INNER JOIN
    ├── Every row of the main table, matched or not → LEFT JOIN
    │   └── filtering the optional table? → condition goes in ON, not WHERE
    ├── Every row of both sides (reconciliation) → FULL OUTER JOIN
    ├── Every combination (scaffolding, calendars) → CROSS JOIN
    ├── Top N rows per outer row → LATERAL
    └ The table joined to itself (hierarchy, pairs) → SELF JOIN
```

Before every join, ask: what's the grain of each side? A one-to-many join multiplies the one side. Measures from the "one" side must be aggregated separately or you double-count.

20.3 GROUP BY and HAVING

	WHERE	HAVING
Filters	rows	groups
Runs	before grouping	after grouping
Aggregates allowed	no	yes
Use for	row-level conditions	aggregate thresholds

```
SELECT dept, COUNT(*) FROM employees
WHERE hire_date >= '2020-01-01' -- row filter
GROUP BY dept
HAVING COUNT(*) > 5;           -- group filter
```

Every non-aggregated SELECT column must be in GROUP BY (exception: columns functionally determined by a grouped primary key).

GROUPING SETS, **ROLLUP**, **CUBE** produce multiple grouping levels including subtotals in one pass. **GROUPING(col)** distinguishes a subtotal NULL from a data NULL.

20.4 CASE

```

CASE WHEN cond THEN val
      WHEN cond THEN val
      ELSE val END

```

- First match wins; later branches implicitly exclude earlier ones.
- **Put the NULL branch first** — NULL fails every comparison and falls into ELSE.
- Always write ELSE explicitly.
- All branches must return compatible types.
- `CASE x WHEN NULL` never matches; use the searched form with `IS NULL`.

Conditional aggregation:

```

SUM(CASE WHEN c THEN 1 ELSE 0 END) -- portable
COUNT(*) FILTER (WHERE c)        -- Postgres, cleaner
SUM(x) FILTER (WHERE c)

```

20.5 NULL handling

Expression	Result
<code>NULL = NULL</code>	NULL
<code>NULL IS NULL</code>	true
<code>1 + NULL</code>	NULL
<code>'a' NULL</code>	NULL
<code>CONCAT('a', NULL)</code>	'a'
<code>NOT NULL</code>	NULL
<code>COUNT(*)</code>	includes NULL rows
<code>COUNT(col), SUM, AVG, MIN, MAX</code>	ignore NULLs
<code>SUM</code> over zero rows	NULL, not 0
<code>GROUP BY</code>	all NULLs form one group
<code>COUNT(DISTINCT col)</code>	excludes NULL entirely
<code>ORDER BY ASC</code>	NULLs last (Postgres default)
<code>UNIQUE</code> constraint	permits multiple NULLs

```

COALESCE(a, b, c) -- first non-NULL
NULLIF(a, b)      -- NULL if a = b; use for /0 guards
a IS DISTINCT FROM b -- NULL-safe <>
a IS NOT DISTINCT FROM b -- NULL-safe =

```

The four traps to recite: `NOT IN` with NULLs returns nothing; `<> value` silently drops NULL rows; `LEFT JOIN + COUNT(*)` returns 1 for non-matches; `AVG` excludes rather than zeroes.

20.6 Date functions

```

CURRENT_DATE, CURRENT_TIMESTAMP, NOW()
DATE_TRUNC('month'|'week'|'day'|'quarter'|'year', ts)
EXTRACT(YEAR|QUARTER|MONTH|DAY|ISODOW|WEEK|HOUR|EPOCH FROM ts)
AGE(a, b) -- readable interval
ts + INTERVAL '30 days' -- arithmetic
date - date -- integer days
timestamp - timestamp -- interval
EXTRACT(EPOCH FROM (b - a))/3600 -- hours as a number
TO_CHAR(ts, 'YYYY-MM' | 'DD/MM/YYYY' | 'Day')
generate_series(start, stop, INTERVAL '1 day')

```

Standard windows:

```

order_ts >= CURRENT_DATE - INTERVAL '30 days' -- rolling 30d
order_ts >= DATE_TRUNC('month', CURRENT_DATE) -- MTD
order_ts >= DATE_TRUNC('year', CURRENT_DATE) -- YTD
order_ts >= DATE_TRUNC('month', CURRENT_DATE) - INTERVAL '1 month'
AND order_ts < DATE_TRUNC('month', CURRENT_DATE) -- last full month

```

- **Half-open ranges always:** `>= start AND < next_start`. Never `BETWEEN` on timestamps.
- Postgres weeks start Monday (ISO). Many US tools start Sunday.
- UK financial year: `DATE_TRUNC('year', ts - INTERVAL '3 months') + INTERVAL '3 months'`.
- `EXTRACT(MONTH)` merges years; `DATE_TRUNC('month')` doesn't.

20.7 String functions

```
a || b          CONCAT(a,b)          CONCAT_WS(sep, a, b, c)
LOWER UPPER INITCAP
TRIM LTRIM RTRIM BTRIM(x, chars)
LENGTH LPAD(x,n,c) RPAD
LEFT(x,n) RIGHT(x,n) SUBSTRING(x FROM n FOR len) SUBSTRING(x FROM 'regex')
POSITION(sub IN x) STRPOS(x, sub)
REPLACE(x, from, to) TRANSLATE(x, from, to)
SPLIT_PART(x, delim, n) STRING_TO_ARRAY UNNEST
x ~ 're' x ~* 're' x !~ 're'
REGEXP_REPLACE(x, 're', 'to', 'g')
STRING_AGG(x, ',' ORDER BY y)
```

`||` returns NULL if either side is NULL; `CONCAT_WS` skips NULLs. That's the one to remember.

20.8 Window functions

```
func() OVER (PARTITION BY a ORDER BY b ROWS BETWEEN x AND y)
```

Function	Returns
<code>ROW_NUMBER()</code>	1,2,3,4 — no ties
<code>RANK()</code>	1,2,2,4 — ties share, then gap
<code>DENSE_RANK()</code>	1,2,2,3 — ties share, no gap
<code>NTILE(n)</code>	equal-count buckets
<code>PERCENT_RANK(), CUME_DIST()</code>	relative position 0–1
<code>LAG(x, n, default)</code>	value n rows back
<code>LEAD(x, n, default)</code>	value n rows forward
<code>FIRST_VALUE(x)</code>	first in frame
<code>LAST_VALUE(x)</code>	last in frame — needs an explicit frame
<code>NTH_VALUE(x, n)</code>	nth in frame
<code>SUM/AVG/COUNT/MIN/MAX ... OVER</code>	aggregate over the frame

Frames:

```
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW -- running total
ROWS BETWEEN 6 PRECEDING AND CURRENT ROW -- 7-row moving window
RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW -- gap-safe by calendar
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING -- whole partition
ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING -- everything before this row
```

Defaults: no `ORDER BY` → whole partition. With `ORDER BY`, no frame → `RANGE UNBOUNDED PRECEDING TO CURRENT ROW` (ties share a value — specify `ROWS` if you don't want that).

Window functions run after `GROUP BY`, so `SUM(COUNT(*)) OVER ()` is legal and useful. They cannot be used in `WHERE` or `HAVING`.

20.9 CTEs and subqueries

```
WITH a AS ( ... ),
     b AS ( SELECT ... FROM a ),
     c AS MATERIALIZED ( ... ) -- force single computation
SELECT ... FROM b JOIN c USING (k);

WITH RECURSIVE t AS (
  anchor_query
```

```

UNION ALL
recursive_query_referencing_t -- add a depth guard
)
SELECT * FROM t;

```

Need	Use
Presence/absence test	<code>EXISTS / NOT EXISTS</code>
Columns from the other table	<code>JOIN</code>
Aggregate before joining	CTE or derived table
Reuse a result	CTE
Per-row correlated logic	correlated subquery — but check a window function first
Top N per outer row	<code>LATERAL</code>

Postgres 12+ inlines single-use CTEs. Older versions always materialise. Some cloud warehouses recompute a CTE per reference.

20.10 Common analytical patterns

```

-- latest per group
SELECT DISTINCT ON (k) * FROM t ORDER BY k, ts DESC, id DESC;

-- top N per group
... WHERE rn <= N -- from ROW_NUMBER/DENSE_RANK OVER (PARTITION BY g ORDER BY m DESC)

-- deduplicate
... WHERE rn = 1 -- ROW_NUMBER OVER (PARTITION BY key ORDER BY updated_at DESC, id DESC)

-- running total
SUM(x) OVER (ORDER BY d ROWS UNBOUNDED PRECEDING)

-- moving average
AVG(x) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)

-- percent of total
100.0 * x / SUM(x) OVER ()

-- period-over-period
x - LAG(x) OVER (ORDER BY period)

-- gaps and islands
d - ROW_NUMBER() OVER (PARTITION BY k ORDER BY d) -- constant within a run

-- sessionise / group on change
SUM(is_new_group) OVER (PARTITION BY k ORDER BY ts)

-- anti-join
WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.k = a.k)

-- relational division ("bought every X")
HAVING COUNT(DISTINCT x) = (SELECT COUNT(*) FROM all_x)

-- zero-fill a series
generate_series(...) LEFT JOIN data ON ... → COALESCE(...,0)

```

20.11 KPI formulas

Metric	Formula	Watch
Revenue	$\sum \text{qty} \times \text{price} \times (1 - \text{discount})$	which statuses; gross vs net; shipping
Gross profit	$\text{revenue} - (\text{qty} \times \text{cost})$	cost at time of sale vs current cost
Gross margin %	$\text{gross profit} \div \text{revenue}$	non-additive — recompute, don't average
AOV	$\text{revenue} \div \text{orders}$	aggregate to order grain first
Conversion rate	$\text{converters} \div \text{eligible}$	sessions or users?; <code>100.0 *</code>
Retention (month N)	$\text{active in N} \div \text{cohort size}$	LEFT JOIN; censoring
Churn rate	$\text{lost} \div \text{active at start}$	define "lost" before writing SQL

Metric	Formula	Watch
CAC	spend ÷ new customers	attribution model dependent
LTV (historic)	Σ revenue per customer	biased by tenure — normalise by cohort age
Repeat rate	customers with ≥2 orders ÷ all	must be cohorted
DAU / MAU	distinct actives per period	distinct counts aren't additive
Stickiness	DAU ÷ MAU	interpret against product type
Growth %	(curr – prev) ÷ prev	meaningless off a tiny base
Share of total	part ÷ SUM(part) OVER ()	internal share ≠ market share
Mean wait	Σ (end – start) ÷ n	completed only = survivorship bias
SLA compliance	within-target ÷ completed	what about still-open cases?
Breach rate	over-target ÷ completed	consider COALESCE(end, now())

Every rate needs `NULLIF(denominator, 0)` and a minimum-denominator guard.

20.12 Dialect differences

Task	Postgres	SQL Server	MySQL	BigQuery
Limit rows	<code>LIMIT n</code>	<code>TOP n / OFFSET...FETCH</code>	<code>LIMIT n</code>	<code>LIMIT n</code>
String concat	<code> / CONCAT</code>	<code>+ / CONCAT</code>	<code>CONCAT</code>	<code> / CONCAT</code>
Case-insensitive match	<code>ILIKE</code>	<code>LIKE (collation)</code>	<code>LIKE (collation)</code>	<code>LOWER()</code>
Current date	<code>CURRENT_DATE</code>	<code>GETDATE()</code>	<code>CURDATE()</code>	<code>CURRENT_DATE()</code>
Date add	<code>+ INTERVAL '1 day'</code>	<code>DATEADD(day,1,d)</code>	<code>DATE_ADD(d,INTERVAL 1 DAY)</code>	<code>DATE_ADD(d,INTERVAL 1 DAY)</code>
Date diff	<code>d2 - d1</code>	<code>DATEDIFF(day,d1,d2)</code>	<code>DATEDIFF(d2,d1)</code>	<code>DATE_DIFF(d2,d1,DAY)</code>
Truncate to month	<code>DATE_TRUNC('month',d)</code>	<code>DATEFROMPARTS(...)</code>	<code>DATE_FORMAT(d,'%Y-%m-01')</code>	<code>DATE_TRUNC(d, MONTH)</code>
Null fallback	<code>COALESCE</code>	<code>ISNULL/COALESCE</code>	<code>IFNULL/COALESCE</code>	<code>IFNULL/COALESCE</code>
Conditional aggregate	<code>FILTER (WHERE ...)</code>	<code>SUM(CASE ...)</code>	<code>SUM(CASE ...)</code>	<code>COUNTIF()</code>
Latest per group	<code>DISTINCT ON</code>	<code>ROW_NUMBER</code>	<code>ROW_NUMBER (8+)</code>	<code>ARRAY_AGG(... LIMIT 1)[OFFSET(0)]</code>
Split string	<code>SPLIT_PART</code>	<code>STRING_SPLIT</code>	<code>SUBSTRING_INDEX</code>	<code>SPLIT</code>
Regex	<code>~, REGEXP_REPLACE</code>	limited	<code>REGEXP</code>	<code>REGEXP_CONTAINS</code>
String aggregation	<code>STRING_AGG</code>	<code>STRING_AGG</code>	<code>GROUP_CONCAT</code>	<code>STRING_AGG</code>

`COALESCE`, `CASE`, window functions, CTEs and standard joins work everywhere — build your habits on those and treat the rest as translation.

FINAL SQL MASTERY CHECKLIST

Five levels. Each one names exactly what you should be able to solve **unaided, without notes, from a written problem statement**. Don't move on until the previous level is automatic — interviews test recall under pressure, and half-known material collapses.

Level 1 — Basic SQL

Target: you can answer simple questions about a single table.

You can, without help:

- ☐ Read a schema and state the grain of each table ("one row per what?")
- ☐ Write SELECT / FROM / WHERE / ORDER BY / LIMIT fluently
- ☐ Filter with =, <>, >, <, IN, BETWEEN, LIKE, IS NULL
- ☐ Combine conditions with AND/OR/NOT and parenthesise OR correctly
- ☐ Use COUNT, SUM, AVG, MIN, MAX
- ☐ Group with GROUP BY and filter groups with HAVING
- ☐ Explain the difference between WHERE and HAVING
- ☐ Explain the difference between COUNT(*) and COUNT(column)
- ☐ Explain why x = NULL never matches
- ☐ Use DISTINCT and say when it's masking a problem

Test yourself: questions 1–20 of the beginner bank, in under two minutes each.

Level 2 — Job-ready SQL

Target: you can produce correct numbers from a real multi-table schema.

Everything above, plus:

- ☐ INNER and LEFT JOIN across three or four tables without hesitation
- ☐ Explain why a LEFT JOIN filter belongs in ON, not WHERE
- ☐ Use COUNT(right.key) rather than COUNT(*) after a LEFT JOIN
- ☐ Recognise fan-out and aggregate to a common grain before combining measures
- ☐ Write CASE for segmentation, banding and conditional aggregation
- ☐ Handle NULLs with COALESCE, NULLIF, IS DISTINCT FROM
- ☐ Explain the NOT IN NULL trap and write the NOT EXISTS alternative
- ☐ Use DATE_TRUNC and EXTRACT correctly, and know which merges years
- ☐ Write half-open date ranges by default
- ☐ Structure a query with CTEs instead of nested subqueries
- ☐ Guard every rate with NULLIF(denominator, 0) and 100.0 *
- ☐ Self-check a result: row counts, uniqueness, one entity verified by hand

Test yourself: the whole beginner bank, plus intermediate questions 1–20.

Level 3 — Interview-ready SQL

Target: you can pass the technical round for a Data Analyst role.

Everything above, plus:

- ☐ Window functions: `OVER`, `PARTITION BY`, `ORDER BY`, and frames
- ☐ `ROW_NUMBER` vs `RANK` vs `DENSE_RANK` — and when each is right
- ☐ Latest-record-per-group three ways, and pick one with a reason
- ☐ Top N per group, including the ties conversation
- ☐ `LAG/LEAD` for period-over-period change
- ☐ Running totals and moving averages, with the correct frame
- ☐ Percentage of total via a window aggregate
- ☐ Deduplicate with `ROW_NUMBER`, including a deterministic tie-breaker
- ☐ `EXISTS/NOT EXISTS` and when to prefer them over joins
- ☐ Explain SQL's logical evaluation order and what follows from it
- ☐ Explain the `LAST_VALUE` frame trap
- ☐ Debug someone else's broken query and name each problem
- ☐ Say "that definition is ambiguous — here's what I'd confirm" naturally

Test yourself: the full intermediate bank. Mock interviews 1–5, out loud.

Level 4 — Strong Data Analyst SQL

Target: you stand out against other candidates for the same role.

Everything above, plus:

- ☐ Build a cohort retention table from scratch, and name the month-0 sanity check
- ☐ Compute retention, churn and repeat rate, and say what each definition assumes
- ☐ Build a funnel at the right grain, ordered or unordered, and know the difference
- ☐ Solve gaps and islands two ways and explain both
- ☐ Sessionise raw events with an inactivity timeout
- ☐ Pareto and contribution analysis, with the parts summing to the whole
- ☐ Anomaly detection that accounts for day-of-week seasonality
- ☐ RFM segmentation, with the NTILE directions right
- ☐ Recognise censoring, survivorship bias and mix effects — unprompted
- ☐ Translate a business KPI definition into SQL and challenge the definition
- ☐ Read `EXPLAIN ANALYZE` well enough to name the bottleneck
- ☐ Know why a query is slow *and* whether it's also wrong
- ☐ Assemble a wide customer-360 table with no fan-out

Test yourself: advanced bank questions 1–40. Mock interviews 6–8.

Level 5 — Advanced SQL

Target: you could take a senior analyst's technical round, or lead analysis independently.

Everything above, plus:

- ☐ Recursive CTEs for hierarchies, with depth guards and cycle detection
- ☐ Point-in-time joins against SCD Type 2 dimensions
- ☐ Price/volume decomposition of a revenue change
- ☐ Cohort-normalised LTV that's fairly comparable across cohorts
- ☐ Market basket lift, not just raw co-occurrence

- [] Reconciliation between two sources with `FULL OUTER JOIN` and `IS DISTINCT FROM`
- [] Fuzzy deduplication with blocking, and know why blocking is mandatory
- [] Multi-touch attribution, and articulate what each model distorts
- [] Design a summary table and an incremental refresh strategy
- [] Explain normalisation, star schemas, grain and additivity confidently
- [] Explain CTE materialisation behaviour and how it differs across engines
- [] Investigate an open-ended business question end to end — decompose, localise, rule out measurement artefacts, and check whether the change is even outside normal variation
- [] Know when SQL is the wrong tool and say so

Test yourself: the full advanced bank. Mock interviews 9 and 10, out loud, with someone pushing back.

The last word

The queries in this handbook are the easy part; you'll internalise them with practice. What actually separates candidates in Data Analyst interviews is smaller and less technical:

Asking what counts as revenue before computing it. Noticing the grain changed after a join. Saying "the average only covers completed pathways, which excludes the longest waiters." Checking whether a metric dropped or the tracking broke. Reporting a median next to a mean. Refusing to give a percentage when the base is four.

Those instincts come from doing the work, not from reading about it. Take the RetailCo schema, load it into a local Postgres, invent your own questions, and get things wrong in private until the checks in Part 19.4 become reflexive.

Then, in the room, think out loud. An interviewer cannot see a silent correct answer any better than a silent wrong one.